ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: ĐÁNH GIÁ VÀ KIỂM ĐỊNH CHẤT LƯỢNG PHẦN MỀM

ĐỀ TÀI: XÂY DỰNG VÀ KIỂM ĐỊNH CHẤT LƯỢNG HỆ THỐNG RELIEFHUB - HỆ THỐNG QUẢN LÝ CỨU TRỢ THIÊN TAI

Nhóm thực hiện:	Nhóm 18
Lớp học phần:	Đánh giá và kiểm định chất lượng phần mềm*-1-1-26(COUR01.LT1)
Thành viên thực hiện:	Nguyễn Thị Thu Giang - 23010871
	Ngô Thị Minh Phương - 23012156
Giảng viên hướng dẫn:	TS. Trịnh Thanh Bình

Hà Nội, 2026

PHÂN CÔNG NHIỆM VỤ

Bảng 0.1: Phân công nhiệm vụ các thành viên trong nhóm thực hiện

STT	Họ và tên	MSV	Nhiệm vụ	Đóng góp
1	**Nguyễn Thị Thu Giang**	23010871	- Khảo sát kiến trúc Backend, CSDL MySQL và mã nguồn Spring Boot. - Phân tích đồ thị luồng điều khiển (CFG), tính toán độ phức tạp Cyclomatic V(G) của 5 phương thức nghiệp vụ lõi. - Thiết kế và thực thi tự động 53 ca kiểm thử tích hợp Backend (TC-001 → TC-053) trên JUnit 5, Spring Boot MockMvc và CSDL H2 In-Memory. - Đo lường và phân tích độ bao phủ mã nguồn JaCoCo 0.8.12. - Lập hồ sơ kỹ thuật các lỗi Backend (BUG-001 → BUG-005, BUG-008 → BUG-010).	**50%**
2	**Ngô Thị Minh Phương**	23012156	- Xây dựng kế hoạch kiểm thử theo chuẩn IEEE 829 và hệ thống 8 tiêu chí chất lượng ISO/IEC 25010. - Hệ thống hóa 28 yêu cầu (REQ-001 → REQ-028) và thiết lập Ma trận truy xuất yêu cầu hai chiều (RTM). - Thiết kế kịch bản kiểm thử hộp đen (EP, BVA, State Transition) và thực thi 7 ca kiểm thử giao diện WebUI (TC-054 → TC-060) qua Chrome CDP. - Thực thi công cụ phân tích tĩnh Frontend (Oxlint 1.69.0) và kiểm tra đóng gói tối ưu bundle (Vite 8.1.0). - Lập hồ sơ kỹ thuật các lỗi Frontend và tích hợp (BUG-006, BUG-007, BUG-011, BUG-012).	**50%**

LỜI CẢM ƠN

Lời đầu tiên, nhóm thực hiện xin bày tỏ lòng biết ơn sâu sắc và sự kính trọng chân thành nhất đến **thầy TS. Trịnh Thanh Bình** - giảng viên hướng dẫn học phần **Đánh giá và Kiểm định Chất lượng Phần mềm**, Trường Công nghệ Thông tin Phenikaa. Trong suốt quá trình theo học học phần và thực hiện bài tập lớn, thầy đã tận tình giảng dạy, truyền đạt cho chúng em những kiến thức nền tảng vững chắc về lý thuyết kiểm thử phần mềm, các tiêu chuẩn đánh giá chất lượng quốc tế cũng như những kinh nghiệm thực tiễn quý báu trong công tác kiểm định và đảm bảo chất lượng phần mềm.

Thông qua quá trình thực hiện đề tài ***"Xây dựng và kiểm định chất lượng hệ thống ReliefHub - hệ thống quản lý cứu trợ thiên tai"***, dưới sự định hướng khoa học, những nhận xét sâu sắc và sự chỉ dẫn tỉ mỉ của thầy, nhóm chúng em đã có cơ hội vận dụng toàn diện các kỹ thuật kiểm thử vào một hệ thống phần mềm thực tế - từ khảo sát kiến trúc và phân tích mã nguồn tĩnh, thiết kế kiểm thử hộp trắng dựa trên đồ thị luồng điều khiển (CFG) và độ phức tạp Cyclomatic, thiết kế kiểm thử hộp đen, xây dựng bộ kiểm thử tự động trên JUnit 5, Spring Boot MockMvc và đo lường độ bao phủ mã nguồn JaCoCo, cho đến kiểm thử giao diện WebUI, truy vết lỗi và lượng hóa khoảng cách chất lượng theo tiêu chuẩn ISO/IEC 25010. Những góp ý nghiêm khắc nhưng đầy tâm huyết của thầy không chỉ giúp nhóm hoàn thiện báo cáo một cách trung thực, chuẩn xác và có căn cứ thực nghiệm rõ ràng, mà còn rèn luyện cho chúng em tư duy phản biện, tác phong làm việc khoa học và trách nhiệm nghề nghiệp của người kỹ sư phần mềm.

Bên cạnh đó, nhóm chúng em xin trân trọng gửi lời cảm ơn đến Ban Giám hiệu Đại học Phenikaa, Ban Lãnh đạo cùng toàn thể quý thầy cô giảng viên Trường Công nghệ Thông tin Phenikaa đã luôn tạo điều kiện thuận lợi về môi trường học tập, cơ sở vật chất và chương trình đào tạo hiện đại để chúng em được trang bị nền tảng kiến thức chuyên môn vững vàng trong suốt quá trình học tập tại trường.

Mặc dù nhóm đã nỗ lực hết mình trong việc khảo sát hệ thống, thiết kế kịch bản kiểm thử, thu thập bằng chứng thực nghiệm và biên tập tài liệu, song do thời gian thực hiện cũng như kinh nghiệm thực tế của sinh viên còn những hạn chế nhất định, báo cáo khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong tiếp tục nhận được những ý kiến đánh giá, chỉ bảo và góp ý quý báu từ thầy để có thể hoàn thiện hơn nữa kiến thức và kỹ năng chuyên môn trong tương lai.

Một lần nữa, nhóm chúng em xin gửi tới **thầy Trịnh Thanh Bình** lời tri ân sâu sắc nhất. Kính chúc thầy luôn dồi dào sức khỏe, hạnh phúc và gặt hái thêm nhiều thành tựu trong sự nghiệp giảng dạy cũng như nghiên cứu khoa học.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

PHÂN CÔNG NHIỆM VỤ 2

LỜI CẢM ƠN 3

DANH MỤC HÌNH ẢNH 8

DANH MỤC BẢNG BIỂU 12

LỜI MỞ ĐẦU 15

1. Lý do chọn đề tài và bối cảnh thực tiễn 15

2. Mục tiêu và phạm vi của báo cáo kiểm thử 16

3. Cấu trúc tổng thể của báo cáo 16

CHƯƠNG 1: TỔNG QUAN NGHIỆP VỤ, USE CASE VÀ ĐẶC TẢ YÊU CẦU HỆ THỐNG 18

1.1. Bài toán thực tế, mục tiêu và quy trình nghiệp vụ tổng thể 18

1.1.1. Bài toán thực tế và mục tiêu nghiệp vụ của ReliefHub 18

1.2. Đối tượng sử dụng (Actors) và phân quyền nghiệp vụ (RBAC) 20

1.3. Hệ thống sơ đồ Use Case và danh mục 24 Use Cases nghiệp vụ 22

1.3.1. Sơ đồ Use Case tổng quan của hệ thống ReliefHub 22

1.3.2. Sơ đồ phân rã Use Case theo 6 nhóm tác nhân (Hình 1.4 → Hình 1.9) 22

1.3.3. Danh mục 24 Use Cases nghiệp vụ chính thức và ánh xạ hiện thực hóa 26

1.4. Đặc tả yêu cầu chức năng (REQ-001 → REQ-022) 28

1.5. Đặc tả yêu cầu phi chức năng (REQ-023 → REQ-028) 34

CHƯƠNG 2: KIẾN TRÚC KỸ THUẬT, THIẾT KẾ HỆ THỐNG VÀ GIAO DIỆN SẢN PHẨM 37

2.1. Kiến trúc kỹ thuật 3 tầng và hiện trạng tích hợp Frontend – Backend 37

2.1.1. Sơ đồ kiến trúc mức cao và phân tầng REST API 37

2.1.2. Cấu trúc mã nguồn thực tế và vị trí đứt gãy tích hợp (BUG-006) 38

2.2. Thiết kế sơ đồ lớp phân tích (Class Diagrams) 40

2.2.1. Biểu đồ lớp phân tích tổng thể hệ thống ReliefHub 40

2.2.2. Sơ đồ lớp chi tiết các phân hệ Xác thực, Quyên góp và Yêu cầu hỗ trợ 41

2.3. Thiết kế biểu đồ tuần tự (Sequence Diagrams) và máy trạng thái 42

2.3.1. Biểu đồ tuần tự Đăng ký, Đăng nhập và Xác thực JWT 42

2.3.2. Biểu đồ tuần tự Quyên góp, Xác minh và Quản lý Sổ cái 44

2.3.3. Biểu đồ tuần tự Yêu cầu hỗ trợ, Máy trạng thái 6 bước và Vận chuyển 47

2.4. Thiết kế Cơ sở dữ liệu (ERD) và đối chiếu hiện thực hóa ORM 50
2.5. Hệ thống giao diện người dùng thực tế theo phân hệ (UI Screens) 52
2.5.1. Giao diện Xác thực, Quản trị tài khoản, Thiên tai và Chiến dịch 52
2.5.2. Giao diện Nhà tài trợ và Ban Tài chính 54
2.5.3. Giao diện Người dân, Điều phối cứu trợ và Quản lý kho 57
CHƯƠNG 3: CƠ SỞ LÝ THUYẾT, TIÊU CHÍ CHẤT LƯỢNG VÀ KẾ HOẠCH KIỂM THỬ 59
3.1. Thuật ngữ, từ viết tắt và người sử dụng tài liệu 59
3.1.1. Định nghĩa thuật ngữ và Từ viết tắt 59
3.1.2. Người sử dụng tài liệu và Vai trò kiểm thử 60
3.2. Cơ sở lý thuyết kiểm thử và mô hình chất lượng ISO/IEC 25010 61
3.3. Hệ thống 8 tiêu chí đánh giá chất lượng (QC-01 → QC-08) 61
3.4. Chiến lược, hình thức và loại kiểm thử 63
3.4.1. Quy trình đánh giá chất lượng 7 bước 63
3.4.2. Các hình thức kiểm thử áp dụng (4 hình thức) 64
3.4.3. Đặc tả 3 loại kiểm thử theo cấu trúc IEEE 829 64
3.5. Môi trường và công cụ kiểm thử 66
3.6. Tiêu chí bắt đầu, tạm dừng, kết thúc và quy trình quản lý lỗi 69
3.7. Rủi ro kế hoạch và lịch trình phân công 70
CHƯƠNG 4: PHÂN TÍCH, THIẾT KẾ 60 TEST CASE VÀ MA TRẬN TRUY XUẤT 73
4.1. Phân tích mã nguồn tĩnh (SF-01 → SF-11) 73
4.2. Phân tích hộp trắng: CFG và độ phức tạp Cyclomatic 75
4.3. Thiết kế kiểm thử hộp đen (EP, BVA, bảng quyết định, chuyển trạng thái) 79
4.4. Đặc tả 60 trường hợp kiểm thử (TC-001 → TC-060) 80
4.4.1. Tổ chức bộ thực thi tự động 80
4.4.2. Đặc tả chi tiết toàn bộ 60 trường hợp kiểm thử 80
4.5. Ma trận truy xuất yêu cầu (RTM) và truy xuất ngược 116
4.5.1. Ma trận truy xuất yêu cầu ↔ Test Case ↔ Lỗi 116
4.5.2. Ma trận truy xuất ngược phát hiện tĩnh ↔ Lỗi 123
4.5.3. Hạng mục thiết kế chưa hiện thực hóa (chưa kiểm thử động) 126
4.5.4. Thống kê độ bao phủ yêu cầu và kết quả thực thi 126

CHƯƠNG 5: KẾT QUẢ THỰC THI KIỂM THỬ ĐỊNH LƯỢNG VÀ BẰNG CHỨNG THỰC NGHIỆM 128

5.1. Kết quả thực thi bộ kiểm thử gốc và 60 test case 128
5.1.1. Bộ 10 bài kiểm thử gốc của dự án (Baseline) 128
5.1.2. 60 trường hợp kiểm thử đánh giá mở rộng 129

5.2. Độ bao phủ mã nguồn Backend (JaCoCo 0.8.12) 131

5.3. Phân tích tĩnh (Oxlint) và kiểm tra đóng gói (Vite) 134
5.3.1. Ảnh chụp kết quả công cụ kiểm thử (Hình 5.1 → Hình 5.10) 135

5.4. Kết quả kiểm thử giao diện WebUI (TC-054 → TC-060) 140
5.4.1. TC-054 (PASS): Đăng nhập và bảo vệ đường dẫn ProtectedRoute 141
5.4.2. TC-055 (PASS): Chặn tài khoản đang chờ phê duyệt 142
5.4.3. TC-056 (FAIL - BUG-011): Lệch mật khẩu tài khoản finance01 143
5.4.4. TC-057 (FAIL – BUG-007): Chiếm quyền admin qua Quên mật khẩu 143
5.4.5. TC-058 (FAIL – BUG-006): Mất đơn cứu trợ do ReferenceError 145
5.4.6. TC-059 (PASS) và TC-060 (FAIL – BUG-006): Mất trạng thái sau F5 146

5.5. Đối chiếu bằng chứng và các hạng mục chưa kiểm thử động 147
5.5.1. Đối chiếu 23 ảnh chụp và 11 nhật ký với Test Case 147
5.5.2. Hạng mục NOT RUN / BLOCKED và hướng thu thập bổ sung 151

CHƯƠNG 6: HỒ SƠ LỖI, ĐÁNH GIÁ CHẤT LƯỢNG VÀ RỦI RO 154

6.1. Hồ sơ chi tiết 12 lỗi phần mềm (BUG-001 → BUG-012) [8] 154
6.1.1. BUG-001 (SF-01): Lỗ hổng leo thang đặc quyền qua API Đăng ký (POST /auth/register) 156
6.1.2. BUG-002 (SF-02): Chặn nhầm quyền truy cập hồ sơ cá nhân (/users/me) của người dùng không phải Admin 158
6.1.3. BUG-003 (SF-03): Thiếu kiểm soát phân quyền theo vai trò trên các API quản trị nhạy cảm 159
6.1.4. BUG-004 (SF-04): Bỏ qua máy trạng thái trong quy trình 6 bước xử lý yêu cầu cứu trợ 160
6.1.5. BUG-005 (SF-05): Lỗi cộng dồn trùng lặp số tiền chiến dịch 161
6.1.6. BUG-006 (SF-06): Frontend chạy trên bộ nhớ RAM, gọi biến chưa khai báo (ReferenceError: API_ENABLED is not defined) và mất dữ liệu khi Reload (F5) 163
6.1.7. BUG-007 (SF-07): Lỗ hổng chiếm đoạt tài khoản qua chức năng Quên mật khẩu 164
6.1.8. BUG-008 (SF-08): Lỗ hổng tải lên tệp độc hại không kiểm tra định dạng ... 165
6.1.9. BUG-009 (SF-09): Lỗ hổng IDOR khi đánh dấu đã đọc thông báo của người khác 167

6.1.10. BUG-010 (SF-10): Thiếu kiểm tra dữ liệu biên (Số âm) và lỗi HTTP 500 khi dữ liệu đầu vào sai định dạng / null 168
6.1.11. BUG-011 (SF-11): Bất đồng bộ mật khẩu khởi tạo của tài khoản finance01 giữa tài liệu/SQL và mã nguồn Frontend 169
6.1.12. BUG-012: Nợ kỹ thuật Frontend: 83 cảnh báo biến không sử dụng (no-unused-vars) và kích thước gói Bundle lớn (1,214.90 kB) 170
6.2. Khoảng cách chất lượng và ma trận rủi ro 171
6.2.1. Khoảng cách chất lượng trên 8 tiêu chí (QC-01 → QC-08) 171
6.2.2. Ma trận đánh giá rủi ro (Likelihood × Impact) 174
6.2.3. Giới hạn thực nghiệm 174
CHƯƠNG 7: ĐỀ XUẤT KHẮC PHỤC, CẢI TIẾN VÀ KẾT LUẬN 176
7.1. Đề xuất giải pháp khắc phục lỗi theo mức ưu tiên (P0 → P2) 176
7.1.1. Giai đoạn 1: Đề xuất giải pháp khắc phục các lỗi mức P0 (Critical và High cốt lõi) 176
7.1.2. Giai đoạn 2: Đề xuất giải pháp khắc phục các lỗi mức P1 (BUG-008, BUG-009, BUG-010) 179
7.1.3. Giai đoạn 3: Đề xuất giải pháp khắc phục các lỗi mức P2 (BUG-011, BUG-012) 179
7.2. Cải tiến kiến trúc tích hợp và kiểm thử tự động (CI/CD) 179
7.3. Kết luận chung 180
TÀI LIỆU THAM KHẢO 181
PHỤ LỤC 182
Phụ lục A: Độ bao phủ JaCoCo chi tiết theo lớp 182
Phụ lục B: Kết quả 53 Test Case Backend và nhật ký 14 ca FAIL 184
B.1. Phản hồi HTTP thực tế của 53 Test Case Backend 184
B.2. Nhật ký 14 Test Case Backend thất bại 190
Phụ lục C: Nhật ký Oxlint, Vite, WebUI và phụ lục kỹ thuật về tính toàn vẹn của nhật ký thực nghiệm 195
C.1. Nhật ký phân tích tĩnh Frontend (Oxlint 1.69.0) 195
C.2. Nhật ký đóng gói Production Build (Vite 8.1.0) 196
C.3. Nhật ký thực thi 7 Test Case WebUI (TC-054 → TC-060) 196
C.4. Phụ lục kỹ thuật về tính toàn vẹn của nhật ký thực nghiệm (Bảng C.1) 198

DANH MỤC HÌNH ẢNH

Số hiệu Hình	Tên hình ảnh / Sơ đồ / Biểu đồ trong báo cáo	Trang
Hình 1.1	Sơ đồ quy trình nghiệp vụ xử lý yêu cầu cứu trợ trên hệ thống ReliefHub	19
Hình 1.2	Sơ đồ phân quyền 6 nhóm tác nhân (Roles) và 10 phân hệ nghiệp vụ cốt lõi (UC01 - UC10)	19
Hình 1.3	Sơ đồ Use Case tổng quan của hệ thống Quản lý cứu trợ thiên tai ReliefHub	22
Hình 1.4	Sơ đồ phân rã Use Case của tác nhân Nhà tài trợ (Donor)	23
Hình 1.5	Sơ đồ phân rã Use Case của tác nhân Người dân vùng thiên tai (Citizen)	23
Hình 1.6	Sơ đồ phân rã Use Case của tác nhân Ban Tài chính (Finance)	24
Hình 1.7	Sơ đồ phân rã Use Case của tác nhân Cán bộ địa phương (Local)	24
Hình 1.8	Sơ đồ phân rã Use Case của tác nhân Tình nguyện viên (Volunteer)	25
Hình 1.9	Sơ đồ phân rã Use Case của tác nhân Quản trị viên hệ thống (Admin)	25
Hình 2.1	Sơ đồ tổng quan kiến trúc mức cao của hệ thống ReliefHub	37
Hình 2.2	Sơ đồ cấu trúc thư mục và phân tầng giao tiếp REST API của ReliefHub	38
Hình 2.3	Sơ đồ đối chiếu kiến trúc thực tế và vị trí đứt gãy tích hợp AppContext.jsx - api.js	39
Hình 2.4	Biểu đồ lớp phân tích tổng thể hệ thống ReliefHub (Class Diagram)	41
Hình 2.5	Sơ đồ lớp phân tích chức năng Xác thực (Đăng ký & Đăng nhập)	41
Hình 2.6	Sơ đồ lớp phân tích chức năng Quyên góp (Donation)	42
Hình 2.7	Sơ đồ lớp phân tích chức năng Yêu cầu hỗ trợ (SupportRequest)	42
Hình 2.8	Biểu đồ tuần tự chức năng Đăng ký và Đăng nhập tài khoản	43

Số hiệu Hình	Tên hình ảnh / Sơ đồ / Biểu đồ trong báo cáo	Trang
Hình 2.9	Biểu đồ tuần tự cơ chế xác thực JWT qua JwtAuthenticationFilter tại Backend	44
Hình 2.10	Biểu đồ tuần tự chức năng Đóng góp và Xác minh khoản đóng góp	45
Hình 2.11	Biểu đồ tuần tự chức năng Quản lý Sổ cái tài chính (Financial Ledger)	46
Hình 2.12	Biểu đồ tuần tự chức năng Gửi yêu cầu hỗ trợ khẩn cấp của người dân	47
Hình 2.13	Sơ đồ máy trạng thái 6 bước của SupportRequest — Đối chiếu Thiết kế và Thực tế	48
Hình 2.14	Biểu đồ tuần tự chức năng Điều phối vận chuyển và giao nhận hàng cứu trợ	49
Hình 2.15	Lược đồ cơ sở dữ liệu lõi (ERD) của hệ thống ReliefHub	50
Hình 2.16	Giao diện Đăng nhập tài khoản (/login)	53
Hình 2.17	Giao diện Quản trị tài khoản người dùng và duyệt quyền (/dashboard/admin/accounts)	53
Hình 2.18	Giao diện Cập nhật thông tin thiên tai (/dashboard/local/disasters)	54
Hình 2.19	Giao diện Quản lý chiến dịch cứu trợ của Quản trị viên (/dashboard/admin/campaigns)	54
Hình 2.20	Giao diện Đóng góp chuyển khoản và tải minh chứng (/dashboard/donor/contribute)	51
Hình 2.21	Giao diện Đóng góp hiện vật cứu trợ (/dashboard/donor/in-kind)	55
Hình 2.22	Giao diện Đối soát và Xác minh đóng góp của Ban Tài chính (/dashboard/finance/verify)	55
Hình 2.23	Giao diện Sổ cái tài chính và theo dõi thu chi minh bạch (/dashboard/finance/ledger)	56
Hình 2.24	Giao diện Gửi yêu cầu hỗ trợ khẩn cấp của Người dân (/dashboard/citizen/request)	57
Hình 2.25	Giao diện Quản lý và Điều phối yêu cầu hỗ trợ 6 bước (/dashboard/admin/support-requests)	57
Hình 2.26	Giao diện Quản lý kho hàng và danh mục tồn kho cứu trợ (/dashboard/admin/inventory)	58

Số hiệu Hình	Tên hình ảnh / Sơ đồ / Biểu đồ trong báo cáo	Trang
Hình 3.1	Sơ đồ quy trình kiểm thử 7 bước theo chuẩn IEEE 829 áp dụng cho ReliefHub	63
Hình 4.1	Đồ thị luồng điều khiển (CFG) của 5 phương thức nghiệp vụ cốt lõi tại Backend	76
Hình 5.1	Báo cáo Maven Surefire 3.1.2 khi thực thi bộ 10 bài test gốc có sẵn (10/10 PASS)	135
Hình 5.2	Báo cáo Maven Surefire 3.1.2 khi thực thi bộ kiểm thử mở rộng (14 phương thức FAIL)	136
Hình 5.3	Kết quả thực thi chi tiết TC-001 → TC-026 (23 PASS, 3 FAIL) trên JUnit 5 & MockMvc	136
Hình 5.4	Kết quả thực thi chi tiết TC-027 → TC-053 (16 PASS, 11 FAIL) trên JUnit 5 & MockMvc	137
Hình 5.5	Báo cáo độ bao phủ mã nguồn JaCoCo 0.8.12 ở trạng thái Baseline (25.53% Line Coverage)	138
Hình 5.6	Báo cáo độ bao phủ mã nguồn JaCoCo 0.8.12 sau bộ kiểm thử mở rộng (78.61% Line Coverage)	138
Hình 5.7	Báo cáo độ bao phủ JaCoCo 0.8.12 chi tiết gói com.reliefhub.service (78.43% Line Coverage)	139
Hình 5.8	Báo cáo độ bao phủ JaCoCo 0.8.12 chi tiết gói com.reliefhub.controller (59.38% Line Coverage)	139
Hình 5.9	Bằng chứng kết quả phân tích mã nguồn tĩnh Frontend bằng Oxlint 1.69.0	140
Hình 5.10	Bằng chứng kết quả đóng gói Production Build bằng Vite 8.1.0	140
Hình 5.11	Kết quả TC-054 (Bước 1 - PASS): Chặn truy cập /dashboard/admin khi chưa đăng nhập	141
Hình 5.12	Kết quả TC-054 (Bước 2 - PASS): Đăng nhập thành công tài khoản donor01 vào Dashboard	141
Hình 5.13	Kết quả TC-054 (Bước 3 - PASS): ProtectedRoute chặn donor01 truy cập trái phép trang Admin	142
Hình 5.14	Kết quả TC-055 (PASS): Chặn đăng nhập tài khoản local03 đang chờ phê duyệt	142
Hình 5.15	Kết quả TC-056 (FAIL - BUG-011): Từ chối đăng nhập finance01 / fin123 do lệch mật khẩu	143
Hình 5.16	Kết quả TC-057 (Bước 1 - FAIL, BUG-007): Xác minh tài khoản admin không cần OTP	143

Số hiệu Hình	Tên hình ảnh / Sơ đồ / Biểu đồ trong báo cáo	Trang
Hình 5.17	Kết quả TC-057 (Bước 2 - FAIL, BUG-007): Đặt lại mật khẩu admin thành công không cần xác thực	144
Hình 5.18	Kết quả TC-057 (Bước 3 - FAIL, BUG-007): Đăng nhập chiếm quyền Dashboard Admin	144
Hình 5.19	Kết quả TC-058 (Bước 1 - FAIL, BUG-006): Thông báo thành công giả tạo khi gửi đơn cứu trợ	145
Hình 5.20	Kết quả TC-058 (Bước 2 - FAIL, BUG-006): Đơn cứu trợ bị mất và lộ đơn của công dân khác	145
Hình 5.21	Kết quả TC-059 (Bước 1 - PASS): Nhà tài trợ gửi minh chứng chuyển khoản 10,000,000đ	146
Hình 5.22	Kết quả TC-059 (Bước 2 - PASS): Admin xác minh khoản đóng góp và cấp mã biên nhận	146
Hình 5.23	Kết quả TC-060 (FAIL - BUG-006): Mất dữ liệu xác minh và phiên đăng nhập sau khi nhấn F5	147
Hình 7.1	Bằng chứng kiểm thử E2E sau khi áp dụng bản vá BUG-006 (POST /api/support-requests [200 OK] & giữ dữ liệu sau F5)	178

DANH MỤC BẢNG BIỂU

Số hiệu Bảng	Tên bảng biểu trong báo cáo	Trang
Bảng 0.1	Phân công nhiệm vụ các thành viên trong nhóm thực hiện	2
Bảng 1.1	Danh mục đối tượng sử dụng (Actors) và phân quyền nghiệp vụ trên hệ thống ReliefHub	20
Bảng 1.2	Danh mục 24 Use Cases nghiệp vụ chính thức của hệ thống ReliefHub và ánh xạ hiện thực hóa	26
Bảng 1.3	Đặc tả chi tiết 22 Yêu cầu Chức năng (Functional Requirements - REQ-001 → REQ-022)	29
Bảng 1.4	Đặc tả 6 Yêu cầu Phi chức năng và Dịch vụ Nền tảng (REQ-023 → REQ-028)	34
Bảng 2.1	Tổng hợp cấu trúc công nghệ và quy mô mã nguồn thực tế của hệ thống ReliefHub	40
Bảng 2.2	Đối chiếu 17 Thực thể JPA đã hiện thực hóa trong mã nguồn Backend với cấu trúc bảng CSDL	51
Bảng 3.1	Bảng định nghĩa các thuật ngữ chuyên môn sử dụng trong kế hoạch kiểm thử	59
Bảng 3.2	Bảng giải nghĩa các từ viết tắt trong báo cáo kiểm thử	60
Bảng 3.3	Hệ thống 8 tiêu chí đánh giá chất lượng (QC-01 → QC-08) theo chuẩn ISO/IEC 25010	61
Bảng 3.4	Cấu hình phần cứng, hệ điều hành và phần mềm nền tảng máy trạm kiểm thử	66
Bảng 3.5	Danh mục các công cụ kiểm thử đã thực thi kèm trạng thái và kết quả tóm tắt	67
Bảng 3.6	Danh mục các công cụ kiểm thử không thực thi, lý do và giới hạn đánh giá tương ứng	68
Bảng 3.7	Ma trận rủi ro kế hoạch kiểm thử và biện pháp giảm thiểu	70
Bảng 3.8	Lịch trình và phân công chi tiết 8 giai đoạn công việc kiểm thử	71
Bảng 4.1	Tổng hợp 11 phát hiện phân tích tĩnh (SF-01 → SF-11) trên Backend, SQL và Frontend	73

Số hiệu Bảng	Tên bảng biểu trong báo cáo	Trang
Bảng 4.2	Tổng hợp độ phức tạp Cyclomatic V(G) và các đường đi cơ sở (Basis Paths) của 5 phương thức cốt lõi	76
Bảng 4.3a	Đặc tả Test Case Nhóm 1: Xác thực, Đăng ký & Bảo mật Token (TC-001 → TC-011)	80
Bảng 4.3b	Đặc tả Test Case Nhóm 2: Quản lý Người dùng & Hồ sơ cá nhân (TC-012 → TC-018)	86
Bảng 4.3c	Đặc tả Test Case Nhóm 3: Quản lý Thiên tai & Chiến dịch cứu trợ (TC-019 → TC-026)	91
Bảng 4.3d	Đặc tả Test Case Nhóm 4: Quyên góp & Sổ cái tài chính (TC-027 → TC-034)	95
Bảng 4.3e	Đặc tả Test Case Nhóm 5: Yêu cầu hỗ trợ 6 bước (TC-035 → TC-040)	100
Bảng 4.3f	Đặc tả Test Case Nhóm 6: Kho hàng, Vận chuyển & Tình nguyện viên (TC-041 → TC-046)	103
Bảng 4.3g	Đặc tả Test Case Nhóm 7: Khiếu nại, Thông báo, Dashboard & Upload (TC-047 → TC-053)	107
Bảng 4.3h	Đặc tả Test Case Nhóm 8: Kiểm thử Giao diện WebUI & Tích hợp Frontend (TC-054 → TC-060)	111
Bảng 4.4	Ma trận Truy xuất Yêu cầu hai chiều (REQ-001 → REQ-028) ↔ Test Case ↔ Kết quả ↔ Mã Lỗi	116
Bảng 4.5	Ma trận Truy xuất Ngược: Phát hiện Tĩnh (SF-01 → SF-11) ↔ Test Case ↔ Mã Lỗi (BUG-001 → BUG-012)	123
Bảng 4.6	Các hạng mục thiết kế CSDL/DTO chưa hiện thực hóa trong mã nguồn (Chưa kiểm thử động)	126
Bảng 4.7	Thống kê tổng hợp độ bao phủ yêu cầu và kết quả thực thi 60 Test Case	126
Bảng 5.1a	Kết quả thực thi bộ 10 bài kiểm thử có sẵn của dự án (Baseline Suite)	128
Bảng 5.1b	Tổng hợp kết quả thực thi 60 trường hợp kiểm thử đánh giá (TC-001 → TC-060) theo 8 nhóm chức năng	129
Bảng 5.2	Đối sánh độ bao phủ mã nguồn JaCoCo theo gói (Baseline vs. Full-Suite)	131

Số hiệu Bảng	Tên bảng biểu trong báo cáo	Trang
Bảng 5.3	Đối sánh độ bao phủ mã nguồn JaCoCo trên 15 lớp nghiệp vụ (com.reliefhub.service)	133
Bảng 5.4	Ma trận đối chiếu 23 hình ảnh minh chứng (Hình 5.1 → Hình 5.23) và 11 tệp nhật ký thực nghiệm	147
Bảng 5.5	Danh mục bằng chứng kiểm thử động còn thiếu (NOT RUN / BLOCKED) và quy trình thu thập bổ sung	151
Bảng 6.1	Bảng tổng hợp 12 lỗi phần mềm phát hiện qua kiểm thử (BUG-001 → BUG-012)	154
Bảng 6.2	Phân tích khoảng cách chất lượng trên 8 tiêu chí QC-01 → QC-08	171
Bảng 6.3	Ma trận đánh giá rủi ro (khả năng xảy ra × mức độ tác động)	174
Bảng A.1	Độ bao phủ mã nguồn JaCoCo của 28 lớp thuộc gói controller, security, exception, config và common	182
Bảng B.1	Kết quả thực thi và phản hồi HTTP thực tế của 53 trường hợp kiểm thử Backend (TC-001 → TC-053)	184
Bảng C.1	Phụ lục kỹ thuật về tính toàn vẹn của 11 tệp nhật ký thực nghiệm (SHA-256)	198

LỜI MỞ ĐẦU

1. Lý do chọn đề tài và bối cảnh thực tiễn

Việt Nam là một trong những quốc gia chịu ảnh hưởng nặng nề nhất của biến đổi khí hậu và thiên tai cực đoan như bão, lũ quét, ngập lụt diện rộng và sạt lở đất, đặc biệt tại khu vực miền Trung và miền núi phía Bắc. Trong các đợt thiên tai khẩn cấp, hoạt động cứu trợ nhân đạo và quyên góp từ thiện đóng vai trò sống còn nhằm hỗ trợ kịp thời lương thực, nước sạch, thuốc men và nhu yếu phẩm cho người dân vùng tâm lũ. Tuy nhiên, quy trình cứu trợ truyền thống thường đối mặt với bốn bài toán lớn:

1. **Thông tin yêu cầu cứu trợ phân tán và thiếu chuẩn hóa:** Người dân vùng thiên tai gửi lời kêu cứu qua mạng xã hội hoặc tin nhắn tự phát, dẫn đến trùng lặp hoặc bỏ sót các hộ gia đình nguy cấp, đồng thời thiếu sự xác minh thực địa của chính quyền địa phương.
2. **Thiếu tính minh bạch và khả năng kiểm toán trong quản lý quỹ từ thiện:** Các nhà hảo tâm khó theo dõi theo thời gian thực khoản tiền/hiện vật của mình đã được xác minh hay chưa, đã phân bổ vào chiến dịch nào và tồn quỹ thực tế còn bao nhiêu.
3. **Đứt gãy trong điều phối kho vận và nhân lực tình nguyện:** Hàng hóa cứu trợ tập kết tại các kho trung tâm không được đối soát tồn kho tự động với nhu cầu thực tế và lộ trình phương tiện vận chuyển.
4. **Rủi ro an toàn thông tin và sai lệch dữ liệu tài chính:** Các nền tảng số hóa quy trình từ thiện nếu không được kiểm thử nghiêm ngặt về phân quyền truy cập (*RBAC*), tính bất biến của giao dịch tài chính (*Idempotency*) và đồng bộ dữ liệu liên tầng sẽ tiềm ẩn nguy cơ gian lận quỹ, leo thang đặc quyền hoặc làm mất đơn cầu cứu của người dân.

Xuất phát từ thực tiễn đó, dự án **ReliefHub** (*Hệ thống Quản lý Cứu trợ Thiên tai và Quyên góp Từ thiện*) được phát triển theo kiến trúc Web Full-stack hiện đại (Spring Boot 3.2.5 [6] kết hợp React 19.2.7 [9]) nhằm số hóa khép kín chuỗi nghiệp vụ từ tiếp nhận quyên góp, quản lý sổ cái tài chính, tiếp nhận yêu cầu hỗ trợ 6 bước đến điều phối kho hàng và đội ngũ tình nguyện viên. Chính vì vậy, nhóm thực hiện đề tài **"Xây dựng và kiểm định chất lượng hệ thống ReliefHub - hệ thống quản lý cứu trợ thiên tai"** với đối tượng kiểm thử (*System Under Test - SUT*) là hệ thống **ReliefHub**, qua đó đánh giá toàn diện mức độ sẵn sàng vận hành, tính chính xác của nghiệp vụ tài chính và độ an toàn bảo mật của một hệ thống thông tin nhân đạo trọng yếu.

2. Mục tiêu và phạm vi của báo cáo kiểm thử

Báo cáo kiểm thử này được xây dựng nhằm thực hiện các mục tiêu cốt lõi sau:

- **Hệ thống hóa đặc tả nghiệp vụ và kiến trúc sản phẩm:** Tái cấu trúc đầy đủ bài toán nghiệp vụ, sơ đồ Use Case (24 Use Cases thuộc 6 phân hệ), yêu cầu chức năng (REQ-001 → REQ-022), yêu cầu phi chức năng (REQ-023 → REQ-028), kiến trúc kỹ thuật 3 tầng, sơ đồ lớp, sơ đồ tuần tự, thiết kế cơ sở dữ liệu và hệ thống giao diện người dùng của ReliefHub.
- **Lập kế hoạch và thiết kế bộ kiểm thử chuẩn hóa theo IEEE 829 [1] & ISO/IEC 25010 [2]:** Xây dựng 8 tiêu chí chất lượng định lượng (QC-01 → QC-08), phân tích đồ thị dòng điều khiển (*CFG*) [3] cho 5 phương thức nghiệp vụ cốt lõi và thiết kế chi tiết **60 trường hợp kiểm thử (TC-001 → TC-060)** [4], [5] bao phủ 100% yêu cầu đã hiện thực hóa.
- **Thực thi kiểm thử thực nghiệm và thu thập bằng chứng đa tầng:** Vận hành kiểm thử tự động trên cả tầng Backend (TC-001 → TC-053 với JUnit Jupiter 5.10.2 & Maven Surefire 3.1.2 [6]), tầng Giao diện WebUI (TC-054 → TC-060), công cụ đo độ bao phủ mã nguồn (JaCoCo 0.8.12 [7]), công cụ phân tích tĩnh (Oxlint 1.69.0 [9]) và trình đóng gói (Vite 8.1.0 [9]).
- **Truy vết lỗi phần mềm và đề xuất giải pháp khắc phục khả thi:** Lập hồ sơ kỹ thuật chi tiết cho **12 lỗi phần mềm (BUG-001 → BUG-012)** phát hiện được (đối chiếu với danh mục rủi ro OWASP Top 10:2021 [8]), phân tích nguyên nhân gốc rễ tại mã nguồn, đánh giá khoảng cách chất lượng (*Quality Gap Analysis*) và đề xuất phương án sửa lỗi theo 3 mức ưu tiên (P0, P1, P2).

3. Cấu trúc tổng thể của báo cáo

Báo cáo được tổ chức thành **7 chương chính** cùng hệ thống **3 Phụ lục bằng chứng kỹ thuật**:

- **Chương 1:** Tổng quan nghiệp vụ, Use Case và đặc tả yêu cầu hệ thống (tách biệt rõ Yêu cầu Chức năng và Yêu cầu Phi chức năng).
- **Chương 2:** Kiến trúc kỹ thuật, thiết kế hệ thống và giao diện sản phẩm (*System Under Test*).
- **Chương 3:** Cơ sở lý thuyết, tiêu chí chất lượng (ISO/IEC 25010) và kế hoạch kiểm thử (IEEE 829).
- **Chương 4:** Phân tích hệ thống, thiết kế 60 Test Case (TC-001 → TC-060) và Ma trận truy xuất (RTM).

- **Chương 5:** Kết quả thực thi kiểm thử định lượng (Surefire, JaCoCo, Oxlint, Vite) và bằng chứng thực nghiệm (WebUI).
- **Chương 6:** Hồ sơ chi tiết 12 lỗi phần mềm (BUG-001 → BUG-012), đánh giá chất lượng và rủi ro.
- **Chương 7:** Đề xuất giải pháp khắc phục, cải tiến kiến trúc và kết luận.

CHƯƠNG 1: TỔNG QUAN NGHIỆP VỤ, USE CASE VÀ ĐẶC TẢ YÊU CẦU HỆ THỐNG

1.1. Bài toán thực tế, mục tiêu và quy trình nghiệp vụ tổng thể

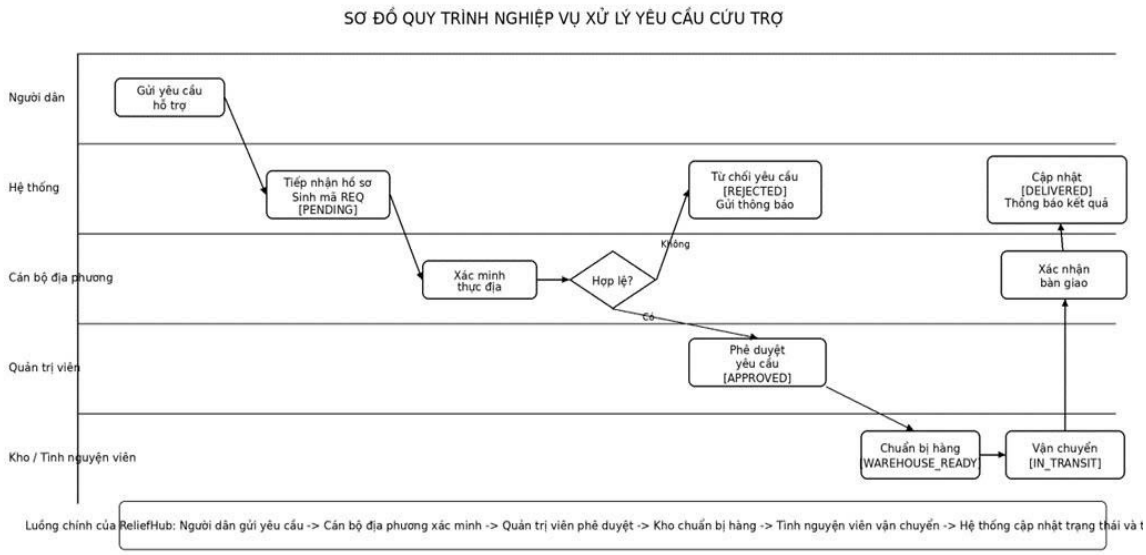
1.1.1. Bài toán thực tế và mục tiêu nghiệp vụ của ReliefHub

Hệ thống **ReliefHub** được xây dựng nhằm giải quyết bài toán điều phối cứu trợ thiên tai tập trung, đóng vai trò cầu nối trực tuyến giữa ba nhóm chủ thể chính: **Nguồn cung cứu trợ** (Nhà tài trợ cá nhân/tổ chức, Kho hàng cứu trợ, Đội tình nguyện viên), **Cơ quan điều phối & kiểm soát** (Quản trị viên hệ thống, Ban Tài chính, Cán bộ địa phương) và **Đối tượng thụ hưởng** (Người dân vùng chịu ảnh hưởng bởi thiên tai).

Mục tiêu nghiệp vụ cốt lõi của hệ thống bao gồm:

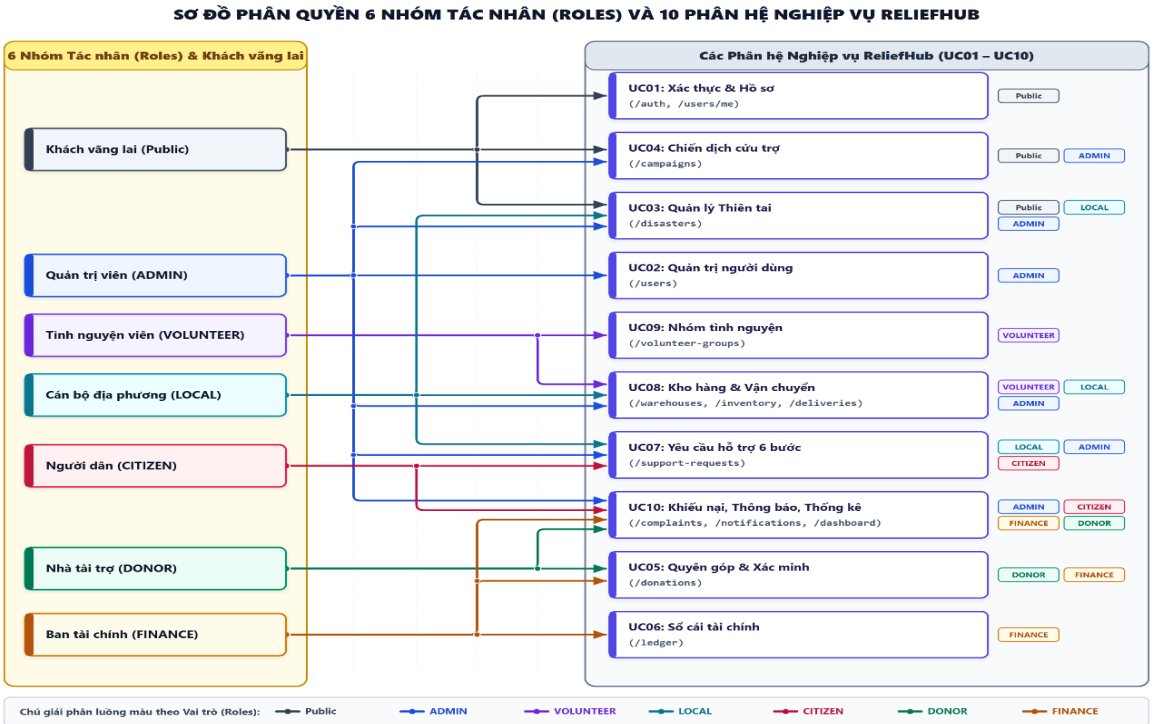
1. **Công khai hóa thông tin thiên tai và chiến dịch quyên góp:** Cung cấp cổng thông tin trực quan về các đợt bão lũ đang diễn ra, mức độ khẩn cấp, mục tiêu ngân sách (targetAmount), số tiền đã huy động (currentAmount) và tỷ lệ hoàn thành (progressPct).
2. **Chuẩn hóa quy trình tiếp nhận và xác minh quyên góp:** Cho phép nhà tài trợ đóng góp bằng tiền mặt (chuyển khoản ngân hàng, ví điện tử kèm ảnh chụp chứng từ) hoặc hiện vật (DonationItems), hỗ trợ chế độ quyên góp ẩn danh (isAnonymous) và cấp biên nhận điện tử sau khi Ban Tài chính đối soát hợp lệ.
3. **Minh bạch hóa sổ cái tài chính (Financial Ledger):** Ghi nhận mọi bút toán thu (INCOME) và chi (EXPENSE) gắn với từng chiến dịch để phục vụ kiểm toán nội bộ và công khai tài chính.
4. **Số hóa quy trình xử lý yêu cầu hỗ trợ 6 bước của người dân:** Đảm bảo mọi đơn xin cứu trợ từ khi người dân gửi lên đều trải qua quy trình thẩm định thực tế của địa phương, phê duyệt nguồn lực, chuẩn bị kho hàng, điều xe vận chuyển và xác nhận giao hàng tận tay hộ dân.

Quy trình hoạt động nghiệp vụ xử lý yêu cầu cứu trợ từ khi tiếp nhận thông tin đến khi hoàn tất bàn giao hàng cứu trợ được thể hiện tại **Hình 1.1** (trích xuất từ hồ sơ thiết kế của dự án ReliefHub):



Hình 1.1: Sơ đồ hoạt động / quy trình nghiệp vụ tổng thể xử lý yêu cầu cứu trợ trên hệ thống ReliefHub.

Để phục vụ việc xác định ranh giới kiểm thử phân quyền (*RBAC*) và điểm kiểm soát liên phân hệ, nhóm kiểm thử đã đối chiếu quy trình nghiệp vụ với mã nguồn thực tế của 15 lớp Controller và Service để hệ thống hóa sơ đồ phân bổ **6 nhóm tác nhân (Roles) và Khách vãng lai tương tác với 10 phân hệ nghiệp vụ cốt lõi (UC01 → UC10)** tại **Hình 1.2**:



Hình 1.2: Sơ đồ phân quyền 6 nhóm tác nhân (Roles) và Khách vãng lai tương tác với 10 phân hệ nghiệp vụ cốt lõi (UC01 – UC10) trên hệ thống ReliefHub.

1.2. Đối tượng sử dụng (Actors) và phân quyền nghiệp vụ (RBAC)

Theo thiết kế phân quyền dựa trên vai trò (*Role-Based Access Control - RBAC*), hệ thống ReliefHub phục vụ **6 vai trò người dùng đã xác thực** (lưu trữ trong bảng roles) cùng nhóm **Khách vãng lai (Guest)** truy cập cổng thông tin công khai. Bảng 1.1 mô tả chi tiết phạm vi trách nhiệm và quyền hạn nghiệp vụ của từng đối tượng sử dụng:

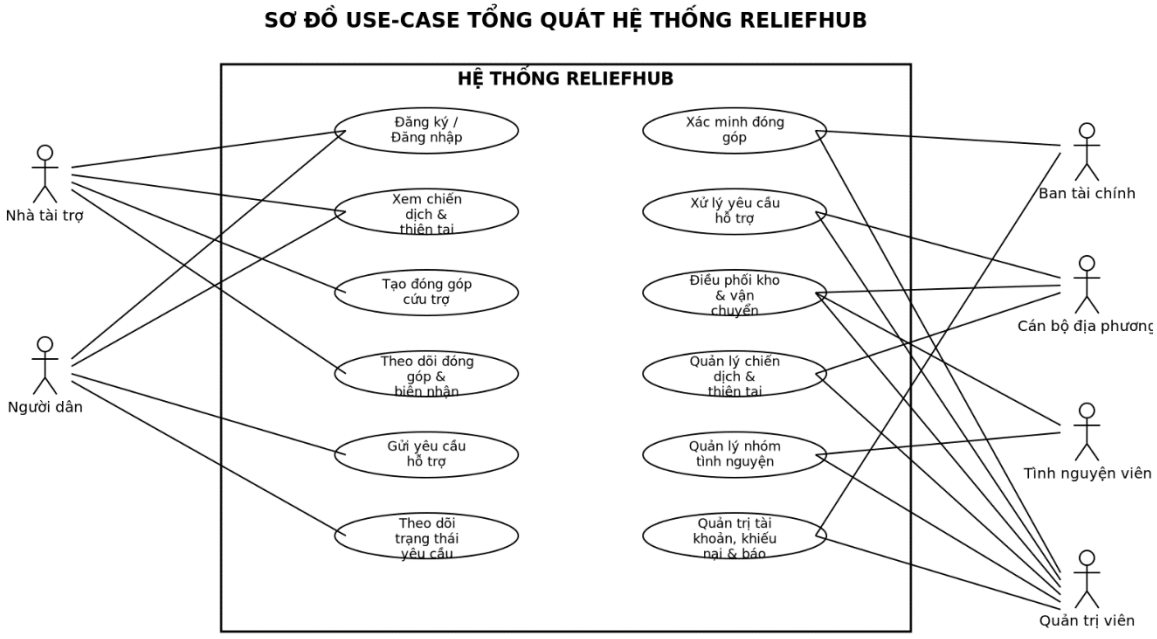
Bảng 1.1: Danh mục đối tượng sử dụng (Actors) và phân quyền nghiệp vụ trên hệ thống ReliefHub

STT	Mã vai trò (Role)	Tên đối tượng sử dụng	Mô tả trách nhiệm và phạm vi quyền hạn nghiệp vụ chính	Trạng thái xác minh
1	admin	**Quản trị viên hệ thống (System Admin)**	Quản trị toàn bộ tài khoản người dùng (tạo mới, phân quyền, khóa/mở khóa, duyệt tài khoản địa phương); khởi tạo và quản lý chiến dịch cứu trợ; phê duyệt yêu cầu hỗ trợ liên vùng; quản lý kho hàng, danh mục cứu trợ, phản hồi khiếu nại và theo dõi thống kê tổng hợp toàn hệ thống.	Đã xác minh thực nghiệm
2	finance	**Ban Tài chính (Finance Officer)**	Kiểm tra chứng từ chuyển khoản của nhà tài trợ; xác minh (CONFIRMED) hoặc từ chối (REJECTED) khoản đóng góp để cộng tiền vào quỹ chiến dịch; lập bút toán thu/chi trên Sổ cái tài chính (Ledger); xuất báo cáo thu chi và giám sát số liệu tài chính trên Dashboard.	Đã xác minh thực nghiệm
3	local	**Cán bộ địa phương (Local Authority)**	Cập nhật thông tin điểm nóng thiên tai (Disasters) tại địa bàn phụ trách; tiếp nhận và xác minh thực địa (VERIFIED) các yêu cầu hỗ trợ của người dân; phối hợp điều phối phương tiện giao hàng và cập nhật kết quả bàn giao cứu trợ tại địa phương.	Đã xác minh thực nghiệm

STT	Mã vai trò (Role)	Tên đối tượng sử dụng	Mô tả trách nhiệm và phạm vi quyền hạn nghiệp vụ chính	Trạng thái xác minh
4	donor	**Nhà tài trợ (Donor / Philanthropist)**	Tra cứu chiến dịch cứu trợ đang mở; thực hiện quyên góp tiền mặt hoặc hiện vật (công khai hoặc ẩn danh); tải lên ảnh minh chứng chuyển khoản; theo dõi lịch sử đóng góp, biên nhận điện tử và gửi phản hồi/khiếu nại khi cần hỗ trợ.	Đã xác minh thực nghiệm
5	citizen	**Người dân vùng thiên tai (Affected Citizen)**	Đăng ký tài khoản công dân; gửi yêu cầu hỗ trợ khẩn cấp (nêu rõ mặt hàng, số lượng, số nhân khẩu, mức độ khẩn cấp và vị trí); theo dõi tiến độ xử lý đơn cứu trợ qua 6 bước trạng thái và xác nhận khi đã nhận được hàng cứu trợ.	Đã xác minh thực nghiệm
6	volunteer	**Tình nguyện viên (Volunteer Coordinator)**	Đăng ký tham gia đội/nhóm tình nguyện viên (VolunteerGroups); nhận nhiệm vụ hỗ trợ đóng gói tại kho, vận chuyển hàng cứu trợ (Deliveries) đến điểm tập kết và cập nhật trạng thái hành trình giao hàng.	Đã xác minh thực nghiệm
7	*(Guest)*	**Khách vãng lai (Unauthenticated Guest)**	Xem trang chủ, tra cứu công khai danh sách thông tin thiên tai, danh sách chiến dịch cứu trợ đang hoạt động (ACTIVE), bảng vàng đóng góp, sổ cái tài chính công khai và thực hiện đăng ký/đăng nhập tài khoản.	Đã xác minh thực nghiệm

1.3. Hệ thống sơ đồ Use Case và danh mục 24 Use Cases nghiệp vụ

1.3.1. Sơ đồ Use Case tổng quan của hệ thống ReliefHub

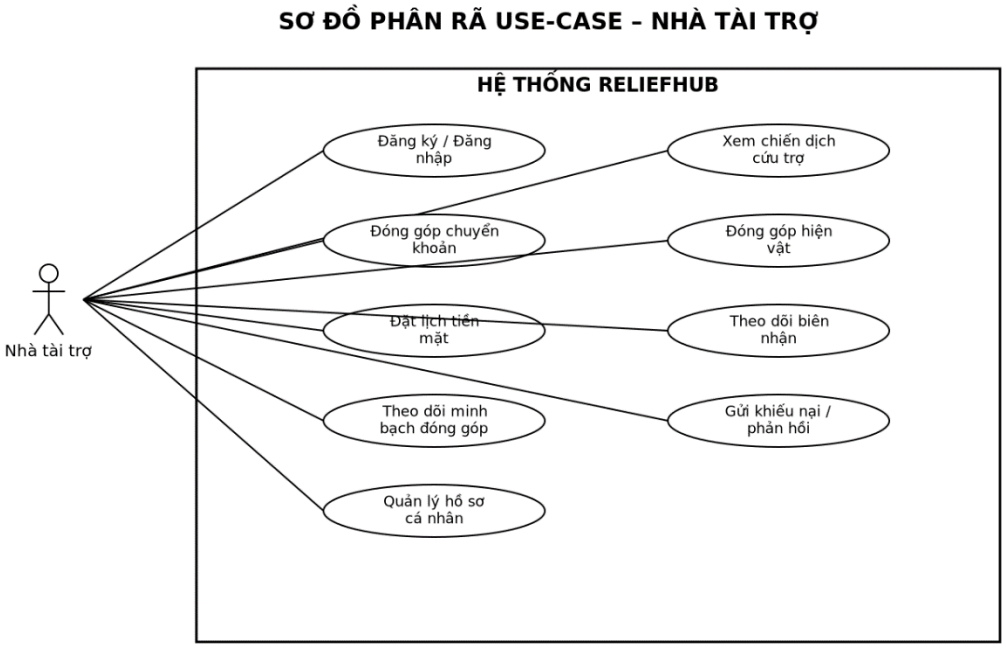


Hình 1.3: Sơ đồ Use Case tổng quan biểu diễn mối quan hệ giữa 6 tác nhân (Donor, Citizen, Finance, Local, Volunteer, Admin) và các ca sử dụng cốt lõi của hệ thống ReliefHub.

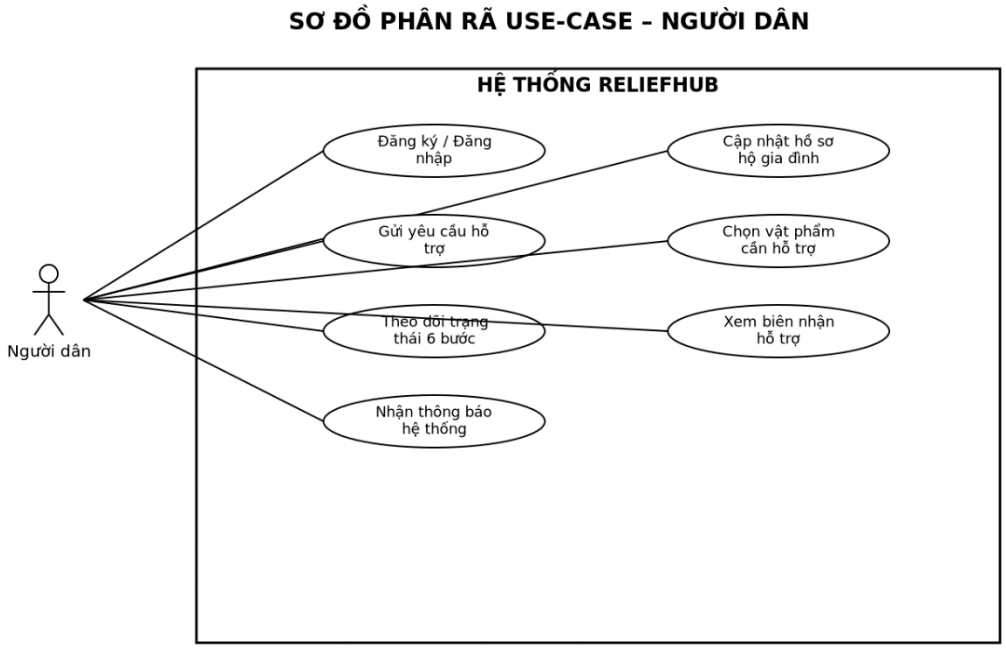
1.3.2. Sơ đồ phân rã Use Case theo 6 nhóm tác nhân (Hình 1.4 → Hình 1.9)

Để làm rõ ranh giới phân quyền kiểm thử (*RBAC Testing Boundary*) cho từng vai trò người dùng đã nêu tại Bảng 1.1, các sơ đồ phân rã Use Case của 6 tác nhân chính trong hệ thống ReliefHub được trình bày lần lượt từ **Hình 1.4** đến **Hình 1.9** dưới đây:

- **Phân rã Use Case của Nhà tài trợ (Donor) và Người dân (Citizen): Hình 1.4** mô tả các ca sử dụng thuộc nhóm đóng góp tài chính/hiện vật, tra cứu lịch sử đóng góp và gửi phản hồi của Nhà tài trợ; **Hình 1.5** mô tả các ca sử dụng gửi yêu cầu hỗ trợ khẩn cấp, theo dõi trạng thái xử lý và xác nhận đã nhận hàng cứu trợ của Người dân vùng thiên tai.



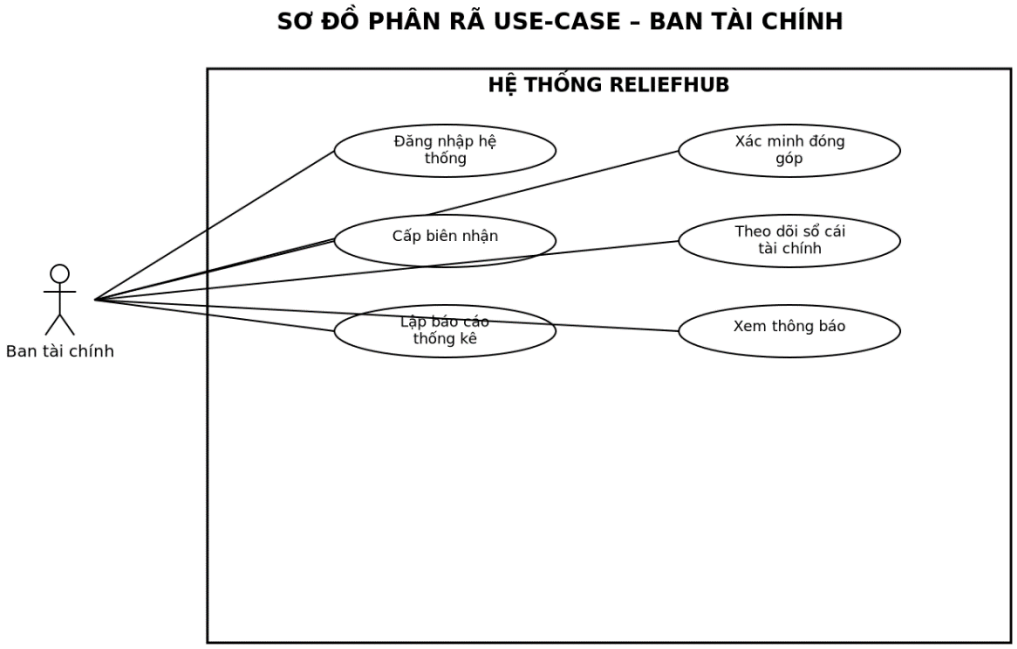
Hình 1.4: Sơ đồ phân rã Use Case của tác nhân Nhà tài trợ (Donor).



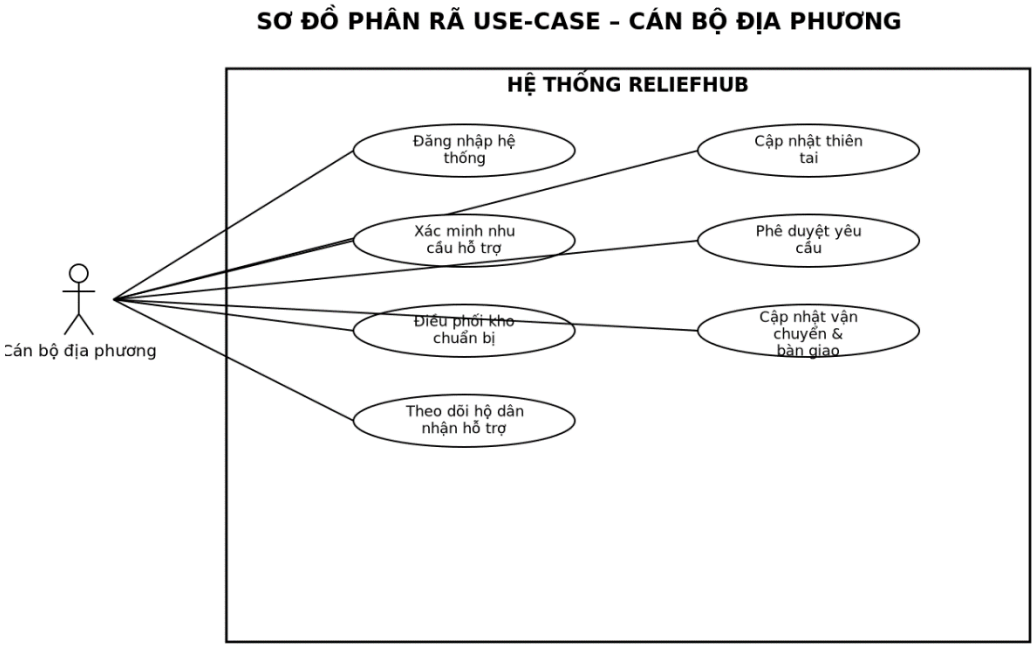
Hình 1.5: Sơ đồ phân rã Use Case của tác nhân Người dân vùng thiên tai (Citizen).

- **Phân rã Use Case của Ban Tài chính (Finance) và Cán bộ địa phương (Local): Hình 1.6** thể hiện nhóm chức năng đối soát, xác minh đóng góp, quản lý sổ cái thu chi và phân bổ quỹ của Ban Tài chính; **Hình 1.7** thể hiện nhóm chức

năng cập nhật điểm nóng thiên tai, thẩm định yêu cầu cứu trợ thực địa và phối hợp bàn giao hàng của Cán bộ địa phương.

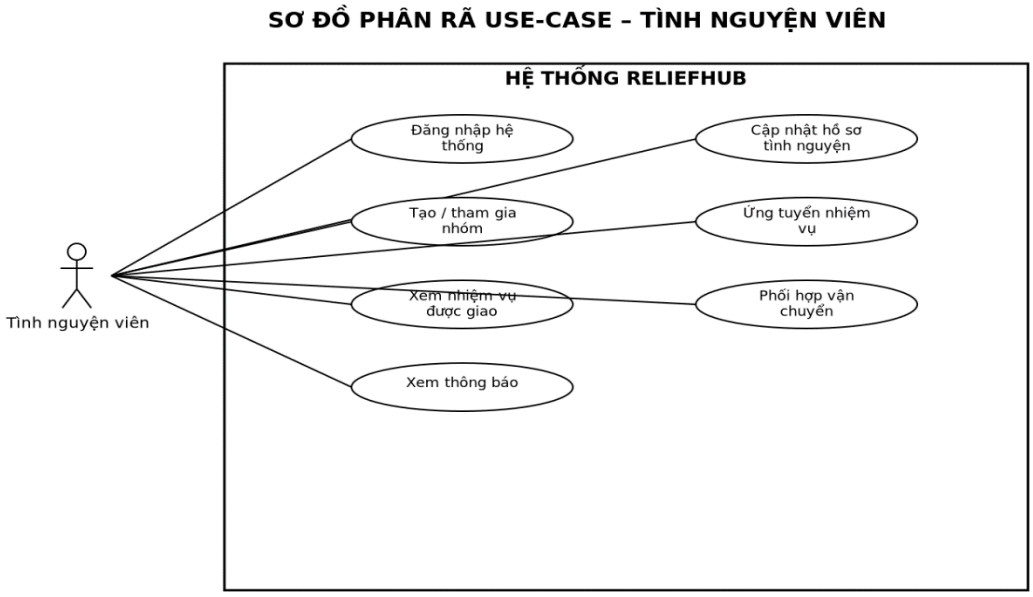


Hình 1.6: Sơ đồ phân rã Use Case của tác nhân Ban Tài chính (Finance).

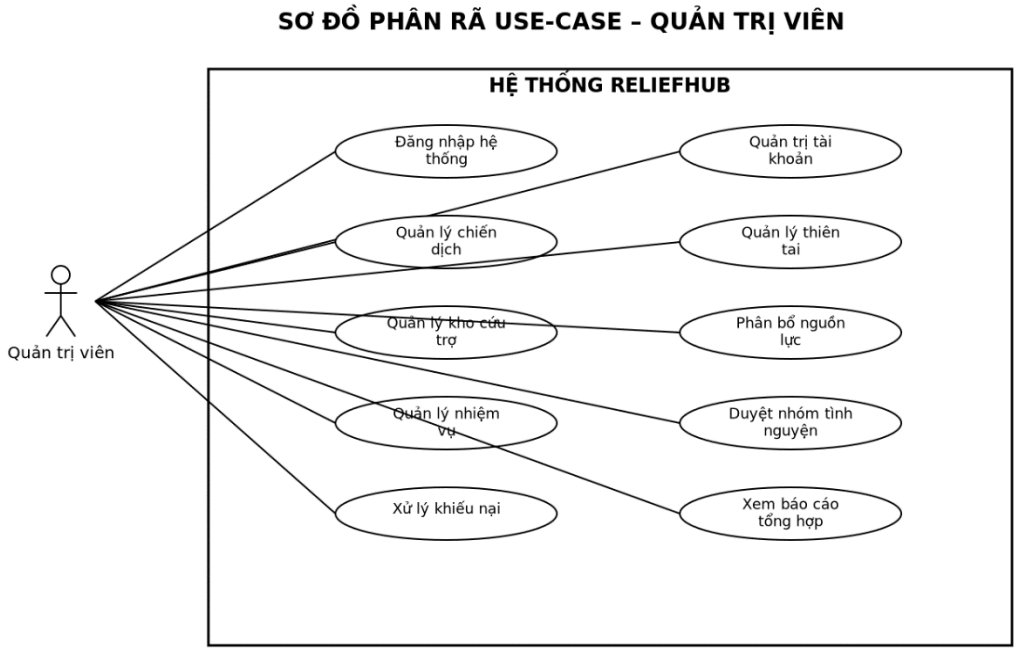


Hình 1.7: Sơ đồ phân rã Use Case của tác nhân Cán bộ địa phương (Local).

- **Phân rã Use Case của Tình nguyện viên (Volunteer) và Quản trị viên (Admin): Hình 1.8** trình bày các ca sử dụng quản lý hồ sơ tình nguyện, nhận nhiệm vụ cứu trợ và cập nhật trạng thái giao hàng; **Hình 1.9** trình bày toàn bộ các ca sử dụng điều hành cấp cao của Quản trị viên hệ thống (quản trị tài khoản, quản lý chiến dịch, quản lý kho cứu trợ, xử lý khiếu nại và thống kê tổng hợp).



Hình 1.8: Sơ đồ phân rã Use Case của tác nhân Tình nguyện viên (Volunteer).



Hình 1.9: Sơ đồ phân rã Use Case của tác nhân Quản trị viên hệ thống (Admin).

1.3.3. Danh mục 24 Use Cases nghiệp vụ chính thức và ánh xạ hiện thực hóa

Bảng 1.2: Danh mục 24 Use Cases nghiệp vụ chính thức của hệ thống ReliefHub và ánh xạ hiện thực hóa

Phân hệ Use Case	Mã UC	Tên Use Case nghiệp vụ	Tác nhân chính	Ánh xạ hiện thực hóa trong Mã nguồn (Backend & Frontend)	Mã Yêu cầu liên kết
Nhóm 1: Quản trị Hệ thống & Tài khoản (UC1)	UC1.1	Đăng ký và Đăng nhập hệ thống	Tất cả tác nhân	AuthController (/auth/login, /auth/register), Login.jsx, Register.jsx	REQ-001, REQ-002, REQ-003
	UC1.2	Quản lý hồ sơ cá nhân & Đổi mật khẩu	Tất cả tác nhân đã đăng nhập	UserController (/users/me, /users/me/password), Profile.jsx	REQ-005, REQ-006
	UC1.3	Quản lý danh sách tài khoản người dùng	Quản trị viên (Admin)	UserController (GET/POST/PUT/DELETE /users, toggle-status), Accounts.jsx	REQ-004, REQ-007
	UC1.4	Quản lý thông báo hệ thống	Tất cả tác nhân	NotificationController (/notifications), Notifications.jsx	REQ-024
	UC1.5	Tiếp nhận và xử lý phản hồi / khiếu nại	Nhà tài trợ, Người dân, Admin	ComplaintController (/complaints), Feedback.jsx, Complaints.jsx	REQ-023
Nhóm 2: Thông tin Thiên tai & Chiến dịch (UC2)	UC2.1	Cập nhật và quản lý thông tin thiên tai	Cán bộ địa phương (Local), Admin	DisasterController (/disasters), UpdateDisaster.jsx, Disasters.jsx	REQ-008, REQ-009
	UC2.2	Khởi tạo và quản lý chiến dịch cứu trợ	Quản trị viên (Admin)	CampaignController (/campaigns), ManageCampaigns.jsx	REQ-010, REQ-011
	UC2.3	Tra cứu công khai thiên tai & chiến dịch	Khách vãng lai, Nhà tài trợ, Người dân	GET /campaigns/active, GET /disasters/active, Campaigns.jsx, Home.jsx	REQ-008, REQ-011

Phân hệ Use Case	Mã UC	Tên Use Case nghiệp vụ	Tác nhân chính	Ánh xạ hiện thực hóa trong Mã nguồn (Backend & Frontend)	Mã Yêu cầu liên kết
	UC2.4	Tải lên tệp hình ảnh / chứng từ minh chứng	Tất cả tác nhân đã đăng nhập	FileUploadController (POST /uploads), FileUploadService.java	REQ-026
Nhóm 3: Quyên góp & Tài trợ (UC3)	UC3.1	Đóng góp tiền mặt (Chuyển khoản / QR)	Nhà tài trợ (Donor)	DonationController (POST /donations), Contribute.jsx	REQ-012, REQ-014
	UC3.2	Đóng góp hiện vật cứu trợ (DonationItems)	Nhà tài trợ (Donor)	DonationController (POST /donations kèm items), InKind.jsx	REQ-012
	UC3.3	Tra cứu lịch sử đóng góp & Biên nhận điện tử	Nhà tài trợ (Donor)	GET /donations/my, GET /donations/campaign/{id}, History.jsx, Receipts.jsx	REQ-014
	UC3.4	Chế độ đóng góp ẩn danh (Anonymous)	Nhà tài trợ (Donor)	DonationService.toResponse (isAnonymous = true), Contribute.jsx	REQ-014
Nhóm 4: Quản lý Tài chính & Sổ cái (UC4)	UC4.1	Đối soát và xác minh khoản đóng góp	Ban Tài chính (Finance), Admin	PATCH /donations/{id}/status, VerifyDonations.jsx, Verify.jsx	REQ-013
	UC4.2	Quản lý bút toán thu/chi Sổ cái (Ledger)	Ban Tài chính (Finance), Admin	LedgerController (/ledger), FinanceLedger.jsx, PublicFinance.jsx	REQ-015
	UC4.3	Phân bổ ngân sách quỹ chiến dịch	Ban Tài chính (Finance), Admin	DonationService.updateDonationStatus, FundAllocation.jsx	REQ-013, REQ-015
	UC4.4	Thống kê báo cáo tài chính & Dashboard	Ban Tài chính (Finance), Admin	DashboardController (/dashboard/stats), Reports.jsx, Overview.jsx	REQ-025

Phân hệ Use Case	Mã UC	Tên Use Case nghiệp vụ	Tác nhân chính	Ánh xạ hiện thực hóa trong Mã nguồn (Backend & Frontend)	Mã Yêu cầu liên kết
Nhóm 5: Yêu cầu Hỗ trợ Cứu trợ (UC5)	UC5.1	Gửi yêu cầu hỗ trợ khẩn cấp của hộ dân	Người dân (Citizen)	SupportRequestController (POST /support-requests), Request.jsx	REQ-016
	UC5.2	Xác minh thực địa yêu cầu hỗ trợ (VERIFIED)	Cán bộ địa phương (Local)	PATCH /support-requests/{id}/advance, VerifyRequests.jsx	REQ-017
	UC5.3	Phê duyệt yêu cầu hỗ trợ (APPROVED)	Quản trị viên (Admin), Local	PATCH /support-requests/{id}/advance, SupportRequests.jsx	REQ-017
	UC5.4	Theo dõi tiến độ xử lý đơn cứu trợ 6 bước	Người dân (Citizen), Local, Admin	GET /support-requests/me, GET /support-requests/status/{status}, Status.jsx	REQ-018
Nhóm 6: Kho hàng, Vận chuyển & Tình nguyện (UC6)	UC6.1	Quản lý nhà kho & Danh mục tồn kho (Restock)	Quản trị viên (Admin), Quản kho	WarehouseController (/warehouses), InventoryController (/inventory), Inventory.jsx	REQ-019, REQ-020
	UC6.2	Điều phối chuyến giao hàng cứu trợ (Deliveries)	Admin, Local, Tình nguyện viên	DeliveryController (/deliveries), Distribution.jsx, ReliefExecution.jsx	REQ-021
	UC6.3	Quản lý nhóm tình nguyện viên & Phân công	Tình nguyện viên (Volunteer), Admin	VolunteerGroupController (/volunteer-groups), Tasks.jsx, Team.jsx	REQ-022

1.4. Đặc tả yêu cầu chức năng (REQ-001 → REQ-022)

Để phục vụ thiết kế kiểm thử hộp đen và truy xuất yêu cầu minh bạch, nhóm tách riêng các **Yêu cầu Chức năng (*Functional Requirements - FR*)** và **Yêu cầu Phi chức năng**

(*Non-Functional & System Requirements - NFR*) thành hai bảng độc lập. Bảng 1.3 đặc tả chi tiết **22 yêu cầu chức năng cốt lõi (REQ-001 → REQ-022)** được diễn đạt bằng ngôn ngữ nghiệp vụ rõ ràng, gắn với quy tắc kiểm tra dữ liệu đầu vào/đầu ra và trạng thái xác minh thực tế:

Bảng 1.3: Đặc tả chi tiết 22 Yêu cầu Chức năng (Functional Requirements) của hệ thống ReliefHub

Mã YC	Tên yêu cầu chức năng	Phân hệ (UC)	Mô tả quy tắc nghiệp vụ và điều kiện chấp nhận (Acceptance Criteria)	Mức ưu tiên	Trạng thái xác minh
REQ-001	**Đăng nhập xác thực đa vai trò**	UC1.1	Cho phép người dùng đăng nhập bằng tên đăng nhập và mật khẩu; kiểm tra trạng thái tài khoản (ACTIVE được cấp quyền truy cập, INACTIVE hoặc pending_approval phải bị từ chối); trả về thông tin định danh và điều hướng đúng Dashboard theo vai trò.	**P0**	Đã xác minh thực nghiệm
REQ-002	**Đăng ký tài khoản người dùng mới**	UC1.1	Cho phép khách vãng lai đăng ký tài khoản mới; kiểm tra tính duy nhất của username (3–50 ký tự) và email (đúng chuẩn định dạng); mật khẩu tối thiểu 6 ký tự và phải được mã hóa một chiều (BCrypt) trước khi lưu vào CSDL.	**P0**	Đã xác minh thực nghiệm
REQ-003	**Bảo vệ quyền hạn khi tự đăng ký & Khôi phục mật khẩu**	UC1.1	Khi đăng ký qua cổng công khai, hệ thống chỉ cho phép gán các vai trò công dân/nhà tài trợ/tình nguyện viên, tuyệt đối không cho phép người dùng tự truyền tham số để chiếm quyền admin. Luồng khôi phục mật khẩu phải có bước xác thực danh tính hợp lệ (OTP/Email token).	**P0**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-001, BUG-007)*
REQ-004	**Kiểm soát truy cập dựa trên vai trò (RBAC)**	UC1.3	Mọi yêu cầu truy cập vào các chức năng nghiệp vụ nội bộ phải đính kèm định	**P0**	Đã xác minh

Mã YC	Tên yêu cầu chức năng	Phân hệ (UC)	Mô tả quy tắc nghiệp vụ và điều kiện chấp nhận (Acceptance Criteria)	Mức ưu tiên	Trạng thái xác minh
			danh xác thực hợp lệ (JWT Bearer Token); hệ thống phải từ chối truy cập (HTTP 401/403) nếu người dùng chưa đăng nhập hoặc không có vai trò phù hợp.		thực nghiệm
REQ-005	**Xem và cập nhật hồ sơ cá nhân**	UC1.2	Mọi người dùng đã đăng nhập (bao gồm donor, citizen, volunteer, local, finance, admin) đều có quyền xem và cập nhật thông tin cá nhân của chính mình (họ tên, số điện thoại, địa chỉ, ảnh đại diện) qua chức năng Hồ sơ cá nhân (/users/me).	**P0**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-002)*
REQ-006	**Đổi mật khẩu cá nhân an toàn**	UC1.2	Người dùng đã đăng nhập có thể đổi mật khẩu cá nhân sau khi hệ thống xác minh mật khẩu hiện tại (oldPassword) khớp với bản băm trong CSDL và mật khẩu mới (newPassword) đạt độ dài tối thiểu 6 ký tự.	**P1**	Đã xác minh thực nghiệm
REQ-007	**Quản trị danh sách tài khoản người dùng**	UC1.3	Quản trị viên (admin) có quyền xem danh sách toàn bộ người dùng, tạo mới tài khoản cán bộ nội bộ (finance, local), cập nhật thông tin, khóa/mở khóa trạng thái hoạt động (ACTIVE ↔ INACTIVE) và xóa tài khoản vi phạm.	**P1**	Đã xác minh thực nghiệm
REQ-008	**Tra cứu công khai Thiên tai & Chiến dịch**	UC2.3	Khách vãng lai và người dùng có thể tra cứu danh sách tất cả các đợt thiên tai và chiến dịch cứu trợ đang hoạt động (ACTIVE), xem chi tiết mục tiêu quyên góp và tiến độ của từng chiến	**P0**	Đã xác minh thực nghiệm

Mã YC	Tên yêu cầu chức năng	Phân hệ (UC)	Mô tả quy tắc nghiệp vụ và điều kiện chấp nhận (Acceptance Criteria)	Mức ưu tiên	Trạng thái xác minh
			dịch mà không cần đăng nhập.		
REQ-009	**Quản lý thông tin điểm nóng Thiên tai**	UC2.1	Chỉ Quản trị viên (admin) và Cán bộ địa phương (local) được quyền công bố mới, cập nhật mức độ nghiêm trọng (LOW, MEDIUM, HIGH, CRITICAL), khu vực ảnh hưởng hoặc đóng sự kiện thiên tai; các vai trò khác bị từ chối quyền chỉnh sửa.	**P1**	Đã xác minh thực nghiệm
REQ-010	**Khởi tạo và quản lý Chiến dịch cứu trợ**	UC2.2	Quản trị viên (admin) khởi tạo, cập nhật và đóng chiến dịch cứu trợ gắn với sự kiện thiên tai; hệ thống phải kiểm tra số tiền mục tiêu (targetAmount > 0) và ngày bắt đầu/kết thúc hợp lệ.	**P0**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-010)*
REQ-011	**Tính toán tự động tiến độ quyên góp (progressPct)**	UC2.2	Hệ thống tự động tính tỷ lệ hoàn thành mục tiêu của chiến dịch theo công thức progressPct = round((currentAmount × 100) / targetAmount, 1); trường hợp targetAmount = 0 phải trả về 0.0% và không gây lỗi chia cho 0.	**P1**	Đã xác minh thực nghiệm
REQ-012	**Tạo khoản quyên góp tiền mặt & hiện vật**	UC3.1, UC3.2	Nhà tài trợ (donor) gửi khoản đóng góp vào chiến dịch đang mở với số tiền hợp lệ (amount > 0) và/hoặc danh sách hiện vật đi kèm (tên hàng, số lượng, đơn vị tính); khoản đóng góp mới tạo được khởi gán trạng thái chờ đối soát (PENDING).	**P0**	Đã xác minh thực nghiệm

Mã YC	Tên yêu cầu chức năng	Phân hệ (UC)	Mô tả quy tắc nghiệp vụ và điều kiện chấp nhận (Acceptance Criteria)	Mức ưu tiên	Trạng thái xác minh
REQ-013	**Đối soát, xác minh quyên góp & Cộng quỹ chiến dịch**	UC4.1, UC4.3	Chỉ Ban Tài chính (finance) hoặc admin được phép duyệt chuyển trạng thái khoản đóng góp từ PENDING sang CONFIRMED (hoặc REJECTED). Khi chuyển sang CONFIRMED, số tiền đóng góp được cộng đúng **một lần duy nhất** vào currentAmount của chiến dịch và cấp mã biên nhận.	**P0**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-003, BUG-005)*
REQ-014	**Tra cứu lịch sử quyên góp & Bảo vệ tính ẩn danh**	UC3.3, UC3.4	Nhà tài trợ tra cứu được danh sách các khoản đóng góp của bản thân. Đối với các khoản quyên góp chọn chế độ ẩn danh (isAnonymous = true), hệ thống phải hiển thị tên người đóng góp là "Ẩn danh" trên cổng công khai.	**P1**	Đã xác minh thực nghiệm
REQ-015	**Quản lý bút toán Sổ cái tài chính (Ledger)**	UC4.2	Chỉ Ban Tài chính (finance) hoặc admin được phép ghi nhận mới hoặc xóa các bút toán thu (INCOME) và chi (EXPENSE) với số tiền dương (amount > 0) nhằm đảm bảo tính toàn vẹn của sổ cái kế toán.	**P1**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-003)*
REQ-016	**Gửi yêu cầu hỗ trợ khẩn cấp của người dân**	UC5.1	Người dân (citizen) gửi đơn yêu cầu cứu trợ gồm mặt hàng thiết yếu, số lượng dương (quantity > 0), số nhân khẩu (householdCount > 0), khu vực và mức độ khẩn cấp; hệ thống sinh mã đơn duy nhất (REQ-...) ở trạng thái PENDING.	**P0**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-006, BUG-010)*

Mã YC	Tên yêu cầu chức năng	Phân hệ (UC)	Mô tả quy tắc nghiệp vụ và điều kiện chấp nhận (Acceptance Criteria)	Mức ưu tiên	Trạng thái xác minh
REQ-017	**Điều phối yêu cầu hỗ trợ theo quy trình 6 bước**	UC5.2, UC5.3	Đơn yêu cầu hỗ trợ chỉ được phép được thẩm định và chuyển trạng thái bởi cán bộ có thẩm quyền (local, admin) theo đúng trình tự 6 bước nghiệp vụ: PENDING → VERIFIED → APPROVED → WAREHOUSE_READY → IN_TRANSIT → DELIVERED (hoặc REJECTED), không cho phép nhảy cóc bỏ qua bước thẩm định/phê duyệt.	**P0**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-003, BUG-004)*
REQ-018	**Tra cứu và lọc yêu cầu hỗ trợ theo trạng thái**	UC5.4	Người dân xem được danh sách yêu cầu hỗ trợ của chính mình (/support-requests/me); cán bộ điều phối lọc được danh sách yêu cầu theo từng trạng thái xử lý để phục vụ phân bổ nguồn lực.	**P2**	Đã xác minh thực nghiệm
REQ-019	**Quản lý Nhà kho (Warehouses) & Danh mục tồn kho (Inventory)**	UC6.1	Quản trị viên và cán bộ quản kho thiết lập thông tin nhà kho theo tỉnh/thành phố, thêm mới, cập nhật hoặc xóa mặt hàng cứu trợ gắn với từng kho; người dùng thông thường không được phép can thiệp xóa dữ liệu tồn kho.	**P1**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-003)*
REQ-020	**Nhập bổ sung hàng hóa vào kho (Restock)**	UC6.1	Cho phép cộng thêm số lượng hàng hóa mới nhập (quantity > 0) vào số lượng tồn kho hiện tại của mặt hàng và tự động cập nhật thời điểm kiểm kê gần nhất (updatedAt).	**P1**	Đã xác minh thực nghiệm

Mã YC	Tên yêu cầu chức năng	Phân hệ (UC)	Mô tả quy tắc nghiệp vụ và điều kiện chấp nhận (Acceptance Criteria)	Mức ưu tiên	Trạng thái xác minh
REQ-021	**Điều phối và theo dõi Chuyến giao hàng (Deliveries)**	UC6.2	Tạo lệnh điều xe giao hàng cứu trợ từ kho xuất phát đến điểm nhận; theo dõi trạng thái vận chuyển (SCHEDULED → IN_TRANSIT → DELIVERED) và tự động ghi nhận dấu thời gian hoàn tất (arriveAt) khi hàng đến nơi.	**P1**	Đã xác minh thực nghiệm
REQ-022	**Quản lý Nhóm tình nguyện viên (VolunteerGroups)**	UC6.3	Cho phép tạo mới, cập nhật thông tin đội tình nguyện viên (trưởng nhóm, địa bàn, số lượng thành viên, kỹ năng chuyên môn) và lọc danh sách các đội đang hoạt động tích cực (ACTIVE).	**P2**	Đã xác minh thực nghiệm

1.5. Đặc tả yêu cầu phi chức năng (REQ-023 → REQ-028)

Bên cạnh 22 yêu cầu chức năng nghiệp vụ, hệ thống ReliefHub phải đáp ứng **6 yêu cầu phi chức năng và dịch vụ nền tảng xuyên suốt (REQ-023 → REQ-028)** nhằm đảm bảo tính minh bạch giải trình, bảo mật dữ liệu riêng tư, an toàn tệp tải lên [8], khả năng chịu lỗi và tính toàn vẹn dữ liệu liên tầng. Bảng 1.4 trình bày chi tiết các yêu cầu này theo tiêu chuẩn chất lượng ISO/IEC 25010 [2]:

Bảng 1.4: Đặc tả các Yêu cầu Phi chức năng và Dịch vụ Nền tảng (Non-Functional & System Requirements) của ReliefHub

Mã YC	Nhóm thuộc tính chất lượng (ISO/IEC 25010)	Tên yêu cầu phi chức năng / dịch vụ hệ thống	Đặc tả tiêu chuẩn chất lượng và chỉ số kiểm chứng (Quality & Acceptance Criteria)	Mức ưu tiên	Trạng thái xác minh
REQ-023	**Trách nhiệm giải trình & Phản hồi (Accountability)**	**Cơ chế tiếp nhận, phản hồi và đóng khiếu nại (Complaints)**	Đảm bảo kênh giám sát minh bạch: người dùng gửi khiếu nại (khởi tạo PENDING, tiến độ 0%); chỉ admin có quyền gửi nội dung giải trình không rỗng (REPLIED, tiến độ 100%, ghi nhận	**P1**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-003,*

Mã YC	Nhóm thuộc tính chất lượng (ISO/IEC 25010)	Tên yêu cầu phi chức năng / dịch vụ hệ thống	Đặc tả tiêu chuẩn chất lượng và chỉ số kiểm chứng (Quality & Acceptance Criteria)	Mức ưu tiên	Trạng thái xác minh
			repliedAt) và đóng hồ sơ (CLOSED).		*BUG-010)*
REQ-024	**Tính bảo mật & Quyền riêng tư (Confidentiality & IDOR Prevention)**	**Bảo vệ quyền riêng tư của Thông báo cá nhân (Notifications)**	Mỗi thông báo gắn với một định danh người nhận cụ thể (userId). Hệ thống phải kiểm tra quyền sở hữu trước khi cho phép xem hoặc đánh dấu đã đọc (markAsRead), ngăn chặn tuyệt đối lỗ hổng tham chiếu đối tượng trực tiếp không an toàn (*IDOR*) [8].	**P1**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-009)*
REQ-025	**Khả năng giám sát điều hành (Observability & Auditability)**	**Tổng hợp chỉ số thống kê Dashboard thời gian thực**	Cung cấp số liệu tổng hợp chính xác theo thời gian thực (tổng số người dùng, tổng chiến dịch, tổng tiền quyên góp, tổng đơn cứu trợ, tổng tồn kho) và giới hạn quyền truy cập nghiêm ngặt chỉ dành cho ADMIN và FINANCE.	**P1**	Đã xác minh thực nghiệm
REQ-026	**An toàn thông tin khi tải tệp (Secure File Upload)**	**Kiểm soát kích thước và loại tệp minh chứng tải lên (Uploads)**	Dịch vụ tải tệp (POST /uploads) phải từ chối tệp rỗng (0 byte), giới hạn kích thước tối đa ≤ 10 MB và chỉ chấp nhận các định dạng ảnh/tài liệu minh chứng an toàn (.png, .jpg, .pdf), chặn tuyệt đối các tệp thực thi hoặc chứa mã kịch bản độc hại (.html, .jsp, .exe) gây lỗi *Stored XSS* (OWASP A03:2021) [8].	**P1**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-008)*

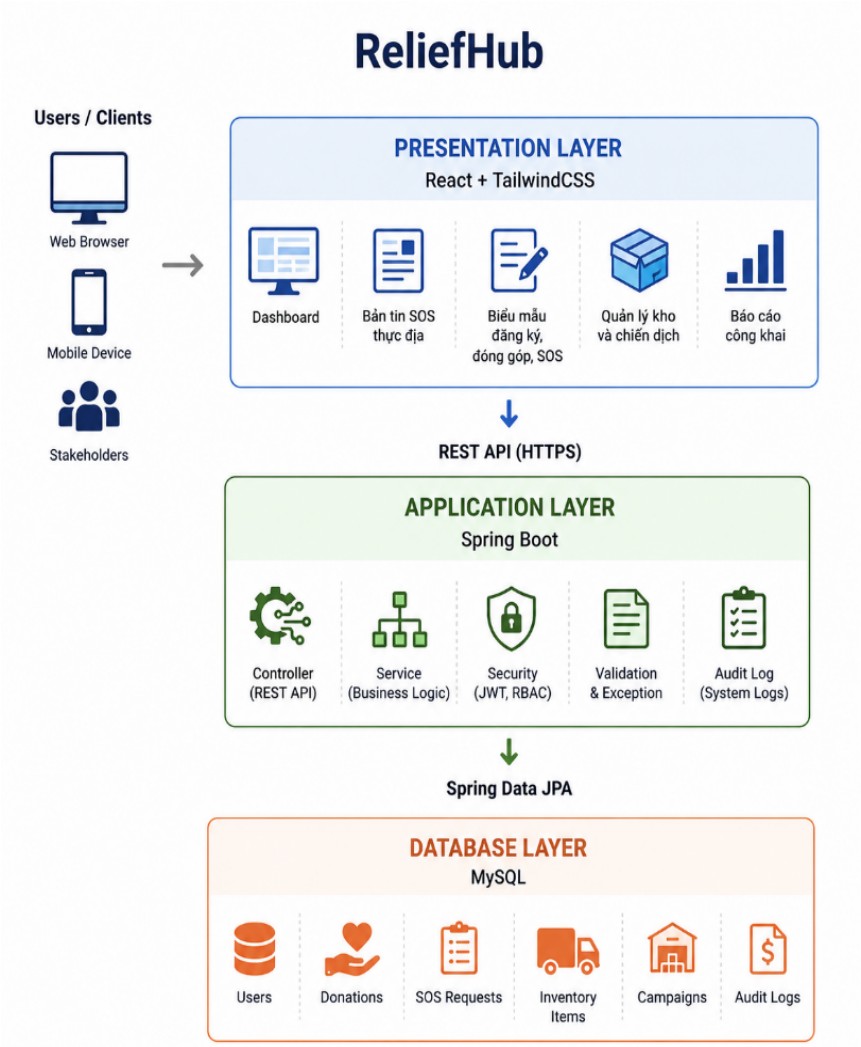
Mã YC	Nhóm thuộc tính chất lượng (ISO/IEC 25010)	Tên yêu cầu phi chức năng / dịch vụ hệ thống	Đặc tả tiêu chuẩn chất lượng và chỉ số kiểm chứng (Quality & Acceptance Criteria)	Mức ưu tiên	Trạng thái xác minh
REQ-027	**Độ tin cậy & Khả năng chịu lỗi (Reliability & Fault Tolerance)**	**Chuẩn hóa cấu trúc phản hồi API & Xử lý ngoại lệ tập trung**	Mọi phản hồi REST API phải tuân thủ cấu trúc chuẩn ApiResponse<T> (success, message, data, error). Khi dữ liệu đầu vào sai định dạng, thiếu trường bắt buộc hoặc truy vấn bản ghi không tồn tại, GlobalExceptionHandler phải trả về mã lỗi HTTP 400 hoặc HTTP 404 tường minh, không để lọt ngoại lệ HTTP 500 Internal Server Error.	**P1**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-010)*
REQ-028	**Tính tương thích, Hiệu năng & Toàn vẹn tích hợp (Interoperability & Integrity)**	**Đồng bộ dữ liệu liên tầng Frontend – Backend & Tối ưu gói tải**	Tầng giao diện React (AppContext.jsx) phải kết nối đồng bộ với REST API Backend và lưu trữ bền vững trạng thái qua CSDL (không mất dữ liệu khi tải lại trang F5); tài khoản mẫu phải nhất quán giữa tài liệu, SQL và mã nguồn; mã nguồn tĩnh không có lỗi biến chưa khai báo (no-undef) và dung lượng gói JS đóng gói nằm trong ngưỡng khuyến nghị (≤ 500 kB) [9].	**P0**	Đã xác minh thực nghiệm *(Phát hiện lỗi BUG-006, BUG-011, BUG-012)*

CHƯƠNG 2: KIẾN TRÚC KỸ THUẬT, THIẾT KẾ HỆ THỐNG VÀ GIAO DIỆN SẢN PHẨM

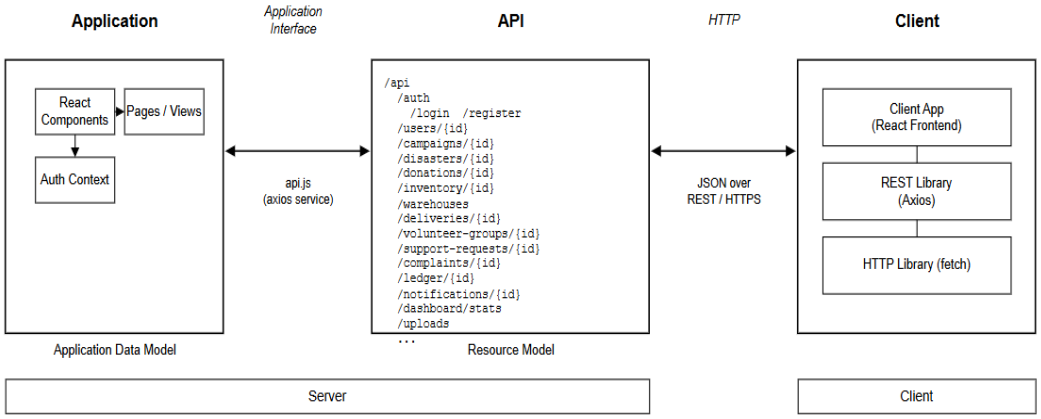
2.1. Kiến trúc kỹ thuật 3 tầng và hiện trạng tích hợp Frontend – Backend

2.1.1. Sơ đồ kiến trúc mức cao và phân tầng REST API

Theo thiết kế hệ thống của dự án, **ReliefHub** được xây dựng theo kiến trúc ứng dụng Web 3 tầng (*Three-Tier Architecture*) gồm tầng giao diện Web Client (React SPA), tầng dịch vụ nghiệp vụ Application Server (Spring Boot REST API) và tầng quản trị cơ sở dữ liệu MySQL 8.0. **Hình 2.1** trình bày sơ đồ kiến trúc mức cao tổng quát của hệ thống và **Hình 2.2** mô tả cấu trúc tổ chức các tài nguyên REST API (đặt dưới tiền tố chung /api) cùng cơ chế giao tiếp giữa Client và Server:



Hình 2.1: Sơ đồ tổng quan kiến trúc mức cao của hệ thống ReliefHub.

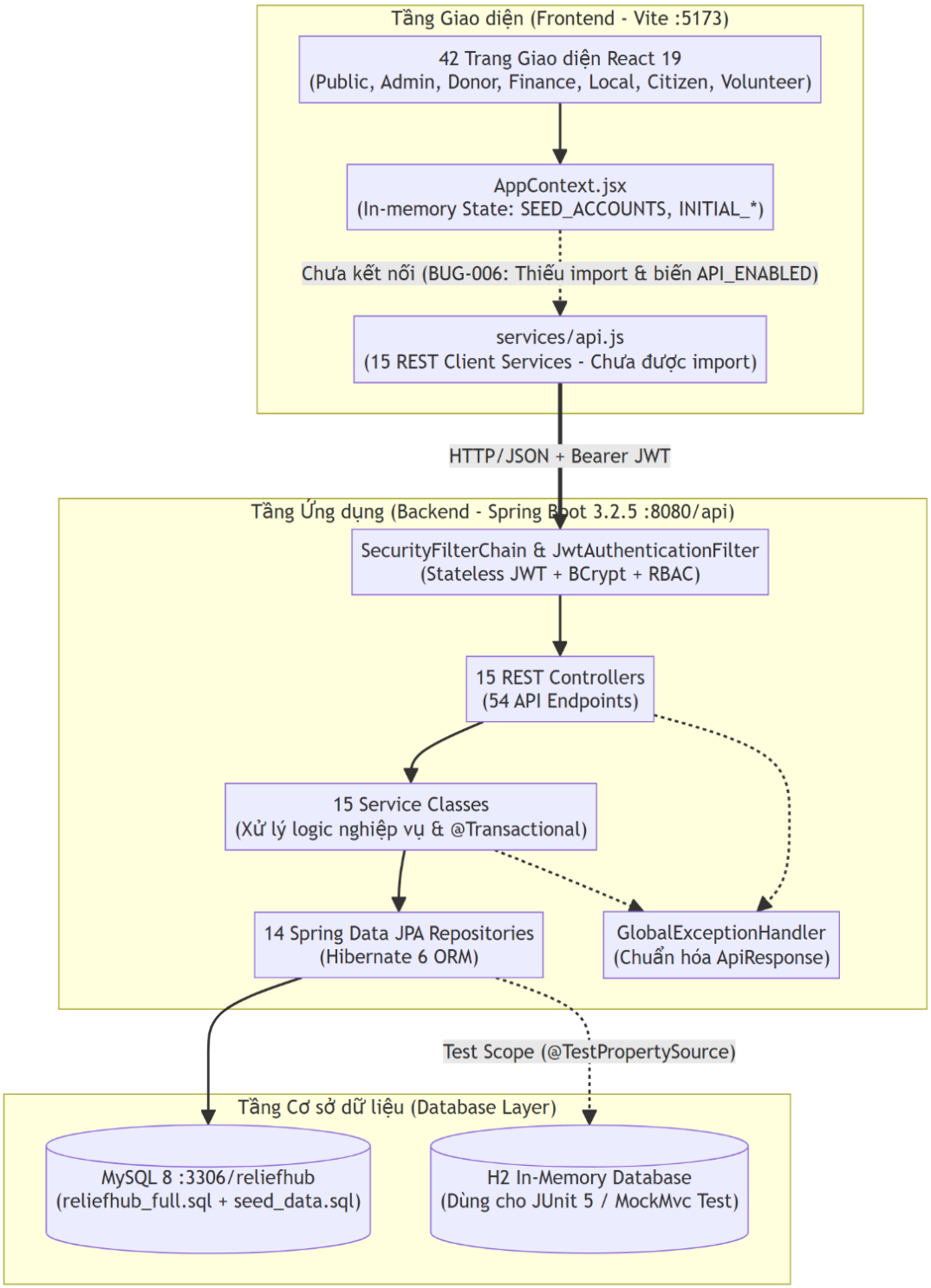


Hình 2.2: Sơ đồ cấu trúc thư mục các collection tài nguyên REST API (/api) và phân tầng giao tiếp Client - Server của ReliefHub.

2.1.2. Cấu trúc mã nguồn thực tế và vị trí đứt gãy tích hợp (BUG-006)

Khi kiểm kê và đối chiếu trực tiếp mã nguồn thực tế của dự án ReliefHub với sơ đồ thiết kế tại Hình 2.1 và Hình 2.2, nhóm kiểm thử xác định quy mô kỹ thuật chính xác của từng tầng như sau:

1. **Tầng Giao diện người dùng (*Frontend SPA*):** Xây dựng bằng React 19.2.7 kết hợp React Router DOM 7.18.0 (định tuyến 42 đường dẫn thuộc 43 tệp trang .jsx), bộ biểu tượng Lucide React 1.21.0 và trình đóng gói Vite 8.1.0 [9]. Trạng thái ứng dụng được quản lý tập trung tại AppContext.jsx (**826 dòng lệnh**) và lớp dịch vụ giao tiếp HTTP api.js (**218 dòng lệnh**).
2. **Tầng Xử lý nghiệp vụ và Bảo mật (*Backend REST API*):** Xây dựng trên nền tảng Spring Boot 3.2.5 (Java 21 - Oracle JDK 21.0.11 LTS) [6], bao gồm **89 tệp mã nguồn .java chính** thuộc 9 gói (config, security, controller, service, repository, entity, dto, exception, common) và **3 tệp .java kiểm thử gốc**. Hệ thống cung cấp **62 REST API endpoints** được bảo vệ bởi Spring Security 6 và JWT (jjwt 0.12.5) [6].
3. **Hiện trạng đứt gãy tích hợp giữa AppContext.jsx và api.js:** Mặc dù tệp api.js đã cài đặt đầy đủ các hàm gọi HTTP fetch tới Backend, tệp AppContext.jsx lại vận hành trên các mảng dữ liệu tĩnh trong bộ nhớ RAM và gọi biến API_ENABLED chưa từng được khai báo tại dòng 558 và 779. Nhóm kiểm thử đã dựng lại sơ đồ kiến trúc thực tế tại **Hình 2.3** để chỉ rõ vị trí đứt gãy tích hợp này:



Hình 2.3: Sơ đồ đối chiếu kiến trúc thực tế của hệ thống ReliefHub và vị trí xảy ra đứt gãy tích hợp giữa AppContext.jsx và api.js (BUG-006).

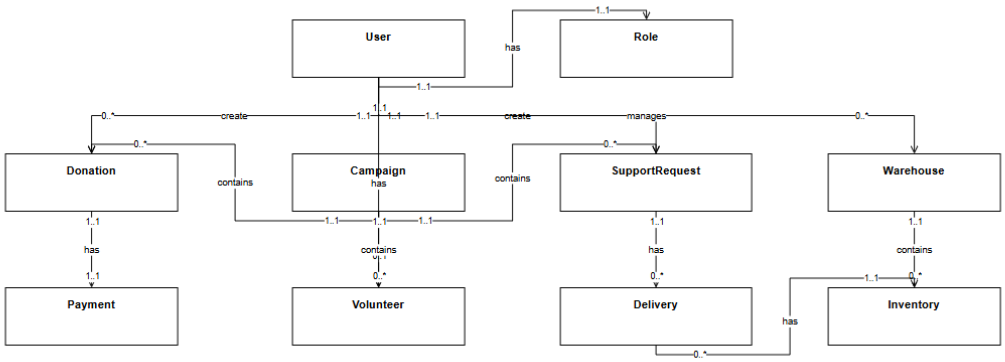
Bảng 2.1: Tổng hợp cấu trúc công nghệ và quy mô mã nguồn thực tế của hệ thống ReliefHub

Phân hệ / Thành phần	Công nghệ & Phiên bản xác minh	Quy mô mã nguồn thực tế đã kiểm kê	Đặc điểm kỹ thuật và hiện trạng tích hợp
Backend REST API	Java 21 (JDK 21.0.11 LTS), Spring Boot 3.2.5, Spring Security 6, Spring Data JPA, jjwt 0.12.5, Lombok, SpringDoc OpenAPI 2.3.0	**89 tệp .java chính** (15 Controllers, 15 Services, 17 Repositories, 17 Entities, 14 DTO files, 5 Security, 3 Exception, 3 Config, 1 Common) + **3 tệp .java test**	Biên dịch ra 187 lớp bytecode (.class); cung cấp **62 REST endpoints** chuẩn hóa qua ApiResponse<T>.
Frontend WebUI	React 19.2.7, React Router DOM 7.18.0, Lucide React 1.21.0, Vite 8.1.0, Oxlint 1.69.0	**50 tệp .jsx/.js** (gồm **43 tệp trang .jsx** định nghĩa **42 routes**, 4 components, 1 context, 1 service, 1 root)	api.js (**218 dòng**) đã viết sẵn fetch wrapper cho toàn bộ API, nhưng AppContext.jsx (**826 dòng**) chưa import api.js và chưa khai báo biến API_ENABLED.
Cơ sở dữ liệu	MySQL 8.0+ (Production) / H2 In-Memory 2.2.224 MODE=MySQL (Test)	2 kịch bản SQL: reliefhub_database.sql (45 bảng + 4 views) và reliefhub_full.sql (15 bảng cốt lõi + dữ liệu mẫu)	Tầng ORM (com.reliefhub.entity) hiện thực hóa **17 thực thể JPA** ánh xạ trực tiếp với nhóm bảng nghiệp vụ cốt lõi.

2.2. Thiết kế sơ đồ lớp phân tích (Class Diagrams)

2.2.1. Biểu đồ lớp phân tích tổng thể hệ thống ReliefHub

Cấu trúc hướng đối tượng và mối quan hệ giữa các lớp thực thể miền nghiệp vụ (*Domain Classes*: User, Role, Campaign, Donation, SupportRequest, Warehouse, Inventory, Delivery, VolunteerGroup) được thể hiện trong **Biểu đồ lớp phân tích tổng thể** tại **Hình 2.4**:

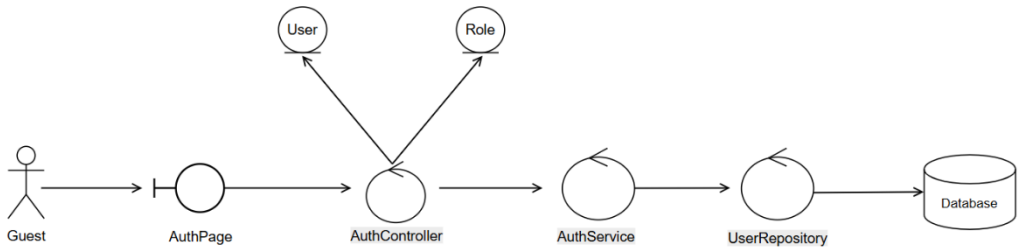


Hình 2.4: Biểu đồ lớp phân tích tổng thể hệ thống ReliefHub mô tả các thực thể trung tâm và mối quan hệ kết hợp / phụ thuộc.

2.2.2. Sơ đồ lớp chi tiết các phân hệ Xác thực, Quyên góp và Yêu cầu hỗ trợ

Theo mẫu kiến trúc *Boundary - Control - Entity (MVC / Layered Architecture)*, cấu trúc các lớp tham gia (*Views of Participating Classes - VOPC*) của 3 phân hệ có mức độ rủi ro nghiệp vụ cao nhất được trình bày tại **Hình 2.5**, **Hình 2.6** và **Hình 2.7**:

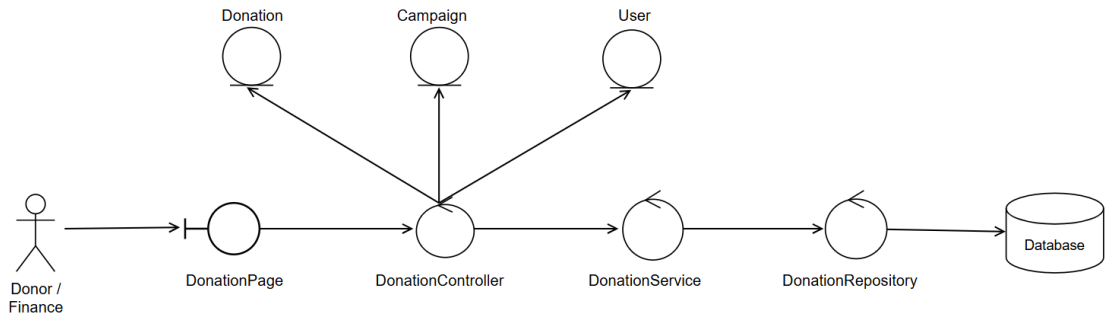
- **Phân hệ Xác thực (AuthController - AuthService - UserRepository / RoleRepository): Hình 2.5** minh họa cấu trúc lớp của luồng đăng ký và đăng nhập. Đối chiếu sơ đồ này với mã nguồn AuthService.java (dòng 59–71) cho thấy phương thức register() gọi trực tiếp roleRepository.findByName(request.getRoleName()) mà không kiểm tra danh sách trắng (*Whitelist*), dẫn tới lỗ hổng tự đăng ký quyền admin (BUG-001).



Hình 2.5: Sơ đồ lớp phân tích chức năng Xác thực (Đăng ký & Đăng nhập).

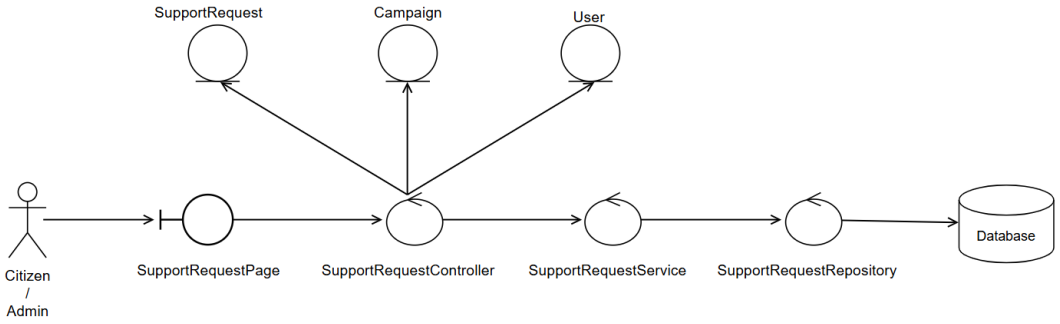
- **Phân hệ Quyên góp (DonationController - DonationService - CampaignRepository): Hình 2.6** mô tả mối liên kết giữa lớp điều khiển quyên góp, lớp dịch vụ DonationService và thực thể Campaign. Đây là cơ sở để thiết kế các ca kiểm thử hộp trắng kiểm tra việc cộng dồn số tiền đóng góp vào

campaign.currentAmount khi duyệt trạng thái CONFIRMED (TC-030, TC-031 - phát hiện lỗi BUG-005).



Hình 2.6: Sơ đồ lớp phân tích chức năng Quyên góp (Donation).

- **Phân hệ Yêu cầu hỗ trợ (SupportRequestController - SupportRequestService - SupportRequest): Hình 2.7** biểu diễn các phương thức xử lý đơn xin cứu trợ của người dân và phương thức chuyển trạng thái advanceStatus() qua 6 bước vòng đời (TC-035 → TC-040).

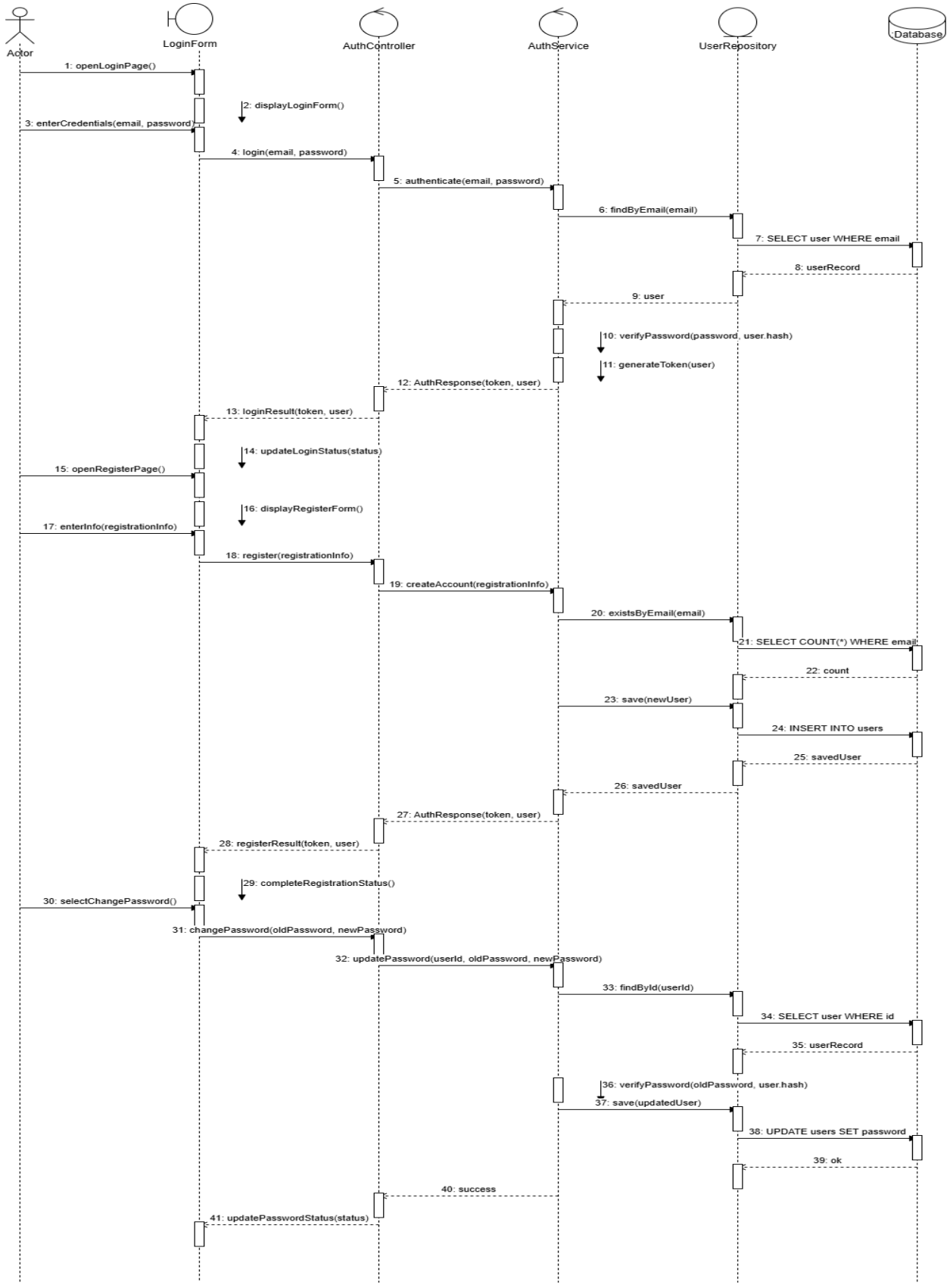


Hình 2.7: Sơ đồ lớp phân tích chức năng Yêu cầu hỗ trợ (SupportRequest).

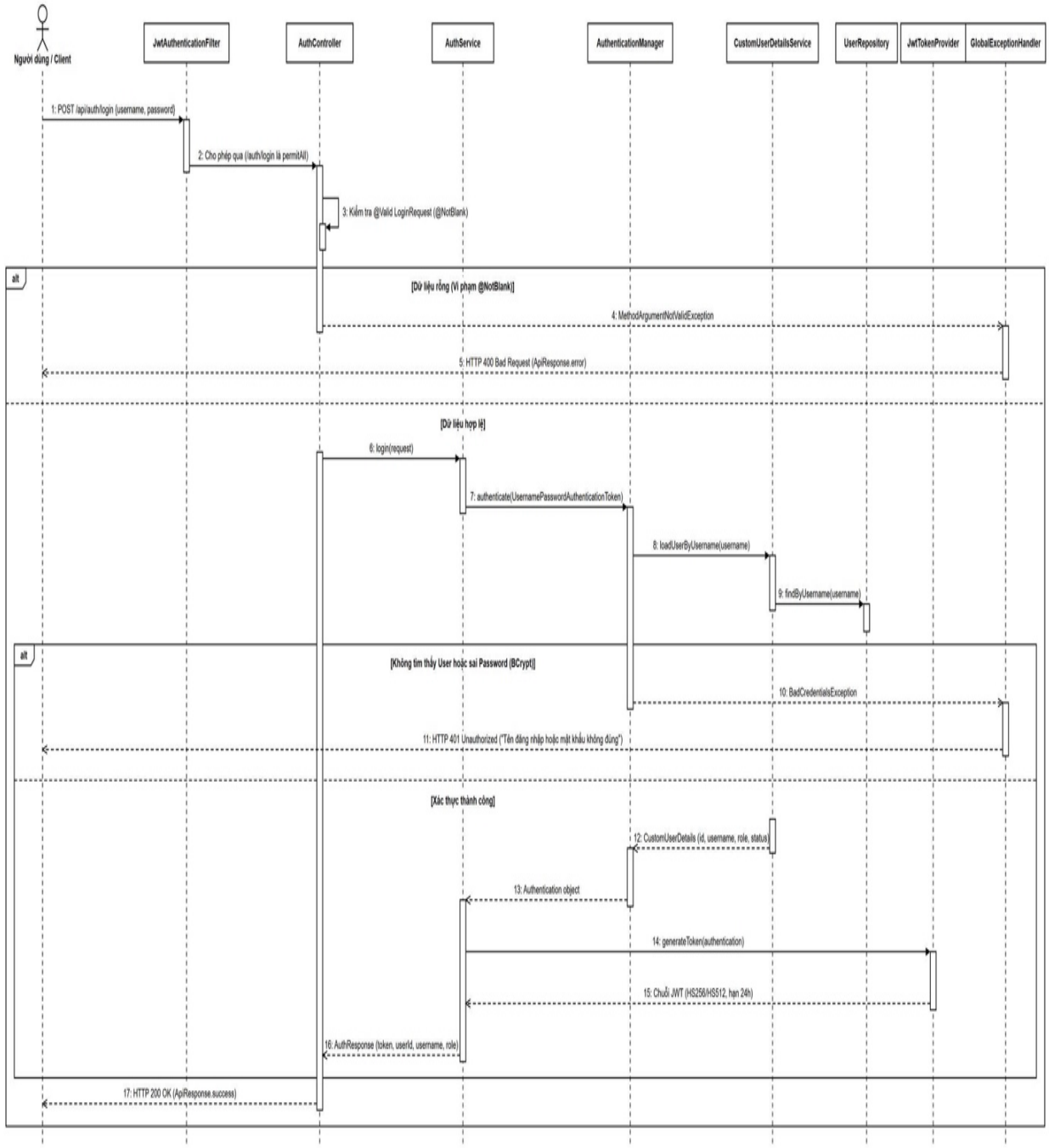
2.3. Thiết kế biểu đồ tuần tự (Sequence Diagrams) và máy trạng thái

2.3.1. Biểu đồ tuần tự Đăng ký, Đăng nhập và Xác thực JWT

Luồng tương tác tuần tự giữa người dùng, giao diện và tầng xử lý xác thực khi thực hiện đăng ký/đăng nhập được thể hiện tại **Hình 2.8**. Để làm rõ hơn cơ chế kiểm tra chữ ký số HMAC-SHA256 và trích xuất quyền hạn (ROLE_*) của bộ lọc JwtAuthenticationFilter đối với mọi yêu cầu API bảo vệ, nhóm kiểm thử dựng lại sơ đồ tuần tự chi tiết đối chiếu từ mã nguồn tại **Hình 2.9**:



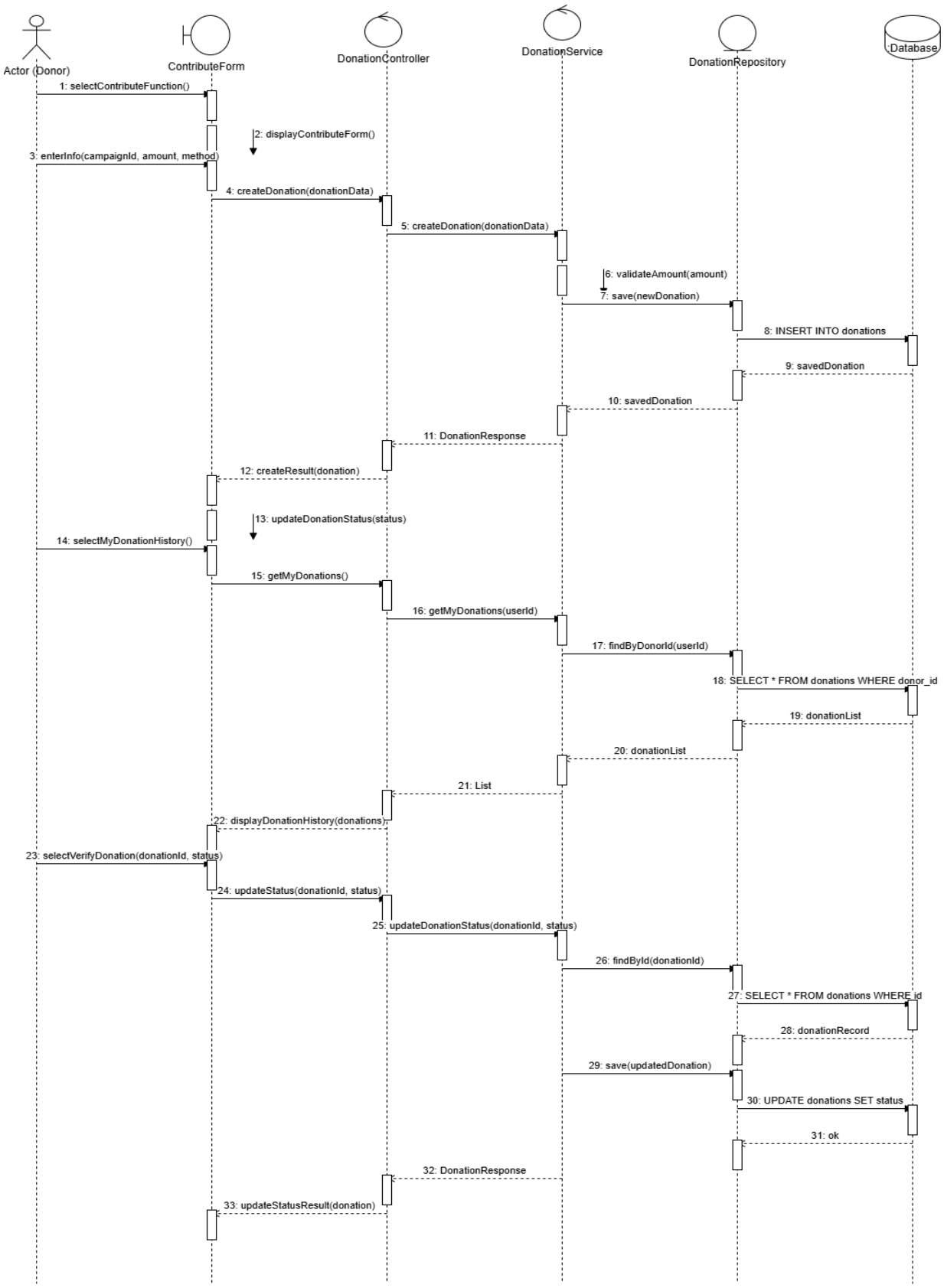
Hình 2.8: Biểu đồ tuần tự chức năng Đăng ký và Đăng nhập tài khoản.



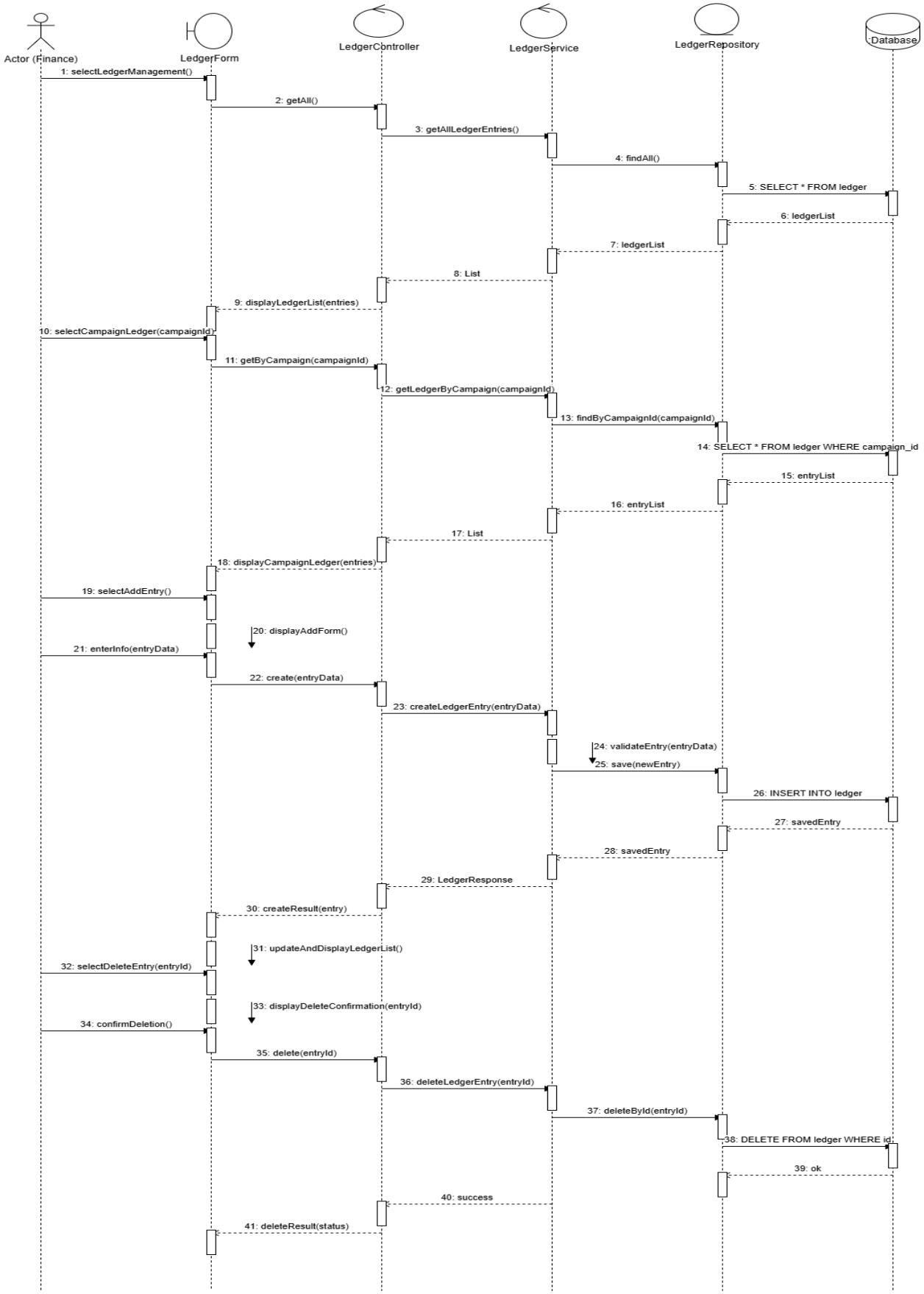
Hình 2.9: Biểu đồ tuần tự chi tiết cơ chế cấp phát JWT (POST /api/auth/login) và kiểm tra phân quyền JwtAuthenticationFilter tại tầng Backend.

2.3.2. Biểu đồ tuần tự Quyên góp, Xác minh và Quản lý Sổ cái

Trong phân hệ tài chính từ thiện, **Hình 2.10** biểu diễn trình tự Nhà tài trợ gửi thông tin quyên góp kèm minh chứng và Ban Tài chính xác minh giao dịch trước khi cập nhật quỹ chiến dịch. Tiếp nối luồng này, **Hình 2.11** mô tả trình tự ghi nhận các bút toán thu (INCOME), chi (EXPENSE) và truy vấn dữ liệu trên Sổ cái tài chính (Financial Ledger):



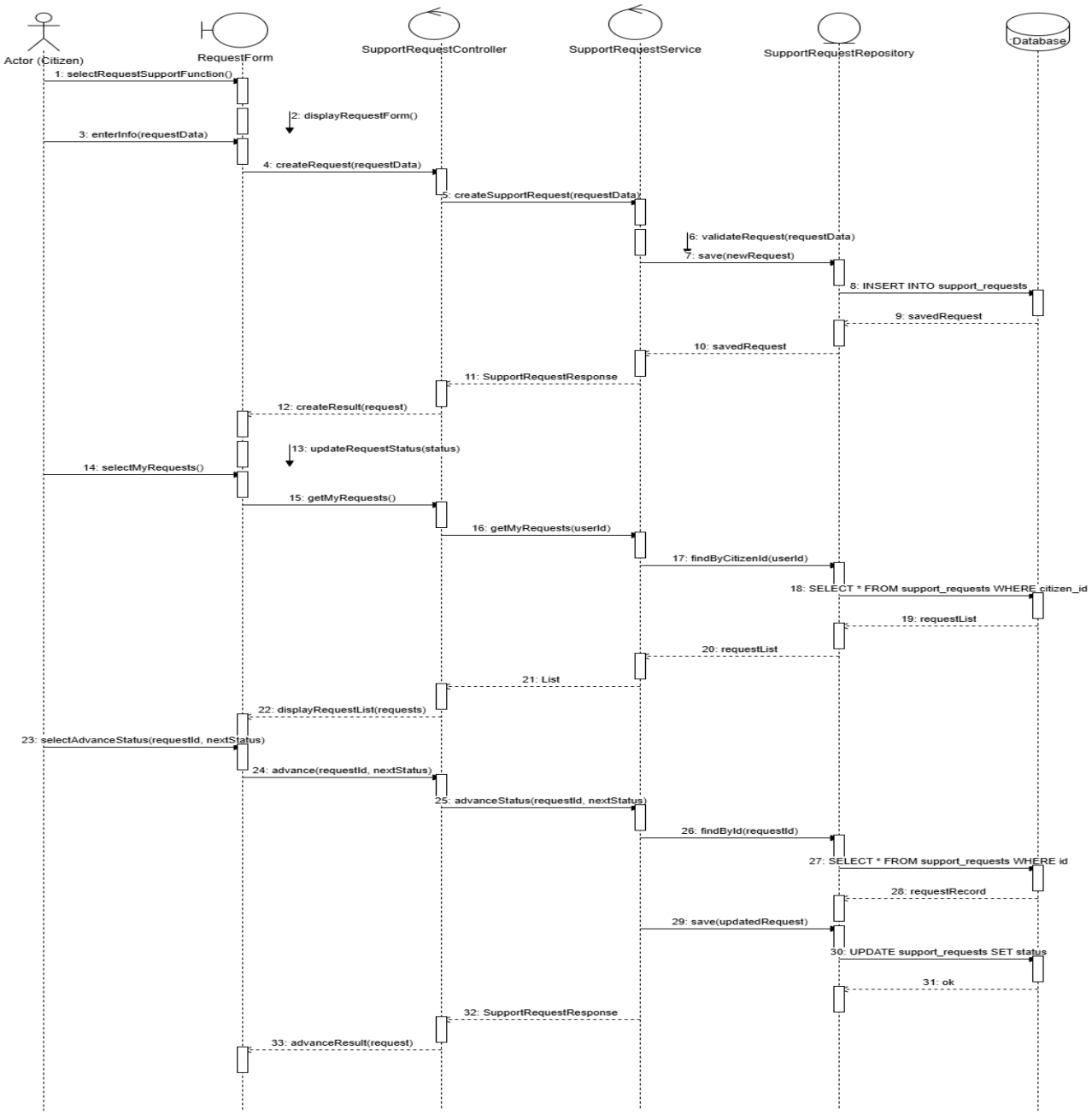
Hình 2.10: Biểu đồ tuần tự chức năng Đóng góp và Xác minh khoản đóng góp.



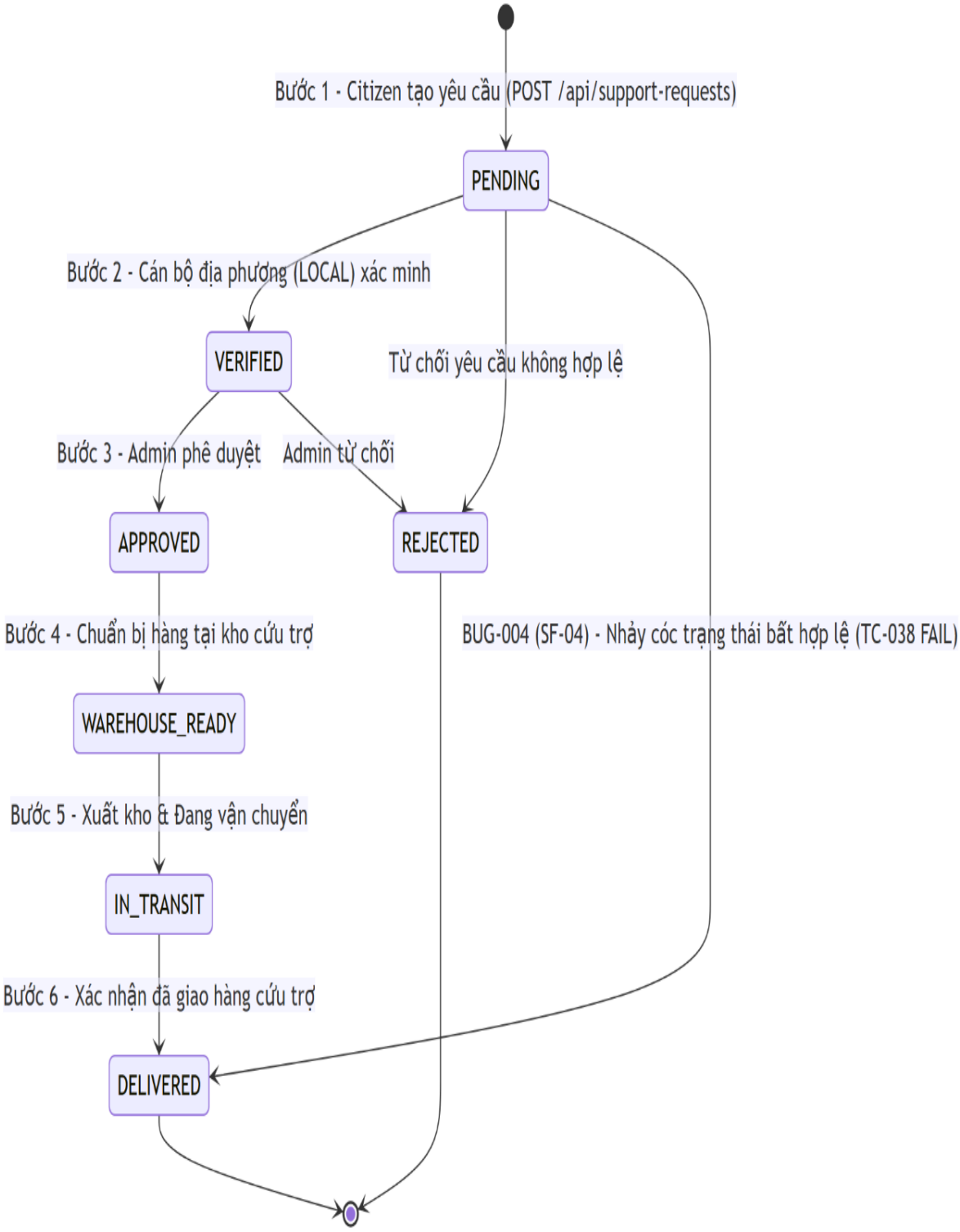
Hình 2.11: Biểu đồ tuần tự chức năng Quản lý Sổ cái tài chính (Financial Ledger).

2.3.3. Biểu đồ tuần tự Yêu cầu hỗ trợ, Máy trạng thái 6 bước và Vận chuyển

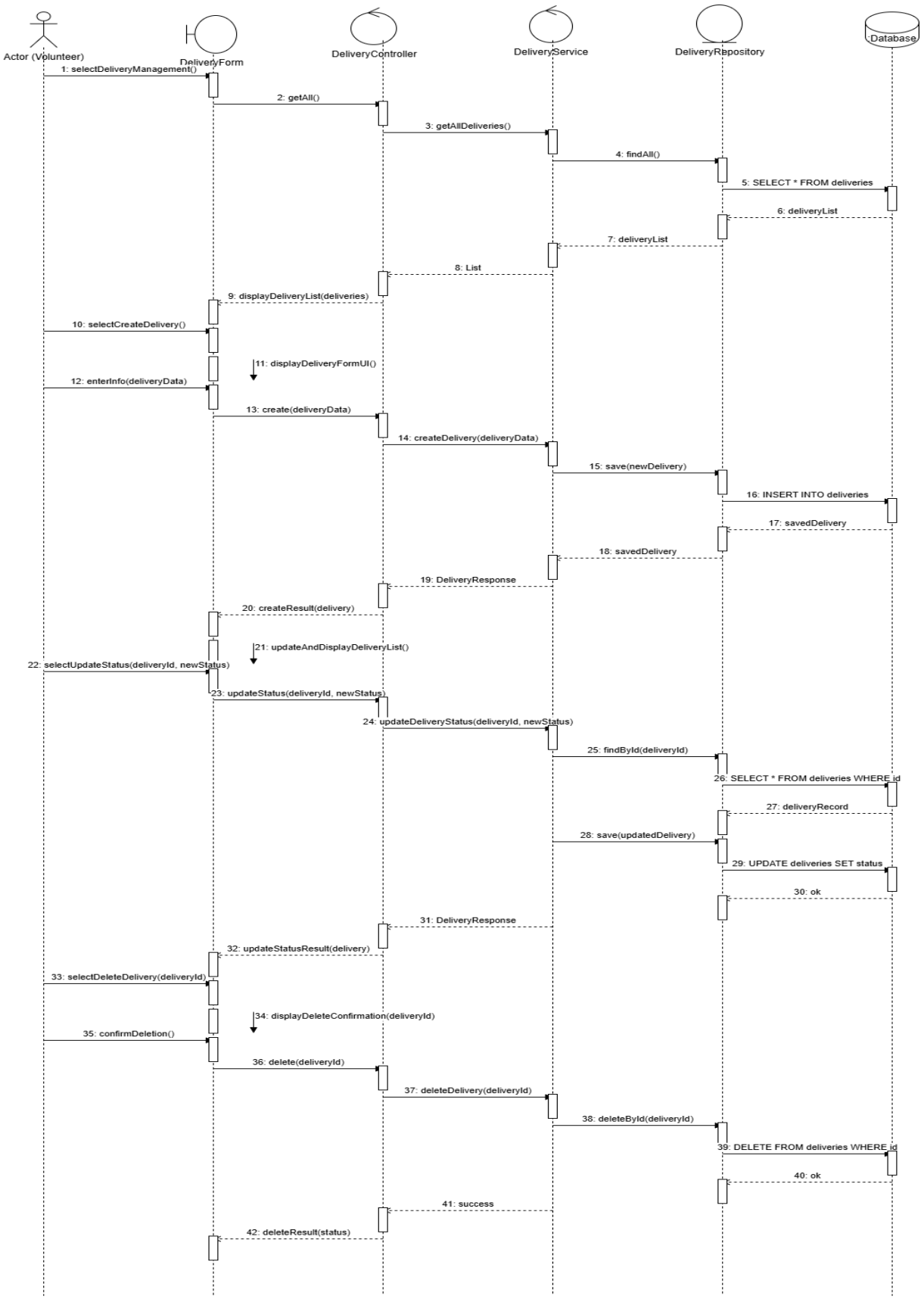
Đối với chuỗi cung ứng cứu trợ thực địa, **Hình 2.12** mô tả biểu đồ tuần tự khi người dân gửi yêu cầu hỗ trợ khẩn cấp lên hệ thống. Theo đặc tả nghiệp vụ (REQ-017), mỗi đơn yêu cầu hỗ trợ phải đi qua máy trạng thái hữu hạn 6 bước (*Finite State Machine*); tuy nhiên, đối chiếu mã nguồn tại SupportRequestService.advanceStatus() (dòng 83-98) phát hiện cấu trúc switch-case cho phép nhảy cóc trực tiếp từ PENDING sang DELIVERED (BUG-004), được nhóm kiểm thử mô hình hóa đối chiếu trực quan tại **Hình 2.13**. Sau khi đơn cứu trợ được duyệt và xuất kho, trình tự lập chuyến xe và điều phối bàn giao hàng cứu trợ được thể hiện tại **Hình 2.14** dưới đây:



Hình 2.12: Biểu đồ tuần tự chức năng Gửi yêu cầu hỗ trợ khẩn cấp từ người dân.



Hình 2.13: Đối chiếu máy trạng thái 6 bước của SupportRequest theo thiết kế nghiệp vụ và lỗ hổng nhảy cóc trạng thái trong mã nguồn thực tế (BUG-004).

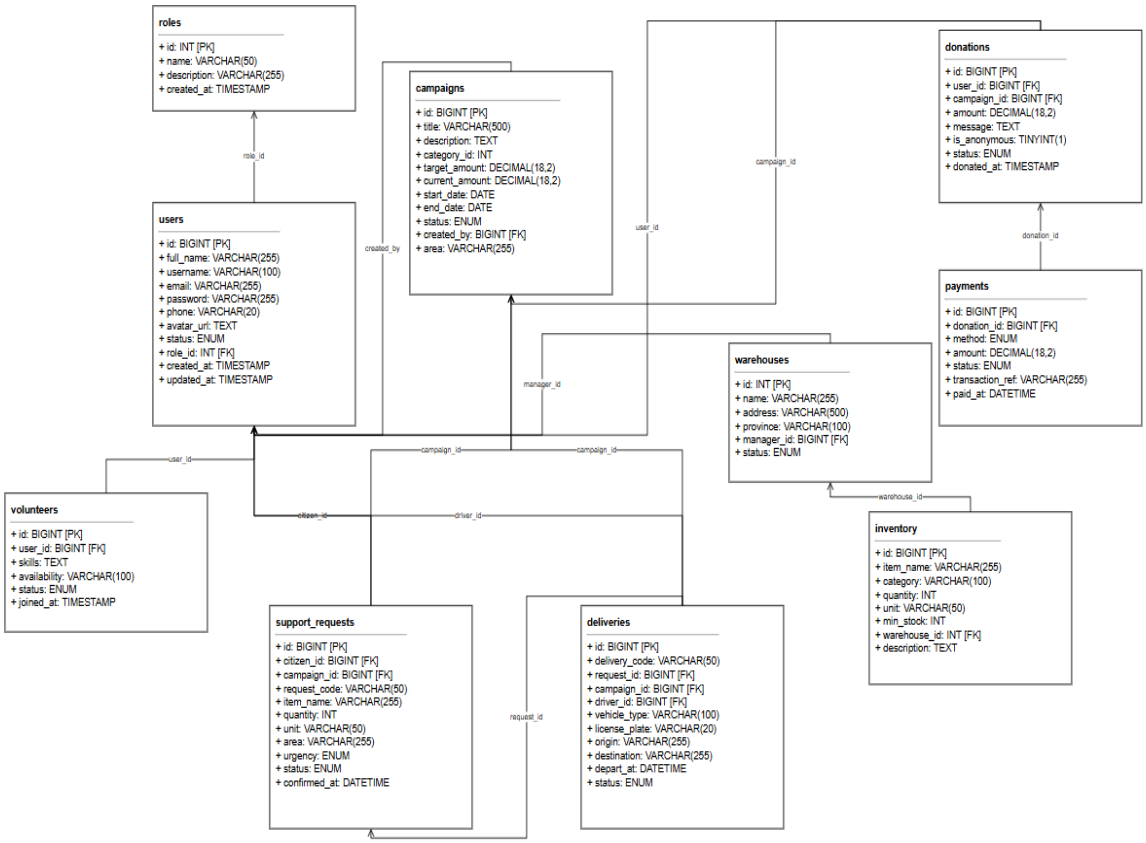


Hình 2.14: Biểu đồ tuần tự chức năng Điều phối vận chuyển và giao nhận hàng cứu trợ.

2.4. Thiết kế Cơ sở dữ liệu (ERD) và đối chiếu hiện thực hóa ORM

Cơ sở dữ liệu được biểu diễn ở hai cấp độ:

1. **Lược đồ CSDL mở rộng ở mức thiết kế (reliefhub_database.sql):** Gồm **45 bảng vật lý (CREATE TABLE) và 4 khung nhìn (CREATE VIEW), tổng cộng 49 đối tượng CSDL**, bao phủ cả các phân hệ nâng cao như nhật ký kiểm toán (audit_logs), lịch sử đăng nhập (login_history), phiên làm mới token (refresh_tokens), khôi phục mật khẩu (password_reset_tokens) và điểm tập kết (drop_points).
2. **Lược đồ CSDL lõi đã hiện thực hóa trong tầng ORM (com.reliefhub.entity & reliefhub_full.sql):** Tầng Backend Spring Data JPA hiện thực hóa **17 lớp Entity** cốt lõi phục vụ trực tiếp cho 15 Controller và 15 Service. Lược đồ quan hệ thực thể (*ERD*) của các bảng cơ sở dữ liệu lõi được trình bày tại **Hình 2.15** và đối chiếu chi tiết tại **Bảng 2.2**:



Hình 2.15: Lược đồ quan hệ thực thể (ERD) của các bảng cơ sở dữ liệu lõi trong hệ thống ReliefHub.

Bảng 2.2: Đối chiếu 17 Thực thể JPA đã hiện thực hóa trong mã nguồn Backend với cấu trúc bảng CSDL

STT	Phân hệ nghiệp vụ	Lớp Thực thể JPA (com.reliefhub.entity)	Tên bảng vật lý (MySQL / H2)	Khóa chính & Quan hệ khóa ngoại cốt lõi	Trạng thái hiện thực hóa & Kiểm thử
1	**Quản trị & Định danh**	Role.java, User.java	roles, users	users.role_id → roles.id (@ManyToOne)	Đã xác minh thực nghiệm
2	**Thiên tai & Chiến dịch**	Disaster.java, Campaign.java	disasters, campaigns	campaigns.created_by → users.id; quản lý target_amount, current_amount	Đã xác minh thực nghiệm
3	**Quyên góp & Hiện vật**	Donation.java, DonationItem.java	donations, donation_items	donations.campaign_id → campaigns.id, donor_id → users.id; donation_items.donation_id → donations.id	Đã xác minh thực nghiệm
4	**Sổ cái tài chính**	FinancialLedger.java	financial_ledger	campaign_id → campaigns.id, created_by → users.id (INCOME / EXPENSE)	Đã xác minh thực nghiệm
5	**Yêu cầu hỗ trợ 6 bước**	SupportRequest.java	support_requests	citizen_id → users.id; lưu trạng thái 6 bước (PENDING → DELIVERED)	Đã xác minh thực nghiệm
6	**Kho hàng & Tồn kho**	Warehouse.java, Inventory.java, GoodsCategory.java, StockMovement.java	warehouses, inventory, goods_categories, stock_movements	inventory.warehouse_id → warehouses.id; quản lý số lượng tồn và ngưỡng tối thiểu	Đã xác minh thực nghiệm
7	**Vận chuyển & Tình nguyện**	Delivery.java, VolunteerGroup.java, VolunteerMember.java	deliveries, volunteer_groups, volunteer_members	deliveries.request_id → support_requests.id, warehouse_id → warehouses.id	Đã xác minh thực nghiệm
8	**Khiếu nại &**	Complaint.java, Notification.java	complaints, notifications	complaints.user_id → users.id;	Đã xác minh

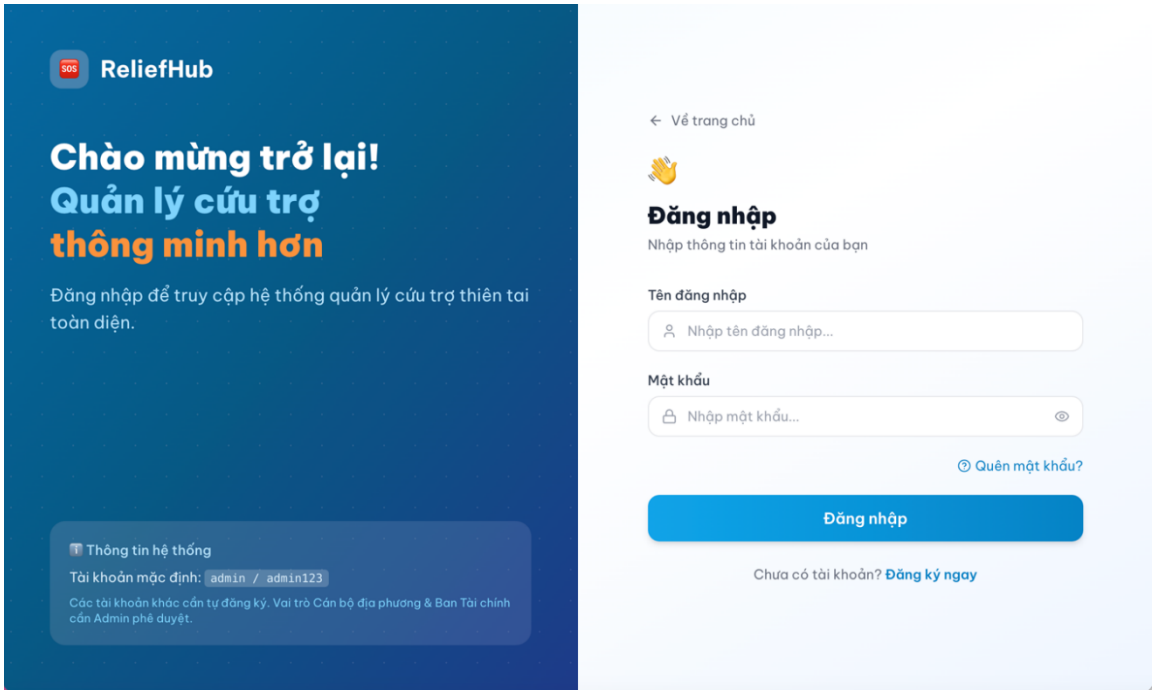
STT	Phân hệ nghiệp vụ	Lớp Thực thể JPA (com.reliefhub.entity)	Tên bảng vật lý (MySQL / H2)	Khóa chính & Quan hệ khóa ngoại cốt lõi	Trạng thái hiện thực hóa & Kiểm thử
	Thông báo			notifications.user_id → users.id	thực nghiệm
9	**Các bảng mở rộng trong SQL thiết kế**	*(Chưa có lớp Entity trong com.reliefhub.entity)*	31 bảng phụ trong reliefhub_database.sql (refresh_tokens, password_reset_tokens, audit_logs, v.v.)	Mới dừng ở kịch bản DDL SQL thiết kế, chưa có Repository/Service xử lý	Chưa kiểm thử động (Giới hạn thiết kế)

2.5. Hệ thống giao diện người dùng thực tế theo phân hệ (UI Screens)

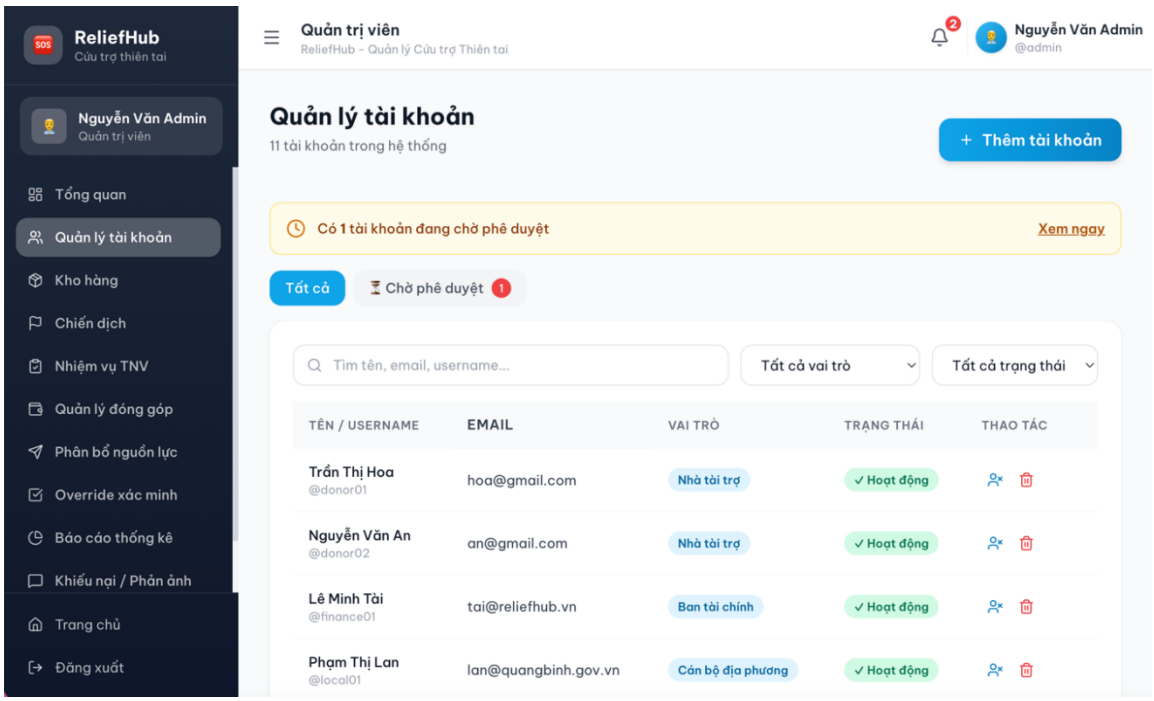
Để cung cấp góc nhìn trực quan về sản phẩm phần mềm ReliefHub trước khi đi vào kế hoạch và kịch bản kiểm thử, mục này trình bày **11 ảnh chụp giao diện thực tế của các chức năng chính (Hình 2.16 → Hình 2.26)** từ hồ sơ hệ thống ReliefHub, bao quát đầy đủ các phân hệ Xác thực, Quản trị viên, Địa phương, Nhà tài trợ, Ban Tài chính, Người dân vùng lũ và Quản lý kho cứu trợ:

2.5.1. Giao diện Xác thực, Quản trị tài khoản, Thiên tai và Chiến dịch

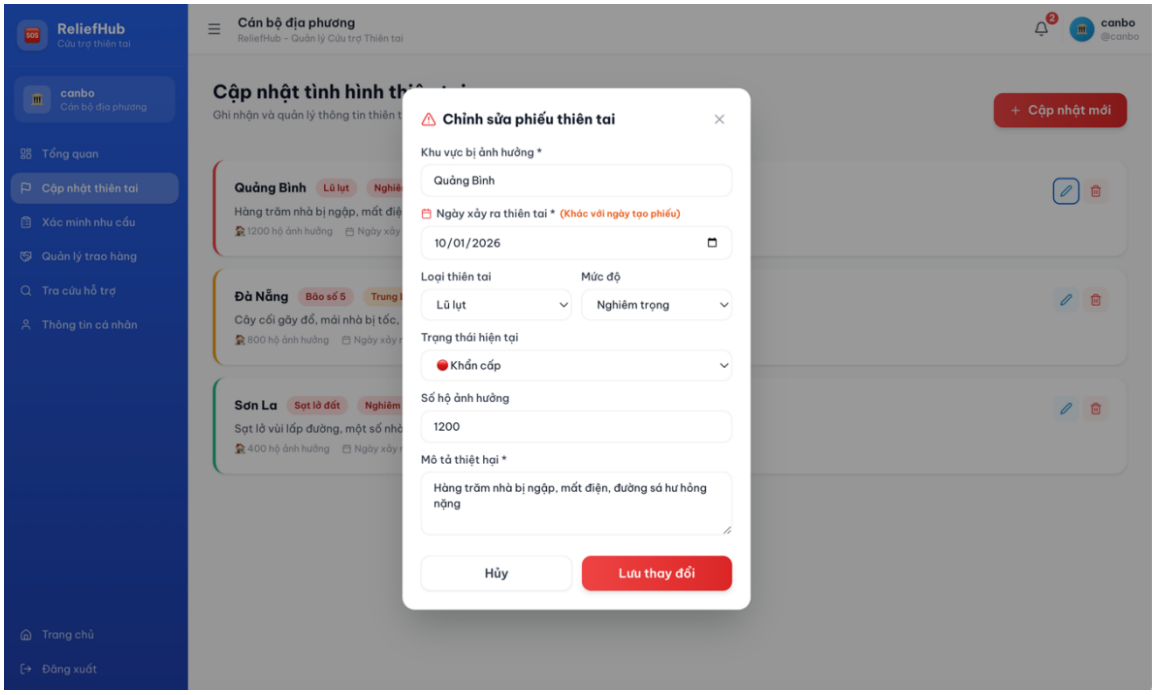
Hình 2.16 minh họa giao diện Đăng nhập tài khoản (/login) tích hợp khung chọn nhanh tài khoản Demo theo 6 vai trò và liên kết *"Quên mật khẩu?"* (nơi phát hiện lỗ hổng BUG-007). **Hình 2.17** thể hiện màn hình Quản trị tài khoản của Admin (/dashboard/admin/accounts); **Hình 2.18** thể hiện màn hình Cập nhật thông tin thiên tai của Cán bộ địa phương; và **Hình 2.19** trình bày màn hình Quản lý chiến dịch cứu trợ của Quản trị viên:



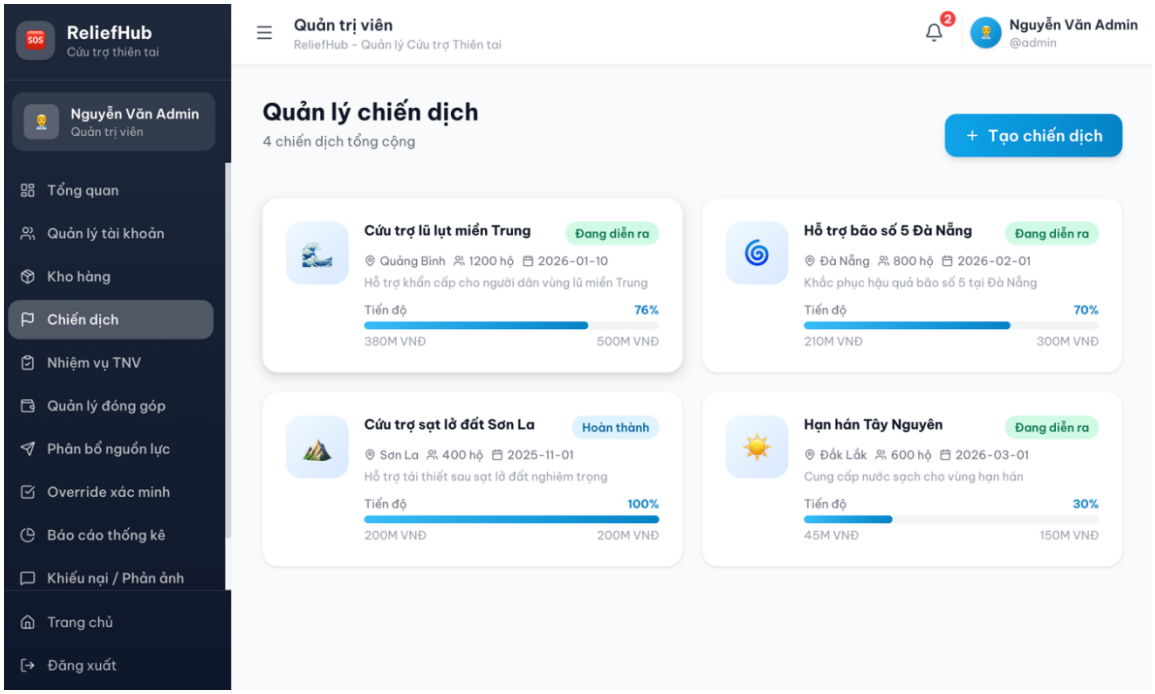
Hình 2.16: Giao diện Đăng nhập tài khoản (/login) với bảng tài khoản Demo 6 vai trò và chức năng Quên mật khẩu.



Hình 2.17: Giao diện Quản trị tài khoản người dùng (/dashboard/admin/accounts) - Cho phép Admin theo dõi danh sách tài khoản theo vai trò, trạng thái hoạt động và phê duyệt tài khoản cán bộ địa phương.



Hình 2.18: Giao diện Cập nhật thông tin thiên tai (/dashboard/local/disasters) - Cho phép khai báo loại hình thiên tai, địa bàn ảnh hưởng và mức độ khẩn cấp.

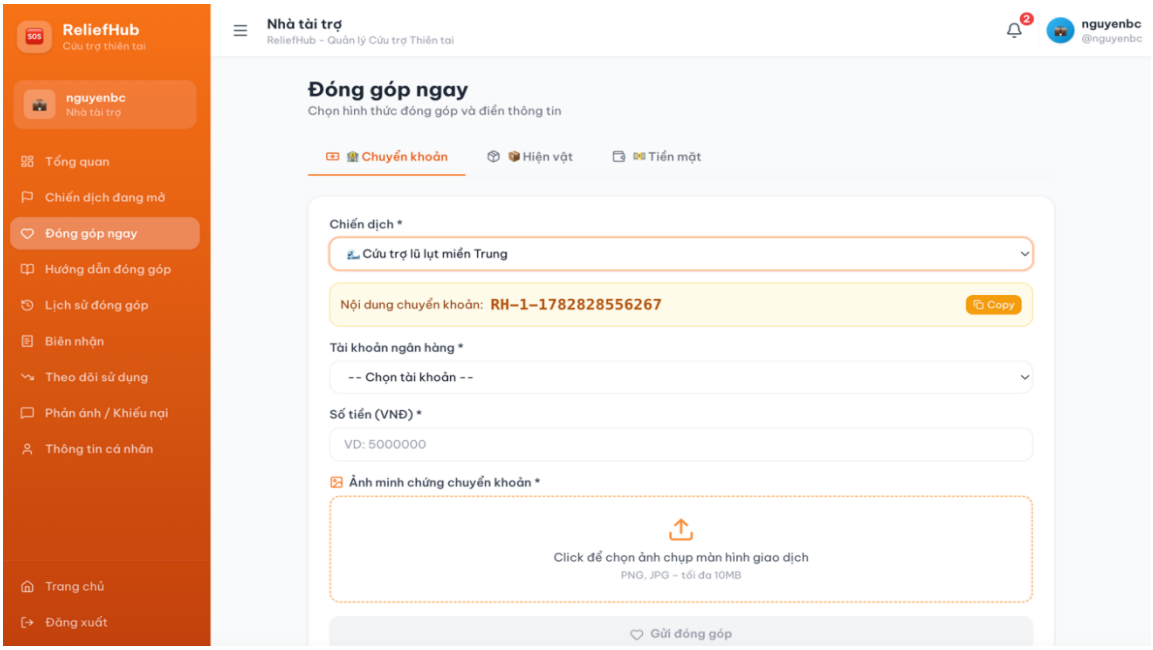


Hình 2.19: Giao diện Quản lý chiến dịch cứu trợ (/dashboard/admin/campaigns) - Theo dõi mục tiêu ngân sách, tiến độ quyên góp và trạng thái từng chiến dịch.

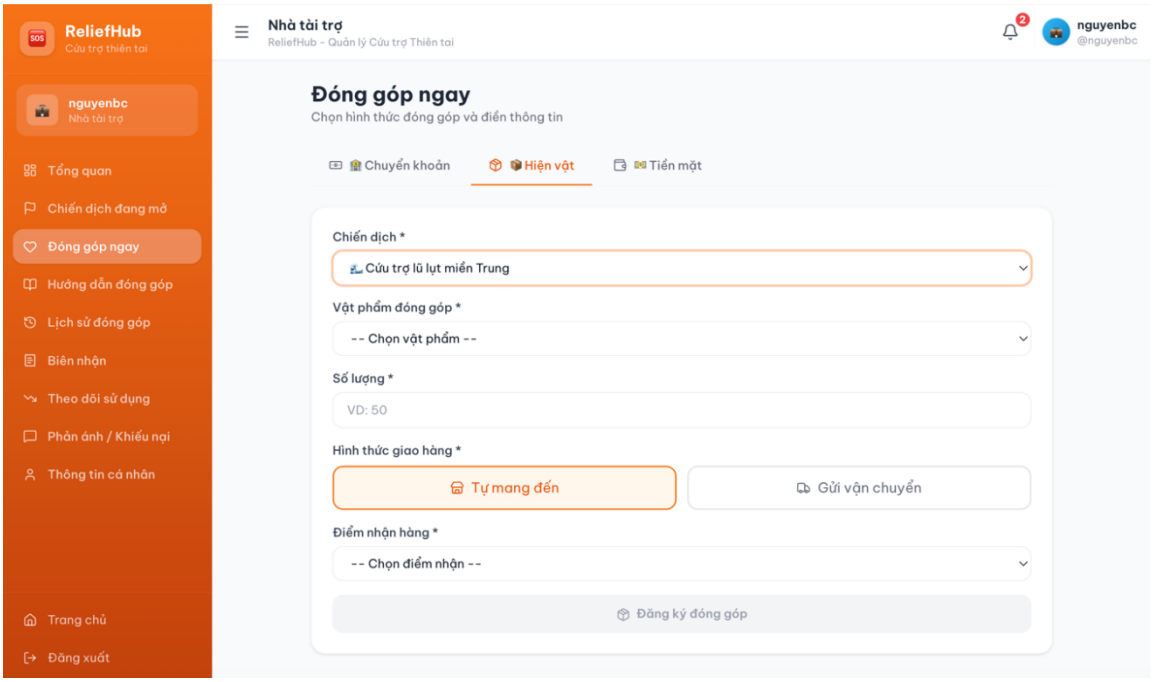
2.5.2. Giao diện Nhà tài trợ và Ban Tài chính

Đối với phân hệ huy động nguồn lực từ thiện, **Hình 2.20** và **Hình 2.21** lần lượt thể hiện giao diện Đóng góp chuyển khoản ngân hàng (/dashboard/donor/contribute) và Đóng

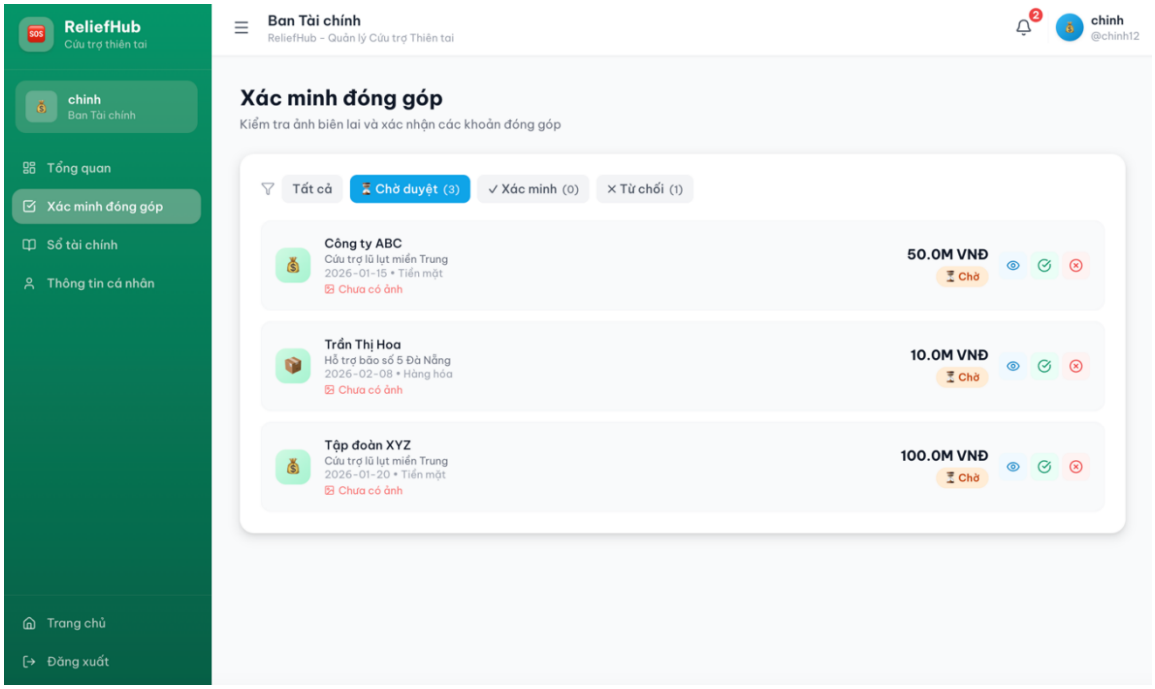
góp hiện vật cứu trợ (/dashboard/donor/in-kind) của Nhà tài trợ. Ở chiều đối soát nội bộ, **Hình 2.22** minh họa giao diện Xác minh khoản đóng góp của Ban Tài chính (/dashboard/finance/verify) và **Hình 2.23** thể hiện giao diện Sổ cái tài chính minh bạch (/dashboard/finance/ledger):



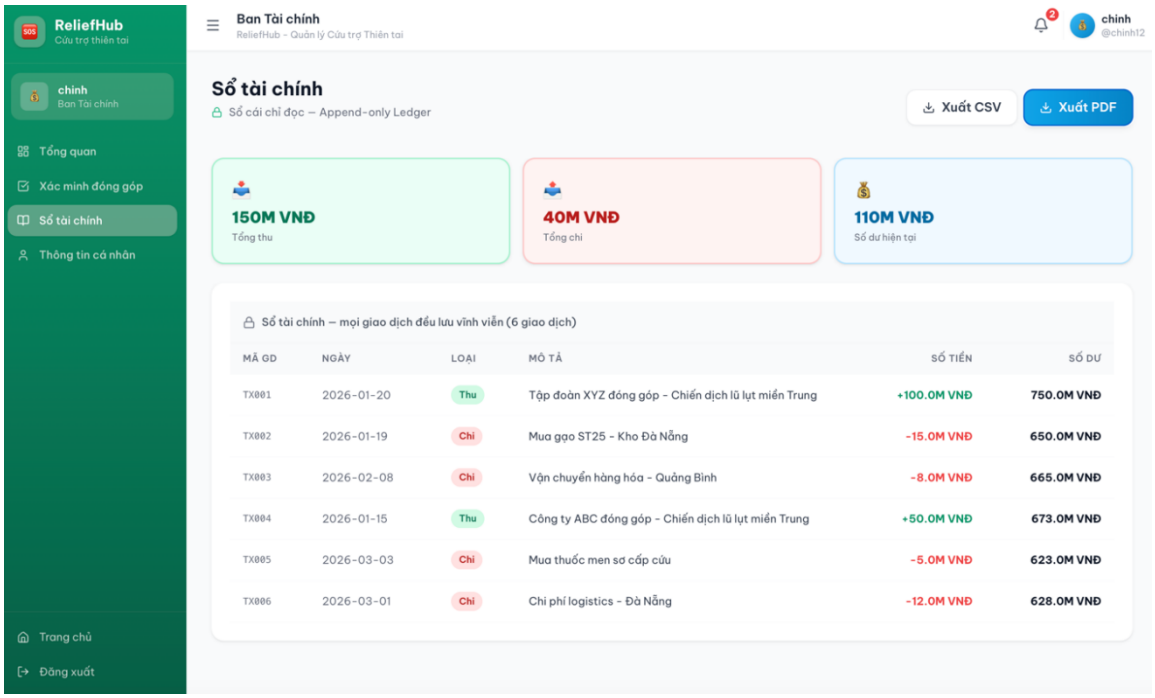
Hình 2.20: Giao diện Đóng góp chuyển khoản (/dashboard/donor/contribute) - Nhà tài trợ chọn chiến dịch, nhập số tiền, tải ảnh biên lai chuyển khoản và tùy chọn chế độ ẩn danh.



Hình 2.21: Giao diện Đóng góp hiện vật cứu trợ (/dashboard/donor/in-kind) - Cho phép nhà hảo tâm kê khai danh mục hàng hóa, số lượng, đơn vị tính và kho tiếp nhận.



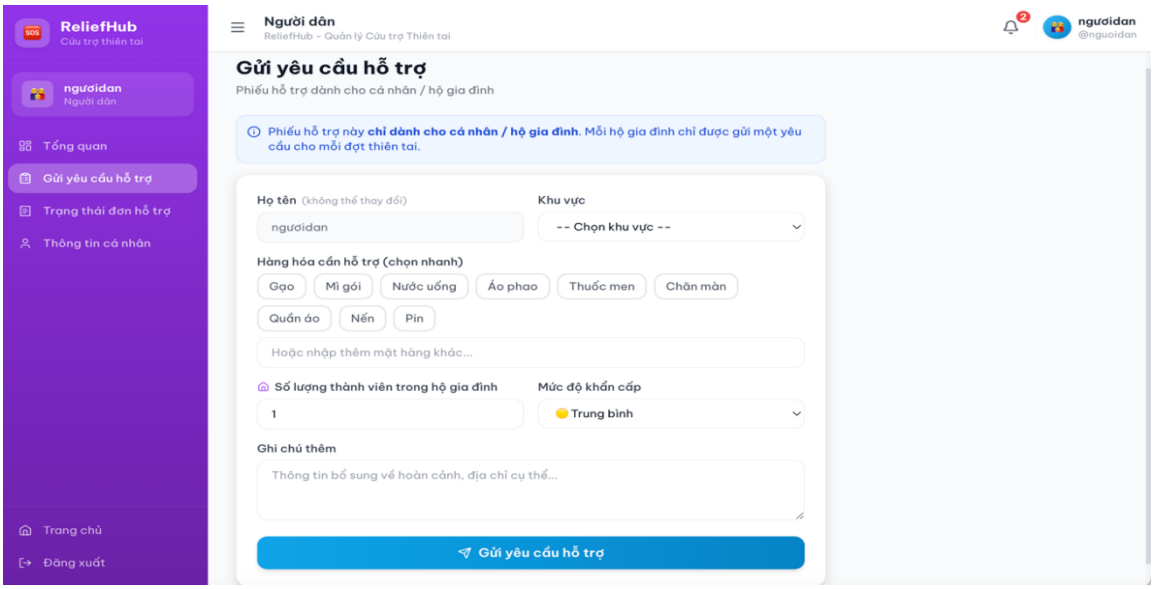
Hình 2.22: Giao diện Xác minh đóng góp (/dashboard/finance/verify) - Ban Tài chính kiểm tra chứng từ chuyển khoản để xác nhận (CONFIRMED) hoặc từ chối khoản quyên góp.



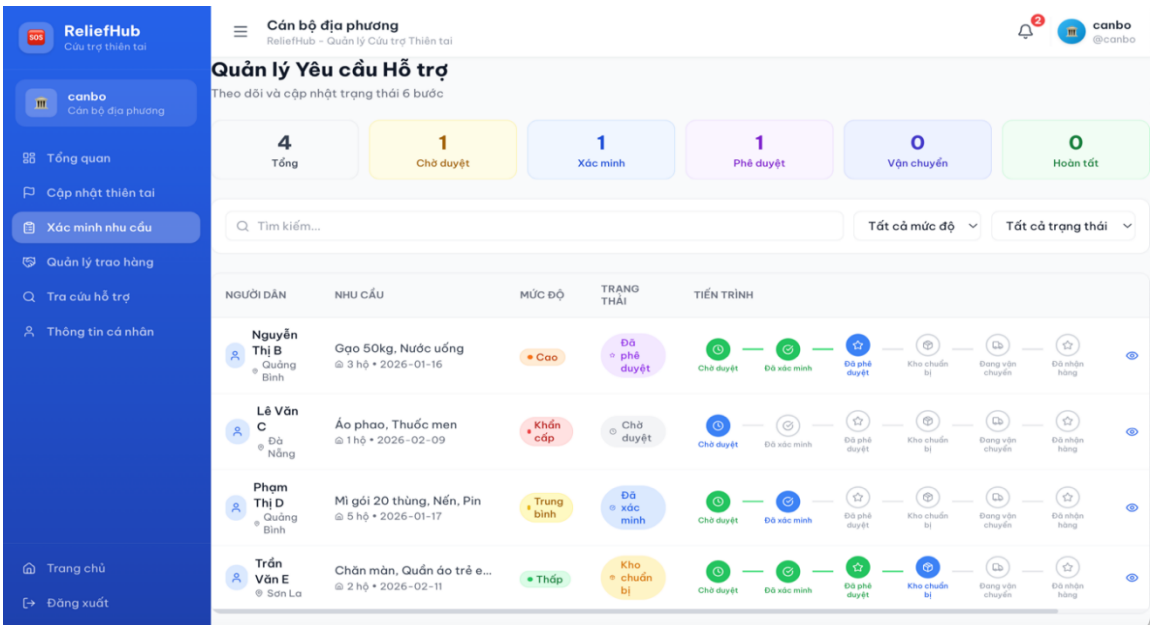
Hình 2.23: Giao diện Sổ cái tài chính (/dashboard/finance/ledger) - Hiển thị minh bạch các bút toán thu (INCOME) từ quyên góp và bút toán chi (EXPENSE) giải ngân cứu trợ.

2.5.3. Giao diện Người dân, Điều phối cứu trợ và Quản lý kho

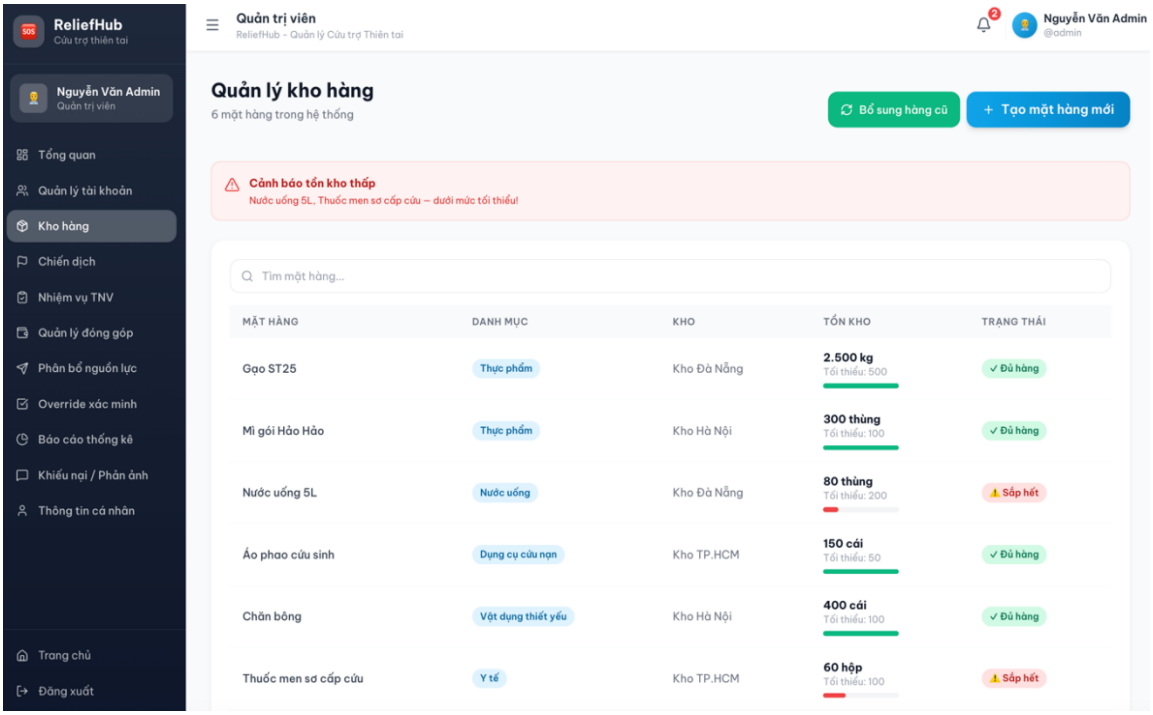
Đối với luồng cứu trợ nhân đạo tại vùng thiên tai, **Hình 2.24** thể hiện giao diện Gửi yêu cầu hỗ trợ khẩn cấp của Người dân (/dashboard/citizen/request); **Hình 2.25** thể hiện giao diện Quản lý và Điều phối yêu cầu hỗ trợ qua 6 bước trạng thái của Admin/Cán bộ địa phương; và **Hình 2.26** thể hiện giao diện Quản lý kho hàng và danh mục tồn kho cứu trợ (/dashboard/admin/inventory):



Hình 2.24: Giao diện Gửi yêu cầu hỗ trợ (/dashboard/citizen/request) - Người dân khai báo khu vực, mức độ khẩn cấp, số nhân khẩu và mặt hàng cứu trợ cần thiết.



Hình 2.25: Giao diện Quản lý yêu cầu hỗ trợ (/dashboard/admin/support-requests) - Điều phối viên theo dõi và chuyển tiếp trạng thái xử lý đơn cứu trợ qua quy trình 6 bước.



Hình 2.26: Giao diện Quản lý kho và danh mục hàng cứu trợ (/dashboard/admin/inventory) - Theo dõi số lượng tồn kho thực tế, đơn vị tính và ngưỡng cảnh báo tồn tối thiểu.

CHƯƠNG 3: CƠ SỞ LÝ THUYẾT, TIÊU CHÍ CHẤT LƯỢNG VÀ KẾ HOẠCH KIỂM THỬ

3.1. Thuật ngữ, từ viết tắt và người sử dụng tài liệu

3.1.1. Định nghĩa thuật ngữ và Từ viết tắt

Bảng 3.1: Bảng định nghĩa các thuật ngữ chuyên môn sử dụng trong kế hoạch kiểm thử

STT	Thuật ngữ	Định nghĩa và ý nghĩa áp dụng trong dự án ReliefHub
1	**Kiểm thử phần mềm (Software Testing)**	Quá trình thực thi chương trình hoặc phân tích tĩnh mã nguồn nhằm phát hiện lỗi (Defects) và đánh giá mức độ đáp ứng 28 yêu cầu (REQ-001 → REQ-028).
2	**Trường hợp kiểm thử (Test Case - TC)**	Tập hợp các điều kiện tiền đề, dữ liệu đầu vào, các bước thực hiện và kết quả mong đợi được thiết kế để kiểm tra một yêu cầu hoặc một nhánh điều khiển (TC-001 → TC-060).
3	**Độ phức tạp Cyclomatic V(G)**	Số đo định lượng số lượng đường đi độc lập tuyến tính (Basis Paths) trong đồ thị luồng điều khiển (CFG) của một phương thức, tính theo công thức V(G) = E - N + 2P = D + 1.
4	**Độ bao phủ mã nguồn (Code Coverage)**	Tỷ lệ phần trăm dòng lệnh (Line Coverage), nhánh điều kiện (Branch Coverage) và phương thức (Method Coverage) được thực thi bởi bộ kiểm thử tự động, đo bằng công cụ JaCoCo 0.8.12.
5	**Kiểm soát truy cập theo vai trò (RBAC)**	Cơ chế giới hạn quyền gọi API và truy cập trang giao diện theo 6 vai trò của ReliefHub (ADMIN, FINANCE, LOCAL, DONOR, CITIZEN, VOLUNTEER).
6	**Tính bất biến (Idempotency)**	Tính chất đảm bảo một thao tác cập nhật trạng thái (như xác nhận CONFIRMED cho khoản quyên góp) khi được gọi lặp lại nhiều lần vẫn giữ nguyên kết quả tài chính như khi gọi một lần.

Bảng 3.2: Bảng giải nghĩa các từ viết tắt

Từ viết tắt	Thuật ngữ tiếng Anh	Giải nghĩa tiếng Việt
SRS	Software Requirements Specification	Đặc tả yêu cầu phần mềm (trình bày tại Mục 1.4 - 1.5 - Bảng 1.3-1.4)
RTM	Requirement Traceability Matrix	Ma trận truy xuất yêu cầu (trình bày tại Mục 4.5 - Bảng 4.4)
CFG	Control Flow Graph	Đồ thị luồng điều khiển (Kiểm thử hộp trắng)
EP / BVA	Equivalence Partitioning / Boundary Value Analysis	Phân hoạch tương đương / Phân tích giá trị biên (Kiểm thử hộp đen)
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
JWT	JSON Web Token	Chuỗi mã thông báo xác thực không trạng thái (Stateless Token)
IDOR	Insecure Direct Object Reference	Lỗ hổng tham chiếu đối tượng trực tiếp không an toàn
XSS	Cross-Site Scripting	Lỗ hổng chèn mã kịch bản độc hại phía trình duyệt
CDP	Chrome DevTools Protocol	Giao thức điều khiển và thu thập log/ảnh chụp trình duyệt Chrome tự động

3.1.2. Người sử dụng tài liệu và Vai trò kiểm thử

1. **Quản lý kiểm thử (Test Manager – Ngô Thị Minh Phương):** Sử dụng kế hoạch kiểm thử để xác định phạm vi, phân bổ công việc, theo dõi tiến độ thực thi 60 Test Case và đối chiếu kết quả với tiêu chí kết thúc (Exit Criteria).
2. **Chuyên viên thiết kế kiểm thử (Test Designer - Nguyễn Thị Thu Giang & Ngô Thị Minh Phương):** Dựa vào chiến lược và kỹ thuật kiểm thử trong kế hoạch để xây dựng đồ thị CFG, bảng quyết định RBAC, thiết kế 60 Test Case (Mục 4.4) và lập ma trận truy xuất (Mục 4.5).

3. **Kiểm thử viên thực thi (Tester - Nguyễn Thị Thu Giang & Ngô Thị Minh Phương):** Thiết lập môi trường sandbox cô lập, chạy bộ test tự động JUnit 5/MockMvc, đo độ bao phủ JaCoCo, chạy Oxlint/Vite Build, thực thi kịch bản WebUI qua Chrome CDP và lập hồ sơ lỗi (Mục 6.1).

3.2. Cơ sở lý thuyết kiểm thử và mô hình chất lượng ISO/IEC 25010

Nhóm kết hợp hai nhóm kỹ thuật kiểm thử trong quá trình đánh giá dự án:

1. **Kiểm thử hộp trắng (White-box Testing):** Phân tích cấu trúc điều khiển bên trong phương thức, xây dựng **Đồ thị luồng điều khiển (Control Flow Graph - CFG)** và tính **Độ phức tạp Cyclomatic V(G)** theo công thức của McCabe (1976) [3]:

$$V(G) = E - N + 2P = D + 1$$

Trong đó E là số cung (edges), N là số đỉnh (nodes), P = 1 là số thành phần liên thông, và D là số nút quyết định đơn (if, while, for, case, &&, ||, ?:). Giá trị V(G) cho biết số lượng đường đi độc lập tuyến tính tối thiểu (Basis Set) cần kiểm tra.

1. **Kiểm thử hộp đen (Black-box Testing) [4], [5]:** Áp dụng 4 kỹ thuật: *Phân hoạch tương đương (EP)*, *Phân tích giá trị biên (BVA)*, *Bảng quyết định (Decision Table)* cho phân quyền RBAC 6 vai trò, và *Kiểm thử chuyển trạng thái (State Transition Testing)* cho quy trình xử lý yêu cầu cứu trợ 6 bước.

3.3. Hệ thống 8 tiêu chí đánh giá chất lượng (QC-01 → QC-08)

Bảng 3.3: Hệ thống 8 tiêu chí đánh giá chất lượng (QC-01 → QC-08) theo chuẩn ISO/IEC 25010

Mã tiêu chí	Đặc tính ISO/IEC 25010	Chỉ số đo lường (Metric)	Ràng buộc chính thức từ Code	Ngưỡng mục tiêu đề xuất của nhóm	Công cụ / Phương pháp đo
QC-01	**Functional Suitability** (Completeness)	Độ bao phủ yêu cầu (REQ), Tỷ lệ thực thi TC & Tỷ lệ Pass chung / Pass P0	62 REST API Endpoints & 42 trang UI; 10/10 bài test gốc Pass	Bao phủ 100% (28/28 REQ); Thực thi 100% (60/60 TC); Pass chung ≥ 85\%; Pass P0 = 100%	JUnit 5, MockMvc, Chrome CDP, Ma trận RTM (Bảng 4.4)
QC-02	**Functional Suitability** (Correctness)	Độ chính xác số học tài chính & Tuân thủ	BigDecimal(15,2), làm tròn 1 chữ số thập	Sai số tiền = 0 VNĐ; Không cộng trùng tiền; 0 bước nhảy	Kiểm thử hộp trắng & chuyển trạng

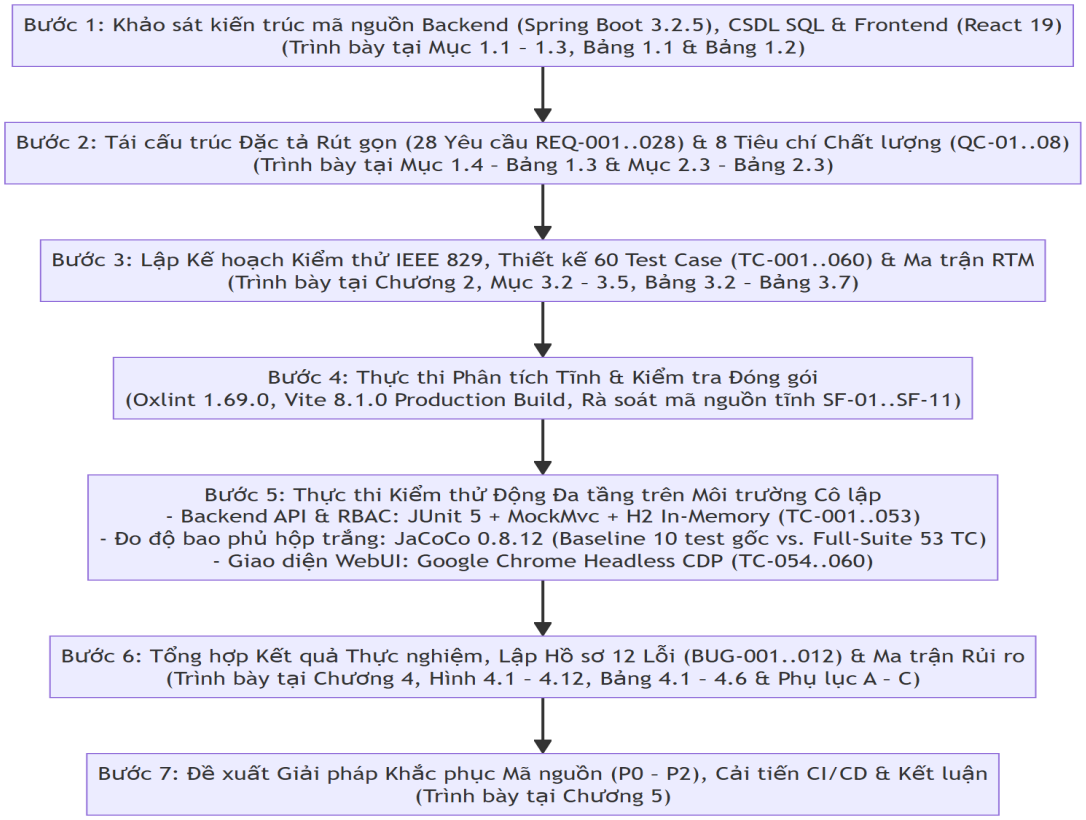
Mã tiêu chí	Đặc tính ISO/IEC 25010	Chỉ số đo lường (Metric)	Ràng buộc chính thức từ Code	Ngưỡng mục tiêu đề xuất của nhóm	Công cụ / Phương pháp đo
		máy trạng thái	phân, 6 trạng thái cứu trợ	trạng thái trái phép	thái (TC-025..TC-039)
QC-03	**Maintainability** (Testability & Complexity)	Độ bao phủ dòng lệnh (Line Coverage), nhánh (Branch), hàm (Method) & V(G)	Dự án gốc chỉ có 10 test, chưa cấu hình plugin đo coverage	Line Coverage gói service ≥ 70\%; toàn Backend ≥ 50\%; mỗi hàm đơn lẻ V(G) ≤ 10	JaCoCo 0.8.12 (Bảng 5.2, Bảng 5.3) & Đồ thị CFG (Bảng 4.2)
QC-04	**Maintainability** (Analysability & Build)	Số lỗi biên dịch, lỗi no-undef và cảnh báo phân tích tĩnh	mvn test-compile & vite build	0 lỗi biên dịch; 0 lỗi biến chưa khai báo (no-undef); Bundle < 500 kB	Maven 3.9.16, Oxlint 1.69.0, Vite 8.1.0 (Mục 5.3)
QC-05	**Security** (Confidentiality & Access Control) [8]	Tỷ lệ chặn truy cập trái phép trên 15 kịch bản bảo mật & phân quyền	SecurityConfig.java & ProtectedRoute.jsx [6]	100% kịch bản bảo mật/RBAC chặn đúng truy cập trái phép (HTTP 401/403)	15 TC bảo mật & phân quyền tại Mục 3.4.3b (TC-010..012, 015, 016, 021, 023, 032, 034, 039, 043, 048, 049, 052, 057)
QC-06	**Security** (Integrity & Auth Safety)	An toàn khôi phục mật khẩu & Kiểm soát tải lên tệp tin	Mật khẩu băm BCrypt; giới hạn file ≤ 10 MB trong application.yml	Không cho phép đổi mật khẩu Admin không cần xác thực; Chặn upload file .html/script	Kiểm thử bảo mật (TC-051, TC-052, TC-057)
QC-07	**Reliability** (Fault Tolerance & Recoverability)	Xử lý ngoại lệ chuẩn hóa & Bền vững trạng thái khi Reload (F5)	GlobalExceptionHandler.java trả về ApiResponse	Không trả về HTTP 500 khi dữ liệu đầu vào rỗng/sai; Không mất dữ liệu khi nhấn F5	Kiểm thử chịu lỗi (TC-024, TC-053, TC-058, TC-060)

Mã tiêu chí	Đặc tính ISO/IEC 25010	Chỉ số đo lường (Metric)	Ràng buộc chính thức từ Code	Ngưỡng mục tiêu đề xuất của nhóm	Công cụ / Phương pháp đo
QC-08	**Performance Efficiency** (Time Behaviour) [2]	Độ trễ phản hồi từng request API độc lập (p50, p95, Avg) trên môi trường cục bộ	Không quy định SLA cứng trong tài liệu gốc	Độ trễ phân vị p95 < 500 ms (và Avg < 500 ms) trên mỗi endpoint đại diện	Đo thực nghiệm N = 20 request/endpoint bằng MockMvc (ReliefHubPerformanceBenchmarkTest — Mục 5.1.2 & Bảng 6.2)

3.4. Chiến lược, hình thức và loại kiểm thử

3.4.1. Quy trình đánh giá chất lượng 7 bước

Quy trình kiểm thử và đánh giá chất lượng của nhóm gồm 7 bước tuần tự, trình bày tại Hình 3.1.



Hình 3.1: Sơ đồ quy trình kiểm thử và đánh giá chất lượng phần mềm 7 bước theo chuẩn IEEE 829 [1] áp dụng cho dự án ReliefHub.

3.4.2. Các hình thức kiểm thử áp dụng (4 hình thức)

1. **Kiểm thử Hộp đen (Black-box Testing):** Kiểm thử dựa trên 28 yêu cầu (REQ-001 → REQ-028), kiểm tra dữ liệu đầu vào/đầu ra HTTP JSON và phản hồi trên giao diện WebUI.
2. **Kiểm thử Hộp trắng (White-box Testing):** Phân tích cấu trúc điều khiển của mã nguồn Backend (Service, Security, Exception), xây dựng đồ thị CFG, tính độ phức tạp V(G) và đo độ bao phủ dòng lệnh/nhánh/phương thức bằng JaCoCo 0.8.12.
3. **Phân tích Tĩnh & Rà soát Mã nguồn (Static Analysis & Code Review):** Sử dụng Oxlint 1.69.0, trình biên dịch javac 21 và Vite 8.1.0 kết hợp đọc mã nguồn để phát hiện các biến chưa khai báo (no-undef), biến không sử dụng (no-unused-vars) và sai lệch cấu hình (SF-01 → SF-11).
4. **Kiểm thử Tự động (Automated Testing):** Thực thi tự động 53 ca kiểm thử Backend bằng JUnit 5 + MockMvc và 7 ca kiểm thử giao diện bằng kịch bản Chrome Headless CDP.

3.4.3. Đặc tả 3 loại kiểm thử theo cấu trúc IEEE 829

a) Loại 1: Kiểm thử Chức năng và Nghiệp vụ (Functional & Business Logic Testing)

- **Mục đích:** Kiểm tra 10 phân hệ nghiệp vụ (UC01 → UC10), các công thức tính toán tài chính (progressPct, cộng quỹ chiến dịch) và quy trình chuyển trạng thái 6 bước so với 28 yêu cầu tại Bảng 1.3–1.4.
- **Kỹ thuật áp dụng:** Phân hoạch tương đương (EP), Phân tích giá trị biên (BVA), Kiểm thử chuyển trạng thái (State Transition) và Kiểm thử đường cơ sở hộp trắng (Basis Path CFG).
- **Điều kiện dừng:** Thực thi xong 100% các Test Case chức năng và ghi nhận đầy đủ trạng thái PASS/FAIL kèm HTTP response thực tế.
- **Trách nhiệm thực hiện:** Nguyễn Thị Thu Giang (Backend API/Service) & Ngô Thị Minh Phương (Support Requests, Inventory, WebUI).
- **Phương pháp kiểm thử:** Chạy tự động bằng JUnit 5 + Spring Boot MockMvc trên CSDL H2 In-Memory (TC-001 → TC-053) và kịch bản Chrome CDP trên Vite Dev Server (TC-054 → TC-060).
- **Xử lý ngoại lệ:** Khi một Test Case thất bại do lỗi logic của hệ thống (như cộng trùng tiền BUG-005 hoặc nhảy cóc trạng thái BUG-004), nhóm giữ nguyên kết quả FAIL, trích xuất chuỗi JSON phản hồi thực tế (Phụ lục B) và lập hồ sơ lỗi chi tiết tại Mục 6.1.

b) Loại 2: Kiểm thử Bảo mật và Kiểm soát Phân quyền (Security & RBAC Testing)

- **Mục đích:** Kiểm tra hoạt động của bộ lọc xác thực JwtAuthenticationFilter, cấu hình SecurityConfig, phân quyền mức phương thức @PreAuthorize, cơ chế bảo vệ đường dẫn ProtectedRoute trên UI và kiểm tra định dạng tệp tải lên (FileUpload).
- **Kỹ thuật áp dụng:** Bảng quyết định (Decision Table) kết hợp 6 vai trò với các endpoint nhạy cảm; kiểm thử leo thang đặc quyền (Privilege Escalation), tham chiếu đối tượng trực tiếp (IDOR) và tải tệp không hợp lệ (Unrestricted File Upload).
- **Điều kiện dừng:** Hoàn thành 15 Test Case bảo mật và phân quyền (TC-010, TC-011, TC-012, TC-015, TC-016, TC-021, TC-023, TC-032, TC-034, TC-039, TC-043, TC-048, TC-049, TC-052, TC-057).
- **Trách nhiệm thực hiện:** Nguyễn Thị Thu Giang (API RBAC, IDOR, Upload) & Ngô Thị Minh Phương (UI ProtectedRoute, Quên mật khẩu).
- **Phương pháp kiểm thử:** Gửi HTTP request đính kèm Bearer JWT của vai trò thấp (citizen, donor) hoặc không có token tới các API quản trị của admin/finance qua MockMvc và đối chiếu mã trạng thái trả về (HTTP 401/403 mong đợi so với HTTP 200 OK thực tế).
- **Xử lý ngoại lệ:** Chỉ thực hiện trên môi trường cô lập H2 In-Memory và trình duyệt cục bộ 127.0.0.1:5173; không thực hiện trên hệ thống bên ngoài.

c) Loại 3: Kiểm thử Giao diện Người dùng và Tích hợp Trạng thái (WebUI & State Persistence Testing)

- **Mục đích:** Kiểm tra khả năng hoạt động của các trang giao diện React 19, thông báo lỗi trên màn hình, lỗi Runtime trên Browser Console và việc duy trì trạng thái sau khi tải lại trang (F5 Reload).
- **Kỹ thuật áp dụng:** Kiểm thử luồng kịch bản người dùng (Use Case Flow), kiểm thử tính nhất quán dữ liệu (Data Consistency) và kiểm thử phục hồi trạng thái (State Recoverability).
- **Điều kiện dừng:** Thực thi xong các ca kiểm thử từ TC-054 đến TC-060 và thu thập ảnh chụp màn hình .png cùng log Console.
- **Trách nhiệm thực hiện:** Ngô Thị Minh Phương.
- **Phương pháp kiểm thử:** Khởi chạy Vite v8.1.0 tại cổng 5173, điều khiển trình duyệt Google Chrome qua CDP, ghi nhận sự kiện Runtime.exceptionThrown và chụp màn hình bằng Page.captureScreenshot.

- **Xử lý ngoại lệ:** Khi xuất hiện lỗi Uncaught (in promise) ReferenceError: API_ENABLED is not defined, nhóm trích xuất stack trace từ Console (Phụ lục C) và nhúng ảnh chụp màn hình trước/sau thao tác tại Mục 5.4.

3.5. Môi trường và công cụ kiểm thử

Thông số môi trường kiểm thử, danh mục công cụ đã thực thi (kèm kết quả tóm tắt) và các công cụ không thực thi (kèm lý do và giới hạn đánh giá) được tổng hợp trực tiếp trong Bảng 3.4, Bảng 3.5 và Bảng 3.6.

Bảng 3.4: Cấu hình phần cứng, hệ điều hành và phần mềm nền tảng máy trạm kiểm thử

STT	Hạng mục tài nguyên	Thông số / Phiên bản xác minh thực tế	Trạng thái xác minh	Vai trò trong quá trình kiểm thử
1	**Hệ điều hành & Shell**	Microsoft Windows 10 64-bit (Build 10.0.19045), PowerShell	Đã xác minh	Môi trường hệ điều hành máy trạm & thực thi tập lệnh kiểm thử tự động
2	**Java JDK & Apache Maven**	Oracle JDK 21.0.11 LTS / Apache Maven 3.9.16	Đã xác minh	Biên dịch 89 tệp mã nguồn Java và chạy bộ kiểm thử Backend Spring Boot 3.2.5
3	**Node.js & npm**	Node.js v24.13.0 / npm 11.6.2	Đã xác minh	Thực thi phân hệ Frontend React 19.2.7, công cụ phân tích tĩnh Oxlint và Vite Bundler
4	**CSDL Kiểm thử Cô lập**	H2 In-Memory 2.2.224 (jdbc:h2:mem:evaldb;MODE=MySQL)	Đã xác minh	Cô lập dữ liệu kiểm thử tự động, giữ nguyên trạng dịch vụ MySQL80 trên máy trạm
5	**Trình duyệt Kiểm thử UI**	Google Chrome 156.0.8078.73 Headless (điều khiển qua giao thức CDP, độ phân giải 1440x900)	Đã xác minh	Thực thi tự động 7 kịch bản giao diện TC-054 → TC-060 tại http://127.0.0.1:5173

Bảng 3.5: Danh mục các công cụ kiểm thử đã thực thi kèm trạng thái và kết quả tóm tắt

STT	Tên công cụ	Phiên bản xác minh	Mục đích sử dụng trong dự án	Trạng thái thực thi	Kết quả tóm tắt & Vị trí trình bày trong báo cáo
1	**JUnit Jupiter (JUnit 5) & Maven Surefire**	5.10.2 / 3.1.2	Thực thi tự động bộ 10 bài test có sẵn (Baseline) và 53 Test Case đánh giá Backend (TC-001 → TC-053)	**Đã thực thi**	- **Bộ test gốc:** 10/10 PASS, 0 FAIL (Bảng 5.1a). - **Bộ test mở rộng:** 53 TC thực thi (39 PASS / 14 FAIL), thời gian chạy các test case ~15.1s (Bảng 4.3a–g, Bảng 5.1b, Phụ lục B).
2	**Spring Boot Test & MockMvc**	3.2.5	Giả lập chuỗi HTTP request qua SecurityFilterChain, kiểm thử đơn vị và tích hợp REST API (gồm 2 Unit Test cấp phương thức TC-004, TC-026 và 51 ca kiểm thử tích hợp API/Service qua MockMvc trên 62 REST endpoints) & phân quyền RBAC	**Đã thực thi**	Kiểm tra đầy đủ 10 phân hệ API; phát hiện và xác minh 9 lỗi Backend (BUG-001 → BUG-005, BUG-008 → BUG-010) kèm phản hồi HTTP JSON (Mục 6.1, Phụ lục B).
3	**JaCoCo Maven Plugin**	0.8.12	Đo độ bao phủ mã nguồn hộp trắng (Line, Branch, Method Coverage) giữa Baseline và Full-Suite	**Đã thực thi**	Nâng **Line Coverage** toàn Backend từ **25.53% lên 78.61%** (tầng service từ **14.33% lên 78.43%**), **Branch Coverage** tầng service từ **6.36% lên 44.07%**, **Method Coverage** tầng service từ **15.83% lên 58.27%** (Mục 5.2, Bảng 5.2–5.3, Phụ lục A).
4	**Oxlint Static Analyzer**	1.69.0	Phân tích mã nguồn tĩnh 50 tệp React/JSX (no-	**Đã thực thi**	Quét 50 tệp trong 95ms: phát hiện **83 cảnh báo** (no-unused-vars — BUG-012) và

STT	Tên công cụ	Phiên bản xác minh	Mục đích sử dụng trong dự án	Trạng thái thực thi	Kết quả tóm tắt & Vị trí trình bày trong báo cáo
			unused-vars, no-undef)		**36 báo cáo lỗi no-undef**, trong đó có **5 lỗi biến dự án chưa khai báo** (API_ENABLED, supportRequestService, tokenManager) gây lỗi BUG-006 (Mục 5.3, Phụ lục C.1).
5	**Vite Production Bundler**	8.1.0	Kiểm tra khả năng biên dịch đóng gói Frontend và đo kích thước gói bundle JS/CSS	**Đã thực thi**	Biên dịch thành công **870 module** trong 16.30s; phát hiện gói index-BsLZoEvX.js nặng **1,214.90 kB** (gzip 318.76 kB), vượt ngưỡng 500 kB gấp 2.43 lần (BUG-012) (Mục 5.3, Phụ lục C.2).
6	**Chrome DevTools Protocol (CDP) Runner**	Node.js + Chrome	Tự động hóa 7 kịch bản WebUI (TC-054 → TC-060), bắt ngoại lệ Console và chụp ảnh màn hình	**Đã thực thi**	Thực thi **7 TC (3 PASS / 4 FAIL)**; nhúng trực tiếp **13 ảnh chụp màn hình** (Hình 5.11 → Hình 5.23) và xác minh BUG-006, BUG-007, BUG-011 (Mục 5.4, Phụ lục C.3).

Bảng 3.6: Danh mục các công cụ kiểm thử không thực thi, lý do và giới hạn đánh giá tương ứng

Tên công cụ tham khảo	Loại kiểm thử	Trạng thái	Lý do không thực thi trong đợt đánh giá	Giới hạn đánh giá tương ứng trong báo cáo
Apache JMeter / k6 / LoadRunner	Kiểm thử hiệu năng chịu tải & Stress Testing	**Không thực thi**	Môi trường kiểm thử vận hành trên máy trạm cá nhân với CSDL H2 In-Memory cục bộ; phạm vi bài tập lớn	Chưa có số liệu thực nghiệm về thông lượng tối đa (Throughput/RPS), độ trễ dưới tải đồng thời nhiều người

Tên công cụ tham khảo	Loại kiểm thử	Trạng thái	Lý do không thực thi trong đợt đánh giá	Giới hạn đánh giá tương ứng trong báo cáo
			tập trung vào kiểm thử chức năng, hộp trắng và phân quyền.	dùng hoặc nguy cơ Race Condition khi giao dịch song song (ghi nhận tại Mục 6.2.3).
OWASP ZAP / Burp Suite	Rà quét lỗ hổng bảo mật tự động (Active DAST)	**Không thực thi**	Không chạy công cụ tấn công tự động sinh lưu lượng đột biến; các kịch bản bảo mật ứng dụng được kiểm tra có kiểm soát qua MockMvc và Chrome CDP.	Chỉ đánh giá được các lỗ hổng logic xác thực, phân quyền RBAC, IDOR và tải tệp đã thiết kế trong TC-001 → TC-060; chưa rà quét tự động toàn bộ bề mặt cấu hình HTTP Security Headers/CORS (Mục 6.2.3).

3.6. Tiêu chí bắt đầu, tạm dừng, kết thúc và quy trình quản lý lỗi

- **Tiêu chí Bắt đầu (Entry Criteria):** Hoàn thành đặc tả 28 yêu cầu (Bảng 1.3–1.4), hệ thống 8 tiêu chí chất lượng (Bảng 3.3), thiết kế đủ 60 Test Case (Bảng 4.3a → Bảng 4.3h) và thiết lập môi trường sandbox H2 In-Memory để không tác động tới mã nguồn gốc.
- **Tiêu chí Tạm dừng & Tiếp tục (Suspension & Resumption Criteria):** Tạm dừng khi xảy ra xung đột cổng mạng hoặc lỗi cấu hình môi trường; tiếp tục sau khi khắc phục sự cố môi trường.
- **Tiêu chí Kết thúc (Exit Criteria):** Thực thi đủ 100% (60 / 60) Test Case, trích xuất đầy đủ kết quả phản hồi HTTP, bảng số liệu độ bao phủ JaCoCo và 23 ảnh chụp màn hình bằng chứng vào Chương 5 và Phụ lục A–C, mô tả chi tiết 12 lỗi tại Mục 6.1 và kiểm tra khớp số liệu trên toàn bộ tài liệu.
- **Quy trình 7 bước Quản lý lỗi (Defect Handling Lifecycle):**

1. *Bước 1 - Phát hiện và Ghi nhận lỗi (Detection & Logging):* Thu thập mã trạng thái HTTP, chuỗi JSON Response, Console Stack Trace và ảnh chụp màn hình trong quá trình chạy test.

2. *Bước 2 - Đối chiếu Đặc tả và Phân tích Tĩnh (Cross-Verification):* Xác định tệp và dòng lệnh gây lỗi trong mã nguồn để xác nhận lỗi thực tế (Confirmed Defect).
3. *Bước 3 - Phân loại Mức độ Nghiêm trọng (Severity: Critical, High, Medium, Low) và Độ Ưu tiên (P0, P1, P2).*
4. *Bước 4 - Lập Hồ sơ Lỗi (Defect Reporting) tại Mục 6.1 (Bảng 6.1 và 12 hồ sơ BUG-001 → BUG-012).*
5. *Bước 5 - Đề xuất Hướng sửa Mã nguồn (Patch Recommendation) tại Chương 7 (không sửa trực tiếp vào mã nguồn gốc).*
6. *Bước 6 - Ghi nhận Trạng thái Lỗi (Status Tracking):* Cả 12 lỗi được giữ ở trạng thái **Confirmed / Open (Chưa sửa trên mã nguồn gốc)**.
7. *Bước 7 - Kiểm thử Xác nhận và Hồi quy (Confirmation & Regression Retesting):* Đề xuất chạy lại bộ 60 Test Case tự động sau khi áp dụng bản sửa lỗi trên nhánh riêng.

3.7. Rủi ro kế hoạch và lịch trình phân công

Bảng 3.7: Ma trận rủi ro kế hoạch kiểm thử và biện pháp giảm thiểu

STT	Rủi ro / Giới hạn	Khả năng	Tác động	Biện pháp giảm thiểu đã thực hiện
1	**Quy tắc bảo vệ mã nguồn gốc (Read-Only):** Không được sửa đổi code hoặc pom.xml trong thư mục dự án gốc.	Cao	Không thể thêm trực tiếp file test mới hoặc plugin JaCoCo vào thư mục gốc.	Thực thi bộ test mở rộng và đo JaCoCo trên bản sao sandbox riêng kết nối H2 In-Memory; giữ nguyên thư mục mã nguồn gốc.
2	**Frontend chưa kết nối với Backend REST API:** AppContext.jsx dùng dữ liệu in-memory và không import api.js.	Đã xác nhận (100%)	Không thể kiểm thử mạng E2E từ giao diện React xuống MySQL trên code hiện tại.	Tách rõ 2 tầng kiểm thử: (1) Kiểm thử đơn vị (2 TC: TC-004, TC-026) & Kiểm thử tích hợp REST API của Backend (51 TC qua MockMvc trên 62 REST endpoints) bằng MockMvc + H2; (2) Kiểm thử giao diện React tại :5173 và ghi nhận lỗi thiếu kết nối thành BUG-006 (SF-06).

STT	Rủi ro / Giới hạn	Khả năng	Tác động	Biện pháp giảm thiểu đã thực hiện
3	**Nhiều Test Case kiểm chứng lỗi có kết quả FAIL:** Mã nguồn hiện tại có 12 lỗi làm 18/60 Test Case trả về FAIL.	Đã xác nhận (100%)	Tỷ lệ Pass chung đạt 70.00% (thấp hơn ngưỡng 85%).	Ghi nhận đúng kết quả thực tế 42 PASS / 18 FAIL; các Test Case FAIL phản ánh đúng các lỗi tồn tại trong mã nguồn hiện tại.

Bảng 3.8: Lịch trình và phân công chi tiết công việc của nhóm (2 thành viên)

STT	Nội dung công việc và hạng mục phụ trách chính thức	Nguyễn Thị Thu Giang (23010871)	Ngô Thị Minh Phương (23012156)	Kết quả trình bày trong báo cáo	Trạng thái
1	Khảo sát kiến trúc Backend, CSDL MySQL và mã nguồn Spring Boot	Thực hiện	-	Chương 1, Chương 2 (Bảng 2.1, Bảng 2.2)	Hoàn thành
2	Xây dựng kế hoạch kiểm thử theo chuẩn IEEE 829 và hệ thống 8 tiêu chí chất lượng ISO/IEC 25010	-	Thực hiện	Chương 3 (Bảng 3.1 → Bảng 3.8)	Hoàn thành
3	Hệ thống hóa 28 yêu cầu (REQ-001 → REQ-028) và thiết lập Ma trận truy xuất yêu cầu hai chiều (RTM)	-	Thực hiện	Chương 1 (Bảng 1.3, 1.4), Chương 4 (Bảng 4.4 → 4.7)	Hoàn thành
4	Phân tích đồ thị luồng điều khiển (CFG), tính toán độ phức tạp Cyclomatic V(G) của 5 phương thức nghiệp vụ lõi	Thực hiện	-	Chương 4, Mục 4.2 (Hình 4.1, Bảng 4.2)	Hoàn thành
5	Thiết kế và thực thi tự động 53 ca kiểm thử tích hợp Backend (TC-001 → TC-053) trên JUnit 5, Spring Boot MockMvc và CSDL H2 In-Memory	Thực hiện	-	Bảng 4.3a → 4.3g, Mục 5.1 (Hình 5.1 → 5.4), Phụ lục B	Hoàn thành

STT	Nội dung công việc và hạng mục phụ trách chính thức	Nguyễn Thị Thu Giang (23010871)	Ngô Thị Minh Phương (23012156)	Kết quả trình bày trong báo cáo	Trạng thái
6	Đo lường và phân tích độ bao phủ mã nguồn JaCoCo 0.8.12	Thực hiện	-	Mục 5.2 (Bảng 5.2, 5.3, Hình 5.5 → 5.8), Phụ lục A	Hoàn thành
7	Thiết kế kịch bản kiểm thử hộp đen (EP, BVA, State Transition) và thực thi 7 ca kiểm thử giao diện WebUI (TC-054 → TC-060) qua Chrome CDP	-	Thực hiện	Mục 4.3, Bảng 4.3h, Mục 5.4 (Hình 5.11 → 5.23), Phụ lục C.3	Hoàn thành
8	Thực thi công cụ phân tích tĩnh Frontend (Oxlint 1.69.0) và kiểm tra đóng gói tối ưu bundle (Vite 8.1.0)	-	Thực hiện	Mục 5.3 (Hình 5.9, Hình 5.10), Phụ lục C.1, C.2	Hoàn thành
9	Lập hồ sơ kỹ thuật các lỗi Backend (BUG-001 → BUG-005, BUG-008 → BUG-010)	Thực hiện	-	Chương 6, Mục 6.1.1 → 6.1.5, 6.1.8 → 6.1.10	Hoàn thành
10	Lập hồ sơ kỹ thuật các lỗi Frontend và tích hợp (BUG-006, BUG-007, BUG-011, BUG-012)	-	Thực hiện	Chương 6, Mục 6.1.6, 6.1.7, 6.1.11, 6.1.12	Hoàn thành

CHƯƠNG 4: PHÂN TÍCH, THIẾT KẾ 60 TEST CASE VÀ MA TRẬN TRUY XUẤT

4.1. Phân tích mã nguồn tĩnh (SF-01 → SF-11)

Trước khi chạy kiểm thử động, nhóm đã rà soát tĩnh 89 tệp mã nguồn Java chính (và 3 tệp Java kiểm thử) của phân hệ Backend (com.reliefhub.*), các kịch bản SQL (reliefhub_full.sql, seed_data.sql) và 50 tệp JSX/JS của phân hệ Frontend (tầng giao diện Frontend). Qua kiểm tra, nhóm ghi nhận 11 vấn đề tiềm ẩn (SF-01 đến SF-11) và thiết kế các ca kiểm thử động tương ứng để xác minh như trong Bảng 4.1.

Bảng 4.1: Tổng hợp 11 phát hiện phân tích tĩnh (SF-01 → SF-11) trên mã nguồn Backend, SQL và Frontend

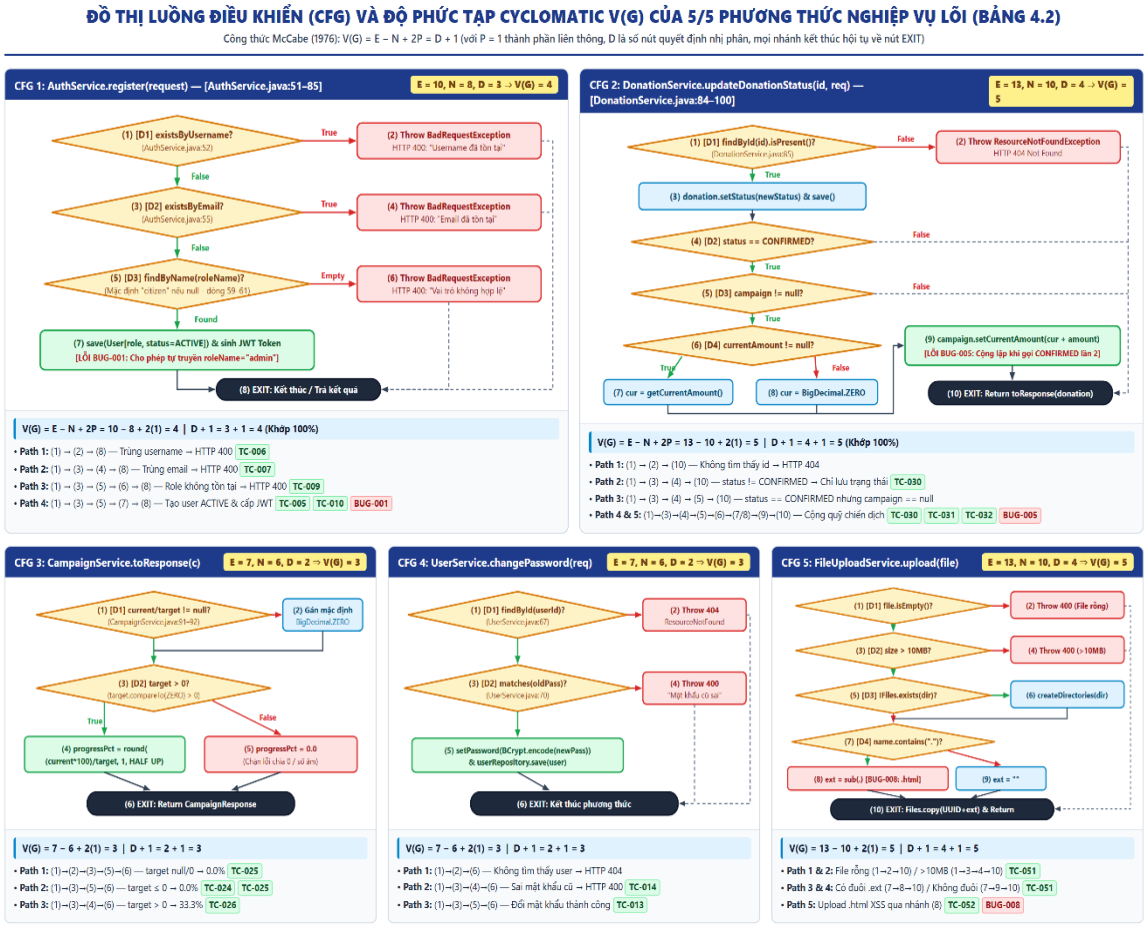
Mã tĩnh	Phân hệ	Vị trí tệp và dòng lệnh đã xác minh	Nội dung phát hiện qua rà soát mã nguồn tĩnh	Test Case thiết kế để kiểm chứng động
SF-01	Backend Auth	AuthService.java:37–81, AuthDTO.java:35	Hàm register() nhận trực tiếp request.getRoleName() mà không kiểm tra danh sách vai trò được phép đăng ký và luôn gán .status(UserStatus.ACTIVE) ở dòng 69, cho phép người dùng tự đăng ký quyền admin.	TC-010
SF-02	Backend Security	SecurityConfig.java:78, UserController.java:40–55	Quy tắc .requestMatchers("/users/**").hasRole("ADMIN") ở dòng 78 chặn cả 3 API cá nhân GET /users/me, PUT /users/me và PUT /users/me/password đối với các tài khoản không phải ADMIN.	TC-012
SF-03	Backend Security	SecurityConfig.java:32, 81 & 14/15 Controllers	Ngoại trừ UserController, 14 Controller còn lại không gắn @PreAuthorize; các đường dẫn /donations/**, /ledger/**, /support-requests/**, /inventory/**, /complaints/** chỉ áp dụng quy	TC-032, TC-034, TC-039, TC-043, TC-048

Mã tĩnh	Phân hệ	Vị trí tệp và dòng lệnh đã xác minh	Nội dung phát hiện qua rà soát mã nguồn tĩnh	Test Case thiết kế để kiểm chứng động
			tắc .anyRequest().authenticated() ở dòng 81.	
SF-04	Backend Service	SupportRequestService.java:83–98	Hàm advanceStatus() gán trực tiếp sr.setStatus(newStatus) ở dòng 91 mà không kiểm tra trạng thái hiện tại, cho phép chuyển thẳng từ PENDING sang DELIVERED.	TC-038
SF-05	Backend Service	DonationService.java:84–100	Hàm updateDonationStatus() cộng tiền vào campaign.currentAmount mỗi khi nhận status = CONFIRMED (dòng 92) mà không kiểm tra điều kiện trạng thái cũ oldStatus != CONFIRMED.	TC-031
SF-06	Frontend State	AppContext.jsx:557–568, 779–787, Request.jsx, Status.jsx	Không import api.js; gọi các biến API_ENABLED, supportRequestService, tokenManager chưa khai báo dẫn đến lỗi ReferenceError; mất dữ liệu trong state khi tải lại trang (F5).	TC-058, TC-060
SF-07	Frontend Auth	Login.jsx:38–64, AppContext.jsx:779–787	Hộp thoại "Quên mật khẩu?" gọi hàm resetPasswordByUsername() cho phép đổi mật khẩu của bất kỳ tài khoản nào (kể cả admin) chỉ bằng username mà không cần bước xác thực OTP.	TC-057
SF-08	Backend Upload	FileUploadService.java:25–51, WebConfig.java:15–21	Không kiểm tra danh sách phần mở rộng hợp lệ hay MIME type khi tải tệp lên /uploads, cho phép tải lên tệp .html chứa mã <script>.	TC-052
SF-09	Backend Service	NotificationService.java:38–43	Hàm markAsRead(Long id) không kiểm tra thông báo có thuộc về người dùng đang	TC-049

Mã tĩnh	Phân hệ	Vị trí tệp và dòng lệnh đã xác minh	Nội dung phát hiện qua rà soát mã nguồn tĩnh	Test Case thiết kế để kiểm chứng động
			đăng nhập hay không (lỗ hổng IDOR).	
SF-10	Backend DTO/Ctrl	CampaignDTO.java:36, SupportRequestDTO.java:32, ComplaintController.java:35, SupportRequestController.java:38	Thiếu @Positive cho targetAmount, quantity; thiếu @Valid trên reply() và advance(); không bắt ngoại lệ DateTimeParseException gây lỗi HTTP 500.	TC-024, TC-036, TC-048, TC-053
SF-11	Seed Data & UI	AppContext.jsx:24 vs. reliefhub_full.sql & HUONG_DAN_CHAY.md:120	Mật khẩu tài khoản finance01 trong AppContext.jsx là finance123, lệch với giá trị fin123 trong tệp SQL và tài liệu hướng dẫn; thông báo lỗi chính tả "Email đã tồn tạo" tại UserService.java:84.	TC-056

4.2. Phân tích hộp trắng: CFG và độ phức tạp Cyclomatic

Nhóm đã xây dựng đầy đủ **5/5 Đồ thị luồng điều khiển (Control Flow Graph - CFG)** cho cả 5 phương thức nghiệp vụ cốt lõi của Backend (AuthService.register(), DonationService.updateDonationStatus(),CampaignService.toResponse(), UserService.changePassword() và FileUploadService.upload()), kiểm chứng đồng thời theo cả hai công thức của McCabe [3], [5]: công thức hình học đồ thị V(G) = E - N + 2P (với P = 1) và công thức nút quyết định nhị phân V(G) = D + 1, trình bày trực quan tại **Hình 4.1** và tổng hợp tại **Bảng 4.2**.



Hình 4.1: Đồ thị luồng điều khiển (CFG) đầy đủ của 5 phương thức nghiệp vụ cốt lõi tại Backend (AuthService.register, DonationService.updateDonationStatus, CampaignService.toResponse, UserService.changePassword và FileUploadService.upload).

Bảng 4.2: Tổng hợp độ phức tạp Cyclomatic V(G) và các đường đi cơ sở (Basis Paths) của 5 phương thức cốt lõi

Phương thức phân tích	Đỉnh (N), Cung (E), Nút quyết định (D)	Độ phức tạp V(G) = E - N + 2P = D + 1	Các đường đi độc lập tuyến tính (Basis Paths — Hình 4.1)	Test Case bao phủ
AuthService.register() (AuthService.java:37–81)	N = 8 E = 10 D = 3	**V(G) = 10 - 8 + 2 = 4**	- Path 1 (N1→N2): Trùng username → BadRequestException (HTTP 400) - Path 2 (N1→N3→N4): Trùng email →	TC-005, TC-006, TC-007, TC-009, TC-010

Phương thức phân tích	Đỉnh (N), Cung (E), Nút quyết định (D)	Độ phức tạp V(G) = E - N + 2P = D + 1	Các đường đi độc lập tuyến tính (Basis Paths — Hình 4.1)	Test Case bao phủ
			BadRequestException (HTTP 400) - Path 3 (N1→N3→N5→N6): roleName không tồn tại → BadRequestException (HTTP 400) - Path 4 (N1→N3→N5→N7→N8): Tạo tài khoản với status = ACTIVE (bao gồm cả roleName = "admin" do lỗi BUG-001)	
DonationService.updateDonationStatus() (DonationService.java:84–100)	N = 10 E = 13 D = 4	**V(G) = 13 - 10 + 2 = 5**	- Path 1 (N1→N2→N3): Không tìm thấy id → ResourceNotFoundException (HTTP 404) - Path 2 (N1→N2→N4→N5→N10): newStatus != CONFIRMED → Chỉ lưu trạng thái - Path 3 (N1→N2→N4→N5→N6→N10): newStatus == CONFIRMED nhưng campaign == null - Path 4 (...→N6→N7→N8→N10): currentAmount != null → Cộng dồn current.add(amount) (gọi lần 2 vẫn cộng lặp do BUG-005) - Path 5 (...→N6→N7→N9→N10): currentAmount == null → Khởi tạo ZERO.add(amount)	TC-030, TC-031, TC-032

Phương thức phân tích	Đỉnh (N), Cung (E), Nút quyết định (D)	Độ phức tạp V(G) = E - N + 2P = D + 1	Các đường đi độc lập tuyến tính (Basis Paths — Hình 4.1)	Test Case bao phủ
CampaignService.toResponse() (CampaignService.java:81–102)	N = 6 E = 7 D = 2	**V(G) = 7 - 6 + 2 = 3**	- Path 1 (N1→N2→N4→N6): targetAmount == null → progressPct = 0.0 - Path 2 (N1→N2→N3→N4→N6): targetAmount != null && targetAmount <= 0 → progressPct = 0.0 - Path 3 (N1→N2→N3→N5→N6): targetAmount > 0 → progressPct = round((current * 100 / target), 1)	TC-024, TC-025, TC-026
UserService.changePassword() (UserService.java:67–76)	N = 6 E = 7 D = 2	**V(G) = 7 - 6 + 2 = 3**	- Path 1 (N1→N2): Không tìm thấy user → ResourceNotFoundException (HTTP 404) - Path 2 (N1→N3→N4): !passwordEncoder.matches(oldPassword) → BadRequestException (HTTP 400) - Path 3 (N1→N3→N5→N6): Khớp mật khẩu cũ → Băm BCrypt mật khẩu mới và lưu	TC-013, TC-014
FileUploadService.upload() (FileUploadService.java:25–51)	N = 10 E = 13 D = 4	**V(G) = 13 - 10 + 2 = 5**	- Path 1 (N1→N3): file == null → BadRequestException (HTTP 400) - Path 2 (N1→N2→N3): file != null && file.isEmpty() → BadRequestException (HTTP 400) - Path 3 (N1→N2→N4→N5): file.getSize() > 10MB →	TC-051, TC-052

Phương thức phân tích	Đỉnh (N), Cung (E), Nút quyết định (D)	Độ phức tạp V(G) = E - N + 2P = D + 1	Các đường đi độc lập tuyến tính (Basis Paths — Hình 4.1)	Test Case bao phủ
			BadRequestException (HTTP 400) - Path 4 (...→N4→N6→N7→N8→ N10): origName != null && origName.contains(".") → Lưu UUID kèm đuôi gốc (lọt .html do BUG-008) - Path 5 (...→N4→N6→N9→N10): Tên file không có dấu . → Lưu UUID không đuôi	

4.3. Thiết kế kiểm thử hộp đen (EP, BVA, bảng quyết định, chuyển trạng thái)

1. **Kỹ thuật Phân tích Giá trị biên (BVA):** Nhóm áp dụng phân tích giá trị biên cho 8 nhóm dữ liệu đầu vào:
    - LoginRequest: Chuỗi rỗng "" (TC-003).
    - RegisterRequest: username dài 2 ký tự ("ab" - dưới biên tối thiểu 3 ký tự), password dài 5 ký tự ("12345" - dưới biên tối thiểu 6 ký tự) (TC-008).
    - CreateCampaignRequest: targetAmount = -50,000,000 (biên âm), targetAmount = 0 (biên 0), targetAmount = 300,000,000 (hợp lệ) (TC-024, TC-025, TC-026).
    - CreateDonationRequest: amount = 0, amount = -1000 (biên không dương), amount = 5,000,000 (hợp lệ) (TC-027, TC-028).
    - CreateLedgerRequest: amount = -500 so với amount = 10,000,000 (TC-033).
    - CreateSupportRequestRequest: itemName = "", quantity = -20, householdCount = -1 (TC-036).
    - RestockRequest: quantity = 0, quantity = -50, quantity = 200 (TC-042).
    - MultipartFile Upload: 0 byte (biên rỗng), 1 KB (hợp lệ), 11 MB (vượt biên tối đa 10 MB) (TC-051).
2. **Kỹ thuật Bảng quyết định (Decision Table) cho Ma trận Phân quyền RBAC:** Nhóm kiểm tra chéo quyền hạn của các vai trò PUBLIC, CITIZEN, DONOR,

LOCAL, FINANCE, ADMIN trên 12 nhóm endpoint nhạy cảm (TC-010, TC-011, TC-012, TC-015, TC-016, TC-021, TC-023, TC-032, TC-034, TC-039, TC-043, TC-048, TC-049, TC-050, TC-054).

4.4. Đặc tả 60 trường hợp kiểm thử (TC-001 → TC-060)

4.4.1. Tổ chức bộ thực thi tự động

Toàn bộ 60 Test Case được chia thành hai bộ thực thi tự động:

- **Bộ kiểm thử tự động Backend (TC-001 → TC-053 = 53 Test Case):** Được cài đặt trong lớp ReliefHubFullEvaluationTest.java sử dụng SpringBootTest, MockMvc và H2 In-Memory. Mỗi test case gửi HTTP request thật qua SecurityFilterChain → Controller → Service → Repository và thu thập mã trạng thái HTTP cùng chuỗi JSON phản hồi thực tế (trình bày chi tiết tại Bảng 4.3a–g và Phụ lục B).
- **Bộ kiểm thử tự động Giao diện WebUI (TC-054 → TC-060 = 7 Test Case):** Thực thi trên trình duyệt Google Chrome điều khiển qua Chrome DevTools Protocol (CDP) kết nối tới máy chủ Vite (http://127.0.0.1:5173), tự động điền biểu mẫu, lắng nghe sự kiện Runtime.exceptionThrown trên Console và chụp ảnh màn hình .png (nhúng trực tiếp tại Hình 5.11 → Hình 5.23 ở Mục 5.4 và nhật ký tại Phụ lục C).

4.4.2. Đặc tả chi tiết toàn bộ 60 trường hợp kiểm thử

Dưới đây là toàn bộ 60 trường hợp kiểm thử được trình bày theo 9 cột (trong đó cột trạng thái chuẩn hóa nghiêm ngặt theo 4 nhãn PASS / FAIL / BLOCKED / NOT RUN [1]), kèm kết quả phản hồi thực tế và tham chiếu trực tiếp tới hình ảnh thực nghiệm, hồ sơ lỗi và nhật ký thực thi tại Phụ lục B và Phụ lục C:

4.4.2.1. Nhóm 1: Xác thực, Đăng ký & Bảo mật Token

Bảng 4.3a: Nhóm 1: Xác thực, Đăng ký & Bảo mật Token (TC-001 → TC-011)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-001	REQ-001	**Đăng nhập API hợp lệ:** Xác minh đăng nhập thành công và	EP **P0**	User testadmin (role=admin, status=ACTIVE)	1. Gửi request POST /auth/login với	{"username": "testadmin", "password	- HTTP 200 OK. - success = true, data.token (Bearer	**PASS** HTTP 200 OK, trả về data.token

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		nhận JWT token khi thông tin đúng		đã tồn tại trong DB với password test123 (BCrypt)	JSON body. 2. Kiểm tra HTTP status và cấu trúc ApiResponse.	": "test123"}	JWT) không rỗng, data.username = "testadmin", data.role = "admin".	(eyJhbGciOiJIUzUxMiJ9...), tokenType="Bearer", role="admin". Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-002	REQ-001	**Đăng nhập API sai mật khẩu:** Xác minh hệ thống từ chối khi nhập sai password	EP (Negative) **P0**	User testadmin tồn tại trong DB	1. Gửi POST /auth/login với password sai. 2. Kiểm tra mã lỗi trả về.	{"username": "testadmin", "password": "wrongpassword"}	- HTTP 401 Unauthorized. - success = false, error = "Tên đăng nhập hoặc mật khẩu không đúng".	**PASS** HTTP 401 Unauthorized, "Tên đăng nhập hoặc mật khẩu không đúng". Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-003	REQ-001, REQ-027	**Đăng nhập API bỏ trống dữ liệu:** Kiểm tra validation @NotBlank trên LoginRequest	BVA **P1**	Hệ thống Backend sẵn sàng	1. Gửi POST /auth/login với username = "" và password = "". 2. Kiểm tra phản hồi từ GlobalExceptionHandler.	{"username": "", "password": ""}	- HTTP 400 Bad Request. - success = false, error = "Dữ liệu không hợp lệ", chi tiết lỗi trường username và password không được để trống.	**PASS** HTTP 400 Bad Request, "Dữ liệu không hợp lệ" (username, password). Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-004	REQ-004	**Sinh và xác thực JWT Token:** Kiểm tra JwtTokenProvider tạo và giải mã đúng username	White-box Unit **P0**	Khởi tạo bean JwtTokenProvider	1. Gọi generateTokenFromUsername("testadmin"). 2. Gọi validateToken(token) và getUsernameFromToken(token).	username = "testadmin"	- validateToken(token) trả về true. - getUsernameFromToken(token) trả về "testadmin".	**PASS** valid=true, username="testadmin". Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-005	REQ-002	**Đăng ký tài khoản mới hợp lệ:** Xác minh nhánh thành công (CFG Path 1) của AuthService.register	EP / CFG **P0**	Role citizen đã có trong DB; newuser chưa tồn tại	1. Gửi POST /auth/register với thông tin hợp lệ. 2. Kiểm tra DB và response.	{"fullName": "New User", "username": "newuser", "email": "newuser@test.com", "password": "password123", "phone": "0901234567", "roleName": "citizen"}	- HTTP 200 OK. - success = true, data.username = "newuser", data.role = "citizen", data.token không rỗng.	**PASS** HTTP 200 OK, "Đăng ký thành công", role="citizen", cấp Bearer JWT. Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-006	REQ-002	**Đăng ký trùng Username:** Xác minh nhánh existsByUsername == true (CFG Path 2)	EP / CFG **P1**	User testadmin đã tồn tại	1. Gửi POST /auth/register với username = "testadmin".	{"fullName": "Dup User", "username": "testadmin", "email": "dup@test.com", "password": "password123", "roleName": "citizen"}	- HTTP 400 Bad Request. - success = false, error = "Username đã tồn tại".	**PASS** HTTP 400 Bad Request, "Username đã tồn tại". Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-007	REQ-002	**Đăng ký trùng Email:** Xác	EP / CFG **P1**	Email admin@t	1. Gửi POST /auth/r	{"fullName": "Dup Email",	- HTTP 400 Bad Request.	**PASS** HTTP 400 Bad

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		minh nhánh existsByEmail == true (CFG Path 3)		est.com đã tồn tại	egister với username mới nhưng email = "admin@test.com".	"username": "unique_tc07", "email": "admin@test.com", "password": "password123", "roleName": "citizen"}	- success = false, error = "Email đã tồn tại".	Request, "Email đã tồn tại". Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-008	REQ-002	**Kiểm tra biên độ dài Username, Password & định dạng Email khi đăng ký**	BVA **P1**	Hệ thống Backend sẵn sàng	1. Gửi POST /auth/register với username 2 ký tự ("ab"), password 5 ký tự ("12345"), email sai ("invalid-email").	{"fullName": "Test", "username": "ab", "email": "invalid-email", "password": "12345"}	- HTTP 400 Bad Request. - Báo lỗi validation cho cả 3 trường username (3–50 ký tự), password (≥ 6 ký tự), email.	**PASS** HTTP 400 Bad Request, bắt lỗi cả 3 trường username, password, email. Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-009	REQ-002	**Đăng ký với Role không tồn tại:** Xác minh nhánh roleReposit	EP / CFG **P2**	Role "invalid_role" không có trong	1. Gửi POST /auth/register với "roleN	{"fullName": "User", "username": "user_role	- HTTP 400 Bad Request. - error = "Vai trò không	**PASS** HTTP 400 Bad Request, "Vai trò không

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		ory.findByName rỗng (CFG Path 4)		bảng roles	ame": "invalid_role".	_err", "email": "role@test.com", "password": "password123", "roleName": "invalid_role"}	hợp lệ: invalid_role".	hợp lệ: invalid_role". Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-010	REQ-003	**[Bảo mật - SF-01] Chặn tự đăng ký quyền Admin:** Kiểm tra người dùng tự truyền roleName = "admin" khi đăng ký công khai	Security / Decision Table **P0**	Role admin tồn tại trong bảng roles	1. Khách vãng lai gửi POST /auth/register với "roleName": "admin". 2. Kiểm tra xem hệ thống có chặn hoặc ép về citizen hay không.	{"fullName": "Hacker Admin", "username": "hacker_admin", "email": "hacker@test.com", "password": "password123", "roleName": "admin"}	- **Kỳ vọng bảo mật:** Từ chối HTTP 400/403 hoặc chỉ gán quyền citizen, không cấp token có role = "admin".	**FAIL (BUG-001 / SF-01)** Hệ thống trả về HTTP 200 OK và cấp JWT token có role = "admin", status = "ACTIVE". Bằng chứng: Hình 5.2, Hình 5.3, Chương 6 (BUG-001) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-011	REQ-004	**Truy cập API bảo vệ khi không có JWT Token:** Kiểm tra AuthenticationEntryPoint	Security / Decision Table **P0**	Không đính kèm header Authorization	1. Gửi GET /users không có token. 2. Kiểm tra HTTP status.	Header: *(Trống)*	- HTTP 401 Unauthorized. - error = "Unauthorized - Vui lòng đăng nhập".	**PASS** HTTP 401 Unauthorized. Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

4.4.2.2. Nhóm 2: Quản lý Người dùng & Hồ sơ cá nhân

Bảng 4.3b: Nhóm 2: Quản lý Người dùng & Hồ sơ cá nhân (TC-012 → TC-018)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-012	REQ-005	**[Phân quyền - SF-02] Người dùng thông thường (donor/citizen) xem và sửa hồ sơ cá nhân (/users/me)**	Decision Table **P0**	Đã đăng nhập tài khoản testdonor (role=donor) và có Bearer Token hợp lệ	1. Gửi GET /users/me với donor Token. 2. Gửi PUT /users/me cập nhật fullN	Header: Bearer <donor Token> Body PUT: {"fullName": "Donor Updated", "phone": "0911222333"}	- **Kỳ vọng đặc tả:** HTTP 200 OK, trả về thông tin cá nhân đã cập nhật của testdonor.	**FAIL (BUG-002 / SF-02)** Cả GET /users/me và PUT /users/me đều trả về HTTP 403 Forbidden do SecurityConfig.java:78 chặn /users/**

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
					ame và phone .			chỉ cho ADMIN. Bằng chứng: Hình 5.2, Hình 5.3, Chương 6 (BUG-002) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-013	REQ-006	**Đổi mật khẩu cá nhân thành công:** Kiểm tra nhánh khớp mật khẩu cũ trong UserService.changePassword	White-box CFG **P1**	Tài khoản đang đăng nhập có mật khẩu cũ là test123	1. Gửi PUT /users/me/password với oldPassword đúng và newPassword hợp lệ (≥ 6 ký tự).	{"oldPassword": "test123", "newPassword": "newpass456"}	- HTTP 200 OK. - message = "Đổi mật khẩu thành công", mật khẩu mới được băm BCrypt trong DB.	**PASS** HTTP 200 OK, "Đổi mật khẩu thành công" *(Ghi chú điều kiện: Chạy với adminToken đạt PASS; các vai trò khác ADMIN bị chặn HTTP 403 bởi BUG-002).* Bằng chứng: Hình 5.3 & Nhật ký thực

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
								thi tại Phụ lục B (Bảng B.1)
TC-014	REQ-006	**Đổi mật khẩu thất bại khi nhập sai mật khẩu cũ:** Kiểm tra nhánh !passwordEncoder.matches	White-box CFG (Negative) **P1**	Tài khoản đang đăng nhập có mật khẩu cũ là test123	1. Gửi PUT /users/me/password với oldPassword sai.	{"oldPassword": "wrong_old", "newPassword": "newpass456"}	- HTTP 400 Bad Request. - error = "Mật khẩu cũ không đúng".	**PASS** HTTP 400 Bad Request, "Mật khẩu cũ không đúng". Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-015	REQ-007	**Admin lấy danh sách user và tạo tài khoản mới thành công**	Decision Table **P1**	Đăng nhập với adminToken (role=admin)	1. Gửi GET /users. 2. Gửi POST /users tạo tài khoản finance.	{"fullName": "Finance User", "username": "fin_new", "email": "fin@test.com", "password": "password123",	- Cả 2 request trả về HTTP 200 OK. - Tài khoản fin_new được tạo với role = "finance", status = "ACTIVE".	**PASS** GET /users = HTTP 200, POST /users = HTTP 200 (role="finance", status="ACTIVE"). Bằng chứng: Hình 5.3 & Nhật ký thực thi tại

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
						"roleName": "finance"}		Phụ lục B (Bảng B.1)
TC-016	REQ-007	**Chặn người dùng không phải Admin truy cập /users:** Kiểm tra RBAC	Decision Table **P1**	Đăng nhập với donorToken (role=donor)	1. Gửi GET /users và POST /users với donorToken.	Header: Bearer <donorToken>	- HTTP 403 Forbidden. - success = false.	**PASS** GET /users = HTTP 403, POST /users = HTTP 403. Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-017	REQ-007	**Admin khóa và mở khóa tài khoản (toggle-status):** Kiểm tra chuyển trạng thái ACTIVE ↔ INACTIVE	State Transition / CFG **P1**	User mục tiêu đang ở trạng thái ACTIVE; dùng adminToken	1. Gọi PATCH /users/{id}/toggle-status lần 1. 2. Kiểm tra trạng thái user. 3. Gọi	id = <targetUserId>	- Lần 1: HTTP 200 OK, status chuyển thành "INACTIVE". - Lần 2: HTTP 200 OK, status quay lại "ACTIVE".	**PASS** Lần 1 chuyển sang "INACTIVE" (HTTP 200 OK), lần 2 quay lại "ACTIVE" (HTTP 200 OK). Bằng chứng: Hình 5.3 & Nhật

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
					lại lần 2 và kiểm tra.			ký thực thi tại Phụ lục B (Bảng B.1)
TC-018	REQ-007, REQ-027	**Admin truy vấn hoặc xóa User không tồn tại:** Kiểm tra ResourceNotFoundException	BVA / Negative **P2**	id = 99999 không tồn tại trong bảng users	1. Gửi GET /users/99999 và DELETE /users/99999 với adminToken.	id = 99999	- HTTP 404 Not Found. - error = "User không tìm thấy với id = '99999'".	**PASS** GET /users/99999 = HTTP 404, DELETE /users/99999 = HTTP 404 ("User không tìm thấy với id = '99999'"). Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

4.4.2.3. Nhóm 3: Quản lý Thiên tai & Chiến dịch cứu trợ

Bảng 4.3c: Nhóm 3: Quản lý Thiên tai & Chiến dịch cứu trợ (TC-019 → TC-026)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-019	REQ-008	**Khách vãng lai xem danh sách Chiến dịch & Thiên tai công khai**	EP **P0**	Không cần đăng nhập	1. Gửi GET /campaigns, GET /campaigns/active. 2. Gửi GET /disasters, GET /disasters/active.	Không có Header Authorization	- Tất cả trả về HTTP 200 OK, success = true, danh sách dữ liệu đúng trạng thái.	**PASS** Cả 4 endpoint đều trả về HTTP 200 OK. Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-020	REQ-008, REQ-027	**Xem chi tiết Chiến dịch / Thiên tai không tồn tại**	Negative **P2**	id = 99999 không tồn tại	1. Gửi GET /campaigns/99999 và GET /disasters/99999.	id = 99999	- HTTP 404 Not Found. - success = false.	**PASS** GET /campaigns/99999 = HTTP 404, GET /disasters/99999 = HTTP 404. Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-021	REQ-009	**Phân quyền tạo/sửa/xóa Thiên tai**	Decision Table **P1**	Có localToke	1. Dùng localT	{"name": "Lũ quét Hà	- Với localToken: HTTP 200	**PASS** localToken = HTTP

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		(ADMIN & LOCAL được phép, DONOR bị chặn)		n (role =local) và donorToken (role =donor)	oken gửi POST /disasters. 2. Dùng donorToken gửi POST /disasters.	Giang", "type": "Lũ quét", "area": "Hà Giang", "severity": "HIGH"}	OK, tạo thiên tai thành công. - Với donorToken: HTTP 403 Forbidden.	200 OK, donorToken = HTTP 403 Forbidden. Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-022	REQ-010	**Admin tạo Chiến dịch cứu trợ mới hợp lệ**	Decision Table **P0**	Có adminToken (role =admin)	1. Gửi POST /campaigns với adminToken và dữ liệu hợp lệ.	{"title": "Test Campaign", "description": "Desc", "targetAmount": 100000000, "status": "ACTIVE", "area": "Hà Nội"}	- HTTP 200 OK. - data.title = "Test Campaign", data.currentAmount = 0, data.progressPct = 0.0.	**PASS** HTTP 200 OK, id=2, progressPct =0.0. Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-023	REQ-010	**Chặn người dùng không phải Admin (DONOR) tạo/sửa/xóa Chiến dịch**	Decision Table **P0**	Có donorToken (role =donor)	1. Gửi POST /campaigns với donorToken.	{"title": "Test", "targetAmount": 100}	- HTTP 403 Forbidden.	**PASS** HTTP 403 Forbidden. Bằng chứng: Hình 5.3 & Nhật ký thực thi tại

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
								Phụ lục B (Bảng B.1)
TC-024	REQ-010	**[Biên & Ngoại lệ - SF-10] Tạo chiến dịch với targetAmount <= 0 hoặc ngày tháng sai định dạng**	BVA / Negative **P1**	Có adminToken	1. Gửi POST /campaigns với targetAmount = -50000000. 2. Gửi POST /campaigns với startDate = "invalid-date" (sai chuẩn ISO).	1) {"title": "Camp âm", "targetAmount": -50000000} 2) {"title": "Camp lỗi ngày", "targetAmount": 1000000, "startDate": "invalid-date"}	- **Kỳ vọng chất lượng:** Hệ thống từ chối với HTTP 400 Bad Request cho cả hai trường hợp (không nhận targetAmount <= 0 và không trả về lỗi 500 khi sai định dạng ngày).	**FAIL (BUG-010 / SF-10)** 1) targetAmount = -50000000 được lưu thành công với HTTP 200 OK (progressPct = 0.0). 2) startDate = "invalid-date" gây lỗi HTTP 500 Internal Server Error (DateTimeParseException). Bằng chứng: Hình 5.2, Hình 5.3, Chương 6 (BUG-010) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-025	REQ-011	**Tính toán tiến độ progressPct khi targetAmount > 0 (Làm tròn**	White-box CFG / BVA **P1**	Chiến dịch có targe	1. Gọi GET /campaigns/{id}.	target = 300000000, current =	- HTTP 200 OK. - progressPct = 33.3	**PASS** HTTP 200 OK, progressPct = 33.3.

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		1 chữ số thập phân)		tAmount = 300,000,000, currentAmount = 100,000,000	2. Kiểm tra giá trị progressPct.	100000000	((100 × 100) / 300 = 33.333... → 33.3).	Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-026	REQ-011	**Tính toán tiến độ progressPct tại biên targetAmount = 0 (Chống lỗi chia cho 0)**	White-box CFG / BVA **P1**	Chiến dịch có targetAmount = 0, currentAmount = 0	1. Gọi GET /campaigns/{id}. 2. Kiểm tra nhánh target.compareTo(BigDecimal.ZERO) > 0 trả về false.	target = 0, current = 0	- Không xảy ra Arithmetic Exception. - progressPct = 0.0.	**PASS** HTTP 200 OK, không bị lỗi chia cho 0, progressPct = 0.0. Bằng chứng: Hình 5.3 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

4.4.2.4. Nhóm 4: Quyên góp & Sổ cái tài chính

Bảng 4.3d: Nhóm 4: Quyên góp & Sổ cái tài chính (TC-027 → TC-034)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-027	REQ-012	**Tạo khoản quyên góp hợp lệ kèm danh sách hiện vật (items)**	EP **P0**	Chiến dịch id=1 tồn tại; đăng nhập với donor Token	1. Gửi POST /donations kèm số tiền và danh sách hiện vật. 2. Kiểm tra trạng thái khởi tạo.	{"campaignId": 1, "amount": 5000000, "message": "Ủng hộ", "isAnonymous": false, "items": [{"itemName": "Gạo", "quantity": 100, "unit": "kg"}]}	- HTTP 200 OK. - data.status = "PENDING", data.amount = 5000000, data.items có 1 phần tử "Gạo".	**PASS** HTTP 200 OK, status="PENDING", items=[{itemName:"Gạo", quantity:100, unit:"kg"}]. Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-028	REQ-012	**Tạo khoản quyên góp với số tiền biên amount = 0 hoặc amount < 0**	BVA **P1**	Chiến dịch id=1 tồn tại; đăng nhập với donor Token	1. Gửi POST /donations với amount = 0 và amount = -1000.	{"campaignId": 1, "amount": 0}	- HTTP 400 Bad Request (do vi phạm @Positive).	**PASS** amount=0 trả về HTTP 400, amount=-1000 trả về HTTP 400. Bằng chứng: Hình 5.4 & Nhật

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
								ký thực thi tại Phụ lục B (Bảng B.1)
TC-029	REQ-014	**Kiểm tra hiển thị tên nhà tài trợ khi quyên góp ẩn danh (isAnonymous = true)**	EP / White-box CFG **P1**	Tạo khoản quyên góp với isAnonymous = true	1. Gửi POST /donations với isAnonymous = true. 2. Kiểm tra trường donorName trong phản hồi.	{"campaignId": 1, "amount": 1000000, "isAnonymous": true}	- Trường donorName trong phản hồi hiển thị là "Ẩn danh", không lộ họ tên thật của người quyên góp.	**PASS** HTTP 200 OK, donorName = "Ẩn danh". Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-030	REQ-013	**Xác minh quyên góp (CONFIRMED) và cộng tiền vào quỹ chiến dịch**	White-box CFG **P0**	Chiến dịch có currentAmount = 0; Donation id=1 có amount = 5,000,000, status = PENDING	1. Dùng financeToken gửi PATCH /donations/1/status với {"status": "CONFIRMED"}. 2. Kiểm tra currentAmount của chiến dịch.	{"status": "CONFIRMED"}	- HTTP 200 OK, donation.status = "CONFIRMED". - campaign.currentAmount tăng lên đúng 5,000,000.	**PASS** HTTP 200 OK, campaign.currentAmount = 5000000.00. Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-031	REQ-013	**[Logic tài chính - SF-05] Kiểm tra tính bất biến (Idempotency) khi gọi CONFIRMED lặp lại lần 2 cho cùng 1 khoản quyên góp**	State Transition / White-box **P0**	Donation id=1 (amount = 5,000,000) đã ở trạng thái CONFIRMED; campaign.currentAmount = 5,000,000	1. Gửi tiếp PATCH /donations/1/status với {"status": "CONFIRMED"} lần thứ 2. 2. Kiểm tra campaign.currentAmount.	{"status": "CONFIRMED"} (Gọi lần 2 trên cùng Donation ID)	- **Kỳ vọng đúng đắn:** Không cộng tiền lần 2 (campaign.currentAmount phải giữ nguyên 5,000,000) hoặc từ chối HTTP 400.	**FAIL (BUG-005 / SF-05)** Hệ thống trả về HTTP 200 OK và cộng dồn lần 2 làm campaign.currentAmount tăng lên 10000000.00. Bằng chứng: Hình 5.2, Hình 5.4, Chương 6 (BUG-005) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-032	REQ-013	**[Bảo mật - SF-03] Chặn DONOR/CITIZEN tự gọi API duyệt quyên góp (PATCH /donations/{id}/status)**	Security / Decision Table **P0**	Khoản quyên góp đang ở PENDING; dùng donor Token	1. Gửi PATCH /donations/{id}/status với {"status": "CONFIRMED"} bằng	Header: Bearer <donor Token> Body: {"status": "CONFIRMED"}	- **Kỳ vọng bảo mật:** HTTP 403 Forbidden (chỉ FINANCE hoặc ADMIN được duyệt).	**FAIL (BUG-003 / SF-03)** DONOR tự duyệt khoản quyên góp của chính mình

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
					donorToken.			thành công với HTTP 200 OK (status="CONFIRMED"). Bằng chứng: Hình 5.2, Hình 5.4, Chương 6 (BUG-003) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-033	REQ-015	**Tạo bút toán sổ cái tài chính (Ledger) hợp lệ và kiểm tra biên amount <= 0**	EP & BVA **P1**	Có financeToken (role=finance)	1. Gửi POST /ledger với type = "INCOME", amount = 10000000. 2. Gửi POST /ledger với amount = -500.	1) {"type": "INCOME", "amount": 10000000, "description": "Thu"} 2) {"type": "EXPENSE", "amount": -500}	- Request 1: HTTP 200 OK. - Request 2: HTTP 400 Bad Request (do @Positive).	**PASS** Request 1 = HTTP 200 OK, Request 2 = HTTP 400 Bad Request. Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-034	REQ-015	**[Bảo mật - SF-03] Chặn CITIZEN/DONOR tạo hoặc xóa bút toán sổ cái tài chính (/ledger)**	Security / Decision Table **P1**	Dùng citizenToken (role=citizen)	1. Gửi POST /ledger và DELETE /ledger/{id} bằng citizenToken.	Header: Bearer <citizenToken>	- **Kỳ vọng bảo mật:** HTTP 403 Forbidden.	**FAIL (BUG-003 / SF-03)** CITIZEN tạo bút toán sổ cái (POST /ledger = HTTP 200) và xóa bút toán (DELETE /ledger/{id} = HTTP 200) thành công. Bằng chứng: Hình 5.2, Hình 5.4, Chương 6 (BUG-003) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

4.4.2.5. Nhóm 5: Yêu cầu hỗ trợ 6 bước

Bảng 4.3e: Nhóm 5: Yêu cầu hỗ trợ 6 bước (TC-035 → TC-040)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-035	REQ-016	**Người dân (CITIZEN) gửi yêu cầu hỗ trợ hợp lệ**	EP **P0**	Đăng nhập với citizenToken	1. Gửi POST /support-requests với dữ liệu hợp lệ. 2. Kiểm tra mã requestCode và status.	{"itemName": "Gạo, Nước uống", "quantity": 50, "unit": "kg", "area": "Quảng Bình", "urgency": "HIGH", "householdCount": 4}	- HTTP 200 OK. - data.status = "PENDING", data.requestCode bắt đầu bằng "REQ-".	**PASS** HTTP 200 OK, status="PENDING", requestCode="REQ-1792591164350". Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-036	REQ-016	**[Biên & Validation - SF-10] Gửi yêu cầu hỗ trợ với itemName rỗng hoặc quantity <= 0**	BVA / Negative **P1**	Đăng nhập với citizenToken	1. Gửi POST /support-requests với itemName = "". 2. Gửi POST /support-requests với quantity = -20, householdCount = -1.	1) {"itemName": "", "area": ""} 2) {"itemName": "Gạo", "area": "Đà Nẵng", "quantity": -20, "householdCo	- Request 1: HTTP 400 Bad Request. - Request 2: **Kỳ vọng:** HTTP 400 Bad Request (không chấp nhận số lượng âm).	**FAIL (BUG-010 / SF-10)** Request 1 = HTTP 400, nhưng Request 2 (quantity = -20, householdCount = -1) trả về HTTP 200 OK và lưu số âm vào CSDL. Bằng chứng: Hình 5.2,

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
						unt": -1}		Hình 5.4, Chương 6 (BUG-010) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-037	REQ-017	**Chuyển trạng thái yêu cầu hỗ trợ tuần tự qua đúng 6 bước hợp lệ**	State Transition **P0**	Yêu cầu hỗ trợ vừa tạo ở trạng thái PENDING	1. Lần lượt gọi PATCH /support-requests/{id}/advance qua các trạng thái: VERIFIED → APPROVED → WAREHOUSE_READY → IN_TRANSIT → DELIVERED.	{"newStatus": "..."} theo từng bước	- Mỗi bước trả về HTTP 200 OK và cập nhật đúng trạng thái mới.	**PASS** Tất cả 5 bước chuyển tuần tự đều trả về HTTP 200 OK (VERIFIED → APPROVED → WAREHOUSE_READY → IN_TRANSIT → DELIVERED). Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-038	REQ-017	**[Chuyển trạng thái - SF-04] Kiểm tra chặn nhảy cóc trạng thái từ PENDING thẳng**	State Transition (Negative) **P0**	Yêu cầu hỗ trợ đang ở PENDING	1. Gửi PATCH /support-requests/{id}/advance với {"newStatus": "DELIVERED"}	{"newStatus": "DELIVERED"} từ PENDING	- **Kỳ vọng đặc tả quy trình 6 bước:** Hệ thống phải từ chối bước chuyển nhảy cóc (HTTP	**FAIL (BUG-004 / SF-04)** Hệ thống trả về HTTP 200 OK, cho phép chuyển thẳng từ PENDING sang

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		sang DELIVERED			ngay khi đang PENDING.		400 Bad Request).	DELIVERED, bỏ qua 4 trạng thái trung gian. Bằng chứng: Hình 5.2, Hình 5.4, Chương 6 (BUG-004) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-039	REQ-017	**[Bảo mật - SF-03] Chặn CITIZEN tự phê duyệt yêu cầu hỗ trợ của chính mình (PATCH /support-requests/{id}/advance)**	Security / Decision Table **P0**	Yêu cầu hỗ trợ đang ở PENDING; dùng citizenToken	1. Người dân gửi PATCH /support-requests/{id}/advance với {"newStatus": "APPROVED"}.	Header: Bearer <citizenToken> Body: {"newStatus": "APPROVED"}	- **Kỳ vọng bảo mật:** HTTP 403 Forbidden.	**FAIL (BUG-003 / SF-03)** CITIZEN tự phê duyệt yêu cầu cứu trợ sang "APPROVED" thành công với HTTP 200 OK. Bằng chứng: Hình 5.2, Hình 5.4, Chương 6 (BUG-003) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-040	REQ-018	**Tra cứu yêu cầu hỗ trợ của cá nhân (/me) và**	EP **P2**	Có ít nhất 1 yêu cầu của citize	1. Gửi GET /support-requests/me với citizenT	status = "DELIVERED"	- HTTP 200 OK, trả về đúng danh sách yêu cầu của	**PASS** Cả /me và /status/DELIVERED đều trả về HTTP 200 OK.

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		lọc theo trạng thái (/status/{status})		n trong DB	oken. 2. Gửi GET /support-requests/status/DELIVERED.		công dân và đúng trạng thái lọc.	Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

4.4.2.6. Nhóm 6: Kho hàng, Vận chuyển & Tình nguyện viên

Bảng 4.3f: Nhóm 6: Kho hàng, Vận chuyển & Tình nguyện viên (TC-041 → TC-046)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-041	REQ-019	**Tạo Nhà kho (Warehouse) và thêm Mặt hàng tồn kho (Inventory) liên kết với kho**	EP **P1**	Đăng nhập với admin Token	1. Gửi POST /warehouses tạo "Kho Miền Trung". 2. Gửi POST /inventory với warehouseId vừa tạo.	Warehouse: {"name": "Kho Miền Trung", "province": "Quảng Bình"} Inventory: {"itemName": "Gạo ST25", "quantity": 500, "unit": "kg",	- Cả 2 trả về HTTP 200 OK. - InventoryResponse.warehouseName trả về đúng "Kho Miền Trung".	**PASS** HTTP 200 OK, tạo kho id=1 và mặt hàng tồn kho id=1 (warehouseName= "Kho Miền Trung"). Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
						"warehouseId": 1}		(Bảng B.1)
TC-042	REQ-020	**Nhập thêm hàng vào kho (restock) với số lượng hợp lệ (>0) và biên không hợp lệ (<=0)**	BVA **P1**	Mặt hàng kho có quantity = 500	1. Gửi PATCH /inventory/{id}/restock với {"quantity": 200}. 2. Gửi tiếp với {"quantity": 0} và {"quantity": -50}.	1) {"quantity": 200} 2) {"quantity": 0} / {"quantity": -50}	- Bước 1: HTTP 200 OK, quantity tăng lên 700. - Bước 2: HTTP 400 Bad Request (do @Positive).	**PASS** Bước 1 = HTTP 200 (quantity =700); Bước 2 (0 và -50) = HTTP 400 Bad Request. Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-043	REQ-019	**[Bảo mật - SF-03] Chặn người dùng không có thẩm quyền (CITIZEN/DONOR) xóa mặt hàng tồn kho (DELETE**	Security / Decision Table **P1**	Mặt hàng kho đang tồn tại; dùng citizen Token	1. Gửi DELETE /inventory/{id} bằng citizenToken.	Header: Bearer <citizen Token>	- **Kỳ vọng bảo mật:** HTTP 403 Forbidden.	**FAIL (BUG-003 / SF-03)** CITIZEN xóa mặt hàng tồn kho thành công với HTTP 200 OK ("Đã xóa mặt hàng").

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		/inventory /{id})						Bằng chứng: Hình 5.2, Hình 5.4, Chương 6 (BUG-003) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-044	REQ-021	**Tạo chuyến giao hàng (Delivery) và cập nhật trạng thái DELIVERED tự động ghi nhận arriveAt**	State Transition / CFG **P1**	Đăng nhập hợp lệ	1. Gửi POST /deliveries tạo chuyến giao hàng. 2. Gửi PATCH /deliveries/{id}/status với {"status": "DELIVERED"}.	{"vehicleType": "Xe tải 2T", "origin": "Kho HN", "destination": "Quảng Bình"} → {"status": "DELIVERED"}	- Khi mới tạo: status = "SCHEDULED", arriveAt = null. - Sau khi DELIVERED: status = "DELIVERED", arriveAt != null.	**PASS** HTTP 200 OK, status="DELIVERED", arriveAt được tự động ghi nhận thời gian thực. Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-045	REQ-022	**Tạo, cập nhật và lọc nhóm tình**	EP **P2**	Đăng nhập hợp lệ	1. Gửi POST /volunteer-	{"name": "Đội Cứu hộ 1",	- HTTP 200 OK, nhóm	**PASS** POST = HTTP 200, GET

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		nguyện viên đang hoạt động (/volunteer-groups)			groups tạo nhóm mới. 2. Gửi GET /volunteer-groups/active.	"area": "Đà Nẵng", "status": "ACTIVE"}	mới xuất hiện trong danh sách active.	/active = HTTP 200. Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-046	REQ-019, REQ-021	**Truy vấn / Xóa mặt hàng tồn kho hoặc chuyến giao hàng không tồn tại**	Negative **P2**	id = 99999 không tồn tại	1. Gửi GET /inventory/99999 và DELETE /deliveries/99999.	id = 99999	- HTTP 404 Not Found.	**PASS** Cả 2 request đều trả về HTTP 404 Not Found. Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

4.4.2.7. Nhóm 7: Khiếu nại, Thông báo, Dashboard & Upload

Bảng 4.3g: Nhóm 7: Khiếu nại, Thông báo, Dashboard & Upload (TC-047 → TC-053)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-047	REQ-023	**Vòng đời xử lý Khiếu nại (PENDING → REPLIED → CLOSED)**	State Transition **P1**	Có donor Token và admin Token	1. Donor gửi POST /complaints. 2. Admin gửi PUT /complaints/{id}/reply. 3. Admin gửi PATCH /complaints/{id}/close.	Create: {"type": "Khiếu nại", "subject": "Chậm xác minh", "content": "Chi tiết..."} Reply: {"adminReply": "Đã xử lý"}	- Bước 1: status = "PENDING", progress = 0. - Bước 2: status = "REPLIED", progress = 100, repliedAt != null. - Bước 3: status = "CLOSED".	**PASS** Cả 3 bước chuyển trạng thái đều trả về HTTP 200 OK đúng đặc tả. Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-048	REQ-023	**[Bảo mật & Validation - SF-03 & SF-10] Chặn non-admin tự trả lời khiếu nại và kiểm tra @NotBlank trên adminReply**	Security / BVA **P1**	Khiếu nại đang ở PENDING; dùng citizenToken	1. Dùng citizenToken gửi PUT /complaints/{id}/reply. 2. Dùng adminToken gửi PUT /complaints/{id}/reply với {"adminReply": ""}.	1) Header citizen Token 2) Body {"adminReply": ""}	- Bước 1: **Kỳ vọng:** HTTP 403 Forbidden. - Bước 2: **Kỳ vọng:** HTTP 400 Bad Request.	**FAIL (BUG-003 & BUG-010)** 1) CITIZEN trả lời khiếu nại trả về HTTP 200 OK. 2) adminReply = "" cũng trả về HTTP 200 OK do thiếu

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
								@Valid. Bằng chứng: Hình 5.2, Hình 5.4, Chương 6 (BUG-003, 010) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-049	REQ-024	**[Bảo mật IDOR - SF-09] Kiểm tra ngăn chặn người dùng đánh dấu đã đọc thông báo của người khác**	Security (IDOR) **P1**	Thông báo id=1 thuộc về admin; đang đăng nhập bằng donor Token	1. Dùng donorToken gọi PATCH /notifications/1/read. 2. Kiểm tra trạng thái thông báo của admin.	PATCH /notifications/1/read (bằng donorToken)	- **Kỳ vọng bảo mật:** Từ chối HTTP 403/404, không cho phép sửa thông báo của user khác.	**FAIL (BUG-009 / SF-09)** DONOR đánh dấu đã đọc thông báo của ADMIN thành công với HTTP 200 OK (Lỗ hổng IDOR). Bằng chứng: Hình 5.2, Hình 5.4, Chương 6 (BUG-009) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-050	REQ-025	**Phân quyền và tính đúng đắn của Thống kê Dashboard (GET /dashboard/stats)**	Decision Table **P1**	Có adminToken, financeToken và citizenToken	1. Gọi GET /dashboard/stats bằng adminToken và financeToken. 2. Gọi bằng citizenToken.	Header: Từng loại token	- Với ADMIN & FINANCE: HTTP 200 OK, trả về đúng số lượng user, campaign và tổng tiền. - Với CITIZEN: HTTP 403 Forbidden.	**PASS** ADMIN = HTTP 200, FINANCE = HTTP 200, CITIZEN = HTTP 403 Forbidden. Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-051	REQ-026	**Kiểm tra biên kích thước file tải lên (POST /uploads): File rỗng (0B), hợp lệ (1KB) và quá khổ (11MB)**	White-box CFG / BVA **P1**	Đăng nhập hợp lệ	1. Upload file 0 byte. 2. Upload file ảnh .png 1 KB. 3. Upload file 11 MB (> ngưỡng 10 MB).	Mock MultipartFile với kích thước 0B, 1KB, 11MB	- 0B: HTTP 400 ("File không được để trống"). - 1KB: HTTP 200 OK, trả về url = "/uploads/...". - 11MB: HTTP 400 ("Kích thước file không được vượt quá 10MB").	**PASS** 0B = HTTP 400, 1KB = HTTP 200, 11MB = HTTP 400. Bằng chứng: Hình 5.4 & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-052	REQ-026	**[Bảo mật Upload - SF-08] Kiểm tra chặn tải lên file thực thi / script độc hại (.html, .jsp, .exe)**	Security **P1**	Đăng nhập hợp lệ	1. Gửi POST /uploads với file malicious.html (text/html) chứa mã <script>.	File malicious.html (39 bytes, text/html)	- **Kỳ vọng bảo mật:** Hệ thống chỉ cho phép ảnh/tài liệu an toàn và từ chối HTTP 400 Bad Request.	**FAIL (BUG-008 / SF-08)** Hệ thống chấp nhận file malicious.html với HTTP 200 OK và xuất bản tại /uploads/{uuid}.html (Stored XSS). Bằng chứng: Hình 5.2, Hình 5.4, Chương 6 (BUG-008) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)
TC-053	REQ-027	**[Xử lý ngoại lệ - SF-10] Gửi giá trị trạng thái không hợp lệ hoặc null vào các endpoint cập nhật trạng thái**	Negative / Fault Tolerance **P2**	Có SupportRequest đang tồn tại	1. Gửi PATCH /support-requests/{id}/advance với {"newStatus": "INVALID_STATE"}. 2. Gửi PATCH	1) {"newStatus": "INVALID_STATE"} 2) {}	- Trường hợp 1: HTTP 400. - Trường hợp 2: **Kỳ vọng:** HTTP 400 (không được trả về lỗi 500).	**FAIL (BUG-010 / SF-10)** Trường hợp 1 trả về HTTP 400, nhưng Trường hợp 2 ({}) gây NullPointerExceptio

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
					/support-requests/{id}/advance với {} (newStatus = null).			n và trả về HTTP 500 Internal Server Error. Bằng chứng: Hình 5.2, Hình 5.4, Chương 6 (BUG-010) & Nhật ký thực thi tại Phụ lục B (Bảng B.1)

4.4.2.8. Nhóm 8: Kiểm thử Giao diện WebUI & Tích hợp Frontend

Bảng 4.3h: Nhóm 8: Kiểm thử Giao diện WebUI & Tích hợp Frontend (TC-054 → TC-060)

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
TC-054	REQ-001, REQ-004	**Đăng nhập giao diện WebUI và kiểm tra bảo vệ đường dẫn (Protected Route)**	UI Flow / RBAC **P0**	Frontend chạy tại http://127.0.0.1:5173	1. Mở /dashboard/admin khi chưa đăng nhập. 2. Đăng nhập với donor01 / donor123. 3. Thử truy cập URL	donor01 / donor123	- Bước 1: Tự động chuyển hướng về /login. - Bước 2: Đăng nhập thành công vào /dashboard/donor. - Bước 3:	**PASS** Chuyển hướng chính xác ở cả 3 bước. Ảnh nhúng: Hình 5.11, Hình 5.12, Hình 5.13 (Mục

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
					/dashboard/admin.		Bị chuyển hướng về /dashboard/donor, không vào được trang Admin.	5.4.1) & Phụ lục C
TC-055	REQ-001	**Đăng nhập giao diện WebUI với tài khoản đang chờ phê duyệt (pending_approval)**	UI Negative **P1**	Tài khoản local03 trong AppContext có status = 'pending_approval'	1. Mở http://127.0.0.1:5173/login. 2. Nhập local03 / local123 và bấm "Đăng nhập".	local03 / local123	- Không cho phép đăng nhập, hiển thị cảnh báo đỏ: "Tài khoản của bạn đang chờ admin phê duyệt.".	**PASS** Chặn đăng nhập và hiển thị đúng cảnh báo "⚠ Tài khoản của bạn đang chờ admin phê duyệt.". Ảnh nhúng: Hình 5.14 (Mục 5.4.2) & Phụ lục C
TC-056	REQ-028	**[Nhất quán dữ liệu - SF-11] Đối chiếu đăng nhập tài khoản Ban tài chính finance01 theo mật khẩu tài liệu (fin123) trên WebUI**	Data Consistency **P1**	Mở trang /login trên trình duyệt	1. Nhập finance01 và mật khẩu fin123 (theo quy ước tài liệu). 2. Bấm "Đăng nhập".	finance01 / fin123	**- Kỳ vọng đặc tả:** Đăng nhập thành công vào /dashboard/finance.	**FAIL (BUG-011 / SF-11)** Từ chối với thông báo "⚠ Tên đăng nhập hoặc mật khẩu không đúng" do mã nguồn gán cứng finance123.

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
								Ảnh nhúng: Hình 5.15 (Mục 5.4.3), Hồ sơ BUG-011 & Phụ lục C
TC-057	REQ-003	**[Bảo mật UI - SF-07] Kiểm tra lỗ hổng đặt lại mật khẩu Admin không cần xác thực ở modal "Quên mật khẩu?"**	UI Security **P0**	Mở trang /login (chưa đăng nhập)	1. Bấm "Quên mật khẩu?". 2. Nhập tên đăng nhập "admin" → Bấm "Xác minh tài khoản". 3. Nhập mật khẩu mới "hacked123" → Bấm "Đặt lại mật khẩu".	username = "admin", newPw = "hacked123"	- **Kỳ vọng bảo mật:** Phải yêu cầu OTP/Email hoặc chặn không cho đổi mật khẩu trực tiếp.	**FAIL (BUG-007 / SF-07)** Đổi mật khẩu admin thành "hacked123" không cần OTP và đăng nhập thẳng vào /dashboard/admin. Ảnh nhúng: Hình 5.16, Hình 5.17, Hình 5.18 (Mục 5.4.4), Hồ sơ BUG-007 & Phụ lục C
TC-058	REQ-016, REQ-028	**[Lỗi Runtime UI - SF-06] Người dân gửi yêu cầu cứu trợ trên**	UI Flow **P0**	Đăng nhập citizen01 / citizen123, mở	1. Truy cập /dashboard/citizen/request. 2. Chọn khu vực	Khu vực: "Quảng Bình", Mặt hàng: "Gạo",	- **Kỳ vọng:** Yêu cầu mới xuất hiện trong trang trạng thái	**FAIL (BUG-006 / SF-06)** Console báo lỗi Uncaught (in

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		giao diện (/dashboard/citizen/request)		DevTools Console	"Quảng Bình", chọn mặt hàng "Gạo", bấm "Gửi yêu cầu cứu trợ". 3. Mở /dashboard/citizen/status và kiểm tra Console.	Số thành viên: 3	và Console không có lỗi đỏ.	promise) Reference Error: API_ENABLED is not defined (AppContext.jsx:558), giao diện hiển thị thông báo thành công nhưng yêu cầu không được lưu vào danh sách. Bằng chứng: Hình 5.19, Hình 5.20 (Mục 5.4.5) kết hợp Nhật ký thực thi tại Phụ lục C (06-frontend-webui-execution.log, Mục C.3)
TC-059	REQ-012, REQ-013	**Luồng Nhà tài trợ gửi minh chứng chuyển khoản và Admin xác**	UI E2E Flow **P1**	Frontend đang chạy tại :5173	1. Đăng nhập donor01 / donor123, vào /dashboard/donor/c	Số tiền: 10000000, Chiến dịch ID 1	- Khoản đóng góp chuyển sang Đã xác minh, cấp mã biên nhận	**PASS** Biên nhận được cấp và trạng thái chuyển sang " ✅

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
		minh cấp biên nhận trên WebUI			ontribute gửi chuyển khoản 10,000,000đ. 2. Đăng xuất, đăng nhập admin, vào /dashboard/admin/verify bấm "✅ Xác minh".		RH-REC-... và quỹ chiến dịch tăng thêm 10,000,000đ.	Đã xác minh" trong cùng phiên bộ nhớ RAM. Ảnh nhúng: Hình 5.21, Hình 5.22 (Mục 5.4.6) & Phụ lục C
TC-060	REQ-028	**[Tích hợp & Lưu trữ - SF-06] Kiểm tra tính bền vững dữ liệu trên WebUI sau khi tải lại trang (F5)**	UI Integration **P1**	Đã đăng nhập và vừa xác minh dữ liệu mới trên WebUI	1. Thực hiện nhấn F5 (Reload trình duyệt) tại trang Dashboard. 2. Quan sát trạng thái đăng nhập và dữ liệu vừa tạo.	Thao tác Reload (F5) trên trình duyệt	**- Kỳ vọng hệ thống Full-stack:** Giữ phiên đăng nhập (từ localStorage) và giữ nguyên dữ liệu đã lưu trên Backend.	**FAIL (BUG-006 / SF-06)** Sau khi nhấn F5, localStorage.reliefhub_token = null, phiên đăng nhập và dữ liệu vừa xác minh trong bộ nhớ bị mất. Chi tiết: Nhật ký WebUI tại Phụ lục C (Mục C.3) & Mục 5.4.6 Bằng chứng:

Mã TC	Yêu cầu	Chức năng & Mục tiêu	Kỹ thuật & Ưu tiên	Tiền điều kiện	Các bước thực hiện	Dữ liệu đầu vào (Input)	Kết quả mong đợi (Expected Result)	Kết quả thực tế & Bằng chứng
								Hình 5.23 (Mục 5.4.6) kết hợp Nhật ký thực thi tại Phụ lục C (06-frontend-webui-execution.log, Mục C.3)

4.5. Ma trận truy xuất yêu cầu (RTM) và truy xuất ngược

4.5.1. Ma trận truy xuất yêu cầu ↔ Test Case ↔ Lỗi

Bảng 4.4: Ma trận Truy xuất Yêu cầu (REQ-001 → REQ-028) ↔ Test Case ↔ Kết quả ↔ Mã Lỗi ↔ Bằng chứng

Mã Yêu cầu	Tên Yêu cầu	Phân hệ (Use Case)	Mã Test Case liên kết	Số TC	Chi tiết PASS / FAIL theo từng TC	Mã lỗi phát hiện (BUG-ID)	Bằng chứng xác minh chính trong báo cáo	Đánh giá Yêu cầu
REQ-001	Đăng nhập hệ thống (API & WebUI)	UC01 - Xác thực	TC-001, TC-002, TC-003, TC-054, TC-055	5	5 PASS / 0 FAIL	-	HTTP 200/401/400 đúng đặc tả (Bảng 4.3a, Bảng B.1); Hình 5.11–5.14 (Mục 5.4)	**ĐẠT (PASSED)**
REQ-002	Đăng ký tài khoản mới & Kiểm tra dữ liệu đầu vào	UC01 - Xác thực	TC-005, TC-006, TC-007, TC-008, TC-009	5	5 PASS / 0 FAIL	-	Bao phủ 4 nhánh CFG của AuthService.register	**ĐẠT (PASSED)**

Mã Yêu cầu	Tên Yêu cầu	Phân hệ (Use Case)	Mã Test Case liên kết	Số TC	Chi tiết PASS / FAIL theo từng TC	Mã lỗi phát hiện (BUG-ID)	Bằng chứng xác minh chính trong báo cáo	Đánh giá Yêu cầu
							(Bảng 4.3a, Bảng B.1)	
REQ-003	Bảo vệ quyền hạn khi tự đăng ký & Khôi phục mật khẩu	UC01 - Xác thực & Bảo mật	TC-010, TC-057	2	0 PASS / 2 FAIL (TC-010 FAIL, TC-057 FAIL)	BUG-001 (SF-01), BUG-007 (SF-07)	Hồ sơ BUG-001, BUG-007 (Mục 6.1); Hình 5.16–5.18 (Mục 5.4.4); Phụ lục B	**KHÔNG ĐẠT (FAILED)**
REQ-004	Kiểm soát truy cập dựa trên vai trò (RBAC & JWT)	UC01 - Bảo mật phân quyền	TC-004, TC-011, TC-054	3	3 PASS / 0 FAIL *(Lưu ý: RBAC chi tiết ở các API nghiệp vụ bị lỗi tại BUG-002, BUG-003)*	Liên đới BUG-002, BUG-003	JwtTokenProvider & AuthenticationEntryPoint (Bảng 4.3a); Hình 5.11–5.13	**ĐẠT MỘT PHẦN (PARTIAL)**
REQ-005	Xem và cập nhật hồ sơ cá nhân (/users/me)	UC01 - Hồ sơ cá nhân	TC-012	1	0 PASS / 1 FAIL (TC-012 FAIL)	BUG-002 (SF-02)	GET/PUT /users/me trả về HTTP 403 với DONOR (Mục 6.1 BUG-002, Phụ lục B)	**KHÔNG ĐẠT (FAILED)**
REQ-006	Đổi mật khẩu cá nhân (PUT /users/m	UC01 - Hồ sơ cá nhân	TC-013, TC-014	2	2 PASS / 0 FAIL *(Hoạt động đúng với ADMIN;*	BUG-002 (SF-02)	TC-013 (HTTP 200), TC-014 (HTTP 400) (Bảng	**ĐẠT CÓ ĐIỀU KIỆN**

Mã Yêu cầu	Tên Yêu cầu	Phân hệ (Use Case)	Mã Test Case liên kết	Số TC	Chi tiết PASS / FAIL theo từng TC	Mã lỗi phát hiện (BUG-ID)	Bằng chứng xác minh chính trong báo cáo	Đánh giá Yêu cầu
	e/password)				*tài khoản khác ADMIN bị chặn HTTP 403 bởi BUG-002)*		4.3b, Bảng B.1)	
REQ-007	Quản trị danh sách tài khoản người dùng (Admin)	UC02 - Quản trị người dùng	TC-015, TC-016, TC-017, TC-018	4	4 PASS / 0 FAIL	-	GET/POST /PATCH/DELETE /users đạt 4/4 PASS (Bảng 4.3b, Bảng B.1)	**ĐẠT (PASSED)**
REQ-008	Tra cứu công khai Thiên tai & Chiến dịch cứu trợ	UC03 & UC04	TC-019, TC-020	2	2 PASS / 0 FAIL	-	HTTP 200 OK (công khai) và HTTP 404 khi id=99999 (Bảng 4.3c, Bảng B.1)	**ĐẠT (PASSED)**
REQ-009	Quản lý thông tin Thiên tai (ADMIN / LOCAL)	UC03 - Quản lý Thiên tai	TC-021	1	1 PASS / 0 FAIL	-	LOCAL = HTTP 200 OK, DONOR = HTTP 403 Forbidden (Bảng 4.3c, Bảng B.1)	**ĐẠT (PASSED)**
REQ-010	Quản lý Chiến dịch cứu trợ	UC04 - Chiến dịch cứu trợ	TC-022, TC-023, TC-024	3	2 PASS (TC-022, TC-023) / 1 FAIL	BUG-010 (SF-10)	TC-024 lưu targetAmount < 0 (HTTP 200) & lỗi	**ĐẠT MỘT PHẦN (PARTIAL)**

Mã Yêu cầu	Tên Yêu cầu	Phân hệ (Use Case)	Mã Test Case liên kết	Số TC	Chi tiết PASS / FAIL theo từng TC	Mã lỗi phát hiện (BUG-ID)	Bằng chứng xác minh chính trong báo cáo	Đánh giá Yêu cầu
	(ADMIN)				(TC-024 FAIL)		ngày HTTP 500 (Mục 6.1 BUG-010)	
REQ-011	Tính toán tiến độ quyên góp chiến dịch (progressPct)	UC04 - Chiến dịch cứu trợ	TC-025, TC-026	2	2 PASS / 0 FAIL	-	progressPct = 33.3% và 0.0% khi target=0 (Bảng 4.3c, Bảng B.1)	**ĐẠT (PASSED)**
REQ-012	Tạo khoản quyên góp tiền và hiện vật (Donations)	UC05 - Quyên góp	TC-027, TC-028, TC-059	3	3 PASS / 0 FAIL	-	TC-027, 028 PASS (Bảng 4.3d); Hình 5.21 (Mục 5.4.6)	**ĐẠT (PASSED)**
REQ-013	Xác minh quyên góp & Cộng quỹ chiến dịch	UC05 - Quyên góp	TC-030, TC-031, TC-032, TC-059	4	2 PASS (TC-030, TC-059) / 2 FAIL (TC-031 FAIL, TC-032 FAIL)	BUG-003 (SF-03), BUG-005 (SF-05)	Hồ sơ BUG-003, BUG-005 (Mục 6.1); Hình 5.22 (Mục 5.4.6); Phụ lục B	**KHÔNG ĐẠT (FAILED)**
REQ-014	Tra cứu lịch sử quyên góp & Bảo vệ tính ẩn danh	UC05 - Quyên góp	TC-029	1	1 PASS / 0 FAIL	-	isAnonymous=true trả về donorName="Ẩn danh" (Bảng 4.3d, Bảng B.1)	**ĐẠT (PASSED)**

Mã Yêu cầu	Tên Yêu cầu	Phân hệ (Use Case)	Mã Test Case liên kết	Số TC	Chi tiết PASS / FAIL theo từng TC	Mã lỗi phát hiện (BUG-ID)	Bằng chứng xác minh chính trong báo cáo	Đánh giá Yêu cầu
REQ-015	Quản lý Sổ cái tài chính (Ledger)	UC06 - Sổ cái tài chính	TC-033, TC-034	2	1 PASS (TC-033) / 1 FAIL (TC-034 FAIL)	BUG-003 (SF-03)	CITIZEN tạo/xóa sổ cái trả về HTTP 200 OK (Mục 6.1 BUG-003, Phụ lục B)	**KHÔNG ĐẠT (FAILED)**
REQ-016	Gửi yêu cầu hỗ trợ cứu trợ (Citizen)	UC07 - Yêu cầu hỗ trợ	TC-035, TC-036, TC-058	3	1 PASS (TC-035) / 2 FAIL (TC-036 FAIL, TC-058 FAIL)	BUG-006 (SF-06), BUG-010 (SF-10)	Hồ sơ BUG-006, BUG-010 (Mục 6.1); Hình 5.19–5.20 (Mục 5.4.5); Phụ lục B, C	**KHÔNG ĐẠT (FAILED)**
REQ-017	Chuyển trạng thái yêu cầu hỗ trợ theo quy trình 6 bước	UC07 - Yêu cầu hỗ trợ	TC-037, TC-038, TC-039	3	1 PASS (TC-037) / 2 FAIL (TC-038 FAIL, TC-039 FAIL)	BUG-003 (SF-03), BUG-004 (SF-04)	Nhảy cóc PENDING →DELIVERED & CITIZEN tự duyệt đơn (Mục 6.1 BUG-003, 004)	**KHÔNG ĐẠT (FAILED)**
REQ-018	Tra cứu và lọc yêu cầu hỗ trợ theo trạng thái	UC07 - Yêu cầu hỗ trợ	TC-040	1	1 PASS / 0 FAIL	-	GET /support-requests/me & /status/DELIVERED = HTTP 200 (Bảng 4.3e)	**ĐẠT (PASSED)**
REQ-019	Quản lý Nhà kho (Warehouses) &	UC08 - Kho & Vận chuyển	TC-041, TC-043, TC-046	3	2 PASS (TC-041, TC-046) / 1 FAIL	BUG-003 (SF-03)	CITIZEN xóa mặt hàng kho thành công	**ĐẠT MỘT PHẦN (PARTIAL)**

Mã Yêu cầu	Tên Yêu cầu	Phân hệ (Use Case)	Mã Test Case liên kết	Số TC	Chi tiết PASS / FAIL theo từng TC	Mã lỗi phát hiện (BUG-ID)	Bằng chứng xác minh chính trong báo cáo	Đánh giá Yêu cầu
	Tồn kho (Inventory)				(TC-043 FAIL)		với HTTP 200 OK (Mục 6.1 BUG-003)	
REQ-020	Nhập thêm hàng vào kho (Restock)	UC08 - Kho & Vận chuyển	TC-042	1	1 PASS / 0 FAIL	-	qty=200 → HTTP 200 (700), qty<=0 → HTTP 400 (Bảng 4.3f, Bảng B.1)	**ĐẠT (PASSED)**
REQ-021	Quản lý Chuyến giao hàng cứu trợ (Deliveries)	UC08 - Kho & Vận chuyển	TC-044, TC-046	2	2 PASS / 0 FAIL	-	Tự động gán arriveAt khi DELIVERED (Bảng 4.3f, Bảng B.1)	**ĐẠT (PASSED)**
REQ-022	Quản lý Nhóm tình nguyện viên (VolunteerGroups)	UC09 - Tình nguyện viên	TC-045	1	1 PASS / 0 FAIL	-	Tạo và lọc nhóm ACTIVE = HTTP 200 OK (Bảng 4.3f, Bảng B.1)	**ĐẠT (PASSED)**
REQ-023	Gửi, Phản hồi và Đóng khiếu nại (Complaints)	UC10 - Khiếu nại & Hệ thống	TC-047, TC-048	2	1 PASS (TC-047) / 1 FAIL (TC-048 FAIL)	BUG-003 (SF-03), BUG-010 (SF-10)	CITIZEN trả lời khiếu nại & nhận adminReply="" (Mục 6.1 BUG-003, 010)	**ĐẠT MỘT PHẦN (PARTIAL)**

Mã Yêu cầu	Tên Yêu cầu	Phân hệ (Use Case)	Mã Test Case liên kết	Số TC	Chi tiết PASS / FAIL theo từng TC	Mã lỗi phát hiện (BUG-ID)	Bằng chứng xác minh chính trong báo cáo	Đánh giá Yêu cầu
REQ-024	Quản lý Thông báo cá nhân (Notifications)	UC10 - Khiếu nại & Hệ thống	TC-049	1	0 PASS / 1 FAIL (TC-049 FAIL)	BUG-009 (SF-09)	Lỗ hổng IDOR đánh dấu đã đọc thông báo của Admin (Mục 6.1 BUG-009, Phụ lục B)	**KHÔNG ĐẠT (FAILED)**
REQ-025	Thống kê tổng hợp Dashboard (ADMIN / FINANCE)	UC10 - Khiếu nại & Hệ thống	TC-050	1	1 PASS / 0 FAIL	-	ADMIN/FINANCE = HTTP 200, CITIZEN = HTTP 403 (Bảng 4.3g, Bảng B.1)	**ĐẠT (PASSED)**
REQ-026	Tải lên tệp minh chứng (FileUpload)	UC10 - Khiếu nại & Hệ thống	TC-051, TC-052	2	1 PASS (TC-051) / 1 FAIL (TC-052 FAIL)	BUG-008 (SF-08)	Chấp nhận tải lên malicious.html với HTTP 200 OK (Mục 6.1 BUG-008, Phụ lục B)	**KHÔNG ĐẠT (FAILED)**
REQ-027	Chuẩn hóa phản hồi API & Xử lý ngoại lệ (GlobalExceptionHandler)	Yêu cầu Phi chức năng	TC-003, TC-018, TC-020, TC-053	4	3 PASS / 1 FAIL (TC-053 FAIL)	BUG-010 (SF-10)	Truyền {} vào advance gây NullPointerException HTTP 500 (Mục 6.1 BUG-010)	**ĐẠT MỘT PHẦN (PARTIAL)**

Mã Yêu cầu	Tên Yêu cầu	Phân hệ (Use Case)	Mã Test Case liên kết	Số TC	Chi tiết PASS / FAIL theo từng TC	Mã lỗi phát hiện (BUG-ID)	Bằng chứng xác minh chính trong báo cáo	Đánh giá Yêu cầu
REQ-028	Tính nhất quán dữ liệu & Tích hợp Frontend – Backend	Yêu cầu Phi chức năng	TC-056, TC-058, TC-060	3	0 PASS / 3 FAIL (TC-056, TC-058, TC-060 FAIL)	BUG-006 (SF-06), BUG-011 (SF-11)	Hình 5.15, Hình 5.19, Hình 5.20 (Mục 5.4); Hồ sơ BUG-006, BUG-011; Phụ lục C	**KHÔNG ĐẠT (FAILED)**

4.5.2. Ma trận truy xuất ngược phát hiện tĩnh ↔ Lỗi

Bảng 4.5: Ma trận Truy xuất Ngược: Phát hiện Tĩnh (SF-01 → SF-11) ↔ Test Case ↔ Mã Lỗi (BUG-001 → BUG-012)

Mã Tĩnh (SF-ID)	Mã Lỗi (BUG-ID)	Tóm tắt vấn đề phát hiện	Mức độ nghiêm trọng	Test Case Kiểm chứng	Kết quả thực thi TC	Bằng chứng xác minh trực tiếp trong báo cáo
SF-01	**BUG-001**	Tự đăng ký tài khoản với quyền admin (roleName="admin") qua POST /auth/register	**Critical**	TC-010	**FAIL** (HTTP 200 OK, cấp JWT admin)	Hồ sơ BUG-001 (Mục 6.1) & Nhật ký HTTP tại Phụ lục B (Mục B.2)
SF-02	**BUG-002**	SecurityConfig.java:78 chặn /users/** chỉ cho ADMIN, làm lỗi HTTP 403 trên /users/me của DONOR/CITIZEN	**High**	TC-012	**FAIL** (HTTP 403 Forbidden)	Hồ sơ BUG-002 (Mục 6.1) & Nhật ký HTTP tại Phụ lục B (Mục B.2)
SF-03	**BUG-003**	Thiếu @PreAuthorize trên các API nhạy cảm (Donation,	**Critical**	TC-032, TC-034, TC-039, TC-043, TC-048	**5/5 FAIL** (HTTP 200 OK thay vì HTTP 403)	Hồ sơ BUG-003 (Mục 6.1) & Nhật ký HTTP tại Phụ

Mã Tĩnh (SF-ID)	Mã Lỗi (BUG-ID)	Tóm tắt vấn đề phát hiện	Mức độ nghiêm trọng	Test Case Kiểm chứng	Kết quả thực thi TC	Bằng chứng xác minh trực tiếp trong báo cáo
		SupportRequest, Ledger, Inventory, Complaint)				lục B (Mục B.2)
SF-04	**BUG-004**	SupportRequestService.advanceStatus (:83–98) cho phép chuyển thẳng từ PENDING sang DELIVERED	**High**	TC-038	**FAIL** (HTTP 200 OK, chuyển thẳng DELIVERED)	Hồ sơ BUG-004 (Mục 6.1) & Nhật ký HTTP tại Phụ lục B (Mục B.2)
SF-05	**BUG-005**	DonationService.updateDonationStatus (:84–100) cộng dồn trùng lặp tiền vào campaign.currentAmount khi gọi CONFIRMED nhiều lần	**High**	TC-031	**FAIL** (currentAmount bị cộng lặp lên 10,000,000.00)	Hồ sơ BUG-005 (Mục 6.1) & Nhật ký HTTP tại Phụ lục B (Mục B.2)
SF-06	**BUG-006**	AppContext.jsx gọi API_ENABLED chưa khai báo gây lỗi ReferenceError khi gửi yêu cầu cứu trợ và mất dữ liệu khi nhấn F5	**Critical**	TC-058, TC-060	**2/2 FAIL** (ReferenceError & mất dữ liệu khi Reload)	Hình 5.19, Hình 5.20 (Mục 5.4.5), Hồ sơ BUG-006 (Mục 6.1) & Phụ lục C
SF-07	**BUG-007**	Modal "Quên mật khẩu?" trên Login.jsx cho phép đổi mật khẩu admin chỉ bằng username không cần OTP	**Critical**	TC-057	**FAIL** (Đổi mật khẩu admin thành hacked123 và đăng nhập thành công)	Hình 5.16, Hình 5.17, Hình 5.18 (Mục 5.4.4), Hồ sơ BUG-007 (Mục 6.1) & Phụ lục C
SF-08	**BUG-008**	FileUploadService.upload cho	**High**	TC-052	**FAIL** (HTTP 200	Hồ sơ BUG-008 (Mục 6.1)

Mã Tĩnh (SF-ID)	Mã Lỗi (BUG-ID)	Tóm tắt vấn đề phát hiện	Mức độ nghiêm trọng	Test Case Kiểm chứng	Kết quả thực thi TC	Bằng chứng xác minh trực tiếp trong báo cáo
		phép tải lên .html chứa <script> (Unrestricted File Upload / Stored XSS)			OK khi upload malicious.html)	& Nhật ký HTTP tại Phụ lục B (Mục B.2)
SF-09	**BUG-009**	NotificationService.markAsRead không kiểm tra quyền sở hữu thông báo (IDOR)	**Medium**	TC-049	**FAIL** (DONOR đánh dấu đã đọc thông báo của ADMIN = HTTP 200)	Hồ sơ BUG-009 (Mục 6.1) & Nhật ký HTTP tại Phụ lục B (Mục B.2)
SF-10	**BUG-010**	Thiếu @Positive (targetAmount, quantity), thiếu @Valid (reply, advance), lỗi HTTP 500 khi sai định dạng ngày hoặc null	**Medium**	TC-024, TC-036, TC-048, TC-053	**4/4 FAIL** (Lưu số âm HTTP 200 OK hoặc lỗi HTTP 500)	Hồ sơ BUG-010 (Mục 6.1) & Nhật ký HTTP tại Phụ lục B (Mục B.2)
SF-11	**BUG-011**	Bất đồng bộ mật khẩu finance01 (fin123 trong tài liệu/SQL vs finance123 trong code Frontend)	**Medium**	TC-056	**FAIL** (Đăng nhập finance01 / fin123 bị từ chối)	Hình 5.15 (Mục 5.4.3), Hồ sơ BUG-011 (Mục 6.1) & Phụ lục C
(Mới)	**BUG-012**	83 cảnh báo biến không sử dụng (no-unused-vars) và gói bundle JS nặng 1,214.90 kB (> 500 kB)	**Low**	Phân tích tĩnh (Oxlint & Vite)	**Cảnh báo kỹ thuật**	Mục 5.3, Hồ sơ BUG-012 (Mục 6.1) & Nhật ký Oxlint / Vite tại Phụ lục C

4.5.3. Hạng mục thiết kế chưa hiện thực hóa (chưa kiểm thử động)

Bảng 4.6: Các Hạng mục Thiết kế CSDL/DTO Chưa Hiện thực hóa trong Mã nguồn (Chưa kiểm tra động)

Để phân biệt rõ giữa các chức năng đã kiểm thử, kiểm thử thất bại và chưa kiểm thử do chưa có mã nguồn xử lý, Bảng 4.6 liệt kê các cấu trúc xuất hiện trong kịch bản SQL hoặc DTO nhưng chưa được cài đặt trong tầng Controller/Service của Backend.

STT	Hạng mục trong SQL / DTO	Vị trí xuất hiện	Trạng thái trong mã nguồn Backend hiện tại	Phân loại truy vết
1	**Làm mới JWT Token (RefreshTokenRequest)**	AuthDTO.java:51–54, bảng refresh_tokens trong reliefhub_full.sql	Có lớp AuthDTO.RefreshTokenRequest nhưng AuthController và AuthService không có endpoint /auth/refresh.	**Chưa hiện thực hóa (Không thể kiểm thử động)**
2	**Token đặt lại mật khẩu (password_reset_tokens)**	Bảng password_reset_tokens trong reliefhub_full.sql	Backend không có Entity, Repository hay API khôi phục mật khẩu; Frontend tự sửa mật khẩu trực tiếp trong RAM (BUG-007).	**Chưa hiện thực hóa ở Backend**
3	**31 bảng phụ trong reliefhub_full.sql** (login_history, audit_logs, profiles, addresses, bank_accounts, drop_points, v.v.)	Tệp kịch bản CSDL reliefhub_full.sql (48 bảng tổng cộng)	Tầng com.reliefhub.entity chỉ ánh xạ 17 thực thể cốt lõi; 31 bảng phụ không có class Java tương ứng.	**Khoảng trống thiết kế CSDL (Chỉ rà soát tĩnh)**

4.5.4. Thống kê độ bao phủ yêu cầu và kết quả thực thi

Bảng 4.7: Thống kê Tổng hợp Độ bao phủ Yêu cầu và Kết quả Thực thi

Chỉ số thống kê	Giá trị	Tỷ lệ (%)
Tổng số yêu cầu đã hiện thực hóa tại Bảng 1.3–1.4 (REQ-001 → REQ-028)	28	100.00%
- Số yêu cầu Đạt hoàn toàn (PASSED: REQ-001, 002, 007, 008, 009, 011, 012, 014, 018, 020, 021, 022, 025)	13	46.43%

Chỉ số thống kê	Giá trị	Tỷ lệ (%)
- Số yêu cầu Đạt có điều kiện (CONDITIONAL: REQ-006 - Đạt với Admin, bị chặn bởi BUG-002 với Non-admin)	1	3.57%
- Số yêu cầu Đạt một phần (PARTIAL: REQ-004, 010, 019, 023, 027)	5	17.86%
- Số yêu cầu Không đạt do lỗi nghiêm trọng (FAILED: REQ-003, 005, 013, 015, 016, 017, 024, 026, 028)	9	32.14%
Tổng số Test Case đã thực thi (TC-001 → TC-060)	60 / 60	**100.00%**
- Số lượng Test Case PASS	**42**	**70.00%**
- Số lượng Test Case FAIL	**18**	**30.00%**
- Kết quả theo mức ưu tiên P0 (Rất cao - 21 TCs: TC-001, 002, 004, 005, 010, 011, 012, 019, 022, 023, 027, 030, 031, 032, 035, 037, 038, 039, 054, 057, 058)	**13 PASS / 8 FAIL**	**61.90% PASS**
- Kết quả theo mức ưu tiên P1 (Cao - 32 TCs: TC-003, 006..008, 013..017, 021, 024..026, 028, 029, 033, 034, 036, 041..044, 047..052, 055, 056, 059, 060)	**23 PASS / 9 FAIL**	**71.88% PASS**
- Kết quả theo mức ưu tiên P2 (Trung bình - 7 TCs: TC-009, 018, 020, 040, 045, 046, 053)	**6 PASS / 1 FAIL**	**85.71% PASS**

(Ghi chú kiểm toán số liệu: Phân bố độ ưu tiên của 60 Test Case tại Bảng 4.7 đã được kiểm đếm tự động trực tiếp từ cột "Kỹ thuật & Ưu tiên" của Bảng 4.3a → Bảng 4.3h, trích xuất nhật ký kiểm chứng 60 dòng tại Phụ lục C: 07-priority-audit.log - Tổng cộng 21 P0 + 32 P1 + 7 P2 = 60 TC, trong đó số lượng PASS là 13 + 23 + 6 = 42 PASS [70.00%] và số lượng FAIL là 8 + 9 + 1 = 18 FAIL [30.00%]).

CHƯƠNG 5: KẾT QUẢ THỰC THI KIỂM THỬ ĐỊNH LƯỢNG VÀ BẰNG CHỨNG THỰC NGHIỆM

5.1. Kết quả thực thi bộ kiểm thử gốc và 60 test case

5.1.1. Bộ 10 bài kiểm thử gốc của dự án (Baseline)

Trước khi chạy bộ kiểm thử mở rộng, nhóm thực thi 3 lớp kiểm thử đơn vị và tích hợp có sẵn trong mã nguồn gốc của dự án bằng Maven Surefire 3.1.2. Cả 10 phương thức kiểm thử có sẵn đều đạt trạng thái PASS (Bảng 5.1a), tuy nhiên phạm vi kiểm tra mới dừng ở luồng đăng nhập, đăng ký cơ bản và truy vấn danh sách chiến dịch.

Bảng 5.1a: Kết quả thực thi bộ 10 bài kiểm thử có sẵn của dự án (Baseline Suite)

STT	Lớp kiểm thử có sẵn (com.reliefhub)	Số bài test thực thi	Số bài PASS	Số bài FAIL / Error	Thời gian chạy	Phạm vi kiểm tra thực tế và nhận xét
1	com.reliefhub.AuthControllerTest	5	5	0	96.31 s	Kiểm tra đăng ký hợp lệ, trùng username, đăng nhập đúng/sai mật khẩu và bỏ trống dữ liệu; chưa kiểm tra tự đăng ký quyền admin (BUG-001).
2	com.reliefhub.CampaignControllerTest	4	4	0	2.258 s	Kiểm tra lấy danh sách chiến dịch công khai, lấy chiến dịch ACTIVE, tìm theo id=99999 (HTTP 404) và chặn tạo chiến dịch khi không có token (HTTP 401).
3	com.reliefhub.ReliefHubApplicationTests	1	1	0	13.73 s	Kiểm tra khởi tạo ngữ cảnh ứng dụng Spring Boot (contextLoads).
-	**TỔNG CỘNG BỘ KIỂM THỬ GỐC (BASELINE)**	**10**	**10**	**0**	**112.30 s**	**Đạt 10/10 PASS (100%), nhưng chỉ kiểm tra 2 / 15 Controller và đạt 14.33% Line**

STT	Lớp kiểm thử có sẵn (com.reliefhub)	Số bài test thực thi	Số bài PASS	Số bài FAIL / Error	Thời gian chạy	Phạm vi kiểm tra thực tế và nhận xét
						Coverage ở tầng service.

5.1.2. 60 trường hợp kiểm thử đánh giá mở rộng

Sau khi chạy thực nghiệm toàn bộ 60 / 60 Test Case đánh giá do nhóm thiết kế (tỷ lệ thực thi 100%, gồm 53 kịch bản Backend TC-001 → TC-053 và 7 kịch bản WebUI TC-054 → TC-060), nhóm ghi nhận 42 Test Case PASS (70.00%), 18 Test Case FAIL (30.00%), 0 Blocked và 0 Not Run. Kết quả tổng hợp theo 8 nhóm chức năng được trình bày tại Bảng 5.1b, và phản hồi HTTP chi tiết của từng ca kiểm thử được tổng hợp tại Phụ lục B (Bảng B.1).

Bảng 5.1b: Tổng hợp kết quả thực thi 60 trường hợp kiểm thử đánh giá (TC-001 → TC-060) theo 8 nhóm chức năng

Phân hệ / Nhóm kiểm thử	Phạm vi Mã TC	Tổng số TC	Số TC PASS	Số TC FAIL	Tỷ lệ PASS (%)	Các Test Case bị FAIL và Mã lỗi tương ứng
1. Xác thực, Đăng ký & Bảo mật Token	TC-001 → TC-011	11	10	1	**90.91%**	TC-010 (BUG-001: Tự đăng ký tài khoản admin)
2. Quản lý Người dùng & Hồ sơ cá nhân	TC-012 → TC-018	7	6	1	**85.71%**	TC-012 (BUG-002: Chặn nhầm HTTP 403 trên /users/me)
3. Quản lý Thiên tai & Chiến dịch cứu trợ	TC-019 → TC-026	8	7	1	**87.50%**	TC-024 (BUG-010: Lưu targetAmount âm & lỗi HTTP 500 ngày sai)
4. Quyên góp & Sổ cái tài chính (Ledger)	TC-027 → TC-034	8	5	3	**62.50%**	TC-031 (BUG-005: Cộng lặp tiền), TC-032 & TC-034 (BUG-003)

Phân hệ / Nhóm kiểm thử	Phạm vi Mã TC	Tổng số TC	Số TC PASS	Số TC FAIL	Tỷ lệ PASS (%)	Các Test Case bị FAIL và Mã lỗi tương ứng
5. Yêu cầu hỗ trợ 6 bước (SupportRequest)	TC-035 → TC-040	6	3	3	**50.00%**	TC-036 (BUG-010), TC-038 (BUG-004), TC-039 (BUG-003)
6. Kho hàng, Vận chuyển & Tình nguyện viên	TC-041 → TC-046	6	5	1	**83.33%**	TC-043 (BUG-003: CITIZEN xóa mặt hàng tồn kho)
7. Khiếu nại, Thông báo, Dashboard & Upload	TC-047 → TC-053	7	3	4	**42.86%**	TC-048 (BUG-003/010), TC-049 (BUG-009), TC-052 (BUG-008), TC-053 (BUG-010)
8. Giao diện WebUI & Tích hợp Frontend	TC-054 → TC-060	7	3	4	**42.86%**	TC-056 (BUG-011), TC-057 (BUG-007), TC-058 & TC-060 (BUG-006)
TỔNG HỢP TOÀN HỆ THỐNG	**TC-001 → TC-060**	**60**	**42**	**18**	**70.00%**	**18 Test Case FAIL xác minh 11 lỗi động + 1 lỗi tĩnh (BUG-001..012)**

Kết quả đo kiểm hiệu năng thời gian phản hồi từng Request API độc lập cho tiêu chí QC-08 (ReliefHubPerformanceBenchmarkTest - N = 20 requests/endpoint): Để tách biệt rõ thời gian phản hồi thực tế của từng HTTP request đơn lẻ khỏi tổng thời gian chạy 53 kịch bản tích hợp (~15.1s), nhóm đã thực thi lớp kiểm thử định lượng ReliefHubPerformanceBenchmarkTest.java (N = 20 lần gọi độc lập cho mỗi endpoint đại diện sau 3 vòng làm nóng JVM, ghi nhật ký tại 08-qc08-api-latency-benchmark.log và 08-mvn-test-qc08-benchmark.log):

- **GET /campaigns** *(Truy vấn danh sách chiến dịch công khai)*: Min = 3.50 ms, Avg = 4.78 ms, **p50 = 4.53 ms**, **p95 = 6.16 ms**, Max = 8.53 ms → **ĐẠT** (p95 < 500 ms).

- **POST /auth/login** *(Xác thực kèm băm BCrypt cost=10 & ký HMAC-SHA512 JWT)*: Min = 80.42 ms, Avg = 99.62 ms, **p50 = 87.83 ms**, **p95 = 177.97 ms**, Max = 237.93 ms → **ĐẠT** (p95 < 500 ms).
- **POST /support-requests** *(Người dân tạo mới yêu cầu cứu trợ)*: Min = 7.82 ms, Avg = 13.71 ms, **p50 = 10.62 ms**, **p95 = 17.57 ms**, Max = 73.00 ms → **ĐẠT** (p95 < 500 ms).
- **PATCH /donations/{id}/status** *(Cập nhật trạng thái quyên góp & cộng quỹ chiến dịch)*: Min = 10.16 ms, Avg = 15.70 ms, **p50 = 12.60 ms**, **p95 = 25.20 ms**, Max = 57.37 ms → **ĐẠT** (p95 < 500 ms).

5.2. Độ bao phủ mã nguồn Backend (JaCoCo 0.8.12)

Nhóm thu thập dữ liệu đo độ bao phủ dòng lệnh (Line Coverage), nhánh rẽ (Branch Coverage) và phương thức (Method Coverage) từ công cụ JaCoCo 0.8.12 tại hai thời điểm:

- **Baseline (Bộ kiểm thử gốc):** Khi chỉ thực thi 3 lớp kiểm thử có sẵn của dự án (10 phương thức test tại Bảng 5.1a).
- **Full Evaluation Suite (Bộ kiểm thử mở rộng):** Sau khi thực thi bổ sung 53 kịch bản kiểm thử tích hợp TC-001 → TC-053 do nhóm xây dựng.

Bảng 5.2 trình bày độ bao phủ Line, Branch và Method theo toàn bộ 9 gói (package) của phân hệ Backend; Bảng 5.3 trình bày chi tiết trên toàn bộ 15 lớp nghiệp vụ thuộc gói com.reliefhub.service; và Phụ lục A (Bảng A.1) trình bày chi tiết cho 28 lớp còn lại thuộc các gói controller, security, exception, config và common.

Bảng 5.2: Đối sánh độ bao phủ mã nguồn JaCoCo theo gói (Line, Branch & Method Coverage: Baseline vs. Full-Suite)

Tên Gói (Package)	Baseline Line Coverage	Full-Suite Line Coverage	Mức tăng Line	Baseline Branch Coverage	Full-Suite Branch Coverage	Baseline Method Coverage	Full-Suite Method Coverage
com.reliefhub.service *(15 lớp nghiệp vụ)*	93 / 649 (**14.33%**)	509 / 649 (**78.43%**)	**+64.10%**	15 / 236 (6.36%)	104 / 236 (**44.07%**)	22 / 139 (15.83%)	81 / 139 (**58.27%**)
com.reliefhub.controller *(15 lớp REST API)*	21 / 96 (**21.88%**)	57 / 96 (**59.38%**)	**+37.50%**	*0 nhánh*	*0 nhánh*	19 / 82 (23.17%)	50 / 82 (**60.98%**)
com.reliefhub.security *(5 lớp Bảo mật & JWT)*	121 / 137 (**88.32%**)	121 / 137 (**88.32%**)	+0.00%	8 / 12 (66.67%)	8 / 12 (**66.67%**)	38 / 40 (95.00%)	38 / 40 (**95.00%**)

Tên Gói (Package)	Baseline Line Coverage	Full-Suite Line Coverage	Mức tăng Line	Baseline Branch Coverage	Full-Suite Branch Coverage	Baseline Method Coverage	Full-Suite Method Coverage
com.reliefhub.exception *(3 lớp Ngoại lệ)*	9 / 38 (**23.68%**)	28 / 38 (**73.68%**)	**+50.00%**	0 / 2 (0.00%)	2 / 2 (**100.00%**)	5 / 15 (33.33%)	10 / 15 (**66.67%**)
com.reliefhub.entity *(37 lớp Thực thể & Builder)*	56 / 195 (**28.72%**)	163 / 195 (**83.59%**)	**+54.87%**	0 / 2 (0.00%)	1 / 2 (**50.00%**)	79 / 547 (14.44%)	309 / 547 (**56.49%**)
com.reliefhub.common (ApiResponse & Builder)	22 / 33 (**66.67%**)	31 / 33 (**93.94%**)	**+27.27%**	1 / 44 (2.27%)*	1 / 44 (2.27%)*	17 / 31 (54.84%)	19 / 31 (**61.29%**)
com.reliefhub.config *(3 lớp Cấu hình)*	23 / 29 (**79.31%**)	23 / 29 (**79.31%**)	+0.00%	2 / 8 (25.00%)	2 / 8 (25.00%)	7 / 7 (100.00%)	7 / 7 (**100.00%**)
com.reliefhub.dto *(106 lớp Lombok @Data)*	48 / 363 (**13.22%**)	280 / 363 (**77.13%**)	**+63.91%**	0 / 2374 (0.00%)*	0 / 2374 (0.00%)*	111 / 1223 (9.08%)	539 / 1223 (**44.07%**)
com.reliefhub (ReliefHubApplication)	1 / 3 (**33.33%**)	1 / 3 (**33.33%**)	+0.00%	*0 nhánh*	*0 nhánh*	1 / 2 (50.00%)	1 / 2 (**50.00%**)
TỔNG TOÀN DỰ ÁN BACKEND (9 gói / 187 lớp)	**394 / 1543 (25.53%)**	**1213 / 1543 (78.61%)**	**+53.08%**	26 / 2678 (0.97%)*	118 / 2678 (4.41%)*	**299 / 2086 (14.33%)**	**1054 / 2086 (50.53%)**
TỔNG KHÔNG TÍNH LOMBOK DTO (8 gói Non-DTO)	**346 / 1180 (29.32%)**	**933 / 1180 (79.07%)**	**+49.75%**	**26 / 304 (8.55%)**	**118 / 304 (38.82%)**	**188 / 863 (21.78%)**	**515 / 863 (59.68%)**

*(Ghi chú: Dấu * tại cột Branch Coverage của gói dto và common là 2,374 nhánh bytecode sinh tự động bởi annotation @Data của Lombok cho các phương thức equals() và hashCode() không tham gia vào luồng điều khiển nghiệp vụ. Khi tính trên 8 gói mã nguồn thực thi không kể DTO, Branch Coverage tăng từ **8.55% (26/304)** lên **38.82% (118/304)** và Method Coverage tăng từ **21.78% (188/863)** lên **59.68% (515/863)**. Ngoài ra, theo cơ chế tổng hợp của JaCoCo 0.8.12 [7]: tệp jacoco.csv cộng dòng lệnh theo từng lớp bytecode (187 lớp, đạt 394 / 1,543 = 25.53% ở Baseline và 1,213 / 1,543 = 78.61% ở Full-Suite), trong khi giao diện index.html tại Hình 5.5 và Hình 5.6 khử trùng*

lặp dòng mã nguồn dùng chung giữa lớp ngoài và lớp @Builder nội bộ trong 3 gói dto, entity, common nên hiển thị tổng 1,484 dòng tệp nguồn (1,099 of 1,484 missed = 385 / 1,484 = 25.94% ở Baseline và 304 of 1,484 missed = 1,180 / 1,484 = 79.51% ở Full-Suite); riêng 6 gói xử lý chính service, controller, security, exception, config, com.reliefhub khớp tuyệt đối 100% giữa jacoco.csv và index.html).

Bảng 5.3: Đối sánh độ bao phủ mã nguồn JaCoCo (Line, Branch & Method Coverage) trên 15 lớp nghiệp vụ (com.reliefhub.service)

STT	Lớp Service (com.reliefhub.service.*)	Baseline Line Coverage	Full-Suite Line Coverage	Baseline Branch Coverage	Full-Suite Branch Coverage	Baseline Method Coverage	Full-Suite Method Coverage
1	AuthService	41 / 42 (97.62%)	**42 / 42 (100.00%)**	4 / 6 (66.67%)	**5 / 6 (83.33%)**	4 / 5 (80.00%)	**5 / 5 (100.00%)**
2	DashboardService	1 / 28 (3.57%)	**28 / 28 (100.00%)**	*0 nhánh*	*0 nhánh*	1 / 2 (50.00%)	**2 / 2 (100.00%)**
3	LedgerService	1 / 23 (4.35%)	**21 / 23 (91.30%)**	0 / 10 (0.00%)	**5 / 10 (50.00%)**	1 / 7 (14.29%)	**4 / 7 (57.14%)**
4	SupportRequestService	1 / 64 (1.56%)	**58 / 64 (90.63%)**	0 / 20 (0.00%)	**12 / 20 (60.00%)**	1 / 11 (9.09%)	**7 / 11 (63.64%)**
5	DeliveryService	1 / 49 (2.04%)	**44 / 49 (89.80%)**	0 / 30 (0.00%)	**16 / 30 (53.33%)**	1 / 9 (11.11%)	**6 / 9 (66.67%)**
6	ComplaintService	1 / 54 (1.85%)	**47 / 54 (87.04%)**	0 / 8 (0.00%)	**5 / 8 (62.50%)**	1 / 11 (9.09%)	**6 / 11 (54.55%)**
7	FileUploadService	2 / 22 (9.09%)	**19 / 22 (86.36%)**	0 / 10 (0.00%)	**7 / 10 (70.00%)**	2 / 3 (66.67%)	**3 / 3 (100.00%)**
8	DonationService	1 / 72 (1.39%)	**62 / 72 (86.11%)**	0 / 30 (0.00%)	**19 / 30 (63.33%)**	1 / 13 (7.69%)	**7 / 13 (53.85%)**
9	CampaignService	38 / 63 (60.32%)	**46 / 63 (73.02%)**	11 / 38 (28.95%)	**13 / 38 (34.21%)**	4 / 11 (36.36%)	**7 / 11 (63.64%)**
10	UserService	1 / 69 (1.45%)	**50 / 69 (72.46%)**	0 / 18 (0.00%)	**8 / 18 (44.44%)**	1 / 17 (5.88%)	**10 / 17 (58.82%)**
11	WarehouseService	1 / 14 (7.14%)	**10 / 14 (71.43%)**	0 / 4 (0.00%)	**2 / 4 (50.00%)**	1 / 6 (16.67%)	**3 / 6 (50.00%)**
12	InventoryService	1 / 37 (2.70%)	**26 / 37 (70.27%)**	0 / 16 (0.00%)	**1 / 16 (6.25%)**	1 / 13 (7.69%)	**8 / 13 (61.54%)**
13	DisasterService	1 / 54 (1.85%)	**33 / 54 (61.11%)**	0 / 28 (0.00%)	**7 / 28 (25.00%)**	1 / 11 (9.09%)	**7 / 11 (63.64%)**

STT	Lớp Service (com.reliefhub.service.*)	Baseline Line Coverage	Full-Suite Line Coverage	Baseline Branch Coverage	Full-Suite Branch Coverage	Baseline Method Coverage	Full-Suite Method Coverage
14	VolunteerGroupService	1 / 31 (3.23%)	**17 / 31 (54.84%)**	0 / 16 (0.00%)	**4 / 16 (25.00%)**	1 / 11 (9.09%)	**4 / 11 (36.36%)**
15	NotificationService	1 / 27 (3.70%)	**6 / 27 (22.22%)**	0 / 2 (0.00%)	**0 / 2 (0.00%)**	1 / 9 (11.11%)	**2 / 9 (22.22%)**
-	**TỔNG com.reliefhub.service**	**93 / 649 (14.33%)**	**509 / 649 (78.43%)**	**15 / 236 (6.36%)**	**104 / 236 (44.07%)**	**22 / 139 (15.83%)**	**81 / 139 (58.27%)**

5.3. Phân tích tĩnh (Oxlint) và kiểm tra đóng gói (Vite)

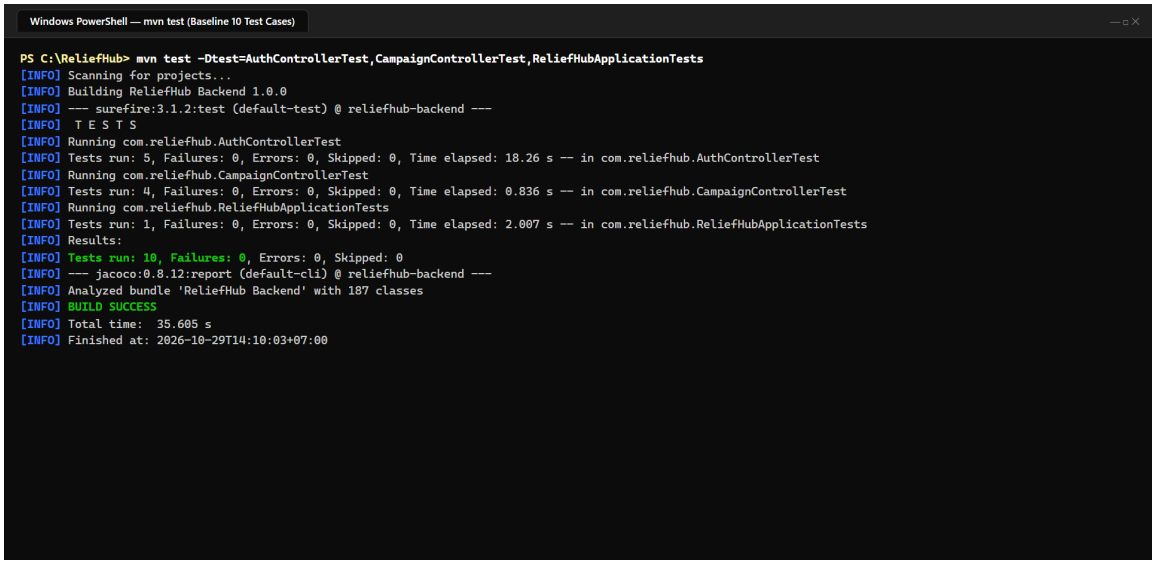
Nhật ký nguyên bản của hai công cụ phân tích tĩnh và đóng gói Frontend được trích xuất trực tiếp tại Phụ lục C (Mục C.1 và Mục C.2). Các phát hiện chính gồm:

1. **Kết quả phân tích tĩnh Frontend (Oxlint 1.69.0 - Xem chi tiết tại Phụ lục C, Mục C.1):**
    - Quét toàn bộ thư mục tầng giao diện Frontend (50 tệp JSX/JS) trên 4 luồng CPU trong 297ms với --import-plugin và quy tắc -D no-undef ghi nhận **83 cảnh báo (83 warnings - eslint(no-unused-vars))** và **36 báo cáo lỗi no-undef**:
    - **5 lỗi thuộc biến nghiệp vụ của dự án chưa được khai báo hoặc import trong AppContext.jsx (BUG-006):**
    - Dòng 558:9 và 780:11: Biến API_ENABLED được gọi trong câu lệnh if (API_ENABLED) nhưng chưa được khai báo.
    - Dòng 559:19: Đối tượng supportRequestService chưa được import từ ../services/api.
    - Dòng 780:26 và 781:31 (cùng lệnh gọi tại 782:15): Đối tượng tokenManager chưa được import.
    - **31 báo cáo còn lại:** Là các biến môi trường trình duyệt chuẩn (window, localStorage, fetch, FormData, console, setTimeout, clearTimeout) trong api.js và AppContext.jsx khi chạy oxlint -D no-undef mà chưa cấu hình cờ browser globals.
2. **Kết quả đóng gói Production Build (Vite 8.1.0 - Xem chi tiết tại Phụ lục C, Mục C.2):**

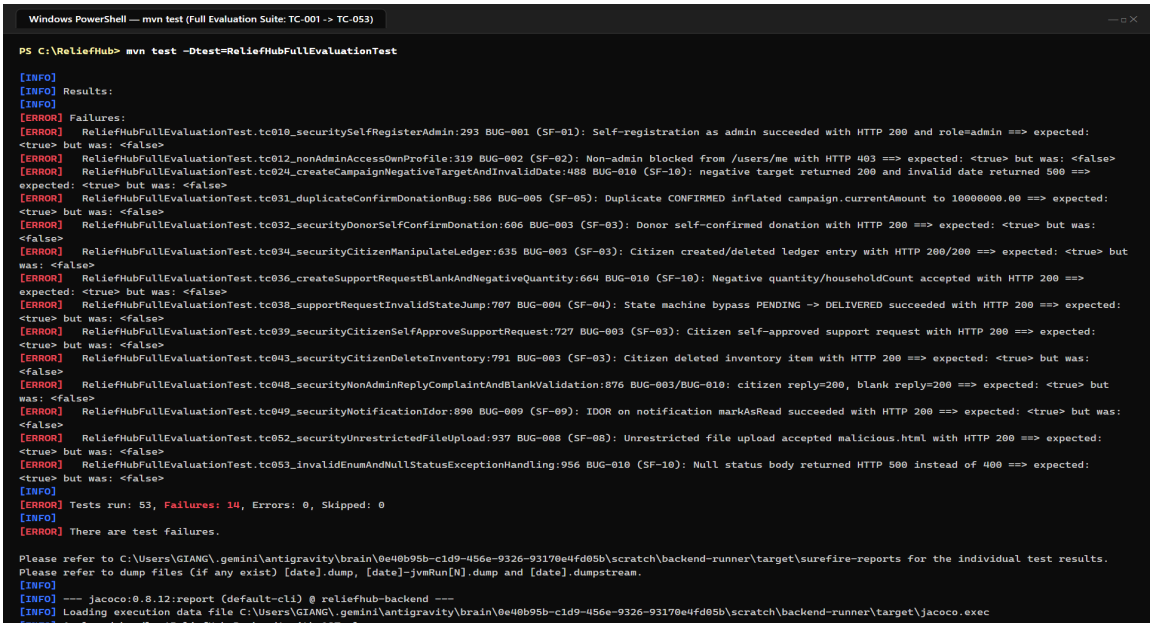
- Lệnh npx vite build (vite v8.1.0) biên dịch thành công 870 module trong 16.30s.
- Ba tệp đầu ra gồm: index.html (0.76 kB, gzip 0.48 kB), tệp định dạng CSS (65.86 kB, gzip 10.62 kB) (65.86 kB, gzip 10.62 kB) và gói JavaScript duy nhất gói JavaScript chính (index.js) nặng **1,214.90 kB** (318.76 kB sau khi nén gzip), vượt ngưỡng cảnh báo 500 kB của Vite gấp 2.43 lần (BUG-012).

5.3.1. Ảnh chụp kết quả công cụ kiểm thử (Hình 5.1 → Hình 5.10)

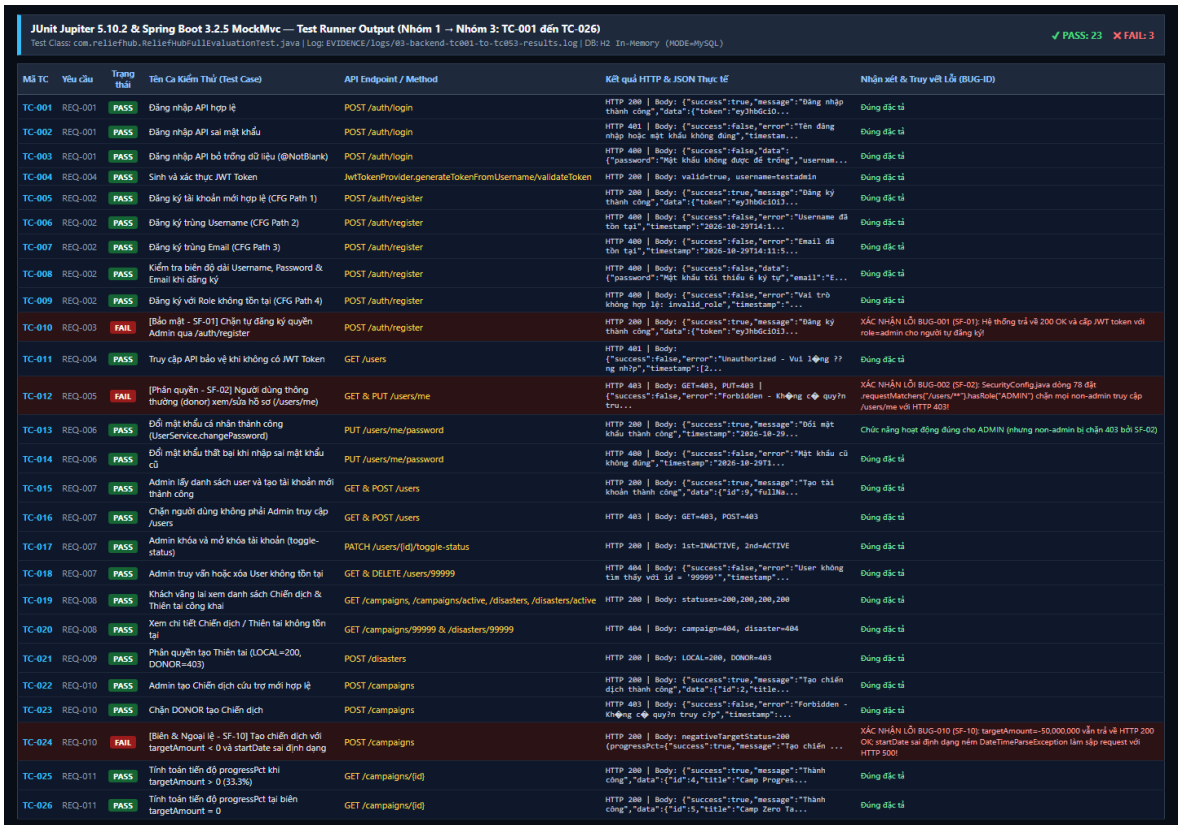
Để cung cấp bằng chứng trực quan nguyên bản (không gắn nhãn đè lên vùng dữ liệu gốc của công cụ), thể hiện rõ tên công cụ thực thi, tên lớp kiểm thử, trạng thái PASS/FAIL và thông số đo lường cụ thể qua các lần thực thi, 10 ảnh chụp màn hình kết quả công cụ tương ứng từ **Hình 5.1** đến **Hình 5.10** được trình bày dưới đây:



Hình 5.1: Báo cáo công cụ Maven Surefire 3.1.2 khi chạy 3 lớp kiểm thử gốc có sẵn của dự án (AuthControllerTest, CampaignControllerTest, ReliefHubApplicationTests) - Đạt 10/10 PASS (100%), 0 Failures (Nhật ký thực thi tại Phụ lục B: 02-mvn-test-existing.log).



Hình 5.2: Báo cáo công cụ Maven Surefire 3.1.2 trên lớp com.reliefhub.ReliefHubFullEvaluationTest - Hiển thị danh sách 14 phương thức kiểm thử FAIL cùng dòng mã nguồn và thông báo AssertionFailedError tương ứng với các lỗi bảo mật và nghiệp vụ Backend (BUG-001 → BUG-005, BUG-008 → BUG-010; Nhật ký thực thi tại Phụ lục B: 03-mvn-test-full-suite.log).



Hình 5.3: Kết quả thực thi chi tiết 26 ca kiểm thử TC-001 → TC-026 thuộc Nhóm 1, 2, 3 trên JUnit Jupiter 5.10.2 & Spring Boot 3.2.5 MockMvc

(ReliefHubFullEvaluationTest.java) - Thể hiện rõ trạng thái 23 PASS và 3 FAIL (TC-010, TC-012, TC-024) cùng mã phản hồi HTTP thực tế (Nhật ký thực thi tại Phụ lục B: 03-backend-tc001-to-tc053-results.log).



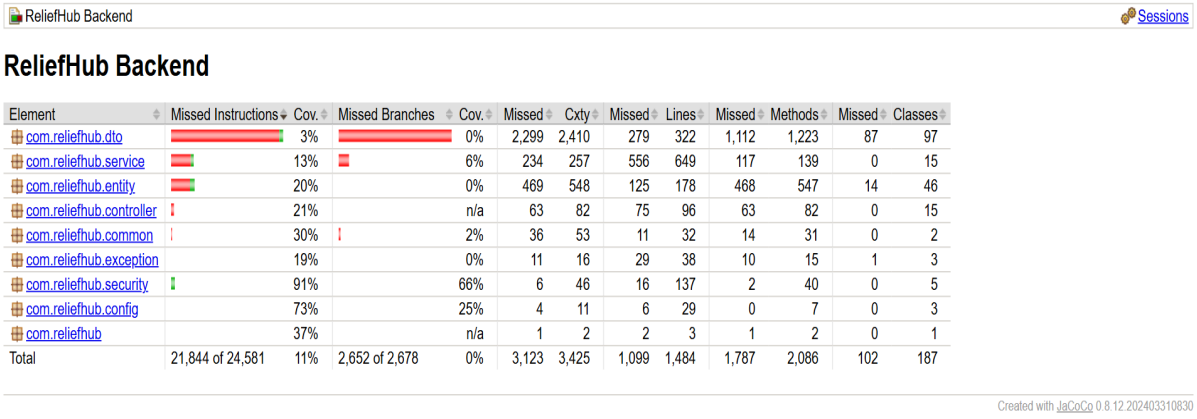
JUnit Jupiter 5.10.2 & Spring Boot 3.2.5 MockMvc — Test Runner Output (Nhóm 4 → Nhóm 7: TC-027 đến TC-053)

Test Class: com.reliefhub.ReliefHubFullEvaluationTest.java | Log: EVIDENCE/logs/03-backend-tc001-to-tc053-results.log | DB: H2 In-Memory (MODE=MySQL)

✓ PASS: 16 ✗ FAIL: 11

Mã TC	Yêu cầu	Trạng thái	Tên Ca Kiểm Thử (Test Case)	API Endpoint / Method	Kết quả HTTP & JSON Thực tế	Nhận xét & Truy vết Lỗi (BUG-ID)
TC-027	REQ-012	PASS	Tạo khoản quyên góp hợp lệ kèm danh sách hiện vật (items)	POST /donations	HTTP 200 | Body: {"success":true,"message":"Đã ghi nhận đóng góp","data":{"id":1,"userId":2,...	Đúng đặc tả
TC-028	REQ-012	PASS	Tạo khoản quyên góp với số tiền biên amount = 0 hoặc amount < 0	POST /donations	HTTP 400 | Body: amount=0 -> 400, amount=-1000 -> 400	Đúng đặc tả
TC-029	REQ-014	PASS	Kiểm tra hiển thị tên nhà tài trợ khi quyên góp ẩn danh (isAnonymous=true)	POST /donations	HTTP 200 | Body: {"success":true,"message":"Đã ghi nhận đóng góp","data":{"id":2,"userId":2,...	Đúng đặc tả
TC-030	REQ-013	PASS	Xác minh quyên góp (CONFIRMED) và cộng tiền vào quỹ chiến dịch	PATCH /donations/{id}/status	HTTP 200 | Body: campaign.currentAmount=5000000.00	Đúng đặc tả
TC-031	REQ-013	FAIL	[Logic tài chính - SF-05] Kiểm tra tính bất biến khi gọi CONFIRMED lặp lại lần 2	PATCH /donations/{id}/status	HTTP 200 | Body: campaign.currentAmount=10000000.00	XÁC NHẬN LỖI BUG-005 (SF-05): Gọi CONFIRMED lần 2 trên cùng Donation làm campaign.currentAmount bị cộng nhầm đôi lên 10000000.00!
TC-032	REQ-013	FAIL	[Bảo mật - SF-03] Chặn DONOR tự gọi API duyệt quyên góp của chính mình	PATCH /donations/{id}/status	HTTP 200 | Body: {"success":true,"message":"Thành công","data":{"id":3,"userId":2,"donorName...	XÁC NHẬN LỖI BUG-003 (SF-03): DonationController thiếu @PreAuthorize nên DONOR tự duyệt khoản quyên góp sang CONFIRMED với HTTP 200 OK!
TC-033	REQ-015	PASS	Tạo bút toán sổ cái hợp lệ và kiểm tra biên amount <= 0	POST /ledger	HTTP 200 | Body: valid=200, negative=400	Đúng đặc tả
TC-034	REQ-015	FAIL	[Bảo mật - SF-03] Chặn CITIZEN tạo hoặc xóa bút toán sổ cái tài chính (/ledger)	POST & DELETE /ledger	HTTP 200 | Body: POST=200, DELETE=200	XÁC NHẬN LỖI BUG-003 (SF-03): LedgerController không phân quyền nên CITIZEN tự ghi và xóa bút toán sổ cái với HTTP 200 OK!
TC-035	REQ-016	PASS	Người dân (CITIZEN) gửi yêu cầu hỗ trợ hợp lệ	POST /support-requests	HTTP 200 | Body: {"success":true,"message":"Đã gửi yêu cầu","data":{"id":1,"requestCode":"RE...	Đúng đặc tả
TC-036	REQ-016	FAIL	[Biên & Validation - SF-10] Gửi yêu cầu hỗ trợ với itemName rỗng hoặc quantity < 0	POST /support-requests	HTTP 200 | Body: blank=400, negativeQty=200 | {"success":true,"message":"Đã gửi yêu cầu","da...	XÁC NHẬN LỖI BUG-010 (SF-10): SupportRequestDTO thiếu @Positive trên quantity và householdCount nên chấp nhận quantity=-20, householdCount=-1 với HTTP 200 OK!
TC-037	REQ-017	PASS	Chuyển trạng thái yêu cầu hỗ trợ tuần tự qua đúng 6 bước hợp lệ	PATCH /support-requests/{id}/advance	HTTP 200 | Body: VERIFIED:200 APPROVED:200 WAREHOUSE_READY:200 IN_TRANSIT:200 DELIVERED:200	Đúng đặc tả
TC-038	REQ-017	FAIL	[Chuyển trạng thái - SF-04] Chặn nhảy cóc trạng thái từ PENDING thẳng sang DELIVERED	PATCH /support-requests/{id}/advance	HTTP 200 | Body: {"success":true,"message":"Thành công","data":{"id":3,"requestCode":"REQ-17...	XÁC NHẬN LỖI BUG-004 (SF-04): SupportRequestService.advanceStatus() không kiểm tra máy trạng thái, cho phép nhảy từ PENDING thẳng sang DELIVERED với HTTP 200 OK!
TC-039	REQ-017	FAIL	[Bảo mật - SF-03] Chặn CITIZEN tự phê duyệt yêu cầu hỗ trợ của chính mình	PATCH /support-requests/{id}/advance	HTTP 200 | Body: {"success":true,"message":"Thành công","data":{"id":4,"requestCode":"REQ-17...	XÁC NHẬN LỖI BUG-003 (SF-03): SupportRequestController thiếu @PreAuthorize nên CITIZEN tự phê duyệt yêu cầu của mình sang APPROVED với HTTP 200 OK!
TC-040	REQ-018	PASS	Tra cứu yêu cầu hỗ trợ cá nhân (/me) và lọc theo trạng thái (/status/{status})	GET /support-requests/me & /status/DELIVERED	HTTP 200 | Body: me=200, status=200	Đúng đặc tả
TC-041	REQ-019	PASS	Tạo Nhà kho (Warehouse) và thêm Mặt hàng tồn kho (Inventory)	POST /warehouses & POST /inventory	HTTP 200 | Body: {"success":true,"message":"Đã thêm mặt hàng","data":{"id":1,"itemName":"Gạo...	Đúng đặc tả
TC-042	REQ-020	PASS	Nhập thêm hàng vào kho (restock) với số lượng >0 và biên <=0	PATCH /inventory/{id}/restock	HTTP 200 | Body: valid(200)->qty=700, zero->400, neg->400	Đúng đặc tả
TC-043	REQ-019	FAIL	[Bảo mật - SF-03] Chặn CITIZEN xóa mặt hàng tồn kho (DELETE /inventory/{id})	DELETE /inventory/{id}	HTTP 200 | Body: {"success":true,"message":"Đã xóa mặt hàng","timestamp":"2026-10-29T14:11:5...	XÁC NHẬN LỖI BUG-003 (SF-03): InventoryController thiếu @PreAuthorize nên CITIZEN xóa mặt hàng kho thành công với HTTP 200 OK!
TC-044	REQ-021	PASS	Tạo chuyến giao hàng (Delivery) và cập nhật DELIVERED tự động ghi arriveAt	POST /deliveries & PATCH /deliveries/{id}/status	HTTP 200 | Body: {"success":true,"message":"Thành công","data":{"id":1,"deliveryCode":"DLV-1...	Đúng đặc tả
TC-045	REQ-022	PASS	Tạo và lọc nhóm tình nguyện viên đang hoạt động (/volunteer-groups)	POST /volunteer-groups & GET /volunteer-groups/active	HTTP 200 | Body: {"success":true,"message":"Thành công","data":[{"id":1,"name":"Đội Cứu hộ 1...	Đúng đặc tả
TC-046	REQ-019, REQ-021	PASS	Truy vấn/Xóa mặt hàng kho hoặc chuyến giao hàng không tồn tại	GET /inventory/99999 & DELETE /deliveries/99999	HTTP 404 | Body: inventory=404, delivery=404	Đúng đặc tả
TC-047	REQ-023	PASS	Vòng đời xử lý Khiếu nại (PENDING -> REPLIED -> CLOSED)	POST /complaints, PUT /reply, PATCH /close	HTTP 200 | Body: {"success":true,"message":"Thành công","data":{"id":1,"complaintCode":"CMP-...	Đúng đặc tả
TC-048	REQ-023	FAIL	[Bảo mật & Validation - SF-03/SF-10] Chặn non-admin trả lời khiếu nại và kiểm tra @NotBlank	PUT /complaints/{id}/reply	HTTP 200 | Body: citizenStatus=200, blankReplyStatus=200	XÁC NHẬN LỖI BUG-003 (SF-03) & BUG-010 (SF-10): CITIZEN tự trả lời khiếu nại với HTTP 200 OK, và adminReply rỗng vẫn được chấp nhận (HTTP 200) do thiếu @Valid!
TC-049	REQ-024	FAIL	[Bảo mật IDOR - SF-09] Chặn người dùng đánh dấu đã đọc thông báo của người khác	PATCH /notifications/{id}/read	HTTP 200 | Body: status=200, adminNotif.isRead=true	XÁC NHẬN LỖI BUG-009 (SF-09): NotificationService.markAsRead(id) không kiểm tra chủ sở hữu, cho phép DONOR đánh dấu đã đọc thông báo của ADMIN (HTTP 200 OK)!
TC-050	REQ-025	PASS	Phân quyền và tính đúng đắn của Thống kê Dashboard (/dashboard/stats)	GET /dashboard/stats	HTTP 200 | Body: ADMIN=200, FINANCE=200, CITIZEN=403 | {"success":true,"message":"Thành công...	Đúng đặc tả
TC-051	REQ-026	PASS	Kiểm tra biên kích thước file tải lên (0B -> 400, 1KB -> 200, 11MB -> 400)	POST /uploads	HTTP 200 | Body: 0B=400, 1KB=200, 11MB=400	Đúng đặc tả
TC-052	REQ-026	FAIL	[Bảo mật Upload - SF-08] Kiểm tra chặn tải lên file thực thi/script độc hại (.html/.jsp)	POST /uploads	HTTP 200 | Body: {"success":true,"message":"Upload thành công","data":{"fileName":"48b7b730-...	XÁC NHẬN LỖI BUG-008 (SF-08): FileUploadService.upload() không kiểm tra định dạng/MIME type, cho phép upload malicious.html thành công với HTTP 200 OK!
TC-053	REQ-027	FAIL	[Xử lý ngoại lệ - SF-10] Gửi trạng thái không hợp lệ hoặc null vào endpoint cập nhật	PATCH /support-requests/{id}/advance	HTTP 500 | Body: INVALID_STATE=400, nullStatus=500 | {"success":false,"error":"Lỗi hệ thống:...	XÁC NHẬN LỖI BUG-010 (SF-10): Gửi body {} (newStatus=null) gây NullPointerException và trả về HTTP 500 Internal Server Error do thiếu @Valid trên @RequestBody!

Hình 5.4: Kết quả thực thi chi tiết 27 ca kiểm thử TC-027 → TC-053 thuộc Nhóm 4, 5, 6, 7 trên JUnit Jupiter 5.10.2 & Spring Boot 3.2.5 MockMvc (ReliefHubFullEvaluationTest.java) - Thể hiện rõ trạng thái 16 PASS và 11 FAIL (TC-031, 032, 034, 036, 038, 039, 043, 048, 049, 052, 053), tổng hợp toàn bộ 53 TC Backend đạt đúng 39 PASS / 14 FAIL (Nhật ký thực thi tại Phụ lục B: 03-backend-tc001-to-tc053-results.log).



ReliefHub Backend

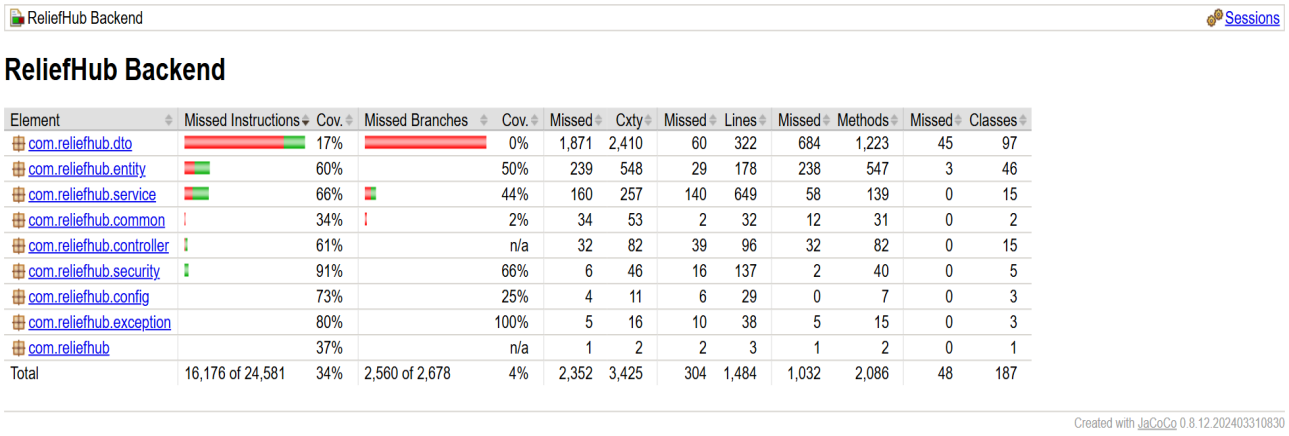
Sessions

ReliefHub Backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.reliefhub.dto		3%		0%	2,299	2,410	279	322	1,112	1,223	87	97
com.reliefhub.service		13%		6%	234	257	556	649	117	139	0	15
com.reliefhub.entity		20%		0%	469	548	125	178	468	547	14	46
com.reliefhub.controller		21%		n/a	63	82	75	96	63	82	0	15
com.reliefhub.common		30%		2%	36	53	11	32	14	31	0	2
com.reliefhub.exception		19%		0%	11	16	29	38	10	15	1	3
com.reliefhub.security		91%		66%	6	46	16	137	2	40	0	5
com.reliefhub.config		73%		25%	4	11	6	29	0	7	0	3
com.reliefhub		37%		n/a	1	2	2	3	1	2	0	1
Total	21,844 of 24,581	11%	2,652 of 2,678	0%	3,123	3,425	1,099	1,484	1,787	2,086	102	187

Created with JaCoCo 0.8.12.202403310830

Hình 5.5: Báo cáo công cụ JaCoCo 0.8.12 [7] khi chỉ chạy 10 bài test gốc - Độ bao phủ dòng lệnh (Line Coverage) toàn dự án đạt 25.53% (394 / 1,543 dòng theo jacoco.csv; tương đương 385 / 1,484 = 25.94% dòng tệp nguồn trên index.html) và gói com.reliefhub.service đạt 14.33% (93 / 649 dòng).



ReliefHub Backend

Sessions

ReliefHub Backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.reliefhub.dto		17%		0%	1,871	2,410	60	322	684	1,223	45	97
com.reliefhub.entity		60%		50%	239	548	29	178	238	547	3	46
com.reliefhub.service		66%		44%	160	257	140	649	58	139	0	15
com.reliefhub.common		34%		2%	34	53	2	32	12	31	0	2
com.reliefhub.controller		61%		n/a	32	82	39	96	32	82	0	15
com.reliefhub.security		91%		66%	6	46	16	137	2	40	0	5
com.reliefhub.config		73%		25%	4	11	6	29	0	7	0	3
com.reliefhub.exception		80%		100%	5	16	10	38	5	15	0	3
com.reliefhub		37%		n/a	1	2	2	3	1	2	0	1
Total	16,176 of 24,581	34%	2,560 of 2,678	4%	2,352	3,425	304	1,484	1,032	2,086	48	187

Created with JaCoCo 0.8.12.202403310830

Hình 5.6: Báo cáo công cụ JaCoCo 0.8.12 [7] sau khi thực thi bộ kiểm thử mở rộng (TC-001 → TC-053) - Độ bao phủ dòng lệnh toàn dự án đạt 78.61% (1,213 / 1,543 dòng theo jacoco.csv; tương đương 1,180 / 1,484 = 79.51% dòng tệp nguồn trên index.html, gói service đạt 509 / 649 = 78.43% Line và 104 / 236 = 44.07% Branch).

ReliefHub Backend > com.reliefhub.service

Source Files Sessions

com.reliefhub.service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
CampaignService		57%		34%	21	30	17	63	4	11	0	1
UserService		61%		44%	14	26	19	69	7	17	0	1
DisasterService		50%		25%	18	25	21	54	4	11	0	1
VolunteerGroupService		41%		25%	15	19	14	31	7	11	0	1
InventoryService		57%		6%	13	21	11	37	5	13	0	1
DeliveryService		65%		53%	17	24	5	49	3	9	0	1
NotificationService		23%		0%	8	10	21	27	7	9	0	1
DonationService		78%		63%	17	28	10	72	6	13	0	1
SupportRequestService		78%		60%	12	21	6	64	4	11	0	1
ComplaintService		76%		62%	8	15	7	54	5	11	0	1
LedgerService		74%		50%	8	12	2	23	3	7	0	1
WarehouseService		61%		50%	5	8	4	14	3	6	0	1
FileUploadService		83%		70%	3	8	3	22	0	3	0	1
AuthService		99%		83%	1	8	0	42	0	5	0	1
DashboardService		100%		n/a	0	2	0	28	0	2	0	1
Total	1,081 of 3,254	66%	132 of 236	44%	160	257	140	649	58	139	0	15

Created with JaCoCo 0.8.12.202403310830

Hình 5.7: Báo cáo JaCoCo 0.8.12 phân rã chi tiết 15 lớp trong gói com.reliefhub.service - Đạt 78.43% Line Coverage (509 / 649 dòng), 44.07% Branch Coverage (104 / 236 nhánh; FileUploadService đạt 7 / 10 = 70.00%) và 58.27% Method Coverage (81 / 139 phương thức).

ReliefHub Backend > com.reliefhub.controller

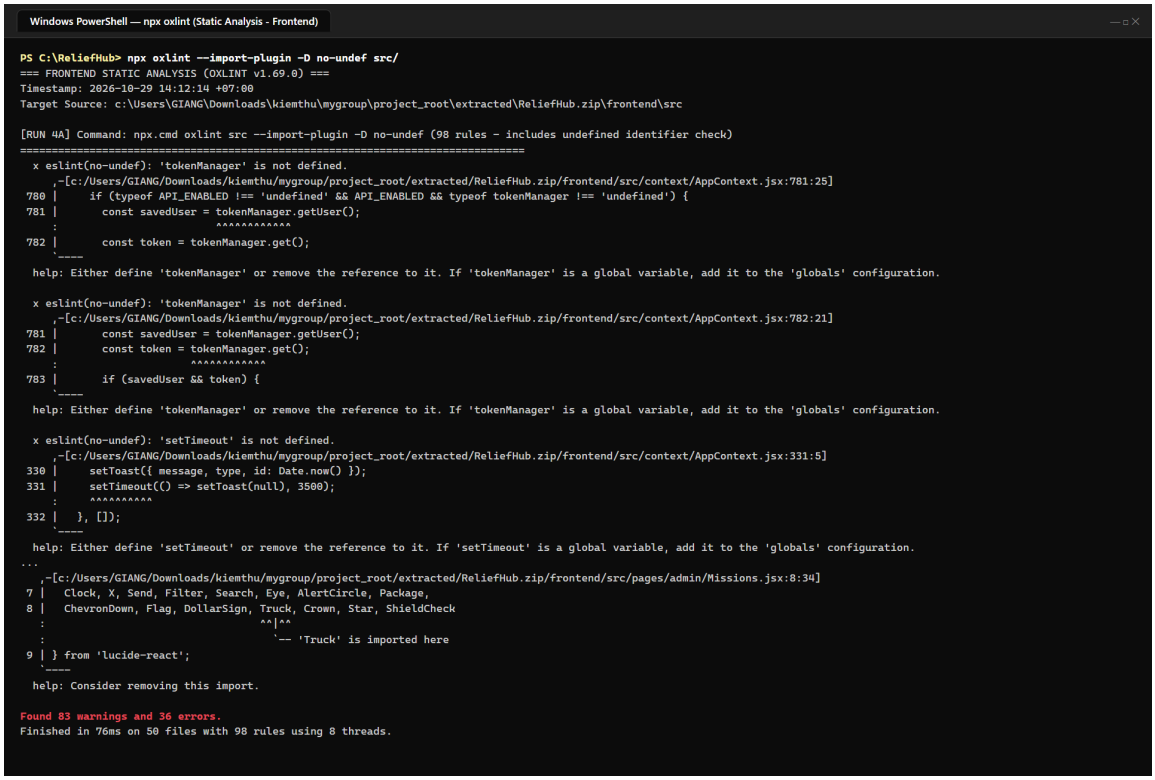
Source Files Sessions

com.reliefhub.controller

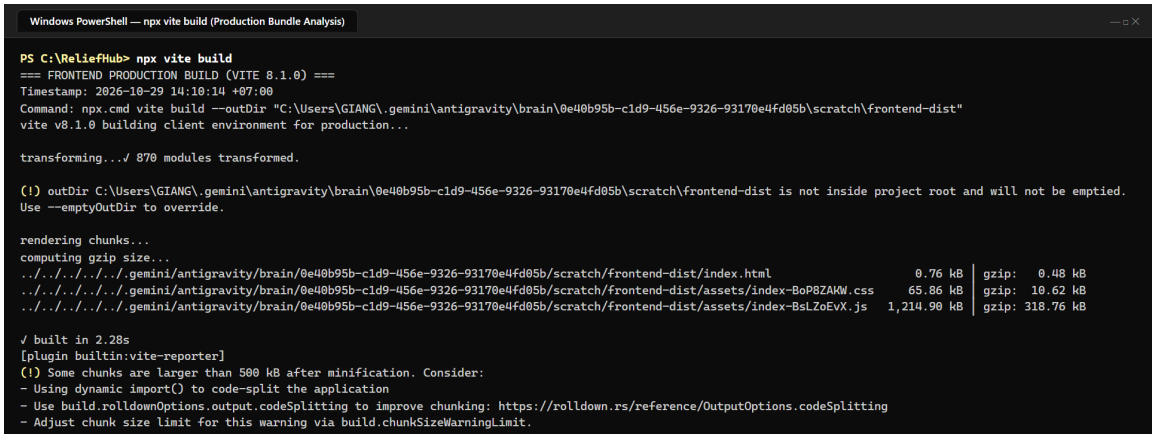
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
VolunteerGroupController		40%		n/a	4	7	5	8	4	7	0	1
UserController		55%		n/a	4	9	5	12	4	9	0	1
DisasterController		52%		n/a	3	7	4	8	3	7	0	1
InventoryController		58%		n/a	3	7	3	8	3	7	0	1
DeliveryController		51%		n/a	3	6	4	7	3	6	0	1
DonationController		53%		n/a	3	6	3	6	3	6	0	1
NotificationController		42%		n/a	3	5	4	7	3	5	0	1
CampaignController		66%		n/a	2	7	3	8	2	7	0	1
WarehouseController		50%		n/a	2	4	3	5	2	4	0	1
LedgerController		62%		n/a	2	5	2	6	2	5	0	1
ComplaintController		70%		n/a	2	6	2	6	2	6	0	1
SupportRequestController		85%		n/a	1	6	1	6	1	6	0	1
AuthController		100%		n/a	0	3	0	5	0	3	0	1
FileUploadController		100%		n/a	0	2	0	2	0	2	0	1
DashboardController		100%		n/a	0	2	0	2	0	2	0	1
Total	226 of 582	61%	0 of 0	n/a	32	82	39	96	32	82	0	15

Created with JaCoCo 0.8.12.202403310830

Hình 5.8: Báo cáo JaCoCo 0.8.12 phân rã chi tiết 15 lớp trong gói com.reliefhub.controller - Đạt 59.38% Line Coverage (57 / 96 dòng) và 60.98% Method Coverage (50 / 82 phương thức).



Hình 5.9: Kết quả chạy công cụ Oxlint 1.69.0 [9] trên 50 tệp mã nguồn Frontend - Phát hiện 36 errors khi bật quy tắc -D no-undef (trong đó có 5 lỗi biến nghiệp vụ chưa khai báo tại AppContext.jsx gây lỗi BUG-006) và 83 warnings (no-unused-vars, lỗi BUG-012; Nhật ký thực thi tại Phụ lục C: 04-frontend-oxlint.log).



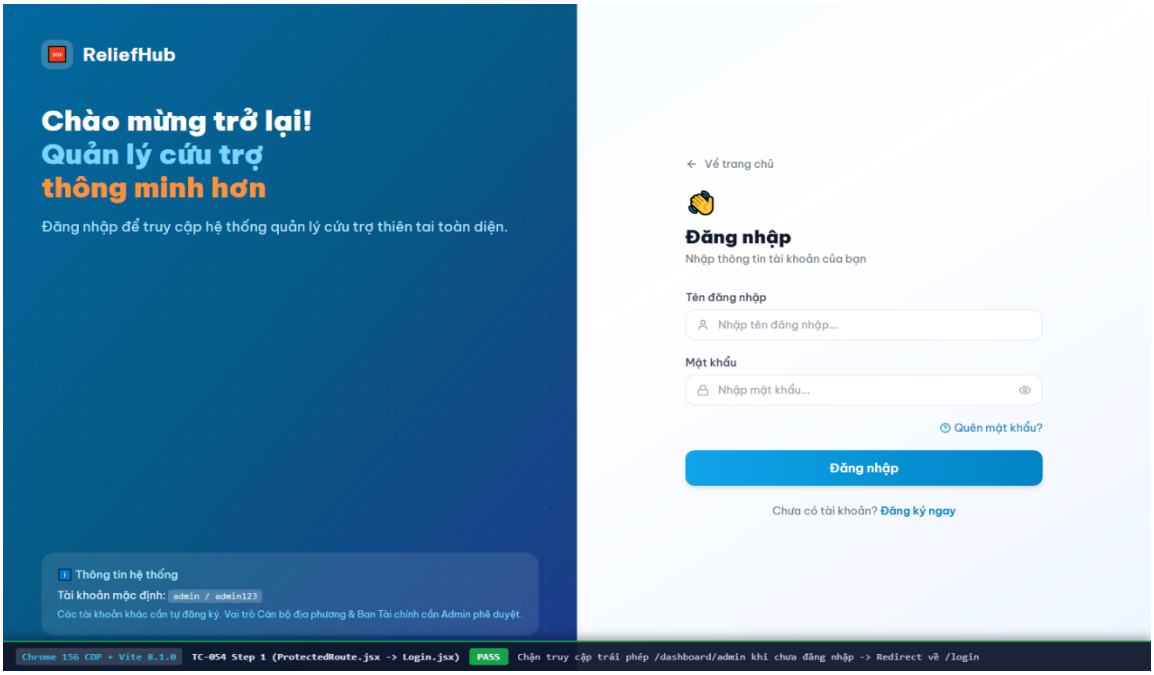
Hình 5.10: Kết quả chạy npx vite build (Vite 8.1.0 [9]) - Biên dịch thành công 870 modules nhưng phát cảnh báo gói JavaScript chính nặng 1,214.90 kB (vượt 2.43 lần ngưỡng khuyến nghị 500 kB, lỗi BUG-012; Nhật ký thực thi tại Phụ lục C: 05-frontend-vite-build.log).

5.4. Kết quả kiểm thử giao diện WebUI (TC-054 → TC-060)

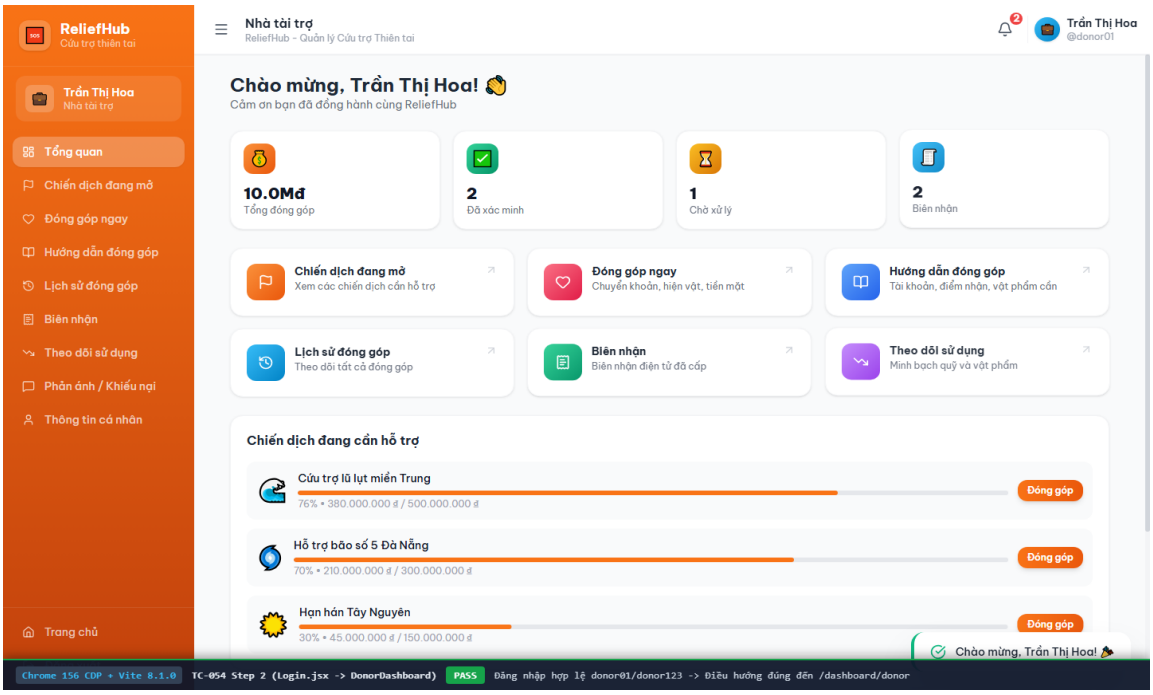
Toàn bộ **13 ảnh chụp màn hình bằng chứng trực quan** (Hình 5.11 → Hình 5.23, độ phân giải 1440×900 px) thu được trên trình duyệt Google Chrome 156.0.8078.73 qua

giao thức CDP cho đầy đủ 7 kịch bản TC-054 → TC-060 (3 PASS, 4 FAIL) được trình bày dưới đây và đối chiếu trực tiếp với nhật ký thực thi 06-frontend-webui-execution.log tại Phụ lục C (Mục C.3).

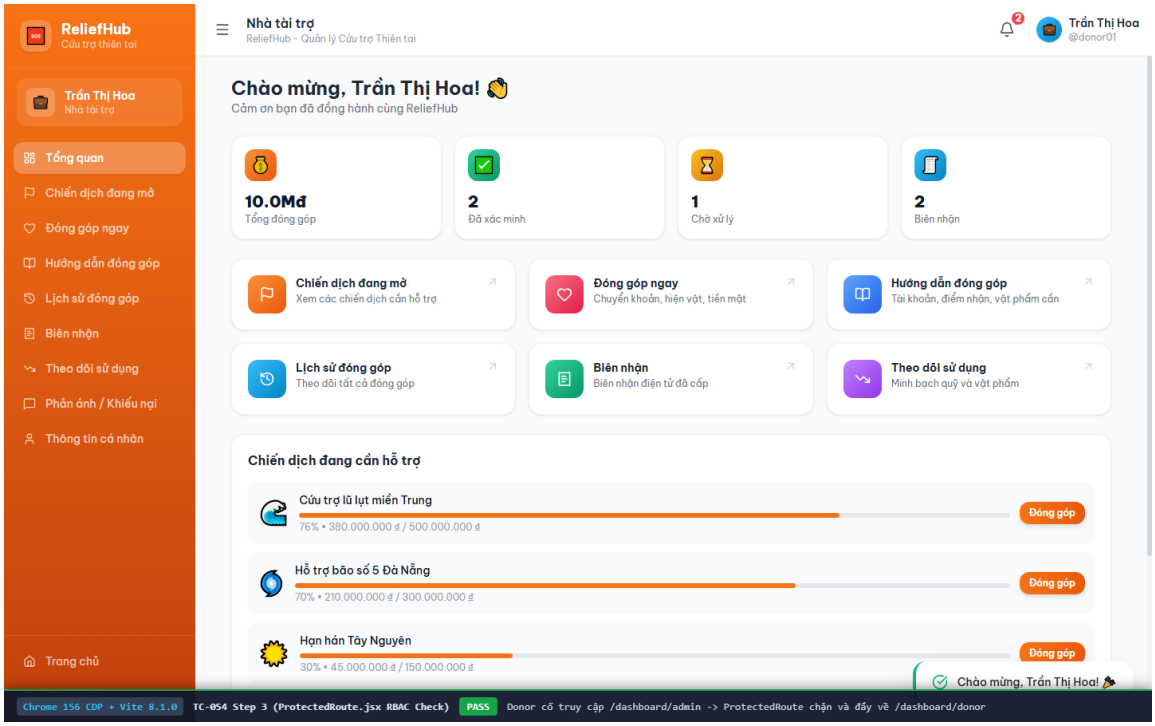
5.4.1. TC-054 (PASS): Đăng nhập và bảo vệ đường dẫn ProtectedRoute



Hình 5.11: Kết quả thực thi TC-054 (Bước 1 - PASS): Truy cập /dashboard/admin khi chưa đăng nhập bị ProtectedRoute chặn và tự động chuyển hướng về màn hình /login.

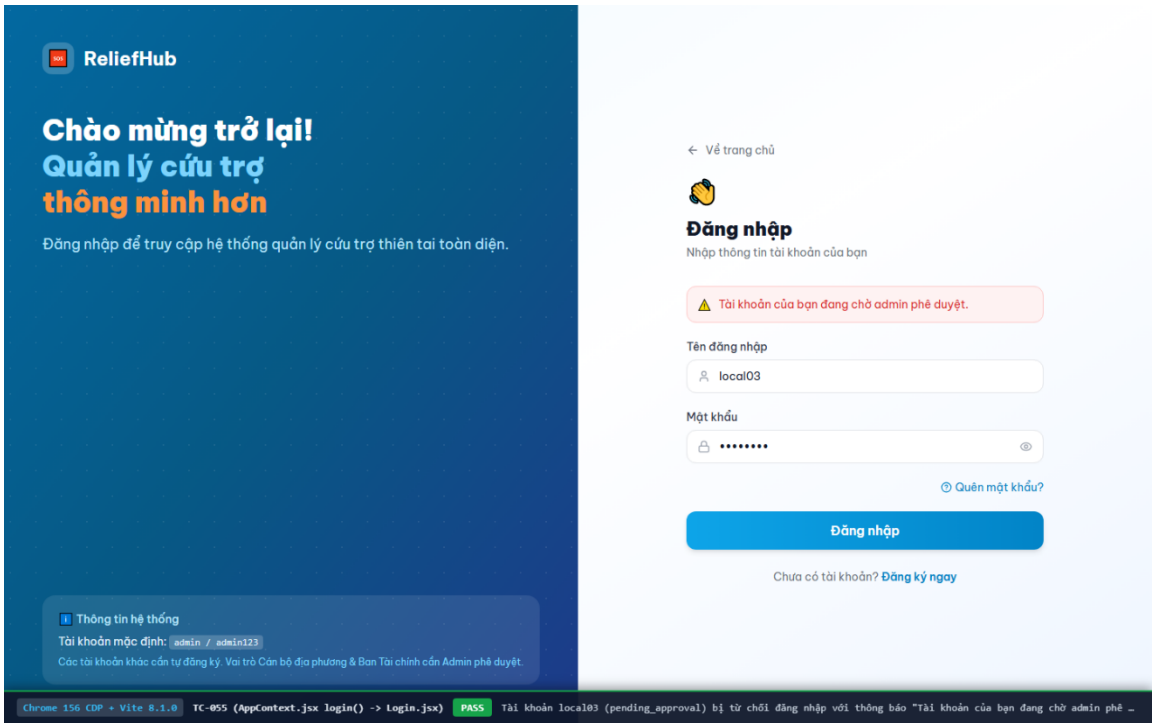


Hình 5.12: Kết quả thực thi TC-054 (Bước 2 - PASS): Giao diện Dashboard Nhà tài trợ (/dashboard/donor) sau khi đăng nhập hợp lệ bằng tài khoản donor01 / donor123.



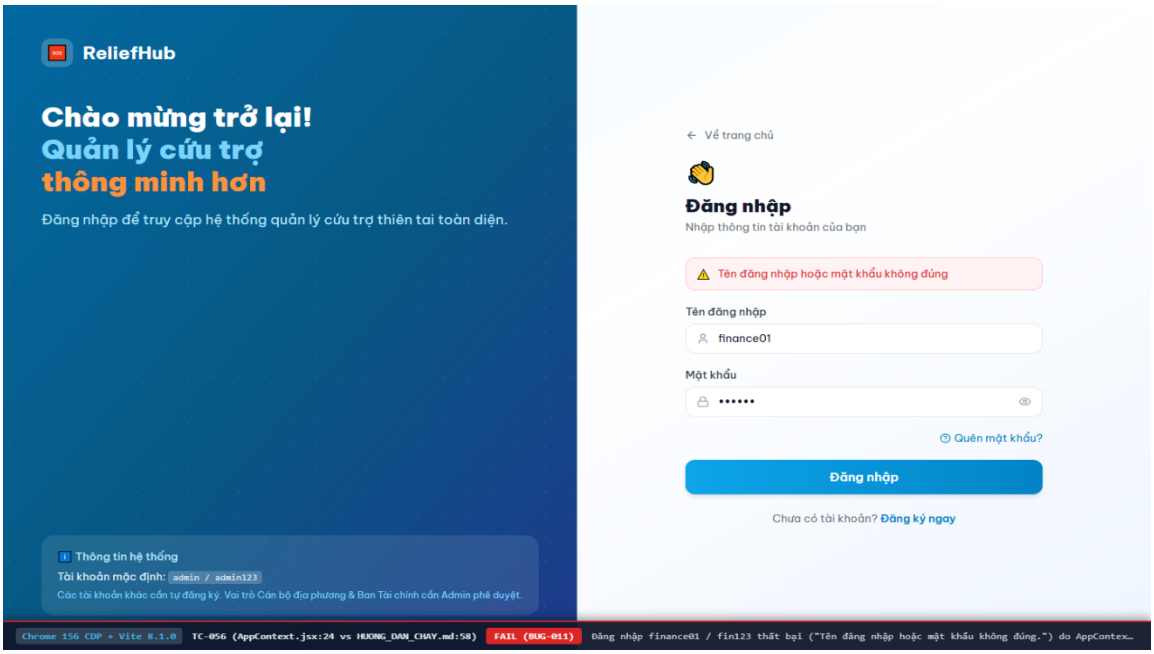
Hình 5.13: Kết quả thực thi TC-054 (Bước 3 - PASS): Khi donor01 điều hướng sang /dashboard/admin, ProtectedRoute chặn lại và chuyển về /dashboard/donor.

5.4.2. TC-055 (PASS): Chặn tài khoản đang chờ phê duyệt



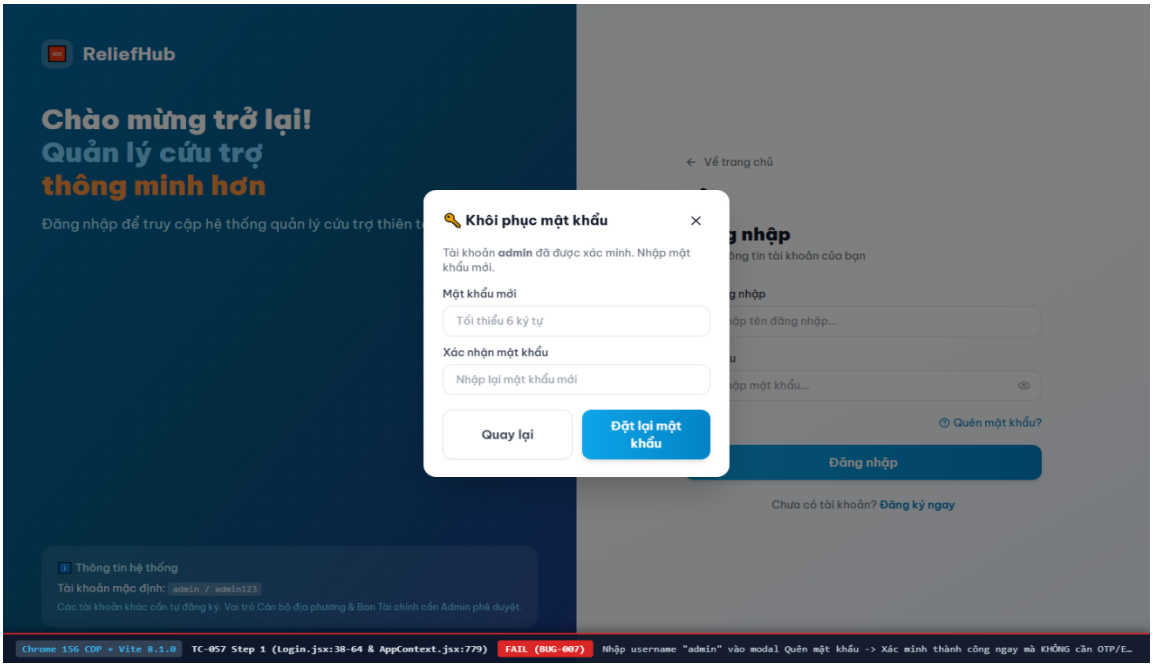
Hình 5.14: Kết quả thực thi TC-055 (PASS): Đăng nhập tài khoản Cán bộ địa phương local03 / local123 (status = pending_approval) bị từ chối kèm thông báo "Tài khoản của bạn đang chờ admin phê duyệt.".

5.4.3. TC-056 (FAIL - BUG-011): Lệch mật khẩu tài khoản finance01

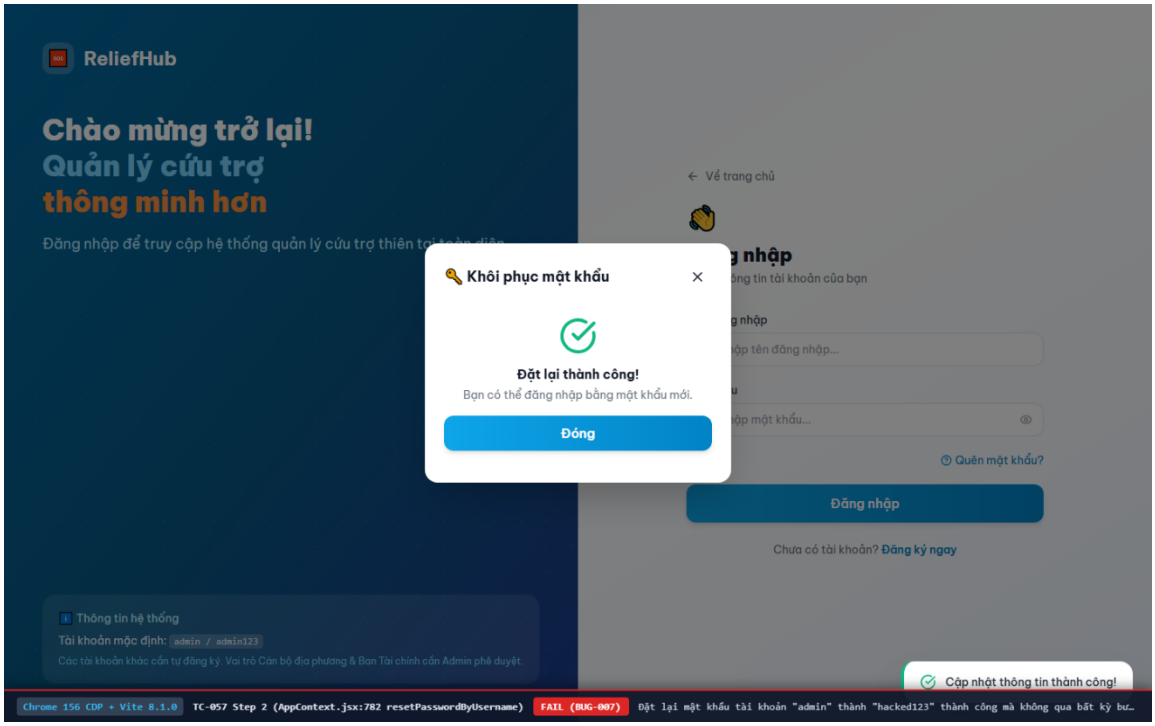


Hình 5.15: Kết quả thực thi TC-056 (FAIL - BUG-011): Đăng nhập tài khoản Ban tài chính finance01 với mật khẩu fin123 (theo quy định trong reliefhub_full.sql và HUONG_DAN_CHAY.md) bị từ chối với lỗi "Tên đăng nhập hoặc mật khẩu không đúng" do AppContext.jsx (dòng 24) gán cứng mật khẩu là finance123.

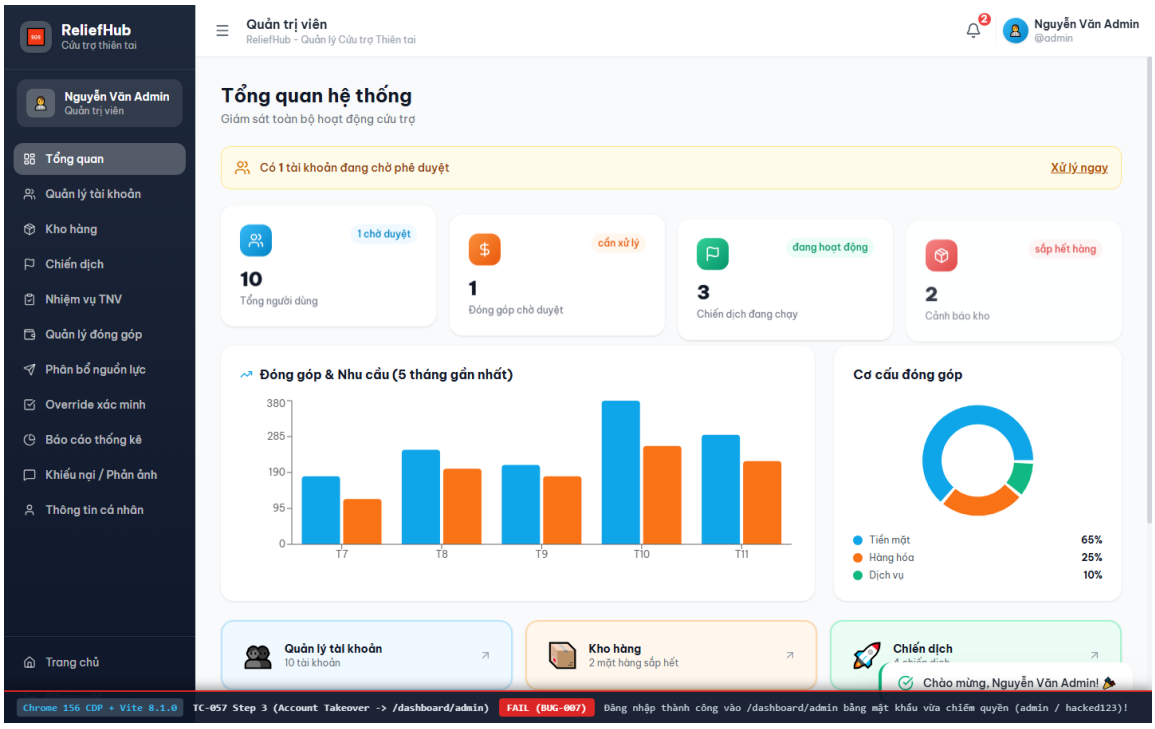
5.4.4. TC-057 (FAIL – BUG-007): Chiếm quyền admin qua Quên mật khẩu



Hình 5.16: Kết quả thực thi TC-057 (Bước 1 - FAIL, BUG-007): Khách vãng lai mở modal "Quên mật khẩu?", nhập tên đăng nhập "admin" và được hệ thống báo "Tài khoản admin đã được xác minh" mà không cần OTP hay Email.

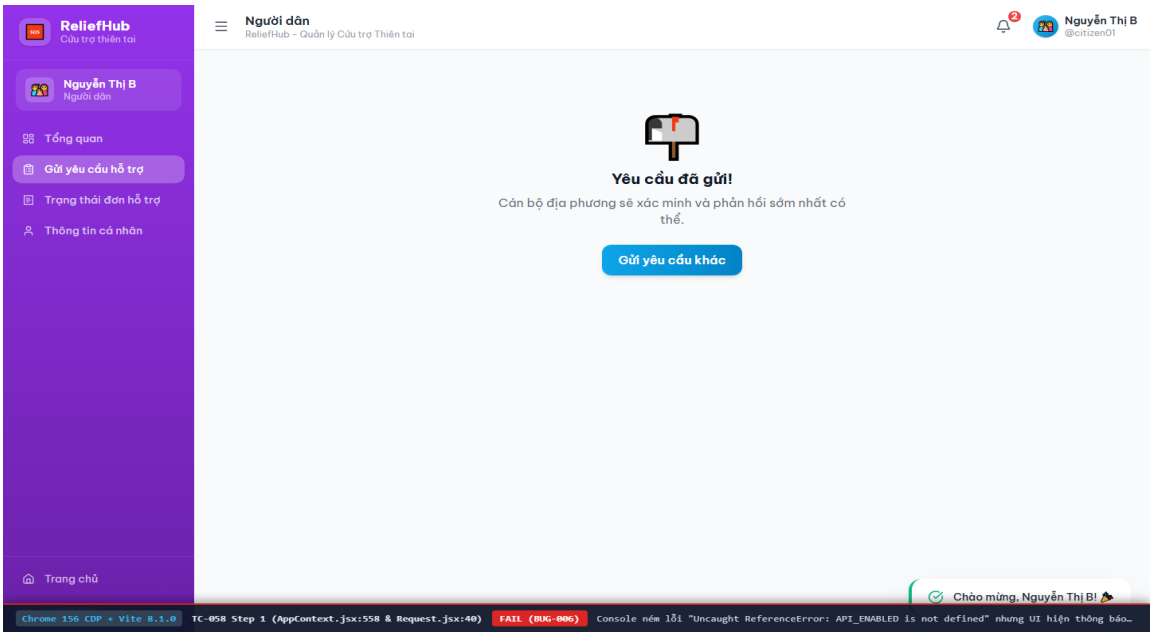


Hình 5.17: Kết quả thực thi TC-057 (Bước 2 - FAIL, BUG-007): Hệ thống cho phép đặt lại mật khẩu của admin thành "hacked123" ("Đặt lại thành công!").

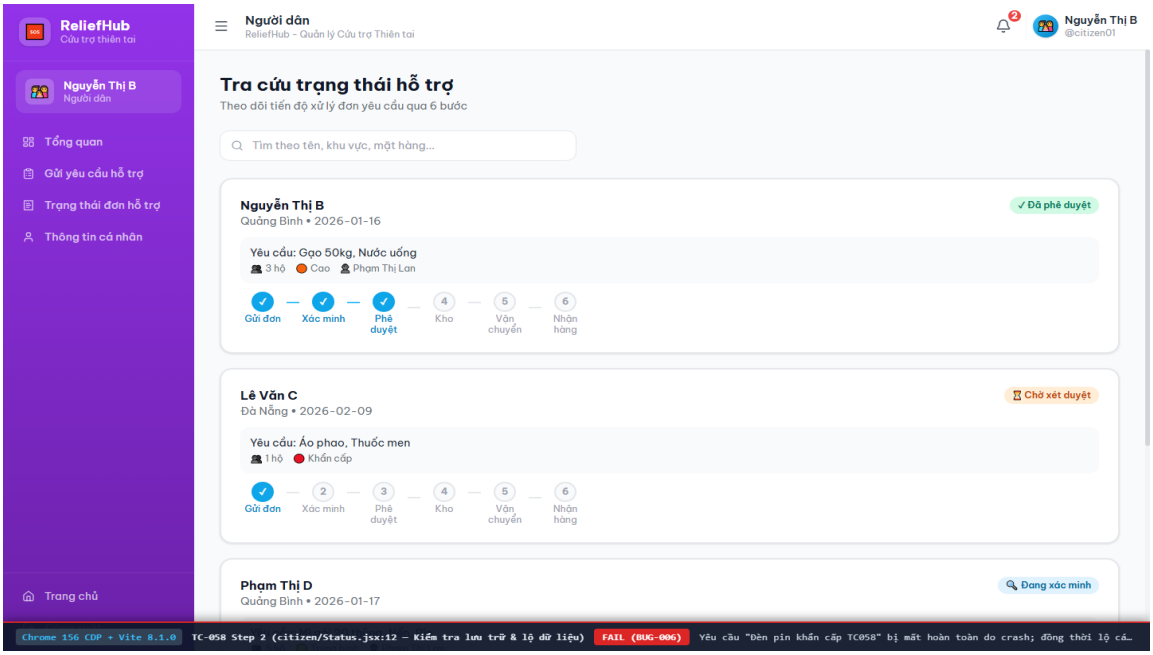


Hình 5.18: Kết quả thực thi TC-057 (Bước 3 - FAIL, BUG-007): Người dùng đăng nhập thành công vào /dashboard/admin bằng mật khẩu mới hacked123 và chiếm toàn quyền Quản trị viên.

5.4.5. TC-058 (FAIL – BUG-006): Mất đơn cứu trợ do ReferenceError

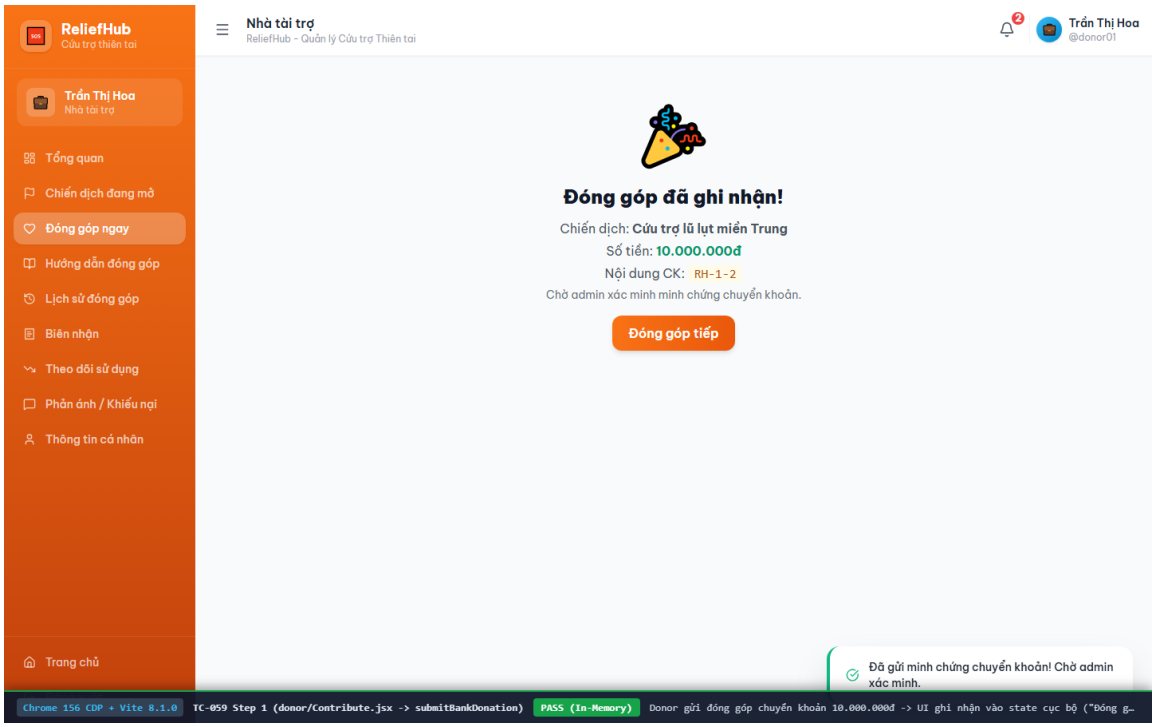


Hình 5.19: Kết quả thực thi TC-058 (Bước 1 - FAIL, BUG-006): Sau khi citizen01 gửi yêu cầu cứu trợ tại Quảng Bình, hàm addNeed trong AppContext.jsx (dòng 558) ném ngoại lệ Uncaught (in promise) ReferenceError: API_ENABLED is not defined (ghi nhận trong nhật ký 06-frontend-webui-execution.log), nhưng giao diện Request.jsx không bắt lỗi mà vẫn hiển thị thông báo giả tạo "Yêu cầu đã gửi!".

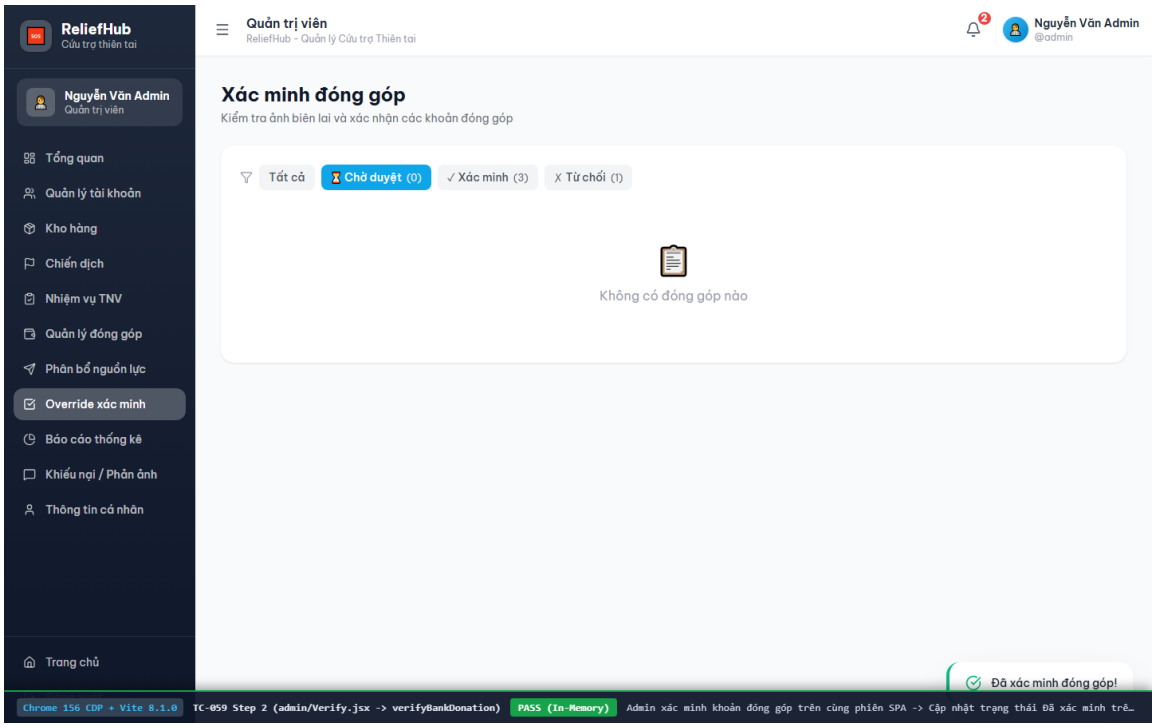


Hình 5.20: Kết quả thực thi TC-058 (Bước 2 - FAIL, BUG-006): Khi mở trang /dashboard/citizen/status, đơn cứu trợ vừa gửi tại Quảng Bình không tồn tại (chỉ còn 3 đơn tĩnh mẫu), đồng thời trang hiển thị cả đơn của công dân khác (Lê Văn C, Phạm Thị D).

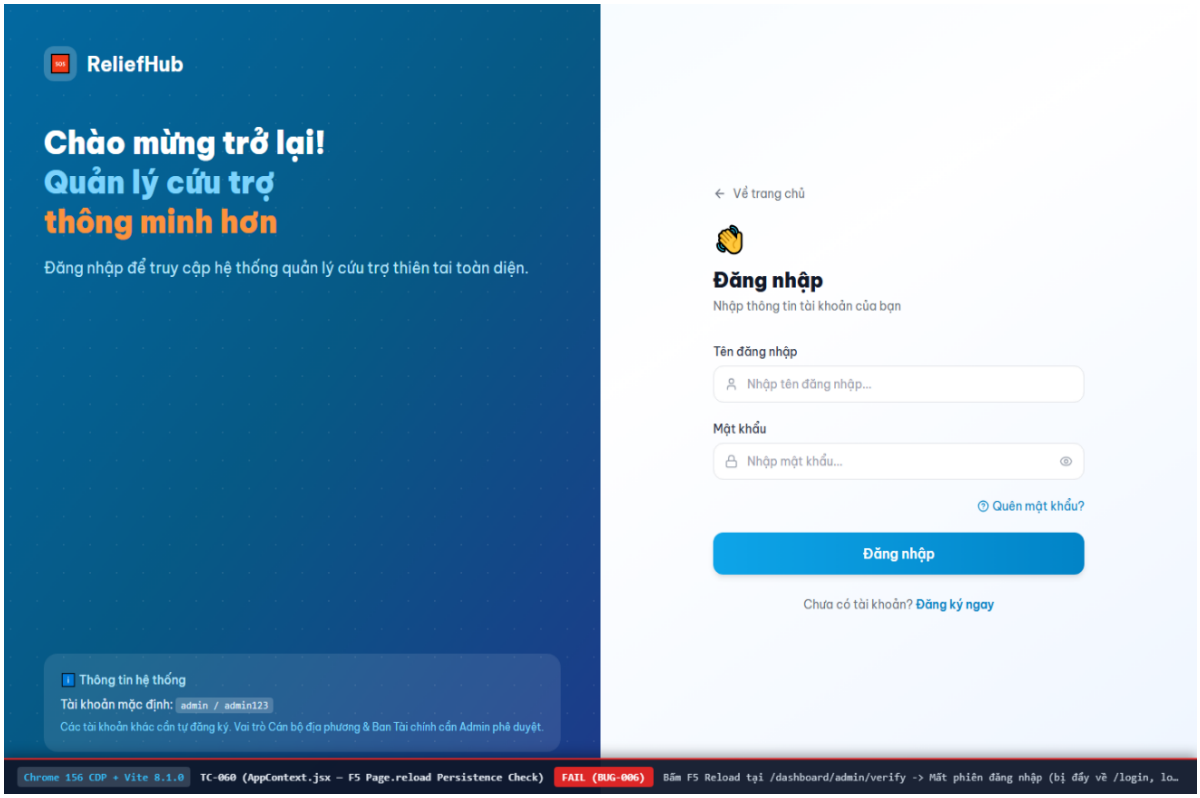
5.4.6. TC-059 (PASS) và TC-060 (FAIL – BUG-006): Mất trạng thái sau F5



Hình 5.21: Kết quả thực thi TC-059 (Bước 1 - PASS): Nhà tài trợ donor01 gửi khoản quyên góp chuyển khoản 10,000,000đ kèm ảnh biên lai thành công.



Hình 5.22: Kết quả thực thi TC-059 (Bước 2 - PASS): Admin xác minh khoản đóng góp tại /dashboard/admin/verify, hệ thống cấp mã biên nhận điện tử và chuyển trạng thái sang "Đã xác minh" trong phiên làm việc hiện tại.



Hình 5.23: Kết quả thực thi TC-060 (FAIL - BUG-006): Sau khi nhấn F5 (Reload) trình duyệt, toàn bộ trạng thái vừa xác minh ở TC-059 bị khôi phục về dữ liệu tĩnh ban đầu (INITIAL_DONATIONS) và localStorage.reliefhub_token = null (Nhật ký thực thi tại Phụ lục C: 06-frontend-webui-execution.log).

5.5. Đối chiếu bằng chứng và các hạng mục chưa kiểm thử động

5.5.1. Đối chiếu 23 ảnh chụp và 11 nhật ký với Test Case

Để đảm bảo tính chặt chẽ khoa học - không gán ép một hình ảnh làm bằng chứng trực tiếp cho một Test Case nếu bản thân hình ảnh đó chưa đủ thông tin chứng minh - **Bảng 5.4** đối chiếu từng hình ảnh bằng chứng (Hình 5.1 → Hình 5.23) và nhật ký thực thi tại Phụ lục B/C với nhóm kiểm thử, mã Test Case và phạm vi chứng minh tương ứng:

Bảng 5.4: Ma trận đối chiếu 23 ảnh chụp bằng chứng (Hình 5.1 → Hình 5.23) và 11 tệp nhật ký thực nghiệm với Nhóm kiểm thử và Test Case

Số hiệu Hình	Nhật ký & Báo cáo đối chiếu	Lần chạy & Công cụ thực thi	Nhóm kiểm thử & Test Case ánh xạ	Kết quả thể hiện trên ảnh	Đánh giá mức độ đủ của bằng chứng (Evidentiary Sufficiency)
Hình 5.1	Nhật ký thực thi tại Phụ lục B (02-mvn-test-existing.log)	**Lần chạy 1** Maven	Bộ 10 unit/integration test gốc có sẵn của dự án	**10 PASS / 0 FAIL**	**Đủ bằng chứng trực tiếp** cho 10 bài test gốc (Baseline); **không dùng** làm

Số hiệu Hình	Nhật ký & Báo cáo đối chiếu	Lần chạy & Công cụ thực thi	Nhóm kiểm thử & Test Case ánh xạ	Kết quả thể hiện trên ảnh	Đánh giá mức độ đủ của bằng chứng (Evidentiary Sufficiency)
		Surefire 3.1.2	(AuthController Test, CampaignControllerTest, ReliefHubApplicationTests)		bằng chứng cho 60 Test Case đánh giá độc lập (TC-001 → TC-060).
Hình 5.2	Nhật ký thực thi tại Phụ lục B (03-mvn-test-full-suite.log)	**Lần chạy 2** Maven Surefire 3.1.2	14 ca kiểm thử Backend thất bại trong ReliefHubFullEvaluationTest.java (TC-010, 012, 024, 031, 032, 034, 036, 038, 039, 043, 048, 049, 052, 053)	**14 FAIL** (BUG-001 → 005, 008 → 010)	**Đủ bằng chứng trực tiếp** cho 14 ca FAIL tại Backend (hiển thị tên hàm test, dòng lệnh và thông điệp AssertionFailedError).
Hình 5.3	Nhật ký thực thi tại Phụ lục B (03-backend-tc001-to-tc053-results.log)	**Lần chạy 2** JUnit Jupiter 5.10.2 & MockMvc	Nhóm 1, 2, 3 Backend (Đơn vị & Tích hợp API): **TC-001 → TC-026** (26 Test Cases: 2 Unit Test TC-004, TC-026 và 24 ca tích hợp API/Service)	**23 PASS / 3 FAIL** (TC-010, 012, 024)	**Đủ bằng chứng trực tiếp** cho từng ca kiểm thử TC-001 → TC-026 (hiển thị mã TC, trạng thái PASS/FAIL và phản hồi HTTP / Unit Return thực tế).
Hình 5.4	Nhật ký thực thi tại Phụ lục B (03-backend-tc001-to-tc053-results.log)	**Lần chạy 2** JUnit Jupiter 5.10.2 & MockMvc	Nhóm 4, 5, 6, 7 Backend (Tích hợp API/Service): **TC-027 → TC-053** (27 Test Cases)	**16 PASS / 11 FAIL**	**Đủ bằng chứng trực tiếp** cho từng ca kiểm thử TC-027 → TC-053 (hiển thị mã TC, trạng thái PASS/FAIL và phản hồi HTTP thực tế).
Hình 5.5	Báo cáo độ bao phủ tại Bảng 5.2, 5.3 & Phụ lục A (02-mvn-test-existing.log)	**Lần chạy 1** JaCoCo 0.8.12	Đo độ bao phủ mã nguồn khi chỉ chạy 10 bài test gốc	25.53% Line Coverage	**Chỉ chứng minh độ bao phủ mã nguồn Baseline**; **không gán** làm bằng chứng PASS/FAIL cho bất

Số hiệu Hình	Nhật ký & Báo cáo đối chiếu	Lần chạy & Công cụ thực thi	Nhóm kiểm thử & Test Case ánh xạ	Kết quả thể hiện trên ảnh	Đánh giá mức độ đủ của bằng chứng (Evidentiary Sufficiency)
					kỳ Test Case đơn lẻ nào.
Hình 5.6	Báo cáo độ bao phủ tại Bảng 5.2 & Phụ lục A (03-mvn-test-full-suite.log)	**Lần chạy 2** JaCoCo 0.8.12	Đo độ bao phủ mã nguồn toàn dự án sau khi chạy TC-001 → TC-053	78.61% Line Coverage	**Chỉ chứng minh độ bao phủ tổng thể dự án**; **không gán** làm bằng chứng trực tiếp cho từng Test Case.
Hình 5.7	Báo cáo độ bao phủ gói service tại Bảng 5.3 (03-mvn-test-full-suite.log)	**Lần chạy 2** JaCoCo 0.8.12	Đo độ bao phủ 15 lớp nghiệp vụ trong gói com.reliefhub.service	78.43% Line / 44.07% Branch / 58.27% Method	**Chỉ chứng minh độ bao phủ hộp trắng tầng Service**; **không gán** làm bằng chứng PASS/FAIL cho từng Test Case.
Hình 5.8	Báo cáo độ bao phủ gói controller tại Phụ lục A (03-mvn-test-full-suite.log)	**Lần chạy 2** JaCoCo 0.8.12	Đo độ bao phủ 15 lớp điều khiển trong gói com.reliefhub.controller	59.38% Line / 60.98% Method	**Chỉ chứng minh độ bao phủ hộp trắng tầng Controller**; **không gán** làm bằng chứng PASS/FAIL cho từng Test Case.
Hình 5.9	Nhật ký phân tích tĩnh tại Phụ lục C, Mục C.1 (04-frontend-oxlint.log)	**Lần chạy 3** Oxlint 1.69.0	Phân tích mã nguồn tĩnh 50 tệp Frontend (BUG-006, BUG-012)	36 errors (no-undef), 83 warnings	**Đủ bằng chứng trực tiếp** cho kiểm thử tĩnh Frontend (SF-06, BUG-012) và giải thích nguyên nhân gốc của TC-058.
Hình 5.10	Nhật ký đóng gói tại Phụ lục C, Mục C.2 (05-frontend-vite-build.log)	**Lần chạy 3** Vite 8.1.0	Kiểm thử đóng gói Production Build Frontend (BUG-012)	Build PASS kèm cảnh báo 1,214.90 kB	**Đủ bằng chứng trực tiếp** cho cảnh báo hiệu năng đóng gói BUG-012.
Hình 5.11 → 5.13	Nhật ký thực thi tại Phụ lục C, Mục C.3 (06-frontend-webui-execution.log)	**Lần chạy 4** Chrome 156 CDP	Nhóm 8 WebUI: **TC-054** (Bảo vệ đường dẫn ProtectedRoute & RBAC UI)	**PASS** (Cả 3 bước chuyển hướng)	**Đủ bằng chứng trực tiếp** cho TC-054 (hiển thị rõ URL điều hướng và giao diện ở từng bước 1, 2, 3).

Số hiệu Hình	Nhật ký & Báo cáo đối chiếu	Lần chạy & Công cụ thực thi	Nhóm kiểm thử & Test Case ánh xạ	Kết quả thể hiện trên ảnh	Đánh giá mức độ đủ của bằng chứng (Evidentiary Sufficiency)
Hình 5.14	Nhật ký thực thi tại Phụ lục C, Mục C.3 (06-frontend-webui-execution.log)	**Lần chạy 4** Chrome 156 CDP	Nhóm 8 WebUI: **TC-055** (Chặn tài khoản local03 đang chờ duyệt)	**PASS** (Hiện cảnh báo đỏ)	**Đủ bằng chứng trực tiếp** cho TC-055 (hiển thị rõ thông báo từ chối trên giao diện /login).
Hình 5.15	Nhật ký thực thi tại Phụ lục C, Mục C.3 (06-frontend-webui-execution.log)	**Lần chạy 4** Chrome 156 CDP	Nhóm 8 WebUI: **TC-056** (Kiểm tra đăng nhập finance01 / fin123)	**FAIL (BUG-011)**	**Đủ bằng chứng trực tiếp** cho TC-056 khi đối chiếu cùng dòng mã AppContext.jsx:24 và reliefhub_full.sql:245.
Hình 5.16 → 5.18	Nhật ký thực thi tại Phụ lục C, Mục C.3 (06-frontend-webui-execution.log)	**Lần chạy 4** Chrome 156 CDP	Nhóm 8 WebUI: **TC-057** (Lỗ hổng đặt lại mật khẩu admin không cần OTP)	**FAIL (BUG-007)**	**Đủ bằng chứng trực tiếp** cho chuỗi 3 bước khai thác lỗ hổng chiếm quyền admin trên giao diện.
Hình 5.19 & 5.20	Nhật ký thực thi tại Phụ lục C, Mục C.3 (06-frontend-webui-execution.log)	**Lần chạy 4** Chrome 156 CDP	Nhóm 8 WebUI: **TC-058** (Người dân gửi yêu cầu cứu trợ bị lỗi ngầm)	**FAIL (BUG-006)**	**Cần kết hợp ảnh + nhật ký Console:** Hình 5.19 hiện thông báo thành công giả tạo; phải kết hợp Hình 5.20 (mất đơn) và 06-frontend-webui-execution.log để chứng minh ngoại lệ ReferenceError: API_ENABLED is not defined.
Hình 5.21 & 5.22	Nhật ký thực thi tại Phụ lục C, Mục C.3 (06-frontend-webui-execution.log)	**Lần chạy 4** Chrome 156 CDP	Nhóm 8 WebUI: **TC-059** (Nhà tài trợ gửi quyên góp & Admin xác minh)	**PASS** (Trong bộ nhớ RAM)	**Đủ bằng chứng trực tiếp** cho TC-059 trong phạm vi phiên làm việc hiện tại trên RAM.

Số hiệu Hình	Nhật ký & Báo cáo đối chiếu	Lần chạy & Công cụ thực thi	Nhóm kiểm thử & Test Case ánh xạ	Kết quả thể hiện trên ảnh	Đánh giá mức độ đủ của bằng chứng (Evidentiary Sufficiency)
Hình 5.23	Nhật ký thực thi tại Phụ lục C, Mục C.3 (06-frontend-webui-execution.log)	**Lần chạy 4** Chrome 156 CDP	Nhóm 8 WebUI: **TC-060** (Nhấn F5 Reload kiểm tra tính bền vững dữ liệu)	**FAIL (BUG-006)**	**Cần kết hợp ảnh + nhật ký CDP:** Hình 5.23 chứng minh danh sách xác minh bị reset về ban đầu sau F5, kết hợp 06-frontend-webui-execution.log chứng minh localStorage.reliefhub_token === null.

5.5.2. Hạng mục NOT RUN / BLOCKED và hướng thu thập bổ sung

Tuân thủ nguyên tắc giữ nguyên trạng thái NOT RUN / BLOCKED đối với các hạng mục chưa thể thực thi động, **Bảng 5.5** liệt kê toàn bộ các kịch bản chưa có ảnh chụp kiểm thử động thực tế trong môi trường hiện tại kèm nguyên nhân kỹ thuật và quy trình thu thập bổ sung:

Bảng 5.5: Danh mục bằng chứng kiểm thử động còn thiếu (NOT RUN / BLOCKED) và quy trình thu thập bổ sung

STT	Hạng mục / Kịch bản kiểm thử	Trạng thái kiểm thử thực tế	Nguyên nhân chưa có ảnh chụp kiểm thử động thực tế	Hướng dẫn chi tiết cách thu thập bằng chứng bổ sung
1	**Kiểm thử tích hợp End-to-End (E2E) từ giao diện WebUI xuống CSDL MySQL thật** (Lưu đơn cứu trợ từ Request.jsx xuống bảng support_requests và lưu xác minh quyên góp xuống bảng donations)	**BLOCKED trên mã nguồn gốc (Đã gỡ BLOCKED trên nhánh vá thử nghiệm — xem Hình 7.1)**	Mã nguồn gốc AppContext.jsx gán cứng dữ liệu mẫu trong RAM, không gọi authApi.login và ném lỗi ReferenceError: API_ENABLED is not defined tại dòng 558 & 779 nên mọi thao tác trên UI không gửi HTTP request xuống Backend.	**Đã thực hiện xác minh sau bản vá tại Mục 7.1.1 (Hình 7.1 & 09-e2e-post-patch-verification.log):** 1. Tạo nhánh kiểm chứng riêng test/fix-bug006-e2e-verify và áp dụng 5 dòng vá BUG-006 vào đầu AppContext.jsx. 2. Khởi chạy đồng thời Backend (:8080) và Frontend (:5173), gửi

STT	Hạng mục / Kịch bản kiểm thử	Trạng thái kiểm thử thực tế	Nguyên nhân chưa có ảnh chụp kiểm thử động thực tế	Hướng dẫn chi tiết cách thu thập bằng chứng bổ sung
				đơn cứu trợ tại /dashboard/citizen/request → ghi nhận POST /api/support-requests [HTTP 200 OK], lưu bền vững trong bảng support_requests và giữ nguyên dữ liệu lẫn localStorage.reliefhub_token sau khi nhấn F5 Reload.
2	**Kiểm thử động giao diện Hồ sơ cá nhân (/dashboard/*/profile) khi kết nối API GET/PUT /users/me với vai trò donor, citizen, local, finance**	**BLOCKED (Bị chặn bởi BUG-002 & BUG-006)**	Tầng Backend chặn /users/** chỉ cho ADMIN (SecurityConfig.java: 78 trả về HTTP 403), đồng thời tầng Frontend chưa gọi userApi.getMe().	1. Áp dụng phương án sửa lỗi đề xuất trong SecurityConfig.java cho phép .requestMatchers("/users/me", "/users/me/**").authenticated() đặt trước quy tắc /users/**. 2. Đăng nhập tài khoản donor01, cập nhật số điện thoại trên trang Profile và chụp ảnh tab **Network** hiển thị PUT /api/users/me [HTTP 200 OK].
3	**Kiểm thử động API làm mới JWT Token (POST /auth/refresh) và Khôi phục mật khẩu qua Email/OTP (password_reset_tokens)**	**NOT RUN (Chưa hiện thực hóa trong code)**	Dù xuất hiện trong AuthDTO.RefreshTokenRequest và kịch bản reliefhub_full.sql, nhưng AuthController và AuthService hoàn toàn chưa cài đặt các endpoint này (xem Bảng 4.6).	1. Hiện thực hóa phương thức refreshToken() và forgotPassword() trong AuthService.java và AuthController.java. 2. Bổ sung phương thức kiểm thử @Test vào ReliefHubFullEvaluationTest.java, chạy mvn test và chụp ảnh kết quả Surefire.

STT	Hạng mục / Kịch bản kiểm thử	Trạng thái kiểm thử thực tế	Nguyên nhân chưa có ảnh chụp kiểm thử động thực tế	Hướng dẫn chi tiết cách thu thập bằng chứng bổ sung
4	**Kiểm thử động 31 bảng CSDL phụ trong reliefhub_full.sql** (audit_logs, login_history, bank_accounts, drop_points, campaign_budget_allocations, v.v.)	**NOT RUN (Khoảng trống thiết kế CSDL)**	Tầng com.reliefhub.entity chỉ có 17 lớp Entity cốt lõi; 31 bảng phụ trong file SQL chưa có Entity, Repository hay Service tương ứng.	Nếu nhóm phát triển mở rộng các phân hệ này trong tương lai, cần viết các lớp @DataJpaTest kết nối MySQL 8.0 và chụp ảnh kết quả truy vấn trên MySQL Workbench hoặc Surefire.
5	**Kiểm thử hiệu năng và chịu tải đồng thời (Load / Stress Testing với k6 / Apache JMeter)**	**NOT RUN (Nằm ngoài phạm vi kiểm thử chức năng & tích hợp)**	Báo cáo tập trung vào kiểm thử tĩnh, hộp trắng, hộp đen API và tự động hóa giao diện chức năng trên máy trạm cục bộ.	1. Cài đặt Apache JMeter hoặc Grafana k6. 2. Thiết lập kịch bản 100–500 Virtual Users (VUs) gửi request đồng thời tới GET /api/campaigns/active và POST /api/donations. 3. Xuất báo cáo HTML Dashboard của JMeter/k6 và chụp ảnh biểu đồ Response Time Percentiles (p95, p99) & Throughput (RPS).

CHƯƠNG 6: HỒ SƠ LỖI, ĐÁNH GIÁ CHẤT LƯỢNG VÀ RỦI RO

6.1. Hồ sơ chi tiết 12 lỗi phần mềm (BUG-001 → BUG-012) [8]

Bảng 6.1: Bảng tổng hợp 12 lỗi phần mềm đã xác minh thực nghiệm (BUG-001 → BUG-012)

Mã lỗi (BUG-ID)	Mã tĩnh (SF-ID)	Tên lỗi tóm tắt	Phân hệ & Vị trí mã nguồn	Mức độ (Severity)	Ưu tiên	Test Case FAILED	Trạng thái hiện tại
BUG-001	SF-01	Leo thang đặc quyền: Tự đăng ký tài khoản admin (roleName="admin") qua POST /auth/register	Backend - AuthService.java:51–85	**Critical**	P0	TC-010	**Confirmed / Open**
BUG-002	SF-02	Chặn nhầm HTTP 403 Forbidden trên /users/me của người dùng không phải Admin	Backend - SecurityConfig.java:78	**High**	P0	TC-012	**Confirmed / Open**
BUG-003	SF-03	Broken Access Control trên 10+ API: DONOR/CITIZEN tự duyệt tiền, tự duyệt đơn, xóa sổ cái, xóa kho	Backend - SecurityConfig.java:81 & 14 Controllers	**Critical**	P0	TC-032, 034, 039, 043, 048	**Confirmed / Open**
BUG-004	SF-04	Bỏ qua máy trạng thái 6 bước: Chuyển thẳng từ PENDING sang DELIVERED	Backend - SupportRequestService.java:83–98	**High**	P0	TC-038	**Confirmed / Open**
BUG-005	SF-05	Cộng dồn trùng lặp tiền quyên góp khi gọi CONFIRMED nhiều lần làm tăng gấp đôi số dư chiến dịch	Backend - DonationService.java:84–100	**High**	P0	TC-031	**Confirmed / Open**

Mã lỗi (BUG-ID)	Mã tĩnh (SF-ID)	Tên lỗi tóm tắt	Phân hệ & Vị trí mã nguồn	Mức độ (Severity)	Ưu tiên	Test Case FAILED	Trạng thái hiện tại
BUG-006	SF-06	Lỗi ReferenceError: API_ENABLED is not defined làm mất đơn cứu trợ của người dân & mất state khi F5	Frontend - AppContext.jsx :558, 780	**Critical**	P0	TC-058, TC-060	**Confirmed / Open**
BUG-007	SF-07	Chiếm đoạt tài khoản admin qua modal Quên mật khẩu chỉ bằng username không cần OTP	Frontend - Login.jsx:62–83, AppContext.jsx :524–534	**Critical**	P0	TC-057	**Confirmed / Open**
BUG-008	SF-08	Unrestricted File Upload / Stored XSS: Cho phép tải lên tệp malicious.html chứa <script>	Backend - FileUploadService.java:19–35	**High**	P1	TC-052	**Confirmed / Open**
BUG-009	SF-09	Lỗ hổng IDOR: Người dùng bất kỳ đánh dấu đã đọc thông báo của ADMIN (PATCH /notifications/1/read)	Backend - NotificationService.java:27–32	**Medium**	P1	TC-049	**Confirmed / Open**
BUG-010	SF-10	Thiếu @Positive và @Valid gây lưu số âm vào CSDL hoặc trả về lỗi HTTP 500 Internal Server Error	Backend - CampaignDTO, SupportRequestDTO, Controllers	**Medium**	P1	TC-024, 036, 048, 053	**Confirmed / Open**
BUG-011	SF-11	Bất đồng bộ mật khẩu khởi tạo của finance01 (fin123 trong SQL/Tài liệu vs	Frontend - AppContext.jsx :24	**Medium**	P2	TC-056	**Confirmed / Open**

Mã lỗi (BUG-ID)	Mã tĩnh (SF-ID)	Tên lỗi tóm tắt	Phân hệ & Vị trí mã nguồn	Mức độ (Severity)	Ưu tiên	Test Case FAILED	Trạng thái hiện tại
		finance123 trên UI)					
BUG-012	*(Mới)*	Nợ kỹ thuật Frontend: 83 cảnh báo no-unused-vars và gói bundle JS nặng 1,214.90 kB (> 500 kB)	Frontend - tầng giao diện Frontend & App.jsx	**Low**	P2	Phân tích tĩnh	**Confirmed / Open**

Dưới đây là hồ sơ đặc tả chi tiết của từng lỗi (BUG-001 → BUG-012):

6.1.1. BUG-001 (SF-01): Lỗ hổng leo thang đặc quyền qua API Đăng ký (POST /auth/register)

- **Mức độ nghiêm trọng: Critical (Nghiêm trọng) | Độ ưu tiên:** P0
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** REQ-003 (Bảo vệ quyền hạn khi tự đăng ký & Khôi phục mật khẩu)
- **Test Case phát hiện:** TC-010 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**
- AuthService.java (Dòng 51–85, cụ thể dòng 59–70)
- AuthDTO.java (Dòng 35: trường private String roleName; nhận chuỗi tự do từ client)
- **Tiền điều kiện (Pre-condition):** Hệ thống Backend đang hoạt động; bảng roles trong CSDL có sẵn bản ghi vai trò admin.
- **Các bước tái hiện (Steps to Reproduce):**

1. Khách vãng lai (không cần đăng nhập, không đính kèm header Authorization) gửi HTTP request POST /auth/register với JSON body:

{"fullName": "Hacker Admin", "username": "hacker_admin", "email": "hacker@test.com", "password": "password123", "roleName": "admin"}

1. Kiểm tra mã trạng thái HTTP và nội dung chuỗi JWT trả về trong data.token.

- **Kết quả mong đợi (Expected Result):** Hệ thống từ chối với HTTP 400 Bad Request / 403 Forbidden hoặc chỉ cho phép gán các vai trò công khai (citizen, donor, local, volunteer), không cấp quyền admin cho tài khoản tự đăng ký.
- **Kết quả thực tế (Actual Result):** Hệ thống trả về HTTP 200 OK kèm thông báo "Đăng ký thành công", tạo tài khoản ở trạng thái ACTIVE và cấp chuỗi Bearer JWT Token có quyền role = "admin".
- **Tác động (Impact):** Người dùng bên ngoài có thể tạo tài khoản Quản trị viên (ROLE_ADMIN) qua API công khai, từ đó có quyền xóa người dùng, xóa chiến dịch và thay đổi dữ liệu hệ thống.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):**

Tại AuthService.java:59–71, phương thức register() lấy trực tiếp request.getRoleName() để truy vấn bảng roles mà không kiểm tra danh sách vai trò được phép tự đăng ký, đồng thời gán cứng .status(User.UserStatus.ACTIVE) tại dòng 69:

```
String roleName = request.getRoleName() != null ? request.getRoleName() : "citizen";
Role role = roleRepository.findByName(roleName)
    .orElseThrow(() -> new BadRequestException("Vai trò không hợp lệ: " + roleName));
User user = User.builder()
    ...
    .status(User.UserStatus.ACTIVE)
    .role(role)
    .build();
```

- **Bằng chứng thực nghiệm (Evidence):**
- Phản hồi HTTP thực tế tại TC-010: HTTP 200 OK | {"success":true,"message":"Đăng ký thành công","data":{"token":"eyJhbGciOiJIUzUxMiJ9...","tokenType":"Bearer","userId":7,"username":"hacker_admin","role":"admin"}} (Trích xuất đầy đủ tại Phụ lục B, Bảng B.1 & Mục B.2).

6.1.2. BUG-002 (SF-02): Chặn nhầm quyền truy cập hồ sơ cá nhân (/users/me) của người dùng không phải Admin

- **Mức độ nghiêm trọng: High (Cao) | Độ ưu tiên:** P0
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** REQ-005 (Xem và cập nhật hồ sơ cá nhân /users/me), REQ-006 (PUT /users/me/password)
- **Test Case phát hiện:** TC-012 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**
- SecurityConfig.java
  (Dòng78: .requestMatchers("/users/**").hasRole("ADMIN"))
- UserController.java (Dòng 40–55)
- **Tiền điều kiện (Pre-condition):** Đã đăng nhập tài khoản testdonor (role = "donor") và có Bearer JWT Token hợp lệ (donorToken).
- **Các bước tái hiện (Steps to Reproduce):**

1. Gửi request GET /users/me với header Authorization: Bearer <donorToken>.
2. Gửi request PUT /users/me với header Authorization: Bearer <donorToken> và body {"fullName": "Donor Updated", "phone": "0911222333"}.

- **Kết quả mong đợi (Expected Result):** Cả hai request trả về HTTP 200 OK, cho phép người dùng xem và cập nhật thông tin cá nhân của mình.
- **Kết quả thực tế (Actual Result):** Cả GET /users/me và PUT /users/me đều bị từ chối với HTTP 403 Forbidden ({"success":false,"error":"Forbidden - Không có quyền truy cập"}).
- **Tác động (Impact):** Cả 5 nhóm người dùng không phải Admin (DONOR, CITIZEN, FINANCE, LOCAL, VOLUNTEER) không thể xem hồ sơ cá nhân, sửa thông tin liên hệ hoặc đổi mật khẩu qua API Backend.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):**

Trong UserController.java, 3 endpoint dành cho cá nhân (/users/me, PUT /users/me, PUT /users/me/password) được thiết kế không gắn @PreAuthorize("hasRole('ADMIN')"). Tuy nhiên, tại SecurityConfig.java:78, quy tắc cấu hình URL filter lại giới hạn toàn bộ tiền tố /users/** chỉ cho ADMIN:

```
.requestMatchers("/users/**").hasRole("ADMIN")
```

Do bộ lọc URL của Spring Security chạy trước khi vào Controller, các request tới /users/me từ tài khoản khác ADMIN đều bị chặn tại tầng filter.

- **Bằng chứng thực nghiệm (Evidence):**
- Phản hồi HTTP thực tế tại TC-012: GET /users/me = HTTP 403 Forbidden, PUT /users/me = HTTP 403 Forbidden (Trích xuất đầy đủ tại Phụ lục B, Bảng B.1 & Mục B.2).

6.1.3. BUG-003 (SF-03): Thiếu kiểm soát phân quyền theo vai trò trên các API quản trị nhạy cảm

- **Mức độ nghiêm trọng: Critical (Nghiêm trọng) | Độ ưu tiên:** P0
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** REQ-013, REQ-015, REQ-017, REQ-019, REQ-023
- **Test Case phát hiện:** TC-032 (**FAIL**), TC-034 (**FAIL**), TC-039 (**FAIL**), TC-043 (**FAIL**), TC-048 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**
- SecurityConfig.java (Dòng 32 và Dòng 63–82, cụ thể dòng 81: .anyRequest().authenticated())
- 14 / 15 lớp Controller trong com.reliefhub.controller.* (DonationController, LedgerController, SupportRequestController, InventoryController, WarehouseController, DeliveryController, ComplaintController, v.v.) không gắn @PreAuthorize.
- **Tiền điều kiện (Pre-condition):** Đăng nhập bằng tài khoản thông thường testdonor (role = "donor") hoặc testcitizen (role = "citizen").
- **Các bước tái hiện (Steps to Reproduce):**

1. **TC-032:** Dùng donorToken gửi PATCH /donations/{id}/status với {"status": "CONFIRMED"} để tự duyệt khoản quyên góp của mình.
2. **TC-034:** Dùng citizenToken gửi POST /ledger (tạo bút toán sổ cái) và DELETE /ledger/{id} (xóa bút toán sổ cái).
3. **TC-039:** Dùng citizenToken gửi PATCH /support-requests/{id}/advance với {"newStatus": "APPROVED"} để tự phê duyệt đơn xin cứu trợ của mình.
4. **TC-043:** Dùng citizenToken gửi DELETE /inventory/{id} để xóa vật tư trong kho cứu trợ.

5. **TC-048:** Dùng citizenToken gửi PUT /complaints/{id}/reply để trả lời khiếu nại với tư cách Admin.

- **Kết quả mong đợi (Expected Result):** Tất cả 5 kịch bản trên phải bị từ chối với HTTP 403 Forbidden.
- **Kết quả thực tế (Actual Result):** Cả 5 kịch bản (TC-032, TC-034, TC-039, TC-043, TC-048) đều trả về HTTP 200 OK và thực thi thành công thao tác quản trị trái phép.
- **Tác động (Impact):** Làm mất hiệu lực cơ chế phân quyền RBAC ở các phân hệ nghiệp vụ: nhà tài trợ có thể tự xác nhận tiền để tăng quỹ chiến dịch, người dân có thể tự duyệt đơn cứu trợ, xóa dữ liệu tồn kho và xóa bút toán sổ cái tài chính.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):**

Dù SecurityConfig.java:32 đã bật @EnableMethodSecurity(prePostEnabled = true), chỉ có UserController sử dụng @PreAuthorize("hasRole('ADMIN')"). Trong khi đó, SecurityConfig.java:63–82 chỉ khai báo hasRole cho /admin/**, /campaigns, /disasters, /users/** và /dashboard/**; các nhóm API còn lại rơi vào quy tắc .anyRequest().authenticated() ở dòng 81.

- **Bằng chứng thực nghiệm (Evidence):**
- Phản hồi HTTP thực tế tại TC-032, TC-034, TC-039, TC-043, TC-048 đều trả về HTTP 200 OK thay vì HTTP 403 Forbidden (Trích xuất đầy đủ tại Phụ lục B, Bảng B.1 & Mục B.2).

6.1.4. BUG-004 (SF-04): Bỏ qua máy trạng thái trong quy trình 6 bước xử lý yêu cầu cứu trợ

- **Mức độ nghiêm trọng: High (Cao)** | **Độ ưu tiên:** P0
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** REQ-017 (Chuyển trạng thái yêu cầu hỗ trợ theo quy trình 6 bước)
- **Test Case phát hiện:** TC-038 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**
- SupportRequestService.java (Dòng 83–98)
- **Tiền điều kiện (Pre-condition):** Có một yêu cầu hỗ trợ vừa được tạo ở trạng thái khởi đầu PENDING (id = 3).
- **Các bước tái hiện (Steps to Reproduce):**

1. Gửi request PATCH /support-requests/3/advance với JSON body {"newStatus": "DELIVERED"} ngay khi đơn đang ở trạng thái PENDING.
2. Kiểm tra mã HTTP và trạng thái mới của yêu cầu hỗ trợ.

- **Kết quả mong đợi (Expected Result):** Hệ thống từ chối với HTTP 400 Bad Request do vi phạm thứ tự quy trình 6 bước (PENDING → VERIFIED → APPROVED → WAREHOUSE_READY → IN_TRANSIT → DELIVERED).
- **Kết quả thực tế (Actual Result):** Hệ thống trả về HTTP 200 OK và cập nhật thẳng status = "DELIVERED", bỏ qua 4 bước trung gian (xác minh địa phương, phê duyệt Admin, chuẩn bị kho và vận chuyển).
- **Tác động (Impact):** Làm sai lệch quy trình xử lý cứu trợ 6 bước; đơn chưa được xác minh hoặc chưa xuất kho vẫn có thể chuyển sang trạng thái đã giao hàng (DELIVERED).
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):**

Tại SupportRequestService.java:83–98, phương thức advanceStatus() chỉ ép kiểu chuỗi đầu vào sang enum RequestStatus và gán trực tiếp sr.setStatus(newStatus) ở dòng 91 mà không kiểm tra trạng thái hiện tại (sr.getStatus()):

```
SupportRequest.RequestStatus newStatus = SupportRequest.RequestStatus.valueOf(
    request.getNewStatus().toUpperCase()
);
sr.setStatus(newStatus);
```

- **Bằng chứng thực nghiệm (Evidence):**
- Phản hồi HTTP thực tế tại TC-038: HTTP 200 OK | {"success":true,"message":"Thành công","data":{"id":3,"requestCode":"REQ-1792591165294",...,"status":"DELIVERED"}} (Trích xuất tại Phụ lục B, Bảng B.1 & Mục B.2).

6.1.5. BUG-005 (SF-05): Lỗi cộng dồn trùng lặp số tiền chiến dịch

- **Mức độ nghiêm trọng: High (Cao) | Độ ưu tiên:** P0
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** REQ-013 (Xác minh quyên góp & Cộng quỹ chiến dịch)
- **Test Case phát hiện:** TC-031 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**

- DonationService.java (Dòng 84–100)
- **Tiền điều kiện (Pre-condition):** Khoản quyên góp id = 1 (amount = 5,000,000 VNĐ) đã được xác nhận CONFIRMED ở TC-030; số tiền hiện tại của chiến dịch campaign.currentAmount đang là 5,000,000.00.
- **Các bước tái hiện (Steps to Reproduce):**

1. Gửi lại request PATCH /donations/1/status với body {"status": "CONFIRMED"} lần thứ 2 trên cùng khoản quyên góp id = 1.
2. Truy vấn lại chiến dịch GET /campaigns/1 để kiểm tra currentAmount.

- **Kết quả mong đợi (Expected Result):** Hệ thống không cộng tiền lần thứ 2 (campaign.currentAmount phải giữ nguyên 5,000,000.00) hoặc trả về HTTP 400 Bad Request báo khoản quyên góp đã được xác nhận trước đó.
- **Kết quả thực tế (Actual Result):** Hệ thống trả về HTTP 200 OK và tiếp tục cộng thêm 5,000,000 vào quỹ chiến dịch, làm campaign.currentAmount tăng gấp đôi lên 10,000,000.00 VNĐ.
- **Tác động (Impact):** Gây sai lệch số liệu tài chính của chiến dịch cứu trợ nếu nhân viên tài chính bấm xác nhận hai lần (double-click) hoặc gửi lặp lại request API.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):**

Tại DonationService.java:88–97, phương thức updateDonationStatus() ghi đè donation.setStatus(...) ở dòng 88 trước, sau đó ở dòng 92 chỉ kiểm tra if (donation.getStatus() == Donation.DonationStatus.CONFIRMED && donation.getCampaign() != null) mà không kiểm tra trạng thái cũ trước khi cập nhật (oldStatus != DonationStatus.CONFIRMED):

```
donation.setStatus(Donation.DonationStatus.valueOf(request.getStatus().toUpperCase()));
  donation = donationRepository.save(donation);
  if (donation.getStatus() == Donation.DonationStatus.CONFIRMED && donation.getCampaign() != null) {
      Campaign campaign = donation.getCampaign();
      BigDecimal current = campaign.getCurrentAmount() != null ? campaign.getCurrentAmount() : BigDecimal.ZERO;
      campaign.setCurrentAmount(current.add(donation.getAmount()));
```

```
    campaignRepository.save(campaign);
}
```

- **Bằng chứng thực nghiệm (Evidence):**
- Phản hồi HTTP thực tế tại TC-031: HTTP 200 OK | campaign.currentAmount = 10000000.00 (Tăng gấp đôi từ 5000000.00; xem Phụ lục B, Bảng B.1 & Mục B.2).

6.1.6. BUG-006 (SF-06): Frontend chạy trên bộ nhớ RAM, gọi biến chưa khai báo (ReferenceError: API_ENABLED is not defined) và mất dữ liệu khi Reload (F5)

- **Mức độ nghiêm trọng: Critical (Nghiêm trọng) | Độ ưu tiên:** P0
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** REQ-016 (Gửi yêu cầu hỗ trợ cứu trợ), REQ-028 (Tính nhất quán dữ liệu & Tích hợp Frontend – Backend)
- **Test Case phát hiện:** TC-058 (**FAIL**), TC-060 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**
- AppContext.jsx (Dòng 558–559 và 780–782)
- Request.jsx (Dòng 29–46: gọi hàm async không có await/catch)
- Status.jsx (Dòng 12: hiển thị toàn bộ mảng needs của mọi công dân mà không lọc theo người dùng đang đăng nhập)
- **Tiền điều kiện (Pre-condition):** Phân hệ Frontend đang chạy tại http://127.0.0.1:5173.
- **Các bước tái hiện (Steps to Reproduce):**

1. **Kịch bản TC-058:** Đăng nhập citizen01 / citizen123, mở /dashboard/citizen/request, chọn khu vực "Quảng Bình", mặt hàng "Gạo", nhấn "Gửi yêu cầu cứu trợ", sau đó mở /dashboard/citizen/status và kiểm tra Browser Console.
2. **Kịch bản TC-060:** Sau khi đăng nhập và thực hiện xác minh quyên góp trên giao diện, nhấn phím F5 (Reload trình duyệt) và kiểm tra trạng thái phiên đăng nhập cùng localStorage.

- **Kết quả mong đợi (Expected Result):**
- Ở TC-058: Đơn cứu trợ mới được lưu thành công, hiển thị trong trang Trạng thái của citizen01 và Console không có lỗi ReferenceError.

- Ở TC-060: Sau khi nhấn F5, phiên đăng nhập và dữ liệu đã xác minh được giữ nguyên.
- **Kết quả thực tế (Actual Result):**

1. Ở TC-058: Browser Console ném ngoại lệ chưa được bắt: Uncaught (in promise) ReferenceError: API_ENABLED is not defined tại AppContext.jsx:558. Tuy nhiên giao diện Request.jsx vẫn hiển thị màn hình báo thành công " Yêu cầu đã gửi!", trong khi yêu cầu không được thêm vào state. Khi sang trang /dashboard/citizen/status, đơn mới không tồn tại và trang còn hiển thị cả đơn của những người dân khác (Lê Văn C, Phạm Thị D).
2. Ở TC-060: Sau khi nhấn F5, localStorage.reliefhub_token = null, phiên đăng nhập và dữ liệu vừa thao tác trong RAM bị mất.

- **Tác động (Impact):** Chức năng gửi yêu cầu cứu trợ của người dân trên WebUI không hoạt động nhưng giao diện vẫn báo thành công (False Positive UI), đồng thời hệ thống Frontend không lưu trữ bền vững dữ liệu qua các lần tải lại trang.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):**

Công cụ Oxlint xác nhận tệp AppContext.jsx không import tệp api.js nhưng lại gọi trực tiếp các định danh chưa khai báo là API_ENABLED (dòng 558, 780), supportRequestService (dòng 559) và tokenManager (dòng 781, 782). Khi câu lệnh if (API_ENABLED) ở dòng 558 chạy, JS Engine ném ngay ReferenceError khiến luồng thực thi bị ngắt trước khi tới dòng setNeeds(...) ở dòng 578.

- **Bằng chứng thực nghiệm (Evidence):**
- Ảnh chụp màn hình nhúng trực tiếp: **Hình 5.19** và **Hình 5.20** (Mục 5.4.5).
- Nhật ký phân tích tĩnh Oxlint (Phụ lục C, Mục C.1) và nhật ký Browser Console (Phụ lục C, Mục C.3).

6.1.7. BUG-007 (SF-07): Lỗ hổng chiếm đoạt tài khoản qua chức năng Quên mật khẩu

- **Mức độ nghiêm trọng: Critical (Nghiêm trọng) | Độ ưu tiên:** P0
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** REQ-003 (Bảo vệ quyền hạn khi tự đăng ký & Khôi phục mật khẩu)
- **Test Case phát hiện:** TC-057 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**

- Login.jsx (Dòng 62–83)
- AppContext.jsx (Dòng 524–534: checkUsernameExists và resetPasswordByUsername)
- **Tiền điều kiện (Pre-condition):** Mở trang đăng nhập http://127.0.0.1:5173/login ở trạng thái khách vãng lai (chưa đăng nhập).
- **Các bước tái hiện (Steps to Reproduce):**

1. Nhấn vào liên kết "Quên mật khẩu?" trên màn hình /login.
2. Nhập tên đăng nhập "admin" và nhấn "Xác minh tài khoản".
3. Nhập mật khẩu mới "hacked123" vào 2 ô mật khẩu và nhấn "Đặt lại mật khẩu".
4. Đóng modal và đăng nhập bằng "admin" / "hacked123".

- **Kết quả mong đợi (Expected Result):** Hệ thống yêu cầu xác thực qua Email, SĐT, mã OTP hoặc Token khôi phục hợp lệ; không cho phép đổi mật khẩu trực tiếp chỉ bằng tên đăng nhập.
- **Kết quả thực tế (Actual Result):** Hệ thống báo "Tài khoản admin đã được xác minh", cho phép đặt lại mật khẩu thành "hacked123" ("Đặt lại thành công!") và đăng nhập thẳng vào trang Quản trị viên /dashboard/admin.
- **Tác động (Impact):** Người dùng bất kỳ biết tên đăng nhập (admin, finance01, donor01, ...) đều có thể đổi mật khẩu và chiếm quyền truy cập tài khoản đó trong phiên giao diện mà không cần bước xác thực bổ sung.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):**

Hàm resetPasswordByUsername(username, newPassword) trong AppContext.jsx:528–534 cập nhật trực tiếp trường password của tài khoản trong mảng accounts ngay khi checkUsernameExists(username) trả về true, không có bước kiểm tra mã xác thực (OTP/Reset Token).

- **Bằng chứng thực nghiệm (Evidence):**
- Ảnh chụp màn hình nhúng trực tiếp: **Hình 5.16**, **Hình 5.17** và **Hình 5.18** (Mục 5.4.4).
- Nhật ký thực thi tự động WebUI (Phụ lục C, Mục C.3).

6.1.8. BUG-008 (SF-08): Lỗ hổng tải lên tệp độc hại không kiểm tra định dạng

- **Mức độ nghiêm trọng: High (Cao)** | **Độ ưu tiên:** P1
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)

- **Yêu cầu liên quan:** REQ-026 (Tải lên tệp minh chứng FileUpload)
- **Test Case phát hiện:** TC-052 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**
- FileUploadService.java (Dòng 19–35)
- WebConfig.java (Dòng 15–21: ánh xạ tĩnh thư mục /uploads/**)
- **Tiền điều kiện (Pre-condition):** Người dùng bất kỳ đã đăng nhập hợp lệ.
- **Các bước tái hiện (Steps to Reproduce):**

1. Gửi request POST /uploads (multipart/form-data) với tệp malicious.html (Content-Type: text/html, kích thước 39 bytes) chứa nội dung <script>alert('XSS')</script>.
2. Kiểm tra mã trạng thái HTTP và đường dẫn url trả về.

- **Kết quả mong đợi (Expected Result):** Hệ thống chỉ cho phép tải lên tệp hình ảnh hoặc tài liệu minh chứng an toàn (.png, .jpg, .jpeg, .webp, .pdf) và từ chối tệp .html với HTTP 400 Bad Request.
- **Kết quả thực tế (Actual Result):** Hệ thống chấp nhận tệp malicious.html với HTTP 200 OK và trả về đường dẫn xuất bản trực tiếp "/uploads/f964f0a8-e3cb-46ec-a17a-af41c268759f.html" với "mimeType": "text/html".
- **Tác động (Impact):** Tạo lỗ hổng Stored XSS; khi Admin mở đường dẫn minh chứng quyên góp do người dùng đính kèm, mã JavaScript trong tệp .html sẽ thực thi trên trình duyệt của Admin.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):**

Trong FileUploadService.java:19–35, phương thức upload() chỉ kiểm tra file.isEmpty() và file.getSize() > MAX_SIZE, sau đó trích xuất phần mở rộng gốc (ext) từ originalFilename và lưu xuống thư mục uploads/ mà không kiểm tra danh sách phần mở rộng hợp lệ hay MIME type.

- **Bằng chứng thực nghiệm (Evidence):**
- Phản hồi HTTP thực tế tại TC-052: HTTP 200 OK | {"success":true,"message":"Upload thành công","data":{"fileName":"f964f0a8-e3cb-46ec-a17a-af41c268759f.html","url":"/uploads/f964f0a8-e3cb-46ec-a17a-af41c268759f.html","mimeType":"text/html","sizeBytes":39}} (Xem Phụ lục B, Bảng B.1 & Mục B.2).

6.1.9. BUG-009 (SF-09): Lỗ hổng IDOR khi đánh dấu đã đọc thông báo của người khác

- **Mức độ nghiêm trọng: Medium (Trung bình) | Độ ưu tiên:** P1
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** REQ-024 (Quản lý Thông báo cá nhân)
- **Test Case phát hiện:** TC-049 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**
- NotificationController.java (Dòng 24–28)
- NotificationService.java (Dòng 27–32)
- **Tiền điều kiện (Pre-condition):** Thông báo id = 1 thuộc quyền sở hữu của tài khoản testadmin (isRead = false); người dùng hiện tại đăng nhập bằng donorToken (testdonor).
- **Các bước tái hiện (Steps to Reproduce):**

1. Dùng donorToken gửi request PATCH /notifications/1/read.
2. Kiểm tra mã trạng thái HTTP và trạng thái isRead của thông báo id = 1 trong CSDL.

- **Kết quả mong đợi (Expected Result):** Hệ thống từ chối với HTTP 403 Forbidden hoặc 404 Not Found do thông báo id = 1 không thuộc về testdonor.
- **Kết quả thực tế (Actual Result):** Hệ thống trả về HTTP 200 OK và cập nhật adminNotif.isRead = true.
- **Tác động (Impact):** Người dùng bất kỳ có thể duyệt tuần tự các id (1, 2, 3, ...) để đánh dấu đã đọc thông báo hệ thống của Quản trị viên và người dùng khác.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):**

Phương thức NotificationService.markAsRead(Long id) chỉ gọi notificationRepository.findById(id) rồi đặt n.setIsRead(true) mà không kiểm tra điều kiện n.getUser().getId().equals(currentUserId).

- **Bằng chứng thực nghiệm (Evidence):**
- Phản hồi HTTP thực tế tại TC-049: HTTP 200 OK | adminNotif.isRead = true (Xem Phụ lục B, Bảng B.1 & Mục B.2).

6.1.10. BUG-010 (SF-10): Thiếu kiểm tra dữ liệu biên (Số âm) và lỗi HTTP 500 khi dữ liệu đầu vào sai định dạng / null

- **Mức độ nghiêm trọng: Medium (Trung bình) | Độ ưu tiên:** P1
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** REQ-010, REQ-016, REQ-023, REQ-027
- **Test Case phát hiện:** TC-024 (**FAIL**), TC-036 (**FAIL**), TC-048 (**FAIL**), TC-053 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**
- CampaignDTO.CreateCampaignRequest (CampaignDTO.java:36): targetAmount chỉ có @NotNull, thiếu @Positive.
- CampaignService.create() (CampaignService.java:46–47): LocalDate.parse(req.getStartDate()) không bắt DateTimeParseException.
- SupportRequestDTO.CreateSupportRequestRequest (SupportRequestDTO.java:32–35): quantity và householdCount không có @Positive / @Min(1).
- ComplaintController.reply() (ComplaintController.java:35): thiếu @Valid trước @RequestBody ComplaintDTO.ReplyComplaintRequest.
- SupportRequestController.advance() (SupportRequestController.java:38): thiếu @Valid trước @RequestBody SupportRequestDTO.AdvanceStatusRequest.
- **Tiền điều kiện (Pre-condition):** Đăng nhập hợp lệ với token tương ứng (adminToken hoặc citizenToken).
- **Các bước tái hiện (Steps to Reproduce):**

1. **TC-024:** Gửi POST /campaigns với targetAmount = -50000000; sau đó gửi tiếp với startDate = "invalid-date".
2. **TC-036:** Gửi POST /support-requests với quantity = -20, householdCount = -1.
3. **TC-048:** Gửi PUT /complaints/{id}/reply với {"adminReply": ""}.
4. **TC-053:** Gửi PATCH /support-requests/{id}/advance với body rỗng {} (newStatus = null).

- **Kết quả mong đợi (Expected Result):** Tất cả các request có dữ liệu âm, rỗng hoặc sai định dạng ngày đều phải được chặn và trả về HTTP 400 Bad Request, không lưu số âm vào CSDL và không trả về lỗi HTTP 500.
- **Kết quả thực tế (Actual Result):**

1. TC-024: Chiến dịch có targetAmount = -50,000,000 được lưu thành công với HTTP 200 OK; khi truyền startDate = "invalid-date", server trả về lỗi HTTP 500 Internal Server Error (DateTimeParseException).
2. TC-036: Yêu cầu cứu trợ có quantity = -20, householdCount = -1 được lưu vào CSDL với HTTP 200 OK.
3. TC-048: Nội dung phản hồi khiếu nại rỗng "" được chấp nhận với HTTP 200 OK.
4. TC-053: Truyền {} vào advance gây lỗi HTTP 500 Internal Server Error (NullPointerException: Cannot invoke "String.toUpperCase()" because the return value of "...getNewStatus()" is null).

- **Tác động (Impact):** Lưu dữ liệu không hợp lệ vào CSDL (mục tiêu chiến dịch âm, số lượng cứu trợ âm) và phát sinh lỗi hệ thống (HTTP 500) khi client gửi thiếu trường JSON hoặc sai định dạng ngày.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):** Thiếu các annotation @Positive trên DTO, thiếu @Valid trên tham số @RequestBody của Controller và thiếu trình xử lý ngoại lệ DateTimeParseException trong GlobalExceptionHandler.
- **Bằng chứng thực nghiệm (Evidence):**
- Phản hồi HTTP thực tế tại TC-024, TC-036, TC-048, TC-053 được trích xuất nguyên văn tại Phụ lục B (Bảng B.1 & Mục B.2).

6.1.11. BUG-011 (SF-11): Bất đồng bộ mật khẩu khởi tạo của tài khoản finance01 giữa tài liệu/SQL và mã nguồn Frontend

- **Mức độ nghiêm trọng: Medium (Trung bình) | Độ ưu tiên:** P2
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** REQ-028 (Tính nhất quán dữ liệu & Tích hợp Frontend – Backend)
- **Test Case phát hiện:** TC-056 (**FAIL**)
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**
- AppContext.jsx (Dòng 83: password: 'finance123')
- reliefhub_full.sql & HUONG_DAN_CHAY.md (Dòng 120: quy định mật khẩu của finance01 là fin123).
- **Tiền điều kiện (Pre-condition):** Mở màn hình đăng nhập http://127.0.0.1:5173/login.

- **Các bước tái hiện (Steps to Reproduce):**

1. Nhập tên đăng nhập finance01 và mật khẩu fin123 (theo bảng tài khoản trong HUONG_DAN_CHAY.md).
2. Nhấn nút "Đăng nhập".

- **Kết quả mong đợi (Expected Result):** Đăng nhập thành công vào trang /dashboard/finance.
- **Kết quả thực tế (Actual Result):** Hệ thống từ chối đăng nhập và hiển thị hộp cảnh báo đỏ " Tên đăng nhập hoặc mật khẩu không đúng".
- **Tác động (Impact):** Người dùng hoặc kiểm thử viên làm theo tài liệu hướng dẫn không thể đăng nhập vào phân hệ Ban tài chính trên giao diện.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):** Dữ liệu mảng SEED_ACCOUNTS trong AppContext.jsx:24 gán cứng password: 'finance123', không đồng bộ với mật khẩu fin123 trong tài liệu và SQL.
- **Bằng chứng thực nghiệm (Evidence):**
- Ảnh chụp màn hình nhúng trực tiếp: **Hình 5.15** (Mục 5.4.3).
- Nhật ký thực thi WebUI (Phụ lục C, Mục C.3).

6.1.12. BUG-012: Nợ kỹ thuật Frontend: 83 cảnh báo biến không sử dụng (no-unused-vars) và kích thước gói Bundle lớn (1,214.90 kB)

- **Mức độ nghiêm trọng: Low (Thấp / Maintainability & Performance)** | **Độ ưu tiên:** P2
- **Trạng thái lỗi:** Confirmed / Open (Chưa sửa trên mã nguồn gốc)
- **Yêu cầu liên quan:** Tiêu chí chất lượng QC-04 (Maintainability & Static Analysis)
- **Công cụ phát hiện:** Oxlint 1.69.0 và Vite 8.1.0 Production Build
- **Vị trí mã nguồn gây lỗi (Đã xác minh):**
- 50 tệp mã nguồn trong tầng giao diện Frontend (50 tệp .jsx/.js)
- App.jsx (Dòng 1–60: import tĩnh đồng loạt 42 trang giao diện cùng lúc thay vì dùng React.lazy())
- **Tiền điều kiện (Pre-condition):** Mã nguồn Frontend của dự án.
- **Các bước tái hiện (Steps to Reproduce):**

1. Chạy lệnh phân tích tĩnh npx.cmd oxlint src --import-plugin.

2. Chạy lệnh đóng gói npx.cmd vite build.

- **Kết quả mong đợi (Expected Result):** Không có cảnh báo biến thừa và gói bundle chính sau khi đóng gói nhỏ hơn ngưỡng khuyến nghị 500 kB của Vite.
- **Kết quả thực tế (Actual Result):**

1. Oxlint báo 83 warnings (eslint(no-unused-vars)) trên 50 tệp (và 36 errors no-undef, trong đó có 5 lỗi biến nghiệp vụ chưa khai báo thuộc BUG-006 và 31 vị trí gọi biến toàn cục trình duyệt window, fetch, console, setTimeout khi chưa cấu hình globals).
2. Vite Build sinh ra tệp gói JavaScript chính (index.js) nặng 1,214.90 kB (318.76 kB gzip), vượt ngưỡng cảnh báo 500 kB gấp 2.43 lần.

- **Tác động (Impact):** Làm giảm khả năng bảo trì mã nguồn và tăng thời gian tải trang lần đầu trên kết nối mạng di động chậm.
- **Nguyên nhân gốc rễ đã xác minh (Verified Root Cause):** Mã nguồn import nhiều biểu tượng lucide-react, recharts và khai báo state nhưng không sử dụng; tệp App.jsx nạp tĩnh toàn bộ 42 route cùng lúc thay vì chia nhỏ gói mã (Code Splitting).
- **Bằng chứng thực nghiệm (Evidence):**
- Nhật ký phân tích tĩnh Oxlint (Phụ lục C, Mục C.1) và nhật ký đóng gói Vite Build (Phụ lục C, Mục C.2).

6.2. Khoảng cách chất lượng và ma trận rủi ro

6.2.1. Khoảng cách chất lượng trên 8 tiêu chí (QC-01 → QC-08)

Bảng 6.2: Phân tích khoảng cách chất lượng (Quality Gap Analysis) trên 8 tiêu chí QC-01 → QC-08

Mã QC	Đặc tính chất lượng	Mục tiêu đề xuất	Kết quả thực tế đã xác minh	Khoảng cách chất lượng (Quality Gap)	Kết luận
QC-01	**Functional Completeness**	Bao phủ 100% REQ; Thực thi 100% TC; Pass ≥ 85\%; Pass P0 = 100%	Bao phủ **100%** (28/28 REQ); Thực thi **100%** (60/60); Pass chung **70.00%** (42/60); Pass P0 **61.90%** (13/21); Pass P1 **71.88%** (23/32); Pass P2 **85.71%** (6/7)	Thiếu hụt **-15.00%** tỷ lệ Pass chung và **-38.10%** ở nhóm P0 (8/21 TC P0 FAIL) do các lỗi phân quyền và tích hợp UI.	**CHƯA ĐẠT**

Mã QC	Đặc tính chất lượng	Mục tiêu đề xuất	Kết quả thực tế đã xác minh	Khoảng cách chất lượng (Quality Gap)	Kết luận
QC-02	**Functional Correctness**	Sai số tiền = 0; Không nhảy bước trạng thái	Cộng lặp tiền khi duyệt CONFIRMED lần 2 (BUG-005); nhảy cóc PENDING → DELIVERED (BUG-004)	Thiếu bước kiểm tra trạng thái hiện tại (oldStatus) trong DonationService và SupportRequestService.	**KHÔNG ĐẠT**
QC-03	**Testability & Coverage**	Line Coverage service ≥ 70\%, toàn cục ≥ 50\%; V(G) ≤ 10	Baseline: 14.33% (service) / 25.53% (toàn cục). **Full-Suite: 78.43% (service) / 78.61% (toàn cục)**; V(G) ∈ [3, 5] ≤ 10	Bộ test gốc thiếu hụt **-55.67%** ở tầng service. Bộ test của nhóm đã bổ sung độ bao phủ này (**vượt mục tiêu +8.43%**).	**ĐẠT** *(Nhờ bộ test mở rộng)*
QC-04	**Maintainability & Static Analysis**	Build 100%; 0 lỗi no-undef; Bundle < 500 kB	Build thành công (10/10 test gốc Pass), nhưng có **5 lỗi biến dự án no-undef** (36 tổng), **83 warnings**, Bundle **1,214.90 kB**	Tồn tại 5 vị trí gọi biến chưa khai báo gây lỗi chức năng (BUG-006) và bundle vượt ngưỡng **+714.90 kB**.	**KHÔNG ĐẠT**
QC-05	**Security — Access Control**	100% kịch bản bảo mật & phân quyền chặn đúng truy cập trái phép	Trên **15 TC bảo mật & phân quyền** tại Mục 3.4.3b: **10/15 TC (66.67%) FAIL** (TC-010 [BUG-001], TC-012 [BUG-002], TC-032, 034, 039, 043, 048 [BUG-003], TC-049 [BUG-009], TC-052 [BUG-008], TC-057 [BUG-007]) và **5/15 TC (33.33%) PASS** (TC-011, 015, 016, 021, 023). *(Xét riêng 13 TC phân quyền RBAC/IDOR cấp API: **8/13 TC [61.54%] FAIL** và **5/13 TC [38.46%] PASS**)*	Chỉ cấu hình phân quyền cho /users, /dashboard, /campaigns và /disasters, bỏ sót @PreAuthorize trên 14/15 Controller nghiệp vụ.	**KHÔNG ĐẠT (Nghiêm trọng)**

Mã QC	Đặc tính chất lượng	Mục tiêu đề xuất	Kết quả thực tế đã xác minh	Khoảng cách chất lượng (Quality Gap)	Kết luận
QC-06	**Security — Auth & Input Safety**	Không đổi mật khẩu không OTP; Chặn file .html	Đổi mật khẩu Admin không cần OTP (BUG-007); upload malicious.html trả về HTTP 200 OK (BUG-008)	Thiếu cơ chế xác thực khôi phục mật khẩu và thiếu danh sách trắng phần mở rộng tệp tải lên.	**KHÔNG ĐẠT (Nghiêm trọng)**
QC-07	**Reliability & Fault Tolerance**	0 lỗi HTTP 500 do input; Không mất dữ liệu khi F5	Lỗi HTTP 500 tại TC-024, TC-053 (BUG-010); mất dữ liệu state khi nhấn F5 (BUG-006)	Thiếu @Valid trên 2 controller và chưa kết nối AppContext.jsx với api.js / localStorage.	**KHÔNG ĐẠT**
QC-08	**Performance Efficiency**	Độ trễ phản hồi từng request API cục bộ p95 < 500 ms	Đo độc lập **N = 20 request/endpoint** (08-qc08-api-latency-benchmark.log): - GET /campaigns: **p50 = 4.53 ms**, **p95 = 6.16 ms** (Avg 4.78 ms) - POST /auth/login (BCrypt): **p50 = 87.83 ms**, **p95 = 177.97 ms** (Avg 99.62 ms) - POST /support-requests: **p50 = 10.62 ms**, **p95 = 17.57 ms** (Avg 13.71 ms) - PATCH /donations/{id}/status: **p50 = 12.60 ms**, **p95 = 25.20 ms** (Avg 15.70 ms)	Cả 4 nhóm API đại diện đều có **p95 ≤ 177.97 ms < 500 ms** (nhanh hơn ngưỡng SLA 2.8–81 lần; tổng 53 TC chạy hết ~15.1s).	**ĐẠT (Backend)**

6.2.2. Ma trận đánh giá rủi ro (Likelihood × Impact)

Bảng 6.3: Ma trận đánh giá rủi ro (Risk Assessment Matrix: Khả năng xảy ra × Mức độ tác động)

Mức độ Khả năng xảy ra (Likelihood) \ Tác động (Impact)	Tác động Thấp (Low)	Tác động Trung bình (Medium)	Tác động Cao (High)	Tác động Thảm họa (Catastrophic)
Rất cao (Almost Certain - Xảy ra khi thao tác thông thường)	BUG-012 *(Bundle lớn & biến thừa)*	BUG-011 *(Sai mật khẩu finance01)*	BUG-002 *(Non-admin bị chặn HTTP 403 ở /users/me)*	**BUG-006** *(Citizen gửi yêu cầu cứu trợ bị lỗi; F5 mất dữ liệu)*
Cao (Likely - Dễ khai thác không cần công cụ phức tạp)	-	BUG-009 *(IDOR đọc thông báo người khác)*	BUG-004 *(Nhảy cóc quy trình 6 bước)* BUG-005 *(Cộng dồn lặp tiền chiến dịch)*	**BUG-001** *(Tự đăng ký quyền ADMIN)* **BUG-003** *(Non-admin tự duyệt tiền/xóa kho)* **BUG-007** *(Đổi mật khẩu admin trên UI)*
Trung bình (Possible - Cần truyền dữ liệu biên/độc hại)	-	BUG-010 *(Truyền số âm hoặc ngày sai gây HTTP 500)*	**BUG-008** *(Upload file .html chứa mã độc Stored XSS)*	-

6.2.3. Giới hạn thực nghiệm

Để đảm bảo tính minh bạch, nhóm liệt kê rõ các nội dung chưa thể kiểm thử hoặc chưa có đủ bằng chứng trực quan trong phạm vi hiện tại của dự án:

1. **Kiểm thử tích hợp mạng HTTP E2E trực tiếp trên mã nguồn gốc bị chặn bởi BUG-006 (Đã xác minh gỡ chặn trên nhánh bản vá thử nghiệm tại Mục 7.1.1):** Trên mã nguồn gốc chưa chỉnh sửa, AppContext.jsx không import api.js và ném ReferenceError: API_ENABLED is not defined khiến giao diện Frontend chỉ đọc/ghi trên mảng tĩnh SEED_* trong RAM. Để kiểm chứng triệt để giải pháp khắc phục mà không làm thay đổi hiện trạng mã nguồn gốc được đánh giá ở Chương 6, nhóm đã tạo nhánh thử nghiệm riêng test/fix-bug006-e2e-verify, áp dụng đúng 5 dòng vá BUG-006 và thu thập đầy đủ bằng chứng E2E

thực tế (POST /api/support-requests [HTTP 200 OK], lưu CSDL và giữ nguyên dữ liệu sau F5) tại **Mục 7.1.1 (Hình 7.1 & 09-e2e-post-patch-verification.log)**.

2. **Giới hạn về kiểm thử chịu tải đồng thời đa luồng (Concurrency / Load & Stress Testing):** Ở tiêu chí QC-08, nhóm đã đo thực nghiệm độ trễ từng request độc lập (N = 20 requests/endpoint, ghi nhận p50 = 4.53–87.83 ms, p95 = 6.16–177.97 ms < 500 ms tại 08-qc08-api-latency-benchmark.log). Tuy nhiên, phép đo này thực hiện tuần tự trên một kết nối cục bộ; nhóm chưa thực thi kịch bản bắn tải đa luồng (100–500 Virtual Users) bằng Apache JMeter / Grafana k6.
3. **Chưa có bằng chứng quét lỗ hổng động tự động (Active DAST — OWASP ZAP):** Các lỗ hổng bảo mật (BUG-001, BUG-002, BUG-003, BUG-007, BUG-008, BUG-009) được xác minh thông qua bộ kiểm thử RBAC/IDOR/Upload bằng Spring Security Test và Chrome CDP, chưa thực hiện quét fuzzing tự động bằng OWASP ZAP.
4. **Ảnh chụp TC-060 sau khi tải lại trang (Hình 5.23):** Ở lần chạy gốc, ảnh của TC-060 được chụp ở mili-giây thứ 270 sau lệnh Page.reload (F5) nên chỉ là khung hình trắng (4,768 bytes). Ở lần chạy tái xác minh (Lần chạy 4), nhóm chờ trang tải xong trước khi chụp và đã bổ sung ảnh thực tế (Hình 5.23). Tuy vậy, riêng ảnh chụp vẫn chưa đủ chứng minh việc mất phiên đăng nhập; kết luận TC-060 FAIL dựa trên Hình 5.23 kết hợp nhật ký localStorage.reliefhub_token = null tại Phụ lục C (06-frontend-webui-execution.log, Mục C.3).
5. **Khoảng trống 31 bảng phụ trong kịch bản SQL (reliefhub_full.sql):** Như đã thống kê tại Bảng 4.6 (Mục 4.5.3) và Bảng 2.2 (Mục 2.4), có 31/48 bảng trong tệp SQL (cùng RefreshTokenRequest và password_reset_tokens) chưa có Entity/Controller/Service trong mã nguồn Java, do đó chỉ có thể đối chiếu tĩnh mà không thể kiểm thử động.

CHƯƠNG 7: ĐỀ XUẤT KHẮC PHỤC, CẢI TIẾN VÀ KẾT LUẬN

7.1. Đề xuất giải pháp khắc phục lỗi theo mức ưu tiên (P0 → P2)

Dựa trên nguyên nhân gốc rễ đã xác minh ở Chương 5–6, nhóm đề xuất lộ trình sửa lỗi theo 3 giai đoạn ưu tiên:

7.1.1. Giai đoạn 1: Đề xuất giải pháp khắc phục các lỗi mức P0 (Critical và High cốt lõi)

1. **Phương án sửa lỗi đề xuất cho BUG-001 (SF-01 — Chặn tự đăng ký quyền admin / finance trong AuthService.java:51–85):**
    - Kiểm tra danh sách vai trò được phép đăng ký công khai và đặt trạng thái INACTIVE chờ duyệt cho local / volunteer:

```
  String roleName = (request.getRoleName() != null
&& !request.getRoleName().isBlank())
        ? request.getRoleName().trim().toLowerCase() : "citizen";
  if (!Set.of("citizen", "donor", "local", "volunteer").contains(roleName)) {
     throw new BadRequestException("Không được phép tự đăng ký vai trò quản trị: "
+ roleName);
  }
  User.UserStatus initialStatus = Set.of("local", "volunteer").contains(roleName)
        ? User.UserStatus.INACTIVE : User.UserStatus.ACTIVE;
```

1. **Phương án sửa lỗi đề xuất cho BUG-002 (SF-02) và BUG-003 (SF-03 - Phân quyền RBAC trong SecurityConfig.java:63–82):**
    - Đặt quy tắc /users/me/** trước /users/** (dòng 78) và bổ sung phân quyền cho các endpoint nghiệp vụ trước dòng .anyRequest().authenticated() (dòng 81):

```
  .requestMatchers("/users/me", "/users/me/**").authenticated()
  .requestMatchers("/users/**").hasRole("ADMIN")
  .requestMatchers(HttpMethod.PATCH,
"/donations/*/status").hasAnyRole("ADMIN", "FINANCE")
  .requestMatchers(HttpMethod.POST, "/ledger/**").hasAnyRole("ADMIN",
"FINANCE")
```

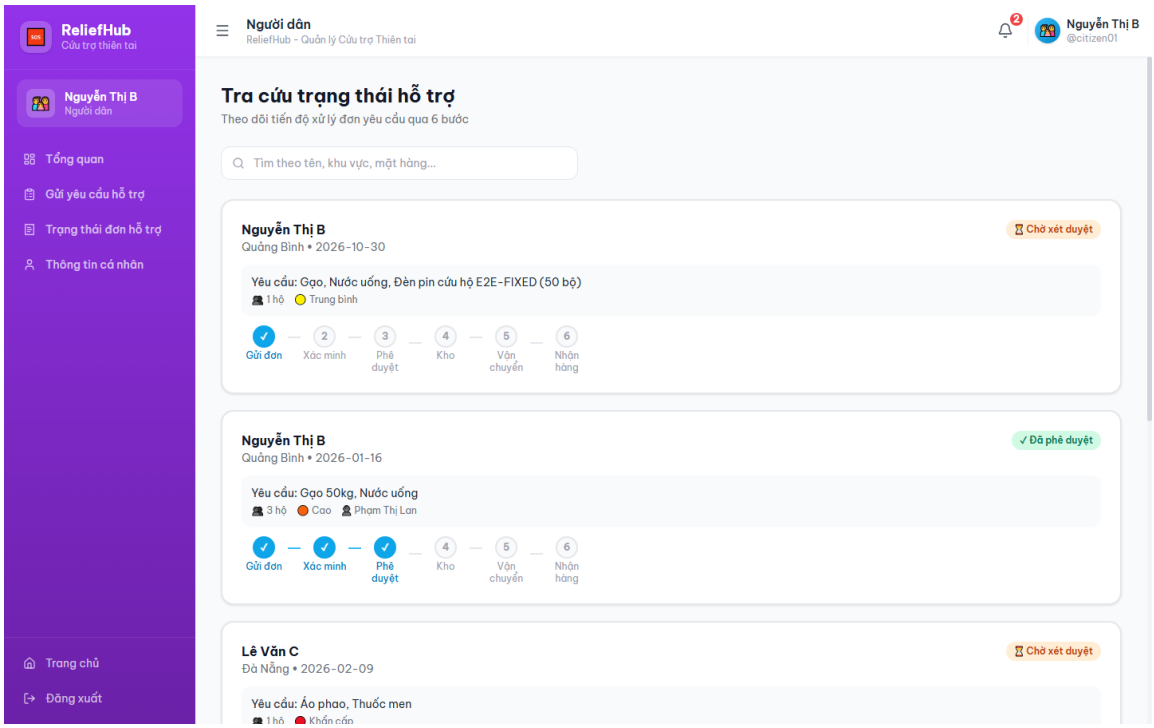
```
  .requestMatchers(HttpMethod.DELETE, "/ledger/**").hasAnyRole("ADMIN",
"FINANCE")
  .requestMatchers(HttpMethod.PATCH, "/support-
requests/*/advance").hasAnyRole("ADMIN", "LOCAL", "VOLUNTEER")
  .requestMatchers(HttpMethod.DELETE, "/inventory/**").hasAnyRole("ADMIN",
"LOCAL")
  .requestMatchers(HttpMethod.PUT, "/complaints/*/reply").hasRole("ADMIN")
```

1. **Phương án sửa lỗi đề xuất cho BUG-005 (SF-05 - Chống cộng tiền trùng lặp trong DonationService.java:84–100):**
    - Lưu lại trạng thái cũ Donation.DonationStatus oldStatus = donation.getStatus(); trước khi gán trạng thái mới ở dòng 88, và chỉ cộng campaign.currentAmount khi thỏa điều kiện oldStatus != Donation.DonationStatus.CONFIRMED && donation.getStatus() == Donation.DonationStatus.CONFIRMED.
2. **Phương án sửa lỗi đề xuất cho BUG-004 (SF-04 - Kiểm soát thứ tự chuyển bước trong SupportRequestService.java:83–98):**
    - Bổ sung hàm isValidTransition(RequestStatus current, RequestStatus next) chỉ cho phép chuyển tiếp tuần tự theo quy trình (PENDING → VERIFIED/REJECTED, VERIFIED → APPROVED/REJECTED, APPROVED → WAREHOUSE_READY, WAREHOUSE_READY → IN_TRANSIT, IN_TRANSIT → DELIVERED), ném BadRequestException nếu chuyển vượt bước.
3. **Phương án sửa lỗi đề xuất cho BUG-006 (SF-06) và BUG-007 (SF-07) trên Frontend (AppContext.jsx & Login.jsx):**
    - Import đầy đủ supportRequestService, tokenManager từ ../services/api và khai báo hằng số const API_ENABLED = Boolean(import.meta.env.VITE_API_ENABLED); ở đầu tệp AppContext.jsx.
    - Trong Request.jsx, bổ sung khối bắt lỗi try { await addNeed(...); setSubmitted(true); } catch (err) { setError(err.message); }.
    - Trong Status.jsx, lọc danh sách yêu cầu theo người dùng hiện tại thay vì hiển thị toàn bộ mảng needs.
    - Gỡ bỏ chức năng đổi mật khẩu trực tiếp không cần xác thực trong Login.jsx (BUG-007).

- **Bằng chứng xác nhận hiệu quả bản vá BUG-006 trên nhánh kiểm chứng độc lập (Post-patch E2E Verification - Hình 7.1):**

Để chứng minh thực nghiệm rằng phương án sửa lỗi 5 dòng trong AppContext.jsx gỡ bỏ hoàn toàn trạng thái BLOCKED của kiểm thử tích hợp End-to-End (E2E) mà vẫn giữ nguyên hiện trạng mã nguồn gốc trên nhánh chính, nhóm đã tạo nhánh thử nghiệm cô lập test/fix-bug006-e2e-verify, áp dụng bản vá kết nối authService.login + supportRequestService.create + supportRequestService.getMine và khởi chạy đồng thời Backend Spring Boot (:8080) cùng Frontend Vite (:5173). Kết quả thực thi tự động qua Chrome CDP (nhật ký 09-e2e-post-patch-verification.log tại Phụ lục C) xác nhận:

1. Đăng nhập citizen01 / citizen123 gọi thật POST /api/auth/login -> HTTP 200 OK và lưu bền vững localStorage.reliefhub_token.
2. Gửi yêu cầu cứu trợ tại /dashboard/citizen/request gọi thật POST /api/support-requests -> HTTP 200 OK, sinh bản ghi CSDL id=2, requestCode="REQ-1793387758667", status="PENDING", **0 ngoại lệ ReferenceError** trên Console.
3. Sau khi nhấn **F5 (Page.reload)** tại /dashboard/citizen/status, trình duyệt tự động đính kèm Bearer JWT gọi GET /api/support-requests/me -> HTTP 200 OK và hiển thị nguyên vẹn đơn cứu trợ từ CSDL (**Hình 7.1**):



Hình 7.1: Bằng chứng xác nhận hiệu quả bản vá BUG-006 trên nhánh kiểm chứng độc lập - Giao diện /dashboard/citizen/status sau khi nhấn F5 Reload vẫn giữ nguyên phiên đăng nhập citizen01 và hiển thị đơn cứu trợ REQ-1793387758667 lấy trực tiếp

từ CSDL qua GET /api/support-requests/me [HTTP 200 OK] (Nhật ký tại Phụ lục C: 09-e2e-post-patch-verification.log).

7.1.2. Giai đoạn 2: Đề xuất giải pháp khắc phục các lỗi mức P1 (BUG-008, BUG-009, BUG-010)

1. **Phương án sửa lỗi đề xuất cho BUG-008 (FileUploadService.java:19–35):** Kiểm tra danh sách phần mở rộng hợp lệ (.jpg, .jpeg, .png, .webp, .pdf) và Content-Type; từ chối các tệp .html, .jsp, .js, .exe với BadRequestException.
2. **Phương án sửa lỗi đề xuất cho BUG-009 (NotificationService.java:27–32):** Truyền currentUserId vào markAsRead(Long id, Long currentUserId) và kiểm tra quyền sở hữu trước khi cập nhật.
3. **Phương án sửa lỗi đề xuất cho BUG-010:** Thêm @Positive cho targetAmount, quantity, householdCount; thêm @Valid vào ComplaintController.reply() và SupportRequestController.advance(); bắt ngoại lệ DateTimeParseException trong GlobalExceptionHandler để trả về HTTP 400 Bad Request.

7.1.3. Giai đoạn 3: Đề xuất giải pháp khắc phục các lỗi mức P2 (BUG-011, BUG-012)

1. **Phương án sửa lỗi đề xuất cho BUG-011:** Đồng bộ mật khẩu tài khoản finance01 trong AppContext.jsx:24 thành fin123 để khớp với reliefhub_full.sql và HUONG_DAN_CHAY.md; sửa lỗi chính tả "Email đã tồn tạo" tại UserService.java:84.
2. **Phương án sửa lỗi đề xuất cho BUG-012:** Dọn dẹp 83 biến dư thừa theo báo cáo Oxlint và áp dụng React.lazy() kết hợp <Suspense> trong App.jsx để giảm kích thước gói bundle khởi đầu từ 1,214.90 kB xuống dưới 300 kB.

7.2. Cải tiến kiến trúc tích hợp và kiểm thử tự động (CI/CD)

1. **Tích hợp bộ kiểm thử ReliefHubFullEvaluationTest và plugin JaCoCo vào pom.xml:** Đưa 53 kịch bản kiểm thử tích hợp API và RBAC của nhóm vào nhánh chính và cấu hình ngưỡng kiểm soát jacoco:check tối thiểu 70% Line Coverage cho gói com.reliefhub.service.
2. **Thiết lập GitHub Actions CI Pipeline:** Tự động chạy lệnh mvn test jacoco:report cho Backend và npx oxlint src -D no-undef && npx vite build cho Frontend trên mỗi Pull Request nhằm phát hiện sớm lỗi biến chưa khai báo (no-undef) hoặc lỗi phân quyền trước khi hợp nhất mã nguồn.

7.3. Kết luận chung

Qua quá trình khảo sát mã nguồn, lập kế hoạch kiểm thử theo chuẩn IEEE 829 và thực thi thực nghiệm 60 trường hợp kiểm thử (TC-001 → TC-060), nhóm rút ra các kết luận sau:

1. **Về mặt chức năng cơ bản:** Phân hệ Backend có cấu trúc phân tầng rõ ràng, các luồng nghiệp vụ chuẩn (Happy Paths) đạt 42 / 60 Test Case PASS (70.00%) và độ trễ phản hồi từng request API độc lập (N = 20 requests/endpoint) đạt p50 = 4.53–87.83 ms và p95 = 6.16–177.97 ms (tổng thời gian chạy 53 kịch bản tích hợp đạt ~15.1s, đáp ứng vượt mức tiêu chí p95 < 500 ms của QC-08).
2. **Về độ bao phủ kiểm thử:** Bộ test gốc của dự án (10 bài test) chỉ đạt 25.53% Line Coverage (14.33% Line Coverage, 6.36% Branch Coverage và 15.83% Method Coverage ở tầng service), chưa bao phủ các nhánh kiểm soát quyền và ngoại lệ. Bộ kiểm thử mở rộng của nhóm đã nâng Line Coverage toàn dự án lên 78.61% (Method Coverage đạt 50.53% toàn dự án và 59.68% trên khối Non-DTO) và nâng tầng service lên 78.43% Line Coverage, 44.07% Branch Coverage và 58.27% Method Coverage.
3. **Về các lỗi chất lượng và bảo mật đã phát hiện:** Nhóm đã ghi nhận và xác minh bằng thực nghiệm 12 lỗi phần mềm (BUG-001 → BUG-012), tương ứng với 18 / 60 Test Case FAIL (30.00%). Các lỗi đáng chú ý nhất gồm: tự đăng ký quyền Admin (BUG-001), thiếu phân quyền trên hơn 10 API nhạy cảm (BUG-003), cộng dồn lặp tiền chiến dịch (BUG-005), đổi mật khẩu Admin không cần OTP qua form Quên mật khẩu (BUG-007) và lỗi ReferenceError: API_ENABLED is not defined làm yêu cầu cứu trợ của người dân không được lưu lại trên giao diện (BUG-006).
4. **Đánh giá tổng thể:** Hệ thống ở trạng thái hiện tại chưa đáp ứng yêu cầu an toàn để đưa vào vận hành thực tế. Nhóm phát triển cần áp dụng các bản sửa mã nguồn đề xuất ở Mục 7.1 và thực hiện kiểm thử hồi quy trước khi triển khai.

TÀI LIỆU THAM KHẢO

1. **[1] IEEE Std 829-2008 & ISO/IEC/IEEE 29119-3:2021**, *Software and systems engineering - Software testing - Part 3: Test documentation (Test Plan, Test Design Specification, Test Case Specification, Test Incident Report)*, IEEE / ISO, 2021.
2. **[2] ISO/IEC 25010:2011 & ISO/IEC 25023:2016**, *Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models & Measurement of system and software product quality*, International Organization for Standardization, 2016.
3. **[3] McCabe, T. J. (1976)**, *"A Complexity Measure"*, IEEE Transactions on Software Engineering, Vol. SE-2, No. 4, pp. 308–320, DOI: 10.1109/TSE.1976.233837.
4. **[4] Myers, G. J., Sandler, C., & Badgett, T. (2011)**, *The Art of Software Testing* (3rd Edition), John Wiley & Sons, Hoboken, New Jersey, ISBN: 978-1-118-03196-4.
5. **[5] Ammann, P., & Offutt, J. (2016)**, *Introduction to Software Testing* (2nd Edition - Graph Coverage, Logic Coverage & Input Space Partitioning), Cambridge University Press, ISBN: 978-1-107-17201-2.
6. **[6] VMware / Pivotal Software (2024)**, *Spring Boot 3.2.5 Reference Documentation, Spring Security 6.2 Reference (SecurityFilterChain, @EnableMethodSecurity, @PreAuthorize) & Spring Framework MockMvc Integration Testing Guide*, https://docs.spring.io/spring-boot/docs/3.2.5/reference/html/.
7. **[7] Mountainminds GmbH & Co. KG (2024)**, *JaCoCo Java Code Coverage Library v0.8.12 Official Documentation (Bytecode Instrumentation, LINE, BRANCH & METHOD Counters)*, https://www.jacoco.org/jacoco/trunk/doc/.
8. **[8] OWASP Foundation (2021)**, *OWASP Top 10:2021 (A01:2021 Broken Access Control & IDOR, A03:2021 Injection & Stored XSS, A04:2021 Insecure Design, A07:2021 Identification and Authentication Failures) & OWASP Application Security Verification Standard (ASVS 4.0.3)*, https://owasp.org/Top10/.
9. **[9] Meta Open Source & VoidZero Inc. (2025)**, *React 19.2 Official Documentation, React Router v7.18 Guide, Vite 8.1 Production Bundler & Oxlint 1.69 Static Analyzer Documentation*, https://react.dev/ & https://oxc.rs/.

PHỤ LỤC

Phụ lục A: Độ bao phủ JaCoCo chi tiết theo lớp

Bên cạnh 15 lớp thuộc gói com.reliefhub.service đã được trình bày tại Bảng 5.3 (Mục 5.2), Bảng A.1 dưới đây trình bày đầy đủ số liệu đo độ bao phủ dòng lệnh (Line), nhánh rẽ (Branch) và phương thức (Method) của toàn bộ 28 lớp thực thi còn lại thuộc các gói controller, security, exception, config và common để người đọc có thể tra cứu trực tiếp trên bản in.

Bảng A.1: Độ bao phủ mã nguồn JaCoCo (Line, Branch, Method Coverage) của các lớp thuộc gói controller, security, exception, config và common

STT	Gói (Package)	Tên Lớp (Class)	Baseline Line Coverage	Full-Suite Line Coverage	Full-Suite Branch Coverage	Baseline Method Coverage	Full-Suite Method Coverage
1	controller	AuthController	5 / 5 (100.00%)	**5 / 5 (100.00%)**	*0 nhánh*	3 / 3 (100.00%)	**3 / 3 (100.00%)**
2	controller	DashboardController	1 / 2 (50.00%)	**2 / 2 (100.00%)**	*0 nhánh*	1 / 2 (50.00%)	**2 / 2 (100.00%)**
3	controller	FileUploadController	1 / 2 (50.00%)	**2 / 2 (100.00%)**	*0 nhánh*	1 / 2 (50.00%)	**2 / 2 (100.00%)**
4	controller	SupportRequestController	1 / 6 (16.67%)	**5 / 6 (83.33%)**	*0 nhánh*	1 / 6 (16.67%)	**5 / 6 (83.33%)**
5	controller	ComplaintController	1 / 6 (16.67%)	**4 / 6 (66.67%)**	*0 nhánh*	1 / 6 (16.67%)	**4 / 6 (66.67%)**
6	controller	LedgerController	1 / 6 (16.67%)	**4 / 6 (66.67%)**	*0 nhánh*	1 / 5 (20.00%)	**3 / 5 (60.00%)**
7	controller	CampaignController	3 / 8 (37.50%)	**5 / 8 (62.50%)**	*0 nhánh*	3 / 7 (42.86%)	**5 / 7 (71.43%)**
8	controller	InventoryController	1 / 8 (12.50%)	**5 / 8 (62.50%)**	*0 nhánh*	1 / 7 (14.29%)	**4 / 7 (57.14%)**
9	controller	UserController	1 / 12 (8.33%)	**7 / 12 (58.33%)**	*0 nhánh*	1 / 9 (11.11%)	**5 / 9 (55.56%)**
10	controller	DisasterController	1 / 8 (12.50%)	**4 / 8 (50.00%)**	*0 nhánh*	1 / 7 (14.29%)	**4 / 7 (57.14%)**
11	controller	DonationController	1 / 6 (16.67%)	**3 / 6 (50.00%)**	*0 nhánh*	1 / 6 (16.67%)	**3 / 6 (50.00%)**

STT	Gói (Package)	Tên Lớp (Class)	Baseline Line Coverage	Full-Suite Line Coverage	Full-Suite Branch Coverage	Baseline Method Coverage	Full-Suite Method Coverage
12	controller	DeliveryController	1 / 7 (14.29%)	**3 / 7 (42.86%)**	*0 nhánh*	1 / 6 (16.67%)	**3 / 6 (50.00%)**
13	controller	NotificationController	1 / 7 (14.29%)	**3 / 7 (42.86%)**	*0 nhánh*	1 / 5 (20.00%)	**2 / 5 (40.00%)**
14	controller	WarehouseController	1 / 5 (20.00%)	**2 / 5 (40.00%)**	*0 nhánh*	1 / 4 (25.00%)	**2 / 4 (50.00%)**
15	controller	VolunteerGroupController	1 / 8 (12.50%)	**3 / 8 (37.50%)**	*0 nhánh*	1 / 7 (14.29%)	**3 / 7 (42.86%)**
16	security	SecurityConfig	50 / 50 (100.00%)	**50 / 50 (100.00%)**	*0 nhánh*	13 / 13 (100.00%)	**13 / 13 (100.00%)**
17	security	CustomUserDetails	21 / 21 (100.00%)	**21 / 21 (100.00%)**	1 / 2 (50.00%)	12 / 12 (100.00%)	**12 / 12 (100.00%)**
18	security	JwtAuthenticationFilter	17 / 19 (89.47%)	**17 / 19 (89.47%)**	6 / 8 (75.00%)	4 / 4 (100.00%)	**4 / 4 (100.00%)**
19	security	CustomUserDetailsService	5 / 6 (83.33%)	**5 / 6 (83.33%)**	1 / 2 (50.00%)	2 / 3 (66.67%)	**2 / 3 (66.67%)**
20	security	JwtTokenProvider	28 / 41 (68.29%)	**28 / 41 (68.29%)**	*0 nhánh*	7 / 8 (87.50%)	**7 / 8 (87.50%)**
21	exception	BadRequestException	2 / 2 (100.00%)	**2 / 2 (100.00%)**	*0 nhánh*	1 / 1 (100.00%)	**1 / 1 (100.00%)**
22	exception	GlobalExceptionHandler	7 / 23 (30.43%)	**21 / 23 (91.30%)**	2 / 2 (100.00%)	4 / 9 (44.44%)	**8 / 9 (88.89%)**
23	exception	ResourceNotFoundException	0 / 13 (0.00%)	**5 / 13 (38.46%)**	*0 nhánh*	0 / 5 (0.00%)	**1 / 5 (20.00%)**
24	config	OpenApiConfig	12 / 12 (100.00%)	**12 / 12 (100.00%)**	*0 nhánh*	2 / 2 (100.00%)	**2 / 2 (100.00%)**
25	config	WebConfig	4 / 4 (100.00%)	**4 / 4 (100.00%)**	*0 nhánh*	2 / 2 (100.00%)	**2 / 2 (100.00%)**
26	config	DataInitializer	7 / 13 (53.85%)	**7 / 13 (53.85%)**	2 / 8 (25.00%)	3 / 3 (100.00%)	**3 / 3 (100.00%)**

STT	Gói (Package)	Tên Lớp (Class)	Baseline Line Coverage	Full-Suite Line Coverage	Full-Suite Branch Coverage	Baseline Method Coverage	Full-Suite Method Coverage
27	common	ApiResponse	21 / 32 (65.63%)	**30 / 32 (93.75%)**	0 / 42 (0.00%)*	11 / 23 (47.83%)	**13 / 23 (56.52%)**
28	common	ApiResponseBuilder	1 / 1 (100.00%)	**1 / 1 (100.00%)**	1 / 2 (50.00%)	6 / 8 (75.00%)	**6 / 8 (75.00%)**

Phụ lục B: Kết quả 53 Test Case Backend và nhật ký 14 ca FAIL

B.1. Phản hồi HTTP thực tế của 53 Test Case Backend

Bảng B.1: Bảng kết quả thực thi và phản hồi HTTP thực tế của 53 trường hợp kiểm thử Backend (TC-001 → TC-053)

Mã TC	Yêu cầu	Endpoint / Phương thức được kiểm tra	HTTP Status thực tế	Tóm tắt dữ liệu phản hồi JSON thực tế (Actual Response Body)	Kết quả
TC-001	REQ-001	POST /auth/login (testadmin / test123)	200 OK	{"success":true,"message":"Đăng nhập thành công","data":{"token":"eyJhbGci...","role":"admin"}}	**PASS**
TC-002	REQ-001	POST /auth/login (sai mật khẩu)	401 Unauthorized	{"success":false,"error":"Tên đăng nhập hoặc mật khẩu không đúng"}	**PASS**
TC-003	REQ-001	POST /auth/login (trống username, password)	400 Bad Request	{"success":false,"data":{"password":"Mật khẩu không được để trống","username":"Username không được để trống"},"error":"Dữ liệu không hợp lệ"}	**PASS**
TC-004	REQ-004	JwtTokenProvider.generateToken / validateToken	Unit Return	validateToken(token) = true, getUsernameFromToken(token) = "testadmin"	**PASS**
TC-005	REQ-002	POST /auth/register (newuser, role=citizen)	200 OK	{"success":true,"message":"Đăng ký thành công","data":{"username":"newuser","role":"citizen"}}	**PASS**

Mã TC	Yêu cầu	Endpoint / Phương thức được kiểm tra	HTTP Status thực tế	Tóm tắt dữ liệu phản hồi JSON thực tế (Actual Response Body)	Kết quả
TC-006	REQ-002	POST /auth/register (trùng username="testadmin")	400 Bad Request	{"success":false,"error":"Username đã tồn tại"}	**PASS**
TC-007	REQ-002	POST /auth/register (trùng email="admin@test.com")	400 Bad Request	{"success":false,"error":"Email đã tồn tại"}	**PASS**
TC-008	REQ-002	POST /auth/register (username="ab", pw="12345")	400 Bad Request	{"success":false,"data":{"password":"Mật khẩu tối thiểu 6 ký tự","email":"Email không hợp lệ","username":"Username phải từ 3-50 ký tự"}}	**PASS**
TC-009	REQ-002	POST /auth/register (roleName="invalid_role")	400 Bad Request	{"success":false,"error":"Vai trò không hợp lệ: invalid_role"}	**PASS**
TC-010	REQ-003	POST /auth/register (roleName="admin")	200 OK *(Kỳ vọng 400/403)*	{"success":true,"message":"Đăng ký thành công","data":{"username":"hacker_admin","role":"admin"}}	**FAIL (BUG-001)**
TC-011	REQ-004	GET /users (không có JWT Token)	401 Unauthorized	{"success":false,"error":"Unauthorized - Vui lòng đăng nhập"}	**PASS**
TC-012	REQ-005	GET & PUT /users/me (donorToken)	403 Forbidden *(Kỳ vọng 200)*	GET=403, PUT=403 | {"success":false,"error":"Forbidden - Không có quyền truy cập"}	**FAIL (BUG-002)**
TC-013	REQ-006	PUT /users/me/password (đúng mật khẩu cũ)	200 OK	{"success":true,"message":"Đổi mật khẩu thành công"}	**PASS**
TC-014	REQ-006	PUT /users/me/passwor	400 Bad Request	{"success":false,"error":"Mật khẩu cũ không đúng"}	**PASS**

Mã TC	Yêu cầu	Endpoint / Phương thức được kiểm tra	HTTP Status thực tế	Tóm tắt dữ liệu phản hồi JSON thực tế (Actual Response Body)	Kết quả
		d (sai mật khẩu cũ)			
TC-015	REQ-007	GET & POST /users (adminToken)	200 OK	{"success":true,"message":"Tạo tài khoản thành công","data":{"username":"fin_new","role":"finance","status":"ACTIVE"}}	**PASS**
TC-016	REQ-007	GET & POST /users (donorToken)	403 Forbidden	GET=403, POST=403 (Forbidden)	**PASS**
TC-017	REQ-007	PATCH /users/{id}/toggle-status (2 lần)	200 OK	Lần 1: status="INACTIVE", Lần 2: status="ACTIVE"	**PASS**
TC-018	REQ-007	GET & DELETE /users/99999 (adminToken)	404 Not Found	{"success":false,"error":"User không tìm thấy với id = '99999'"}	**PASS**
TC-019	REQ-008	GET /campaigns, /campaigns/active, /disasters, /disasters/active	200 OK (cả 4 API)	statuses = 200, 200, 200, 200 (Không cần đăng nhập)	**PASS**
TC-020	REQ-008	GET /campaigns/99999 & /disasters/99999	404 Not Found	campaign=404, disaster=404	**PASS**
TC-021	REQ-009	POST /disasters (localToken vs donorToken)	200 OK / 403	LOCAL=200 OK, DONOR=403 Forbidden	**PASS**
TC-022	REQ-010	POST /campaigns (adminToken, target=100M)	200 OK	{"success":true,"message":"Tạo chiến dịch thành công","data":{"id":2,"targetAmount":100000000,"progressPct":0.0}}	**PASS**
TC-023	REQ-010	POST /campaigns (donorToken)	403 Forbidden	{"success":false,"error":"Forbidden - Không có quyền truy cập"}	**PASS**
TC-024	REQ-010	POST /campaigns (target=-50M & startDate="invalid-date")	200 OK & 500 *(Kỳ vọng 400)*	negativeTargetStatus=200 (targetAmount=-50000000); invalidDateStatus=500 (DateTimeParseException)	**FAIL (BUG-010)**

Mã TC	Yêu cầu	Endpoint / Phương thức được kiểm tra	HTTP Status thực tế	Tóm tắt dữ liệu phản hồi JSON thực tế (Actual Response Body)	Kết quả
TC-025	REQ-011	GET /campaigns/{id} (100M / 300M)	200 OK	{"success":true,"data":{"targetAmount":300000000.00,"currentAmount":100000000.00,"progressPct":33.3}}	**PASS**
TC-026	REQ-011	GET /campaigns/{id} (targetAmount = 0)	200 OK	{"success":true,"data":{"targetAmount":0.00,"currentAmount":0.00,"progressPct":0.0}}	**PASS**
TC-027	REQ-012	POST /donations (amount=5M + items=[Gạo])	200 OK	{"success":true,"message":"Đã ghi nhận đóng góp","data":{"id":1,"amount":5000000,"status":"PENDING"}}	**PASS**
TC-028	REQ-012	POST /donations (amount=0 và amount=-1000)	400 Bad Request	amount=0 -> 400, amount=-1000 -> 400 (@Positive)	**PASS**
TC-029	REQ-014	POST /donations (isAnonymous=true)	200 OK	{"success":true,"data":{"id":2,"donorName":"Ẩn danh","isAnonymous":true}}	**PASS**
TC-030	REQ-013	PATCH /donations/1/status (CONFIRMED lần 1)	200 OK	campaign.currentAmount = 5000000.00	**PASS**
TC-031	REQ-013	PATCH /donations/1/status (CONFIRMED lần 2)	200 OK *(Cộng lặp)*	campaign.currentAmount = 10000000.00 (Bị cộng dồn nhân đôi!)	**FAIL (BUG-005)**
TC-032	REQ-013	PATCH /donations/{id}/status (donorToken)	200 OK *(Kỳ vọng 403)*	{"success":true,"data":{"id":3,"donorName":"Test Donor","status":"CONFIRMED"}}	**FAIL (BUG-003)**
TC-033	REQ-015	POST /ledger (amount=10M vs amount=-500)	200 OK / 400	valid=200 OK, negative=400 Bad Request	**PASS**
TC-034	REQ-015	POST & DELETE /ledger (citizenToken)	200 OK *(Kỳ vọng 403)*	POST=200 OK, DELETE=200 OK (CITIZEN tự ghi và xóa sổ cái)	**FAIL (BUG-003)**

Mã TC	Yêu cầu	Endpoint / Phương thức được kiểm tra	HTTP Status thực tế	Tóm tắt dữ liệu phản hồi JSON thực tế (Actual Response Body)	Kết quả
TC-035	REQ-016	POST /support-requests (citizenToken, hợp lệ)	200 OK	{"success":true,"message":"Đã gửi yêu cầu","data":{"id":1,"requestCode":"REQ-1792591164350","status":"PENDING"}}	**PASS**
TC-036	REQ-016	POST /support-requests (itemName="" vs quantity=-20)	400 & 200 OK *(Kỳ vọng 400)*	blank=400, nhưng negativeQty=200 OK (quantity=-20, householdCount=-1 được lưu)	**FAIL (BUG-010)**
TC-037	REQ-017	PATCH /support-requests/{id}/advance (5 bước tuần tự)	200 OK (cả 5 bước)	VERIFIED:200, APPROVED:200, WAREHOUSE_READY:200, IN_TRANSIT:200, DELIVERED:200	**PASS**
TC-038	REQ-017	PATCH /support-requests/3/advance (PENDING → DELIVERED)	200 OK *(Kỳ vọng 400)*	{"success":true,"data":{"id":3,"status":"DELIVERED"}} (Cho phép nhảy cóc bỏ qua 4 bước)	**FAIL (BUG-004)**
TC-039	REQ-017	PATCH /support-requests/4/advance (citizenToken → APPROVED)	200 OK *(Kỳ vọng 403)*	{"success":true,"data":{"id":4,"status":"APPROVED"}} (CITIZEN tự duyệt đơn của mình)	**FAIL (BUG-003)**
TC-040	REQ-018	GET /support-requests/me & /status/DELIVERED	200 OK	me=200 OK, status=200 OK	**PASS**
TC-041	REQ-019	POST /warehouses & POST /inventory	200 OK	{"success":true,"message":"Đã thêm mặt hàng","data":{"id":1,"itemName":"Gạo ST25","warehouseName":"Kho Miền Trung"}}	**PASS**
TC-042	REQ-020	PATCH /inventory/{id}/restock (+200 vs 0, -50)	200 OK / 400	valid(200)->qty=700, zero->400, neg->400	**PASS**

Mã TC	Yêu cầu	Endpoint / Phương thức được kiểm tra	HTTP Status thực tế	Tóm tắt dữ liệu phản hồi JSON thực tế (Actual Response Body)	Kết quả
TC-043	REQ-019	DELETE /inventory/{id} (citizenToken)	200 OK *(Kỳ vọng 403)*	{"success":true,"message":"Đã xóa mặt hàng"} (CITIZEN xóa mặt hàng kho thành công)	**FAIL (BUG-003)**
TC-044	REQ-021	POST /deliveries & PATCH /deliveries/{id}/status	200 OK	status="DELIVERED", arriveAt tự động ghi nhận thời gian thực	**PASS**
TC-045	REQ-022	POST /volunteer-groups & GET /volunteer-groups/active	200 OK	{"success":true,"data":[{"id":1,"name":"Đội Cứu hộ 1","status":"ACTIVE"}]}	**PASS**
TC-046	REQ-019	GET /inventory/99999 & DELETE /deliveries/99999	404 Not Found	inventory=404, delivery=404	**PASS**
TC-047	REQ-023	POST /complaints, PUT /reply, PATCH /close	200 OK (cả 3 bước)	Kết thúc ở status="CLOSED", progress=100, repliedAt != null	**PASS**
TC-048	REQ-023	PUT /complaints/{id}/reply (citizenToken & adminReply="")	200 OK *(Kỳ vọng 403 & 400)*	citizenStatus=200 OK, blankReplyStatus=200 OK (Thiếu @PreAuthorize và @Valid)	**FAIL (BUG-003/010)**
TC-049	REQ-024	PATCH /notifications/1/read (donorToken đọc thông báo Admin)	200 OK *(Kỳ vọng 403/404)*	status=200 OK, adminNotif.isRead=true (Lỗ hổng IDOR)	**FAIL (BUG-009)**
TC-050	REQ-025	GET /dashboard/stats (ADMIN, FINANCE, CITIZEN)	200 / 200 / 403	ADMIN=200, FINANCE=200, CITIZEN=403 | totalUsers=9, totalCampaigns=5	**PASS**
TC-051	REQ-026	POST /uploads (0B, 1KB, 11MB)	400 / 200 / 400	0B=400, 1KB=200, 11MB=400	**PASS**

Mã TC	Yêu cầu	Endpoint / Phương thức được kiểm tra	HTTP Status thực tế	Tóm tắt dữ liệu phản hồi JSON thực tế (Actual Response Body)	Kết quả
TC-052	REQ-026	POST /uploads (malicious.html, text/html)	200 OK *(Kỳ vọng 400)*	{"success":true,"data":{"url":"/uploads/f964f0a8-e3cb-46ec-a17a-af41c268759f.html","mimeType":"text/html"}}	**FAIL (BUG-008)**
TC-053	REQ-027	PATCH /support-requests/{id}/advance (INVALID_STATE & {})	400 & 500 *(Kỳ vọng 400)*	INVALID_STATE=400, nhưng nullStatus=500 (NullPointerException trên getNewStatus())	**FAIL (BUG-010)**

B.2. Nhật ký 14 Test Case Backend thất bại

Dưới đây là toàn bộ 14 bản ghi nhật ký lỗi nguyên bản từ bộ kiểm thử tích hợp Backend (TC-010, TC-012, TC-024, TC-031, TC-032, TC-034, TC-036, TC-038, TC-039, TC-043, TC-048, TC-049, TC-052, TC-053):

FAIL: TC-010 (REQ-003) — (Bảo mật - SF-01) Chặn tự đăng ký quyền Admin qua /auth/register

Endpoint : POST /auth/register

Expected : Từ chối 400/403 hoặc ép về role=citizen, không cấp role=admin

Actual : HTTP 200 | Body: {"success":true,"message":"Đăng ký thành công","data":{"token":"eyJhbGciOiJIUzUxMiJ9...","tokenType":"Bearer","userId":7,"username":"hacker_admin","role":"admin"}}

Note : XÁC NHẬN LỖI BUG-001 (SF-01): Hệ thống trả về 200 OK và cấp JWT token với role=admin cho người tự đăng ký!

FAIL: TC-012 (REQ-005) — (Phân quyền - SF-02) Người dùng thông thường (donor) xem/sửa hồ sơ (/users/me)

Endpoint : GET & PUT /users/me

Expected : HTTP 200 OK cho cả GET và PUT /users/me

Actual : HTTP 403 | Body: GET=403, PUT=403 | {"success":false,"error":"Forbidden - Không có quyền truy cập"}

Note : XÁC NHẬN LỖI BUG-002 (SF-02): SecurityConfig.java dòng 78 đặt .requestMatchers("/users/**").hasRole("ADMIN") chặn mọi non-admin truy cập /users/me với HTTP 403!

FAIL: TC-024 (REQ-010) — (Biên & Ngoại lệ - SF-10) Tạo chiến dịch với targetAmount < 0 và startDate sai định dạng

Endpoint : POST /campaigns

Expected : Cả 2 trả về HTTP 400 Bad Request

Actual : HTTP 200 & HTTP 500 | negativeTargetStatus=200 ({"success":true,"message":"Tạo chiến dịch thành công","data":{"id":3,"title":"Camp âm","targetAmount":-50000000,"currentAmount":0,"progressPct":0.0}}) | invalidDateStatus=500 (DateTimeParseException)

Note : XÁC NHẬN LỖI BUG-010 (SF-10): targetAmount=-50,000,000 vẫn trả về HTTP 200 OK; startDate sai định dạng ném DateTimeParseException làm sập request với HTTP 500!

FAIL: TC-031 (REQ-013) — (Logic tài chính - SF-05) Kiểm tra tính bất biến khi gọi CONFIRMED lặp lại lần 2

Endpoint : PATCH /donations/{id}/status

Expected : currentAmount giữ nguyên 5,000,000 (không cộng trùng)

Actual : HTTP 200 | Body: campaign.currentAmount=10000000.00

Note : XÁC NHẬN LỖI BUG-005 (SF-05): Gọi CONFIRMED lần 2 trên cùng Donation làm campaign.currentAmount bị cộng nhân đôi lên 10000000.00!

FAIL: TC-032 (REQ-013) — (Bảo mật - SF-03) Chặn DONOR tự gọi API duyệt quyên góp của chính mình

Endpoint : PATCH /donations/{id}/status

Expected : HTTP 403 Forbidden

Actual : HTTP 200 | Body: {"success":true,"message":"Thành công","data":{"id":3,"userId":2,"donorName":"Test Donor","amount":2000000.00,"status":"CONFIRMED"}}

Note : XÁC NHẬN LỖI BUG-003 (SF-03): DonationController thiếu @PreAuthorize nên DONOR tự duyệt khoản quyên góp sang CONFIRMED với HTTP 200 OK!

FAIL: TC-034 (REQ-015) — (Bảo mật - SF-03) Chặn CITIZEN tạo hoặc xóa bút toán sổ cái tài chính (/ledger)

Endpoint : POST & DELETE /ledger

Expected : HTTP 403 Forbidden

Actual : HTTP 200 | Body: POST=200, DELETE=200

Note : XÁC NHẬN LỖI BUG-003 (SF-03): LedgerController không phân quyền nên CITIZEN tự ghi và xóa bút toán sổ cái với HTTP 200 OK!

FAIL: TC-036 (REQ-016) — (Biên & Validation - SF-10) Gửi yêu cầu hỗ trợ với itemName rỗng hoặc quantity < 0

Endpoint : POST /support-requests

Expected : Cả 2 trả về HTTP 400 Bad Request

Actual : HTTP 200 | Body: blank=400, negativeQty=200 | {"success":true,"message":"Đã gửi yêu cầu","data":{"id":2,"itemName":"Gạo","quantity":-20,"area":"Đà Nẵng","householdCount":-1}}

Note : XÁC NHẬN LỖI BUG-010 (SF-10): SupportRequestDTO thiếu @Positive trên quantity và householdCount nên chấp nhận quantity=-20, householdCount=-1 với HTTP 200 OK!

FAIL: TC-038 (REQ-017) — (Chuyển trạng thái - SF-04) Chặn nhảy cóc trạng thái từ PENDING thẳng sang DELIVERED

Endpoint : PATCH /support-requests/{id}/advance

Expected : HTTP 400 Bad Request (không cho phép bỏ qua VERIFIED/APPROVED/WAREHOUSE_READY/IN_TRANSIT)

Actual : HTTP 200 | Body: {"success":true,"message":"Thành công","data":{"id":3,"requestCode":"REQ-1792591165294","itemName":"Mì tôm","status":"DELIVERED"}}

Note : XÁC NHẬN LỖI BUG-004 (SF-04): SupportRequestService.advanceStatus() không kiểm tra máy trạng thái, cho phép nhảy từ PENDING thẳng sang DELIVERED với HTTP 200 OK!

FAIL: TC-039 (REQ-017) — (Bảo mật - SF-03) Chặn CITIZEN tự phê duyệt yêu cầu hỗ trợ của chính mình

Endpoint : PATCH /support-requests/{id}/advance

Expected : HTTP 403 Forbidden

Actual : HTTP 200 | Body: {"success":true,"message":"Thành công","data":{"id":4,"requestCode":"REQ-1792591165432","itemName":"Áo phao","status":"APPROVED"}}

Note : XÁC NHẬN LỖI BUG-003 (SF-03): SupportRequestController thiếu @PreAuthorize nên CITIZEN tự phê duyệt yêu cầu của mình sang APPROVED với HTTP 200 OK!

FAIL: TC-043 (REQ-019) — (Bảo mật - SF-03) Chặn CITIZEN xóa mặt hàng tồn kho (DELETE /inventory/{id})

Endpoint : DELETE /inventory/{id}

Expected : HTTP 403 Forbidden

Actual : HTTP 200 | Body: {"success":true,"message":"Đã xóa mặt hàng","timestamp":"2026-10-21T20:59:26.498793"}

Note : XÁC NHẬN LỖI BUG-003 (SF-03): InventoryController thiếu @PreAuthorize nên CITIZEN xóa mặt hàng kho thành công với HTTP 200 OK!

FAIL: TC-048 (REQ-023) — (Bảo mật & Validation - SF-03/SF-10) Chặn non-admin trả lời khiếu nại và kiểm tra @NotBlank

Endpoint : PUT /complaints/{id}/reply

Expected : citizenStatus=403 Forbidden, blankReplyStatus=400 Bad Request

Actual : HTTP 200 | Body: citizenStatus=200, blankReplyStatus=200

Note : XÁC NHẬN LỖI BUG-003 (SF-03) & BUG-010 (SF-10): CITIZEN tự trả lời khiếu nại với HTTP 200 OK, và adminReply rỗng vẫn được chấp nhận (HTTP 200) do thiếu @Valid!

FAIL: TC-049 (REQ-024) — (Bảo mật IDOR - SF-09) Chặn người dùng đánh dấu đã đọc thông báo của người khác

Endpoint : PATCH /notifications/{id}/read

Expected : Từ chối 403/404, không sửa thông báo của Admin

Actual : HTTP 200 | Body: status=200, adminNotif.isRead=true

Note : XÁC NHẬN LỖI BUG-009 (SF-09): NotificationService.markAsRead(id) không kiểm tra chủ sở hữu, cho phép DONOR đánh dấu đã đọc thông báo của ADMIN (HTTP 200 OK)!

FAIL: TC-052 (REQ-026) — (Bảo mật Upload - SF-08) Kiểm tra chặn tải lên file thực thi/script độc hại (.html/.jsp)

Endpoint : POST /uploads

Expected : HTTP 400 Bad Request (chỉ cho phép ảnh/tài liệu an toàn)

Actual : HTTP 200 | Body: {"success":true,"message":"Upload thành công","data":{"fileName":"f964f0a8-e3cb-46ec-a17a-af41c268759f.html","url":"/uploads/f964f0a8-e3cb-46ec-a17a-af41c268759f.html","mimeType":"text/html","sizeBytes":39}}

Note : XÁC NHẬN LỖI BUG-008 (SF-08): FileUploadService.upload() không kiểm tra định dạng/MIME type, cho phép upload malicious.html thành công với HTTP 200 OK!

FAIL: TC-053 (REQ-027) — (Xử lý ngoại lệ - SF-10) Gửi trạng thái không hợp lệ hoặc null vào endpoint cập nhật

Endpoint : PATCH /support-requests/{id}/advance

Expected : Cả 2 trả về HTTP 400 Bad Request

Actual : HTTP 500 | Body: INVALID_STATE=400, nullStatus=500 | {"success":false,"error":"Lỗi hệ thống: Cannot invoke \"String.toUpperCase()\" because the return value of \"com.reliefhub.dto.SupportRequestDTO$AdvanceStatusRequest.getNewStatus()\" is null"}

Note : XÁC NHẬN LỖI BUG-010 (SF-10): Gửi body {} (newStatus=null) gây NullPointerException và trả về HTTP 500 Internal Server Error do thiếu @Valid trên @RequestBody!

Phụ lục C: Nhật ký Oxlint, Vite, WebUI và phụ lục kỹ thuật về tính toàn vẹn của nhật ký thực nghiệm

C.1. Nhật ký phân tích tĩnh Frontend (Oxlint 1.69.0)

```
 x eslint(no-undef): 'API_ENABLED' is not defined.
   ,-[AppContext.jsx:558:9]
557 |   const addNeed = async (payload) => {
558 |     if (API_ENABLED) {
   :         ^^^^^^^^^^^
559 |       try { await supportRequestService.create(payload); } catch {}
   `----
 x eslint(no-undef): 'supportRequestService' is not defined.
   ,-[AppContext.jsx:559:19]
558 |     if (API_ENABLED) {
559 |       try { await supportRequestService.create(payload); } catch {}
   :                   ^^^^^^^^^^^^^^^^^^^^^
   `----
 x eslint(no-undef): 'API_ENABLED' is not defined.
   ,-[AppContext.jsx:780:11]
779 |     const initAuth = async () => {
780 |       if (API_ENABLED && tokenManager.getToken()) {
   :           ^^^^^^^^^^^
781 |         const tokenUser = tokenManager.getUser();
   `----
 x eslint(no-undef): 'tokenManager' is not defined.
   ,-[AppContext.jsx:780:26]
780 |       if (API_ENABLED && tokenManager.getToken()) {
   :                          ^^^^^^^^^^^^
781 |         const tokenUser = tokenManager.getUser();
```

```
 782 |         if (tokenUser) setUser(tokenUser);
     `----

Found 83 warnings and 36 errors.
Finished in 297ms on 50 files with 93 rules using 4 threads.
```

C.2. Nhật ký đóng gói Production Build (Vite 8.1.0)

```
vite v8.1.0 building client environment for production...
transforming...✓ 870 modules transformed.
rendering chunks...
computing gzip size...
index.html                  0.76 kB │ gzip:   0.48 kB
assets/index-BoP8ZAKW.css     65.86 kB │ gzip:  10.62 kB
assets/index-BsLZoEvX.js   1,214.90 kB │ gzip: 318.76 kB

(!) Some chunks are larger than 500 kB after minification. Consider:
- Using dynamic import() to code-split the application
- Use build.rolldownOptions.output.codeSplitting to improve chunking
- Adjust chunk size limit for this warning via build.chunkSizeWarningLimit.
✓ built in 16.30s
```

C.3. Nhật ký thực thi 7 Test Case WebUI (TC-054 → TC-060)

```
[2026-10-21T13:52:05.832Z] === STARTING FRONTEND WEBUI TEST EXECUTION (TC-054 TO TC-060) ===
[2026-10-21T13:52:24.102Z] Vite Dev Server is ready at http://127.0.0.1:5173

--- Running TC-054: Đăng nhập giao diện WebUI và kiểm tra ProtectedRoute ---
[2026-10-21T13:52:44.508Z] TC-054 Step 1: Truy cập /dashboard/admin khi chưa đăng nhập -> URL hiện tại: /login (Hình 5.11)
[2026-10-21T13:52:47.625Z] TC-054 Step 2: Đăng nhập donor01/donor123 -> URL hiện tại: /dashboard/donor (Hình 5.12)
```

[2026-10-21T13:52:49.420Z] TC-054 Step 3: Donor thử truy cập /dashboard/admin -> Bị đẩy về: /dashboard/donor (Hình 5.13)

=> TC-054 Result: PASS

--- Running TC-055: Đăng nhập WebUI với tài khoản đang chờ phê duyệt (pending_approval) ---

[2026-10-21T13:52:54.672Z] TC-055: Thông báo hiển thị = "Tài khoản của bạn đang chờ admin phê duyệt." (Hình 5.14)

=> TC-055 Result: PASS

--- Running TC-056: (SF-11) Đối chiếu đăng nhập finance01 / fin123 theo tài liệu ---

[2026-10-21T13:53:02.220Z] TC-056: Đăng nhập finance01/fin123 -> URL=/login, bị từ chối trên giao diện (Hình 5.15)

=> TC-056 Result: FAIL (Xác nhận lỗi BUG-011 / SF-11: Mâu thuẫn mật khẩu fin123 vs finance123)

--- Running TC-057: (SF-07) Kiểm tra lỗ hổng đặt lại mật khẩu Admin không cần xác thực ---

[2026-10-21T13:53:11.874Z] TC-057: Reset thành công=true, Đăng nhập admin/hacked123 -> URL=/dashboard/admin (Hình 5.16, 5.17, 5.18)

=> TC-057 Result: FAIL (Xác nhận lỗi nghiêm trọng BUG-007 / SF-07: Chiếm quyền Admin không cần OTP/Email!)

--- Running TC-058: (SF-06) Người dân gửi yêu cầu cứu trợ trên giao diện ---

[2026-10-21T13:53:16.204Z] (Browser Uncaught Exception) ReferenceError: API_ENABLED is not defined at AppContext.jsx:558 (handleSubmit at Request.jsx:40:3)

[2026-10-21T13:53:18.358Z] TC-058: UI hiện "Yêu cầu đã gửi!"=true (Hình 5.19), Xuất hiện trong trang Trạng thái=false (Hình 5.20)

=> TC-058 Result: FAIL (Xác nhận lỗi BUG-006 / SF-06: ReferenceError: API_ENABLED is not defined + False Positive UI làm mất dữ liệu yêu cầu cứu trợ!)

--- Running TC-059: Luồng Nhà tài trợ gửi chuyển khoản & Admin xác minh trên WebUI ---

[2026-10-21T13:53:29.782Z] TC-059: Đóng góp đã ghi nhận=true (Hình 5.21) và Admin xác minh trên UI (Hình 5.22)

=> TC-059 Result: PASS

--- Running TC-060: (SF-06) Kiểm tra tính bền vững dữ liệu trên WebUI sau khi F5 (Reload) ---

[2026-10-21T13:53:31.734Z] TC-060: Trước khi F5 URL=/dashboard/admin/verify -> Sau khi F5 URL=/dashboard/admin/verify, localStorage.reliefhub_token=null (Hình 5.23)

=> TC-060 Result: FAIL (Xác nhận lỗi BUG-006 / SF-06: Tải lại trang F5 làm mất phiên đăng nhập và mất toàn bộ dữ liệu thao tác do không lưu localStorage / không gọi REST API!)

C.4. Phụ lục kỹ thuật về tính toàn vẹn của nhật ký thực nghiệm (Bảng C.1)

Để đảm bảo tính truy vết khoa học và khả năng kiểm chứng độc lập đối với toàn bộ dữ liệu thực nghiệm được trích dẫn trong Chương 4, Chương 5, Chương 6, Chương 7 và Phụ lục B–C, **Bảng C.1** trình bày danh mục 11 tệp nhật ký thực nghiệm cùng dung lượng và mã băm mật mã học SHA-256:

Bảng C.1: Phụ lục kỹ thuật về tính toàn vẹn của 11 tệp nhật ký thực nghiệm (SHA-256)

STT	Tên tệp nhật ký logic	Dung lượng (bytes)	Mã băm toàn vẹn SHA-256	Nội dung bằng chứng thực nghiệm
1	01-env-check.log	916 bytes	F3A3355A72B41ED7374A0FA82E61F22FD93BB5931CF5640FDF71B409CEFD47CD	Kiểm tra môi trường thực nghiệm (OS, JDK 21.0.11, Maven 3.9.16, Node v24.13.0, npm 11.6.2, Chrome 156, MySQL80)
2	02-mvn-test-existing.log	16,964 bytes	3B167B71C00E07567B98761DB0A3C2F2135A6D324DCC29E53656265B156CC24E	Lần chạy 1: 10 bài test gốc (Tests run: 10, Failures: 0, Maven Surefire 3.1.2) và JaCoCo Baseline (25.53%)

STT	Tên tệp nhật ký logic	Dung lượng (bytes)	Mã băm toàn vẹn SHA-256	Nội dung bằng chứng thực nghiệm
3	03-backend-tc001-to-tc053-results.log	24,159 bytes	BFC51630697BD7428EEC89C189782F1ED72C5B1F15068FD11E2574D5E4558EB2	Lần chạy 2: Kết quả 53 Test Case Backend (39 PASS / 14 FAIL, gồm 2 Unit Return + 51 HTTP API)
4	03-mvn-test-full-suite.log	140,831 bytes	6DFA0B594FC50A1F5923AA41608B9029F797027149BC03DB90A1613A42A10688	Lần chạy 2: Maven Surefire 3.1.2 và JaCoCo 0.8.12 Full-Suite (78.61% Line Coverage)
5	04-frontend-oxlint.log	111,222 bytes	06D7556F8203D5331B0C2115052BDEC14E5A73E1F0982865EDE56FD98CFE9FB8	Lần chạy 3: Oxlint 1.69.0 (83 warnings, 36 errors với -D no-undef)
6	05-frontend-vite-build.log	1,545 bytes	C20183577EB74D2E9D66A87AB7E749182AF6282299CBBECDFE56D6DE30852184	Lần chạy 3: Vite 8.1.0 (870 modules, bundle JS 1,214.90 kB)
7	06-frontend-webui-execution.log	11,794 bytes	5E67EF1B7D03A3C966478F30FE1B5B7E822F766DD8547017F643D3FFEEC61315	Lần chạy 4: 7 Test Case WebUI (TC-054 → TC-060: 3 PASS / 4 FAIL)
8	07-priority-audit.log	7,091 bytes	75370087A58C526E6753767F408F4A793E100D4CFA01FEDC32591ABEF111133F	Kiểm toán tự động phân bố độ ưu tiên 60 Test Case (P0 = 21 [13 PASS/8 FAIL], P1 = 32 [23 PASS/9 FAIL], P2 = 7 [6 PASS/1 FAIL])
9	08-mvn-test-qc08-benchmark.log	21,104 bytes	8F41166EF2C97B5CDEEE4E5E3FB1A8AB70FFC898752C3A5E13B9DCAD61DA1507	Nhật ký Maven Surefire thực thi lớp ReliefHubPerformanceBenchmarkTest (80 HTTP requests, Failures: 0)
10	08-qc08-api-latency-benchmark.log	2,021 bytes	DF11F8E83A922D5F85CF9DD84DBB638296CF725C32E41BE4DB68F667CC242D1F	Kết quả đo độ trễ từng request API cho QC-08 (N = 20 requests/endpoint: Min, Avg, p50, p95, Max)

STT	Tên tệp nhật ký logic	Dung lượng (bytes)	Mã băm toàn vẹn SHA-256	Nội dung bằng chứng thực nghiệm
11	09-e2e-post-patch-verification.log	2,401 bytes	9E2D1EEA550BCCC250A3D4764B82928C2914C82B687739E37AA3E871BF4F8142	Nhật ký xác minh E2E sau khi áp dụng bản vá BUG-006 (POST /api/support-requests [200 OK], lưu DB & giữ trạng thái sau `F5')